

Vivado 设计套件教程（UG871）

HLS
---modified by Jonny.W

UG871 (v2021.1)2021 年 3 月 26 日



修订历史

下表显示了本文档的修订历史。

部分	复习总结
05/22/2019 2019.1 版本	
查找教程设计文件	更新链接。

目录

第一章 教程中描述.....	1
第二章 高层次综合介绍.....	3
第三章 C 验证.....	28
第四章 接口综合.....	39
第五章 任意精度类型.....	62
第六章 设计分析.....	71
第七章 设计优化.....	94
第八章 RTL 验证.....	111
第九章 在 IP 集成器中使用 HLS IP.....	124
第十章 在 Zynq Soc 设计中使用 HLS IP.....	141
第十一章 在 DSP 系统生成器中使用 HLS IP.....	175
附录 A 附加资源和法律通知.....	178

教程中描述

概述

本 Vivado®教程是一组较小的教程，解释和演示了使用高级综合将 C、C++和 SystemC 代码转换为 RTL 实现过程中的所有步骤。本教程展示了如何创建一个初始的 RTL 实现，然后使用优化指令 (*optimization directives*)将其转换为低面积 (*low_area*) 和高吞吐量 (*high-throughput*) 的实现，而无需更改 C 代码。以下部分描述了每个教程的摘要。

高层次综合介绍

本教程介绍 Vivado 高级综合工具(HLS)。您可以学习使用图形用户界面(GUI)和 Tcl 环境执行高级综合的主要任务。

C 验证

本教程回顾了良好的 C 测试平台的各个方面，并演示了 Vivado 高级综合 C 调试环境的基本操作。本教程还展示了如何调试任意精度数据类型。

接口综合

本接口综合教程回顾了为 RTL 设计创建端口的所有方面。您可以了解如何控制块级 I/O 端口协议和端口 I/O 协议，如何将 C 函数中的数组实现为多个端口和接口协议类型(*RAM*、*FIFO*、*AXI4-Stream*)，以及如何实现 AXI4 总线接口。

为了创建设计的最佳实现，本教程最后给出一个设计示例，其中 I/O 访问和逻辑一起优化。

优化设计 (*Design Optimization*)

本教程使用一个矩阵乘法器 (*matrix multiplier*) 示例，回顾了两种设计优化技术。设计优化实验中解释了如何流水线 (*pipeline*) 设计，对比了流水化循环和流水线函数。

本教程向您展示了如何使用从分析中获得的见解来更新最初的 C 代码并创建更优化的设计实现。

RTL 验证

本教程展示了如何使用 RTL Cosimulation(*C-RTL 联合仿真*)特性自动验证合成创建的 RTL。本教程演示了 C测试平台的重要性，并向您展示了如何使用 RTL 验证的输出来查看 Vivado 和 MentorGraphics ModelSim 模拟器中的波形图。

在 IP 整合器中使用 *HLS IP*

本教程展示如何将高级综合创建的 RTL 设计打包为 IP，添加到 Vivado IP 目录中，并在 Vivado 设计套件中使用。

在 Zynq SoC 设计中使用 HLS IP

除了在 Zynq®-7000 SoC 设计中使用一个 HLS IP 块外，本教程还展示了如何将高级综合创建的 C 驱动程序文件合并到 Zynq 处理系统(PS)的软件中。

在 DSP 系统生成器中使用 HLS IP

本教程展示了如何通过高级综合创建的 RTL 设计可以打包成 IP，并在 DSP 的系统生成器中使用。

软件需求

本教程要求安装 Vivado Design Suite 2017.1 或更高版本。

硬件需求

Xilinx 建议使用 Vivado 工具时至少要有 2GB 的 RAM。

查找教程设计文件

教程设计文件位于 Xilinx 网站上的压缩归档文件中。接受许可协议后，zip 文件将自动下载。

重要提示:Vivado 高级合成的所有教程示例都可以在参考设计文件中找到



准备教程设计文件

将 zip 文件内容解压缩到任何可写的位置。本教程假设您已经将解压缩的设计文件放置在

位置 C:\Vivado_HLS_Tutorial 中。



重要提示:如果 Vivado_HLS_Tutorial 目录被解压缩到不同的位置，或者它驻留在 Linux 上，那么将路径名调整到您放置目录的位置。*Vivado_HLS_Tutorial*



第二章

1. 高层次综合介绍

概述

本教程介绍 Vivado®高级合成(*High-Level Synthesis, HLS*)。您可以学习使用图形用户界面(GUI)和 Tcl 环境执行高级综合的主要任务。

本教程展示了如何使用优化指令将初始 RTL 实现转换为低面积和高吞吐量的实现。

实验室1描述

解释如何建立一个高级综合(HLS)项目，并执行 HLS 设计流程中的所有主要步骤:

- 验证 C 代码。
- 创建并合成一个解决方案。
- 验证 RTL 并打包 IP。

实验室2描述

演示如何使用 Tcl 接口。

实验室3描述

演示如何使用优化指令优化设计。这个实验创建 RTL 实现的多个版本，并比较不同的解决方案。

教程设计描述

要获得教程设计文件，请参阅查找教程设计文件。

本教程使用教程目录中的设计文件。

Vivado_HLS_Tutorial\Introduction.

本教程中使用的示例设计是一个 *FIR* 滤波器。*FIR* 设计项目的硬件目标是:

- 创建该设计的最高吞吐量版本。

最终设计必须处理输入有效信号提供的的数据，并产生输出有效信号伴随的输出数据。滤波器系数将存储在 *FIR* 设计的外部(一个单一端口 RAM 中)。

实验 1:创建一个高级合成项目

介绍

这个实验演示了如何创建一个高级的综合项目，验证 C 代码，将设计合成为 RTL，并验证 RTL。



重要提示:本教程中的图形和命令假定教程数据目录文件已解压缩并放置在相应位置 `Vivado_HLS_Tutorial`
`C:\Vivado_HLS_Tutorial`.

步骤 1:创建一个新项目

1.打开 Vivado®HLS 图形用户界面(GUI):

在 Windows 系统上，双击 Vivado HLS 2019.1 桌面图标打开 Vivado HLS。



图 2-1:Vivado HLS 桌面图标° 在 Linux 系统上，在命令提示符(command prompt)处输入: `vivado_hls`

提示:你也可以打开 Vivado HLS 使用 Windows 菜单启动>所有程序>Xilinx 设计工具>Vivado 2019.1>Vivado HLS>Vivado HLS 2019.1。

Vivado HLS 将打开欢迎屏幕，如下所示。如果以前打开过任何项目，它们会显示在“最近项目”窗格中，否则此窗口不会显示在“欢迎”屏幕中。



图 2-2:Vivado HLS 欢迎页面

2. 在欢迎页面中，选择创建新项目以打开项目向导。

3. 如图 2-3 所示：

- a. 输入项目名称：*fir_prj*。
- b. 单击 *Browse* 导航到(介绍)目录的位置
- c. 选择目录路径并单击 OK
- d. 单击 Next。

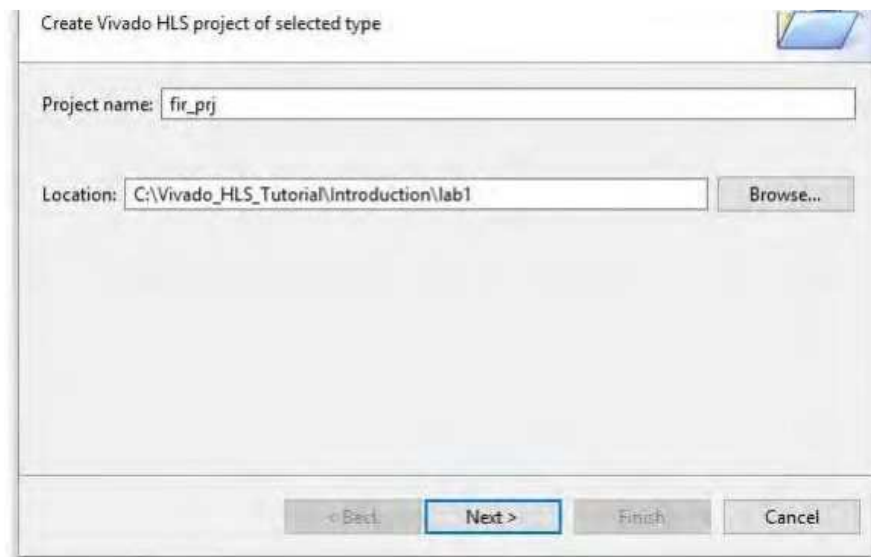


图 2-3: 项目配置

此信息定义 VivadoHLS 项目目录的名称和位置。这种情况下，项目目录是，它驻留在 *fir_prj* 文件夹中。

4. 输入以下信息来指定 C 设计文件：

- a. 点击添加文件 (*Add Files*)。
- b. 选择 *fir.c* 并单击 OK。
- c. 使用 *Browse* 按钮指定 *fir* 作为顶层函数。
- d. 单击 Next。

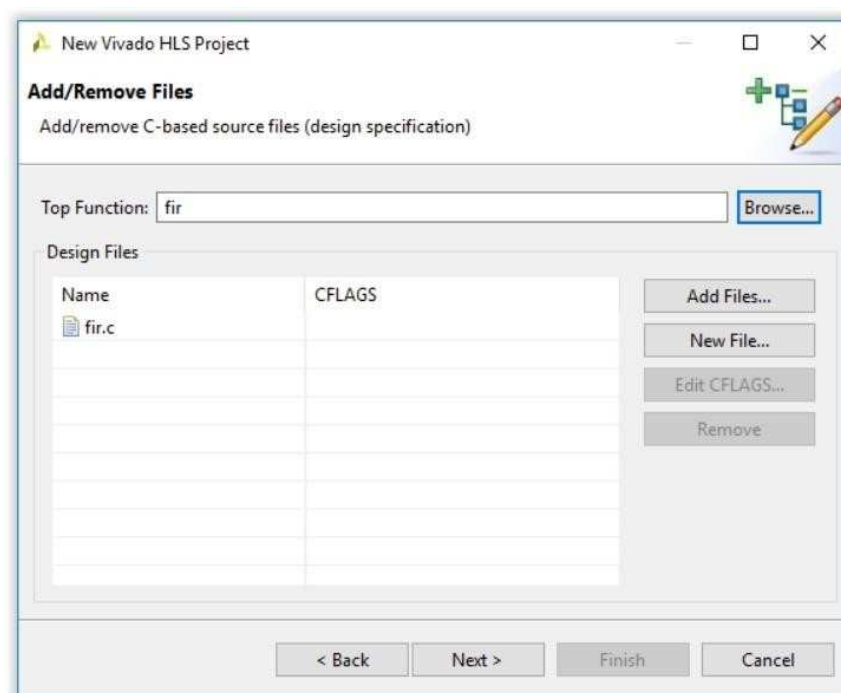


图 2-4: 项目设计文件



重点:在这个实验室里只有一个 C 设计文件。当有多个 C 文件要合成时,您必须在此阶段将它们全部添加到项目中。存在于本地目录中的任何头文件都会自动包含在项目中。*lab1* 如果标题位于不同的位置,使用 *Edit CFLAGS* 按钮添加标准的 gcc/g++ 搜索路径信息(例如, *-I<path_to_header_file_dir>*)。

图 2-5 显示了用于指定测试工作台文件的输入窗口。测试工作台和测试工作台使用的所有文件(头文件除外)必须包括在内。您可以一次添加一个文件,或者使用 Ctrl 和 Shift 键选择要添加的多个文件。

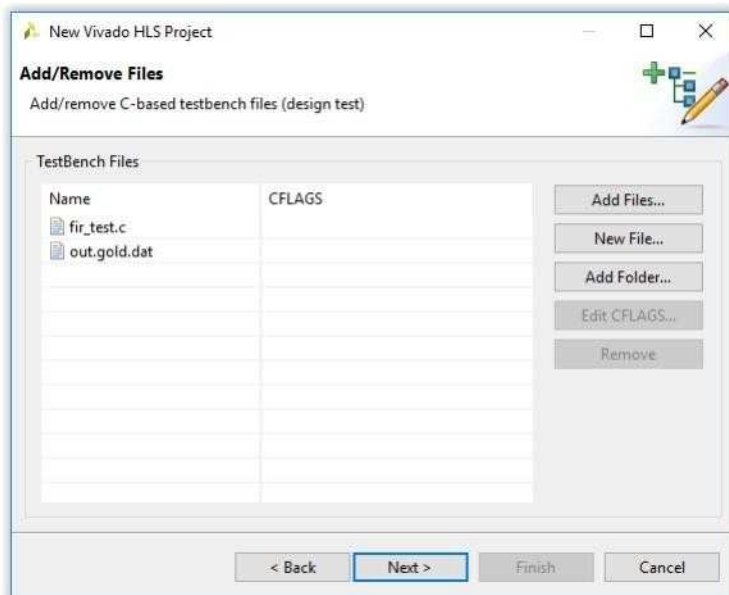


图 2-5:测试工作台文件

5.单击 **Add Files** 按钮来包含测试工作台文件: **fir_test.c** 和 **out.gold.dat**。

6.单击 Next。

C 仿真 (和 RTL CosSimulation)都在解决方案的子目录中执行。

如果您没有包含测试工作台使用的所有文件(例如,由测试工作台读取的数据文件,例如 **out.gold.dat**, C 和 RTL 模拟可能会因为无法找到数据文件而失败。解决方案配置窗口(如图 2-6 所示)指定第一个解决方案的技术规范。

一个项目可以有多个解决方案,每个解决方案使用不同的目标技术、包 (*package*)、约束 (*constraints*) 和/或综合指令 (*synthesis directives*)。

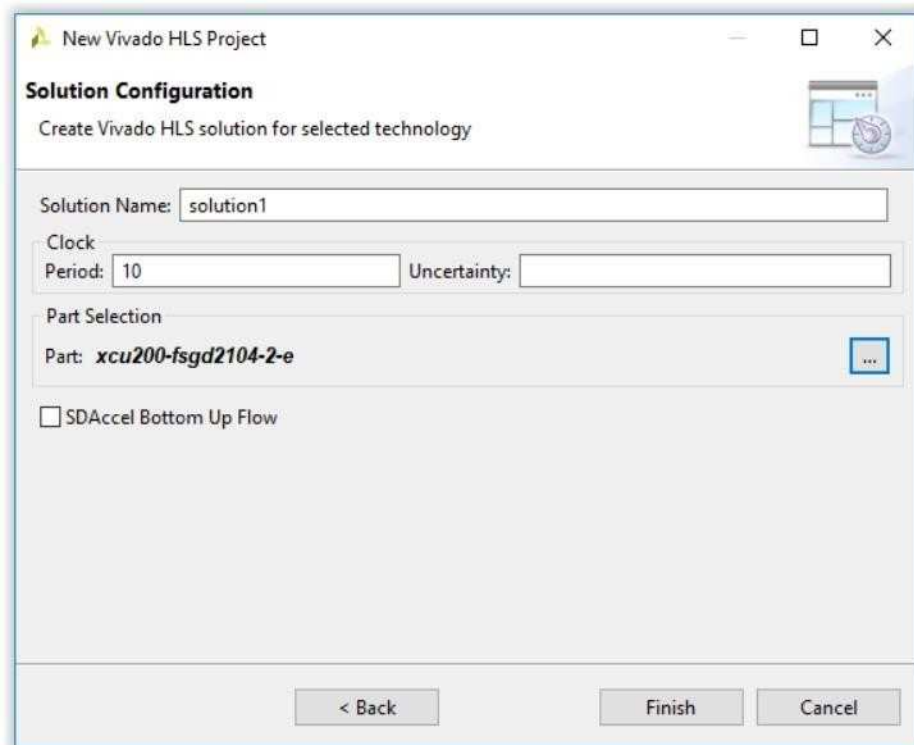


图 2-6: 解决方案配置

7. 接受默认的解决方案名称(*solution1*)、时钟周期(*10ns*)和时钟不确定性(默认为12.5%的时钟周期, 留空/未定义)。
8. 单击零件选择按钮, 打开零件选择窗口。
9. 从可用设备列表中选择设备 *xcvu9p-flgb2104-1*。从下拉筛选器中选择以下内容以帮助细化部件列表:
 - a. 产品类别: 通用
 - b. 家族: *Virtex®UltraScale™*
 - c. 亚科: *Virtex UltraScale+*
 - d. 包装: *flgb2104*
 - e. 速度等级
 - f. 临时等级: 全部
10. 选择 *xcvu9p-flgb2104-1*。
11. 单击 OK。

在“解决方案配置”对话框中(如图2-6所示), 选择的部件名称现在出现在部件选择标题下。

12. 单击 Finish 打开 Vivado HLS 项目, 如图 2-7 所示。

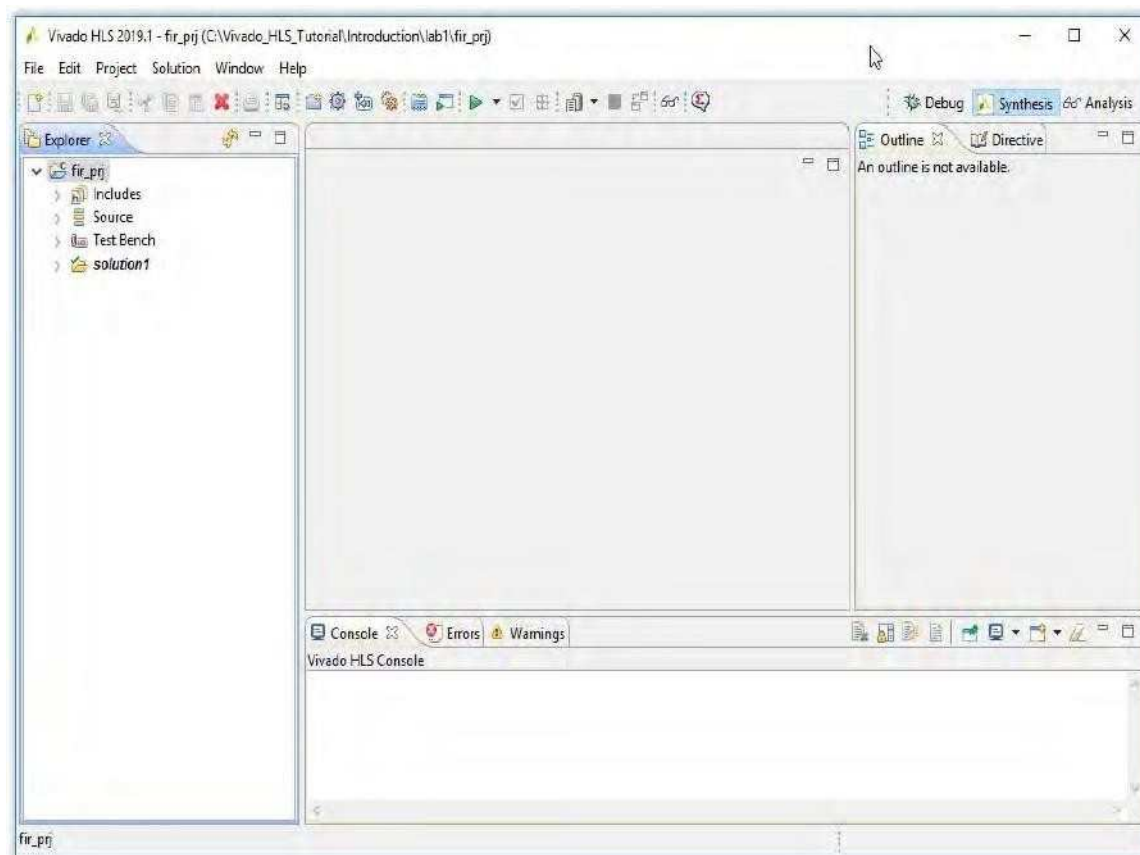


图 2-7: Vivado HLS 项目

- 项目名称出现在资源管理器窗口的顶部一行。
- Vivado HLS 项目以层次结构的形式排列信息。
- 项目包含关于设计源(*design source*)、测试台(*test bench*)和解决方案(*solution*)的信息。
- 解决方案包含目标技术、设计指令和 结果的信息。
- 在一个项目中可能有多个解决方案, 每个解决方案是一个实现相同的源代码。

提示: 您可以随时使用工具栏中相应的项目设置和/或解决方案设置按钮更改项目或解决方案设置。



了解图形用户界面(GUI)

在继续之前，请查看图形用户界面(GUI)中的区域及其功能。图 2-8 显示了区域的概述，并在下面描述了每个区域。

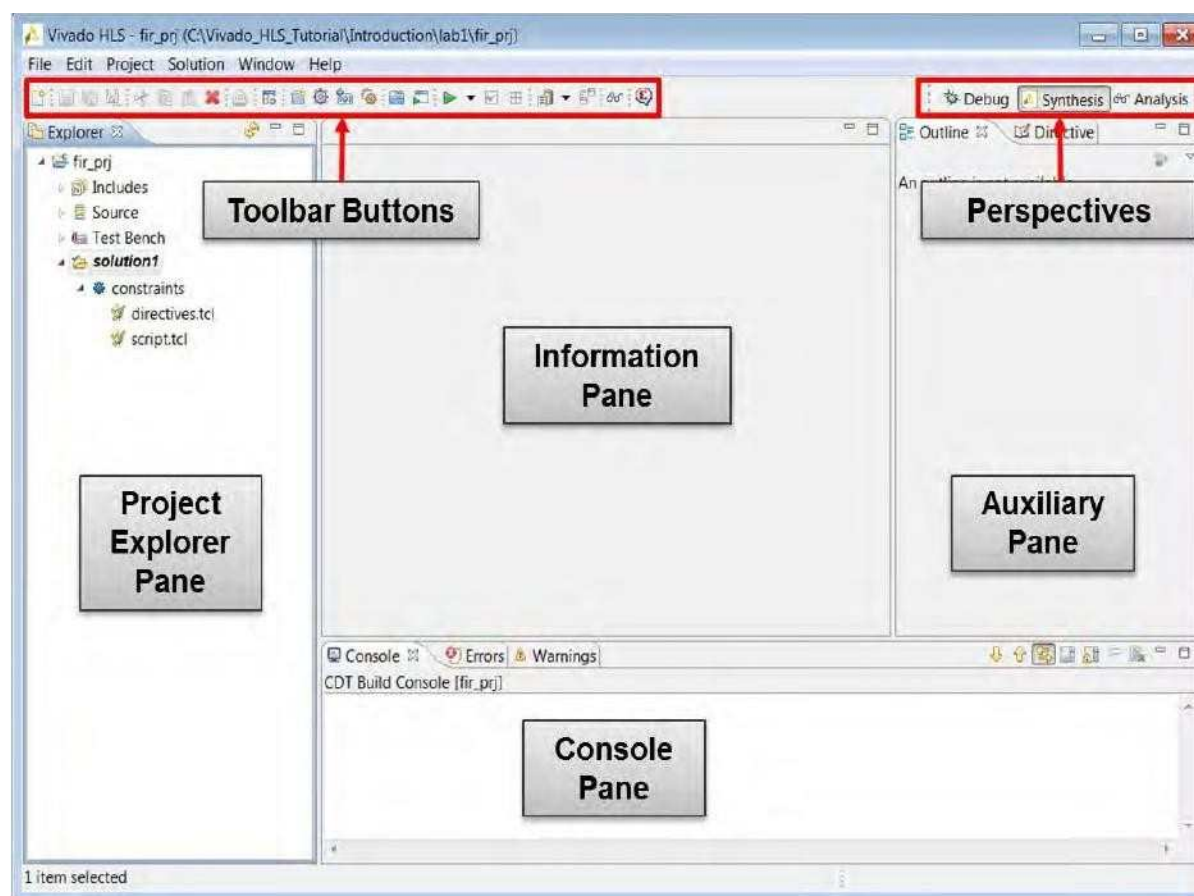


图 2-8: Vivado HLS 图形用户界面

浏览窗格

显示项目层次结构 (*project hierarchy*)。在执行验证 (*validation*)、综合 (*synthesis*)、验证 (*verification*) 和 IP 打包步骤时，将在解决方案目录中自动创建包含每个步骤结果的子文件夹(分别命名和创建 *csim*, *syn*, *sim*, *impl*)。

当您创建新的解决方案时，它们将与 *solution1* 一起出现在项目层次结构中。

信息面板

显示从资源管理器窗格打开的任何文件的内容。当操作完成时，报告文件将在此窗格中自动打开。

辅助面板 (Auxiliary Pane)

与信息窗格交叉链接。此窗格中显示的信息根据信息窗格中打开的文件动态调整。

控制台面板 (Console Pane)

显示 Vivado HLS 运行时生成的消息。错误和警告出现在控制台窗格选项卡中。

工具栏按钮

您可以使用工具栏按钮执行最常见的操作。

当你把光标放在按钮上时，一个弹出的工具提示会打开，解释这个功能。每个按钮还有一个相关的菜单项，可以从下拉菜单中获得。

视角 (Perspectives)

透视图提供了方便的方法来调整 Vivado HLS GUI 中的窗口。

- **综合 (Synthesis) 角度:** 默认透视图允许您综合设计、运行模拟和打包 IP。
- **Debug 透视图**

包括与调试 C 代码相关的窗格。您可以在 C 代码编译之后打开 Debug 透视图(除非使用优化编译模式，因为这会禁用调试信息)。

- **分析 (Analysis) 视角**

此透视图中的窗口配置为支持综合结果分析。您只能在合成完成后使用 Analysis 透视图。

步骤2:验证C源代码

HLS 项目的第一步是确认 C 代码是正确的。这个过程称为 C 验证 (Validation) 或 C 仿真。

在这个项目中，测试台将函数的输出数据与已知的好值进行比较。fir

1. 展开“资源管理器”窗格中的 **Test Bench** 文件夹。
2. 双击 **fir_test.c** 文件以在信息窗格中查看它。
3. 在辅助窗格 (Auxiliary pane) 中，选择 **Outline** 选项卡直接跳转到 **main()** 函数。

图 2-9 显示了这些操作的结果

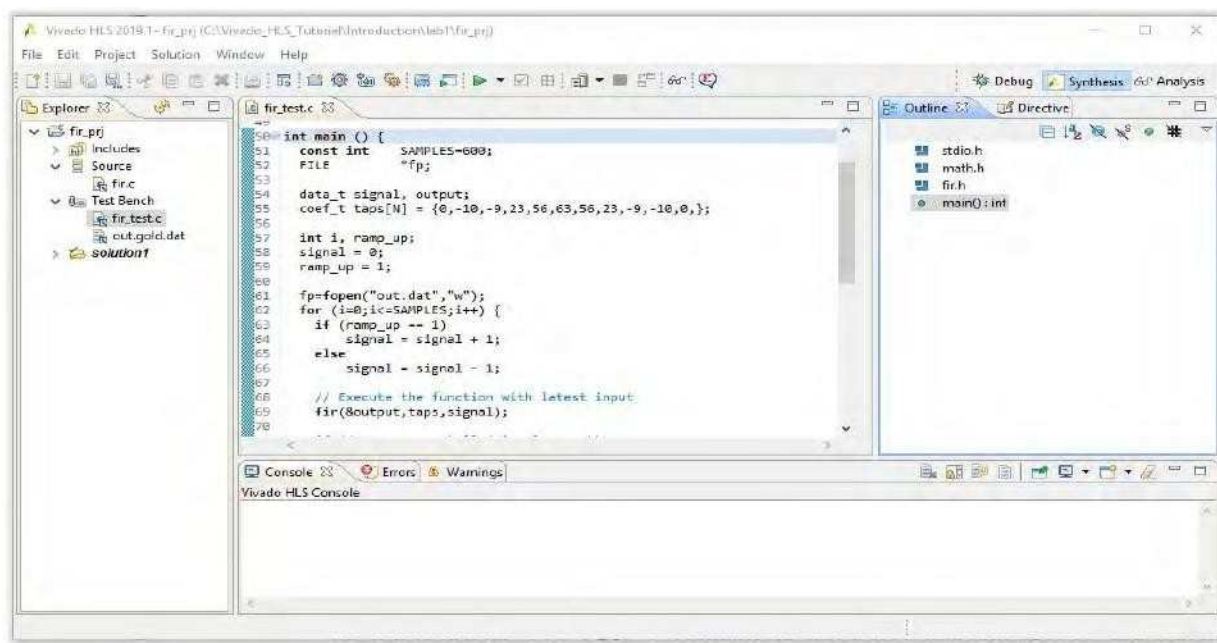


图 2-9: 检查测试工作台代码

测试工作台文件包含顶级 C 函数 **main()**，**fir_test.c**，该函数反过来调用要综合函数(**fir**)。这个测试台的一个有用的特点是它是自我检查：

- 试验台保存函数的输出到输出文件 **out.dat**
- 输出文件是与黄金的结果相比,存储在文件 **out.gold.dat**
- 如果输出匹配黄金数据,证实结果是正确的,则测试台函数的返回值被设置为 0
- 如果输出与黄金结果不同，则会显示一条消息，并且返回值设置为 1。

Vivado HLS 工具可以重用 C 测试平台来执行 RTL 的验证。

如果测试台具有前面描述的自检查特性，则在 RTL 验证期间将自动检查 RTL 结果。Vivado HLS 在 RTL 验证期间重用测试工作台，并在测试工作台返回 0 值时确认 RTL 验证成功。

如果返回值任何非零值，则表明 RTL 验证失败。不需要创建 RTL 测试工作台。这提供了一种健壮(*robust*)和有效的验证方法。

- 单击运行 **Run C Simulation** 按钮，或使用 **Project>Run C Simulation**，以编译并执行 C 设计。
- 在“**C 仿真**”对话框中，单击“OK”。控制台窗格(图2-10)确认模拟成功执行。

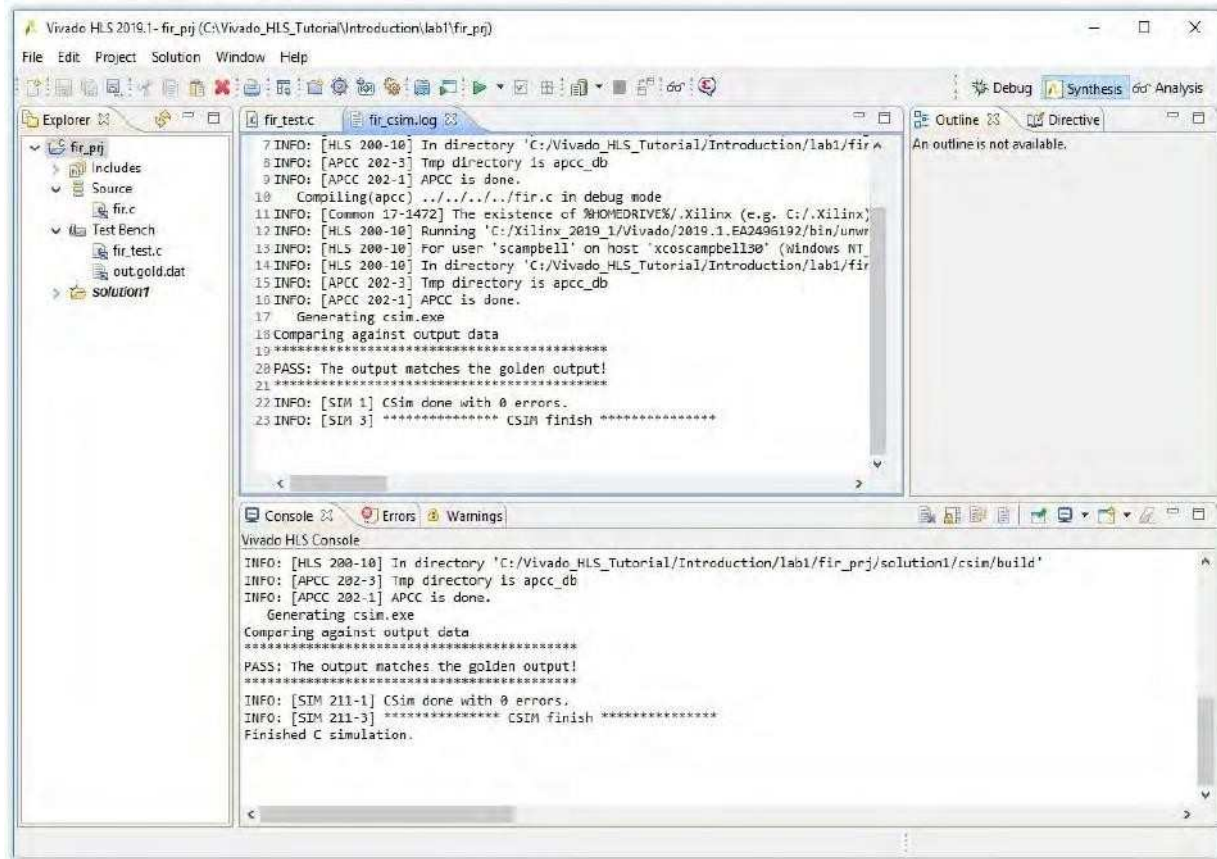


图2-10 C 仿真结果

 提示:如果C模拟失败，在C模拟对话框中选择 **Launch Debugger** 选项，编译设计，然后自动切换到 Debug 透视图。在那里，您可以使用 C 调试器来修复任何问题。

C 验证教程模块提供了有关使用调试环境的更多细节。这项设计现在可以进行综合了。

第三步:高级合成 (*High-Level Synthesis*)

在此步骤中，您将 C 设计合成为 RTL 设计，并查看合成报告

- 单击运行 **Run C Synthesis** 工具栏按钮或使用 **Solution>Run C Synthesis>Active Solution**。

综合完成后，报告文件将自动打开。因为综合报告在信息窗格中打开，所以辅助窗格中的 **Outline** 选项卡会自动更新，以反映报告信息。

- 单击 Outline 选项卡中的 **Performance Estimates**(图 2-11)。
- 在性能评估的详细部分中，展开 Loop 视图。

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	5.772	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
34	34	34	34	none

Detail

Instance

Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- Shift_Accum_Loop	33	33	3	-	-	11	no

图 2-11:性能评估

在如图 2-11 所示的 *Performance Estimates* 窗格中，您可以看到时钟周期被设置为 10 ns 。
Vivado HLS 的目标时钟周期为时钟目标减去时钟不确定性(在本例中为 $10.00 - 1.25 = 8.75\text{ ns}$)。

时钟不确定性确保(在此阶段)由于位置和路由造成的未知净延迟有一些可用的时间裕度。

估计的时钟周期(最坏情况下的延迟)为 5.772 ns ，满足 8.75 ns 的定时要求。

在**总结部分**，您可以看到：

- 该设计有 34 个时钟周期的延迟：它需要 34 个时钟来输出结果。
- 间隔是 34 个时钟周期：下一组输入在 34 个时钟之后读取。设计不是流水线。此函数(或下一个事务)的下一执行只能在当前事务完成时开始。

细节部分显示：

- **本设计中没有子块。**展开 Instance 部分时，层次结构中没有显示任何子模块。
- **所有的延迟都是由 *Shift_Accum_Loop* 的 RTL 逻辑综合造成的**，该逻辑是由一个名为 “*This logic*” 的循环执行 11 次(*Trip Count*)而合成的。每个执行

需要 3 个时钟周期(迭代延迟)，总共 33 个时钟周期才能执行从这个循环合成的逻辑的所有迭代(延迟)。

- **总延迟比循环延迟大一个时钟周期。**它需要一个时钟周期来进入和退出循环(在本例中，设计在循环结束时结束，因此不存在退出周期)。

4.在 Outline 选项卡中，单击 **Utilization estimate (利用率估算)** (图 2-12)。

- 该设计使用一个作为 LUTRAM 实现的单一内存(因为它包含的元素少于 1024 个)、3 个 dsp48 和大约 200 个触发器和 LUTs。在这个阶段，设备资源数量是估计值。
- 资源利用率是估计数，因为 RTL 综合可能会执行额外的优化，而且这些数字在 RTL 综合之后可能会改变。



Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	3	0	85	-
FIFO	-	-	-	-	-
Instance	-	-	-	-	-
Memory	0	-	64	6	0
Multiplexer	-	-	-	105	-
Register	-	-	111	-	-
Total	0	3	175	196	0
Available	4320	6840	2364480	1182240	960
Available SLR	1440	2280	788160	394080	320
Utilization (%)	0	~0	~0	~0	0
Utilization SLR (%)	0	~0	~0	~0	0

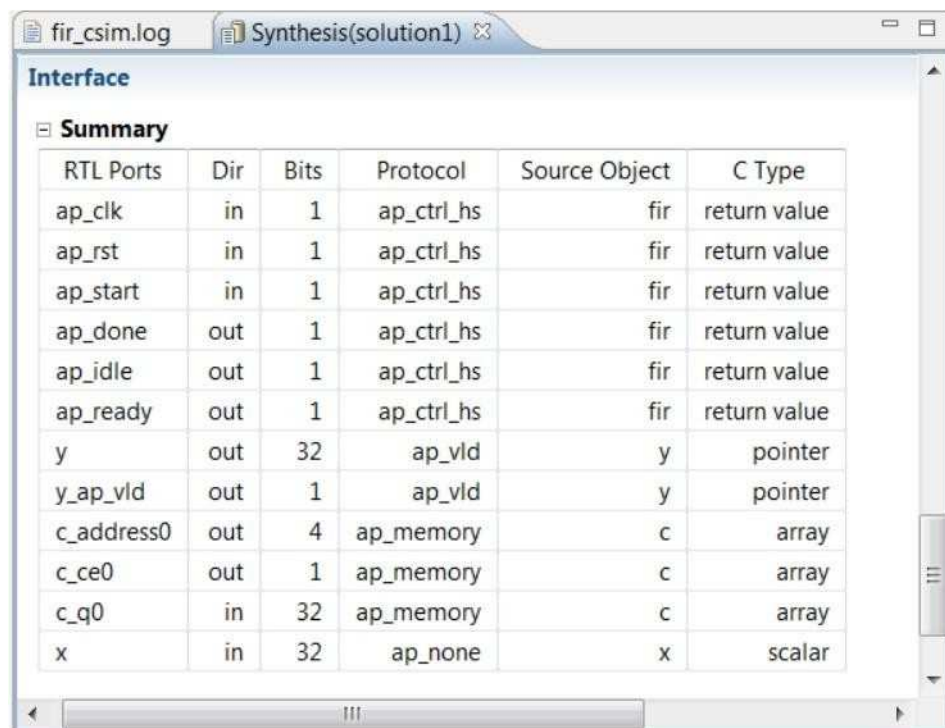
图 2-12: 利用率估计

5. 在利用率估计的详细部分中，展开 **Instance** 视图。

- DSP48s 的数量看起来大于期望的 FIR 滤波器。这是因为数据是 C 语言的整型 32 为类型，他需要多于 1 个 DSP48 去完成 32 位的乘法运算。
- 该乘法器是一个流水线乘法器。它出现在 *instance* 部分，表示它是一个子块 (sub_block)。标准组合乘法器 (*Standard combinational multipliers*) 没有层次结构，并在表达式部分中列出(表示此层次结构级别的组件)。

实验室 3: 使用设计优化的解决方案，优化这个设计。

6. 在 *Outline* 选项卡中，单击 **Interface**(图 2-13)。



RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	fir	return value
ap_rst	in	1	ap_ctrl_hs	fir	return value
ap_start	in	1	ap_ctrl_hs	fir	return value
ap_done	out	1	ap_ctrl_hs	fir	return value
ap_idle	out	1	ap_ctrl_hs	fir	return value
ap_ready	out	1	ap_ctrl_hs	fir	return value
y	out	32	ap_vld	y	pointer
y_ap_vld	out	1	ap_vld	y	pointer
c_address0	out	4	ap_memory	c	array
c_ce0	out	1	ap_memory	c	array
c_q0	in	32	ap_memory	c	array
x	in	32	ap_none	x	scalar

图 2-13: 接口报告

接口部分显示了接口综合 (*interface synthesis*) 创建的端口和 I/O 协议：

- 设计有一个时钟和复位端口(**ap_clk**, **ap_reset**)。这些都与设计本身的源对象 **fir** 相关联。
- 有与设计相关的额外端口，如源对象 **fir** 所示。综合已自动添加了一些块级别控制端口(**ap_start**, **ap_done**, **ap_idle**, and **ap_ready**)
- 接口综合教程提供了关于这些端口的更多信息。
- 函数输出 y 现在是一个 32 位的数据端口，带有一个相关的输出有效信号 **y_ap_vld**。

- 函数输入参数 `c`(一个数组)已经被实现为一个块 RAM 接口，它具有 4 位输出地址端口、一个输出 CE 端口和一个 32 位输入数据端口。
- 最后，输入标量参数 `x` 被实现为一个没有 I/O 协议(*ap_none*)的数据端口。

在本教程的后面部分：Lab 3:使用设计优化解决方案解释如何优化端口 `x` 的 I/O 协议。

步骤4:RTL 验证

高级综合可以重用 C 测试平台，通过仿真验证 RTL。

- 1.单击 **Run C/RTL CoSimulation** 工具栏按钮或使用 **Solution> Run C/RTL CoSimulation**。
- 2.在 C/RTL 协同仿真对话框中单击 **OK** 以执行 RTL 仿真。RTL 联合仿真的默认选项是使用 Vivado 模拟器和 Verilog RTL 执行仿真。要使用不同的模拟器或语言执行验证，请使用 C/RTL 联合模拟对话框中的选项。

当 RTL 联合模拟完成时，报告将在信息窗格中自动打开，控制台将显示如图 2-14 所示的消息。这是在 C 仿真结束时产生的相同消息。

- C 测试平台为 RTL 设计生成输入向量。
- 对 RTL 设计进行了仿真。
- 将 RTL 的输出向量应用回 C 测试台中，测试台的结果检查验证结果是否正确。
- Vivado HLS 表明，如果测试工作台返回的值为 0，仿真就通过。它是测试台中返回变量的值，仅这一点就表示模拟是否成功。重要的是，测试工作台仅在结果正确时才返回值 0。

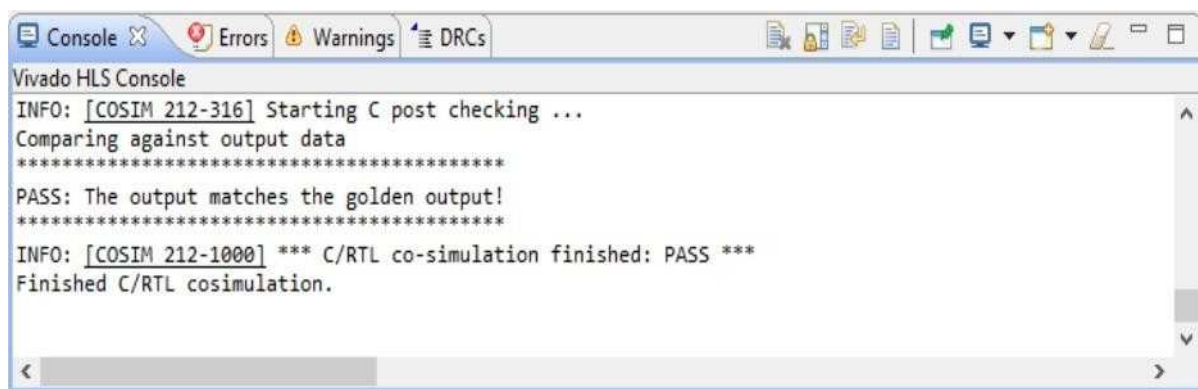


图 2-14:RTL 验证结果

第 8 章 RTL 验证教程提供了更多的信息。

步骤5:创建IP

高级综合流程的最后一步是将设计打包为 IP 块，以便与 Vivado 设计套件中的其他工具一起使用。

- 1.单击 **Export RTL** 工具栏按钮或使用 **Solution> Export RTL**。
- 2.确保格式选择下拉菜单显示 **IP Catalog**。
- 3.单击 OK。

IP 打包程序为 Vivado IP 目录创建一个包。(下拉菜单中的其他选项允许您为 DSP 的 System Generator 创建 IP 包，为 Vivado 创建综合检查点格式，或为 Xilinx Platform Studio 创建 Pcore。)

- 4.在浏览器中扩展 **Solution1**。
- 5.展开 **Export RTL** 命令创建的 **impl** 文件夹。
- 6.展开文件夹，找到打包成 **zip** 文件的 **ip**，准备添加到 Vivado IP 目录(图 2-15)。

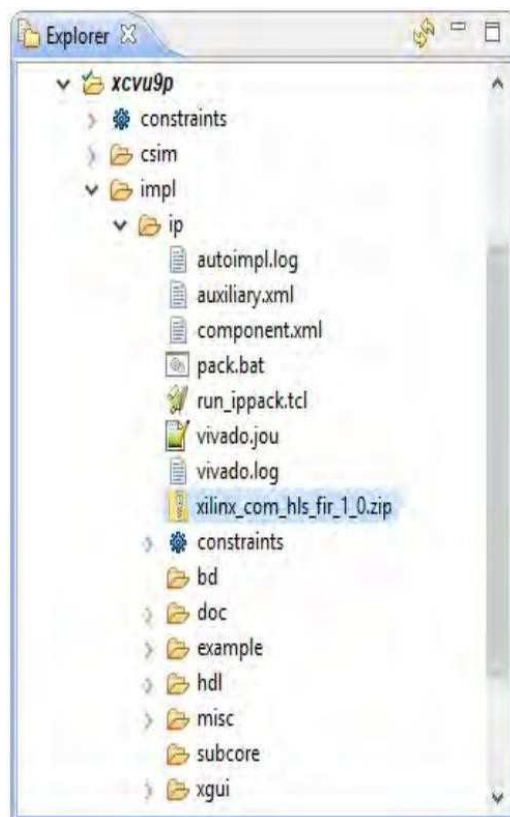


图 2-15: RTL 验证结果

在这个阶段，保持 Vivado HLS GUI 打开。你会在下一次的实验练习中回到这个问题。

实验 2: 使用 Tcl 指令介面

介绍

这个实验演示了如何基于现有的 Vivado HLS 项目创建 Tcl 命令文件并使用 Tcl 接口。

步骤 1: 创建一个 Tcl 文件

1. 打开 Vivado HLS 命令提示符。

在 Windows 上，使用 **Start > All Programs > Xilinx Design Tools > Vivado 2018.3 > Vivado HLS > Vivado HLS 2018.3 Command Prompt** (命令提示符) (图 2-16)。在 Linux 上，打开一个新的 shell。

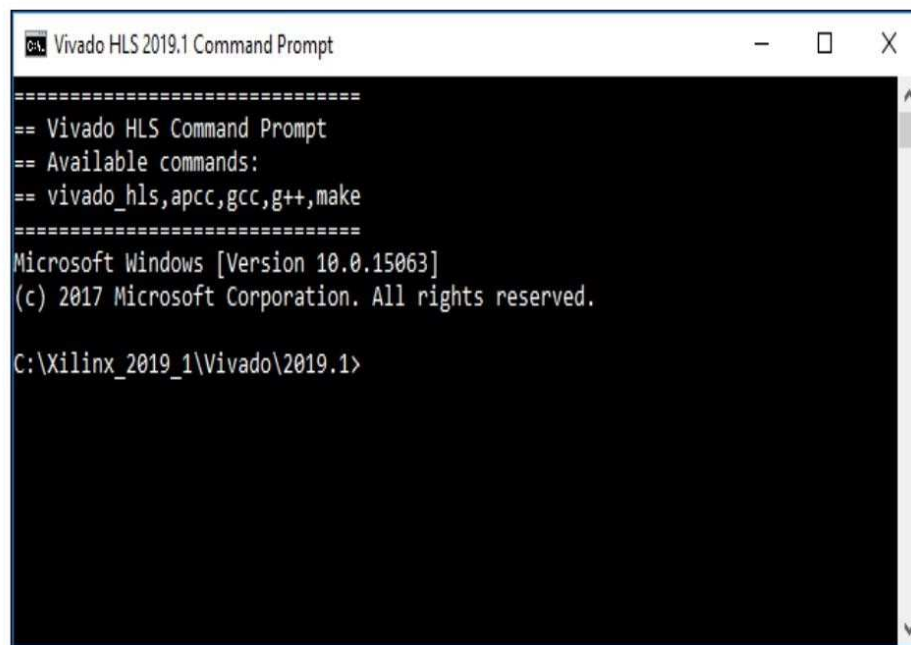


图 2-16: Vivado HLS 命令提示符

当您创建 Vivado HLS 项目时, Tcl 文件将自动保存在项目层次结构中。在 **实验 1** 中仍然打开的 GUI 中, 项目回顾显示了项目层次结构中的 **两个 Tcl 文件**(图 2-17)。

2. 在仍然从 **Lab 1** 打开的 GUI 中, 展开 *solution1* 中的 *Constraints* 文件夹, 双击文件 ***script.tcl***, 在 **Information** 窗格中查看它。

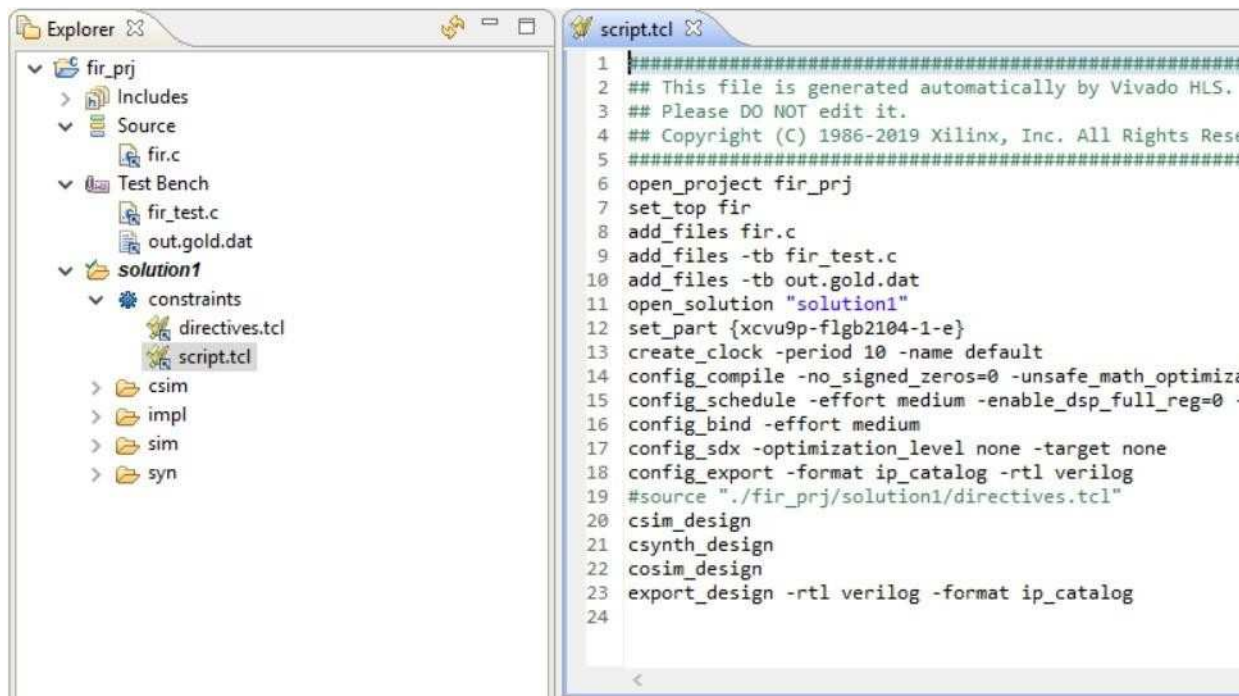


图 2-17: Vivado HLS 项目 Tcl 文件

- 该文件 ***script.tcl*** 包含 Tcl 命令, 用于使用项目设置期间指定的文件创建项目, 并运行 HLS 流的所有阶段。
- 该文件 ***directives.tcl*** 包含应用于设计解决方案的任何优化。在实验室 1 没有使用任何优化指令, 所以这个文件是空的。

在这个实验练习中, 您使用 ***script.tcl*** 从 lab1 为 lab2 项目创建一个 Tcl 文件。

3. 从实验 1 关闭 Vivado HLS GUI。这个项目不再需要了。
4. 在 Vivado HLS 命令提示符中, 使用以下命令(如图 2-18 所示)为 lab2 创建一个新的 Tcl 文件。
 - a) 将目录更改为 Introduction tutorial 目录: **C:\Vivado_HLS_Tutorial\Introduction**
 - b) 使用命令 ***cp lab1\fir_prj\solution1\script.tcl lab2\run_hls.tcl***。将现有的 tcl 文件复制到实验室 2。(Windows 命令提示符支持使用 Tab 键自动完成: 重复按 Tab 键可以看到新的选择)。
 - c) 使用命令 ***cd lab2*** 切换到 lab2 目录。

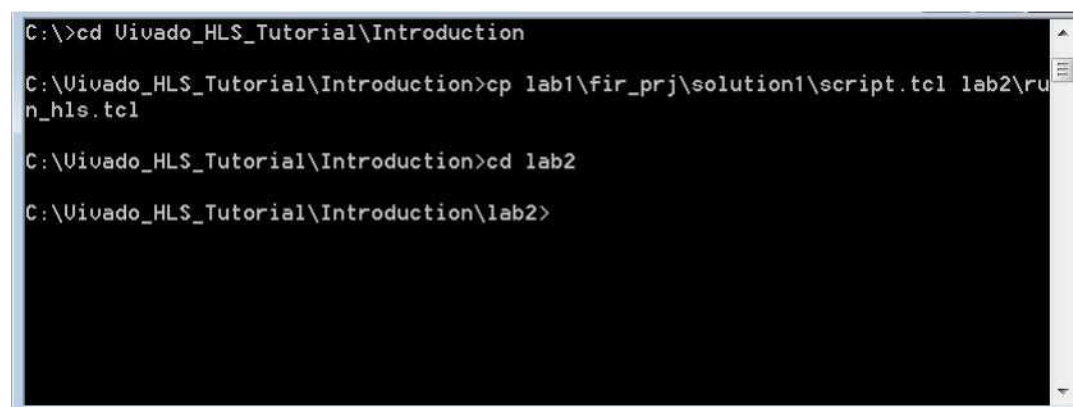
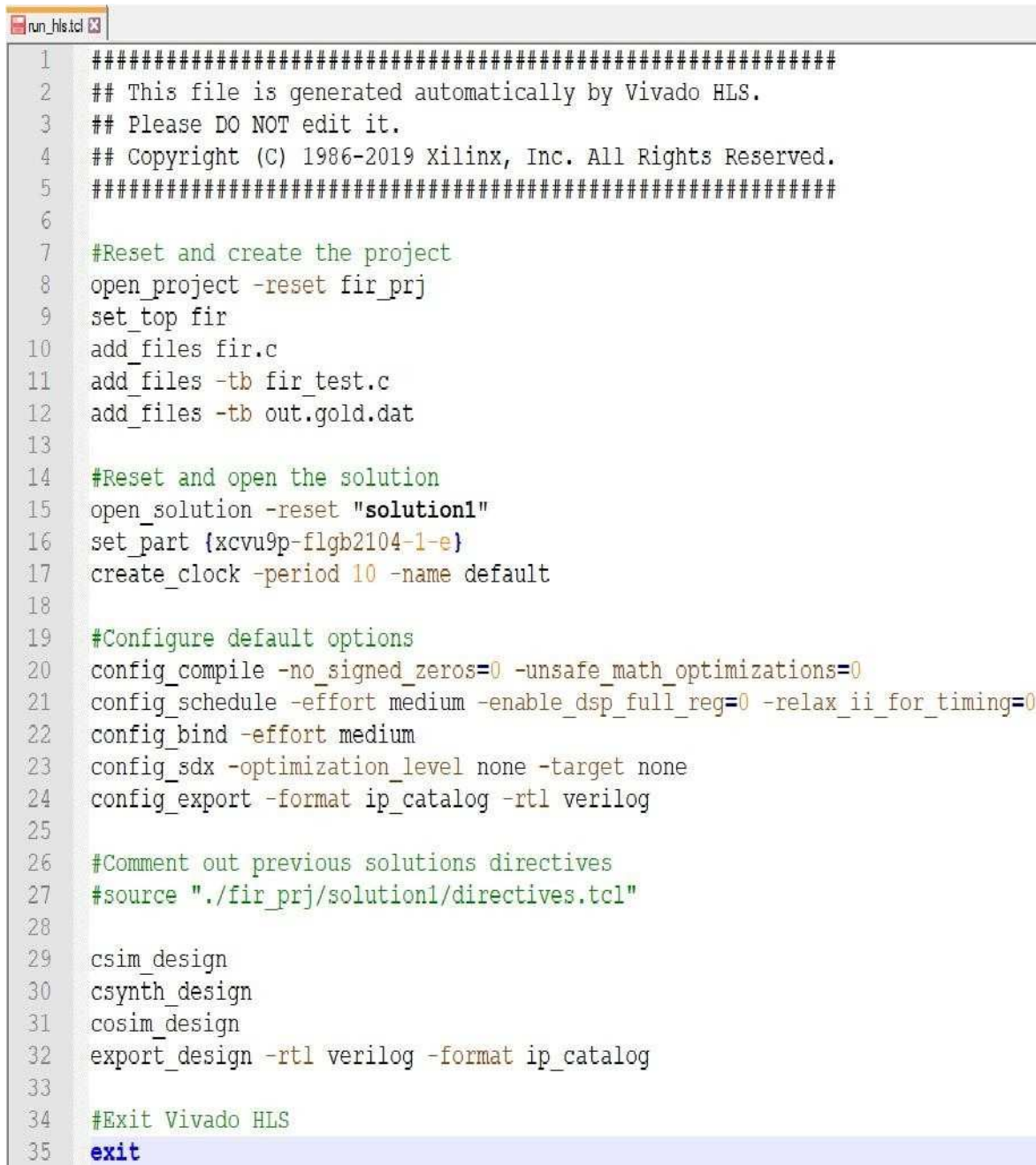


图 2-18: 将 lab1 Tcl 文件复制到 lab2

5. 使用任何文本编辑器, 对 lab2 目录中的文件 ***run_hls.tcl*** 执行以下编辑。最后的编辑如图 2-

19所示。

- 添加一个 **-reset** 选项到 **open_project** 命令。因为您通常在同一个项目上重复运行 Tcl 文件，所以最好覆盖所有现有的项目信息。
- 添加一个 **-reset** 选项到 **open_solution** 命令。这将删除任何现有的在相同解决方案上重新运行 Tcl 文件时的解决方案信息。
- 保留源命令的注释。如果上一个项目包含您希望重复使用的指令，则可以将指令直接复制到此文件中。
- 将 **exit** 命令添加到 Tcl 文件的最后一行
- 保存并退出。



```

1 #####
2 ## This file is generated automatically by Vivado HLS.
3 ## Please DO NOT edit it.
4 ## Copyright (C) 1986-2019 Xilinx, Inc. All Rights Reserved.
5 #####
6
7 #Reset and create the project
8 open_project -reset fir_prj
9 set_top fir
10 add_files fir.c
11 add_files -tb fir_test.c
12 add_files -tb out.gold.dat
13
14 #Reset and open the solution
15 open_solution -reset "solution1"
16 set_part {xcvu9p-flgb2104-1-e}
17 create_clock -period 10 -name default
18
19 #Configure default options
20 config_compile -no_signed_zeros=0 -unsafe_math_optimizations=0
21 config_schedule -effort medium -enable_dsp_full_reg=0 -relax_ii_for_timing=0
22 config_bind -effort medium
23 config_sdx -optimization_level none -target none
24 config_export -format ip_catalog -rtl verilog
25
26 #Comment out previous solutions directives
27 #source "./fir_prj/solution1/directives.tcl"
28
29 csim_design
30 csynth_design
31 cosim_design
32 export_design -rtl verilog -format ip_catalog
33
34 #Exit Vivado HLS
35 exit

```

图2-19:更新的run_hls。实验室2的tcl文件

您可以使用此 Tcl 文件以批处理模式 (**batch mode**) 运行 Vivado HLS。

6.在 Vivado HLS 命令提示窗口中，键入 Vivado HLS 执行 lab1 中涉及的所有步骤。

vivado_hls -f run_hls.tcl. 完成后，可以在项目目录 **fir_prj** 中获得结果。

- 综合报告在: **fir_prj\solution1\syn\report**.
- 仿真结果在: **fir_prj\solution1\sim\report**.
- 输出包在: **fir_prj\solution1\impl\ip**.

•最后的输出 RTL 在: **fir_prj\solution1\impl Verilog** and **then Verilog 或 VHDL** 中可用。

谨慎! 在从 Vivado HLS 项目复制 RTL 结果时, 必须使用目录中的 RTL。impl 在您可以在其他设计工具中使用 RTL 之前, Vivado HLS 会执行额外的处理。 **export_design**



实验三:使用设计最佳化的解决方案

介绍

这个实验使用了来自实验 1 的设计并进行了优化。

步骤 1:创建一个新项目

1.打开 Vivado HLS 命令提示符。

- 在 Windows 上, 使用 **Start > All Programs > Xilinx Design Tools > Vivado 2018.3 > Vivado HLS > Vivado HLS 2018.3 Command Prompt**.
- 在 Linux 上, 打开一个新的 shell。

2.切换到实验室 3 目录:

cd C: \ Vivado_HLS_Tutorial \ Introduction \ lab3。

3. 在命令提示窗口中, 键入: **vivado_hls -f run_hls. tcl** 这将设置项目。

4. 在命令提示窗口中, 在 Vivado HLS GUI 中键入 **vivado_hls -p fir_prj** 以打开项目。

VivadoHLS 打开, 如图 2-20 所示, solution1 的综合已经完成。

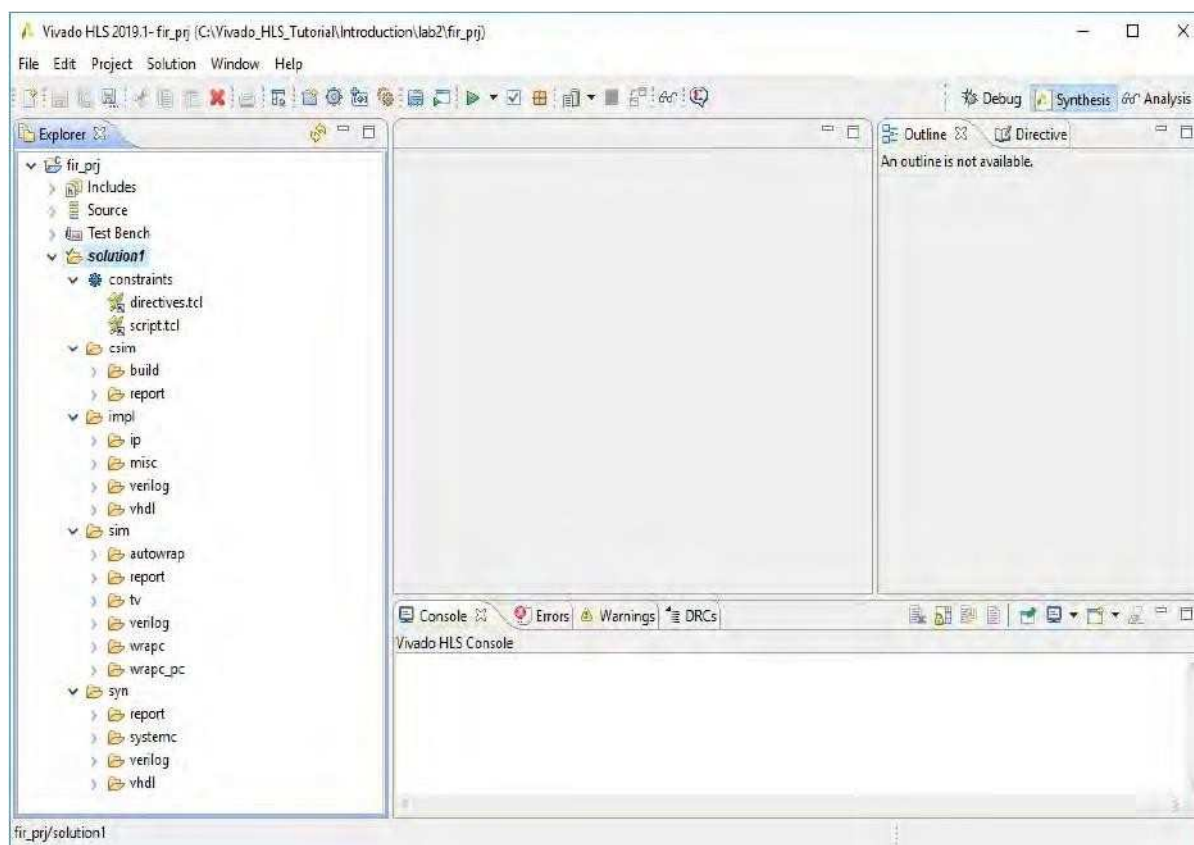


图 2-20: 介绍实验室 3 的初始解决方案

如前所述, 本设计的设计目标是:

- 创建该设计的最高吞吐量版本。
- 最终设计应该能够处理输入有效信号提供的的数据。
- 生成输出数据, 伴随输出有效信号。

- 过滤器系数将存储在 FIR 设计的外部，在一个单独的端口 RAM。

步骤 2:优化 I/O 接口

因为设计规范包含 I/O 协议，所以您执行的第一个优化将创建正确的 I/O 协议和端口。您选择的 I/O 协议类型可能会影响可能的设计优化。**如果存在 I/O 协议需求，则应在设计周期中尽早设置 I/O 协议。**

您在实验室 1 中查看了本设计的 I/O 协议(图 2-13)，并且您可以通过导航到文件夹 **solution1\syn** 中的 report 文件夹中再次查看综合报告。规定如下：

- 端口 C 必须有一个**单端口 RAM 访问**。
- 端口 X 必须有一个**输入数据有效信号**。
- 端口 Y 必须有一个**输出数据有效信号**。

端口 C 已经是单端口 RAM 访问。但是，如果没有显式地指定 RAM 访问类型，则高级综合可能使用双端口接口。如果这样做创建了一个具有更高吞吐量的设计，那么 HLS 将采取此操作。如果需要单端口，您应该在设计中明确添加使用单端口 RAM 的 I/O 协议要求。

输入端口 X 默认是一个简单的 32 位数据端口。通过指定 I/O 协议—*ap_vld*，可以将其实现为具有关联数据有效信号的输入数据端口。

输出端口 Y 已经有一个相关的输出有效信号。这是指针参数的默认值。您不必为这个端口指定一个显式的端口协议，因为默认的实现是必需的，但是如果它是一个需求，那么最好指定它。

为了保持现有的结果，创建一个新的解决方案，**solution2**。

- 1.单击 **Project>New Solution** 工具栏按钮创建新解决方案。
- 2.保留默认的解决方案名称，**solution2**，不要更改任何技术或时钟设置。
- 3.单击 **Finish**。

这将创建 **solution2**并设置它为默认解决方案。要确认，您可以验证当前活动 **solution2** 在资源管理器窗格中以粗体突出显示。

要添加优化指令来定义解决方案所需的 I/O 接口，请执行以下步骤。

- 4.在资源管理器窗格 (*Explorer pane*) 中展开源容器(如图 2-21 所示)。
- 5.双击 **fir.c** 打开信息窗格中的文件。
- 6.激活辅助窗格中的指令选项卡，并选择顶级函数 **fir** 跳转到源代码视图中 fir 函数的顶部。

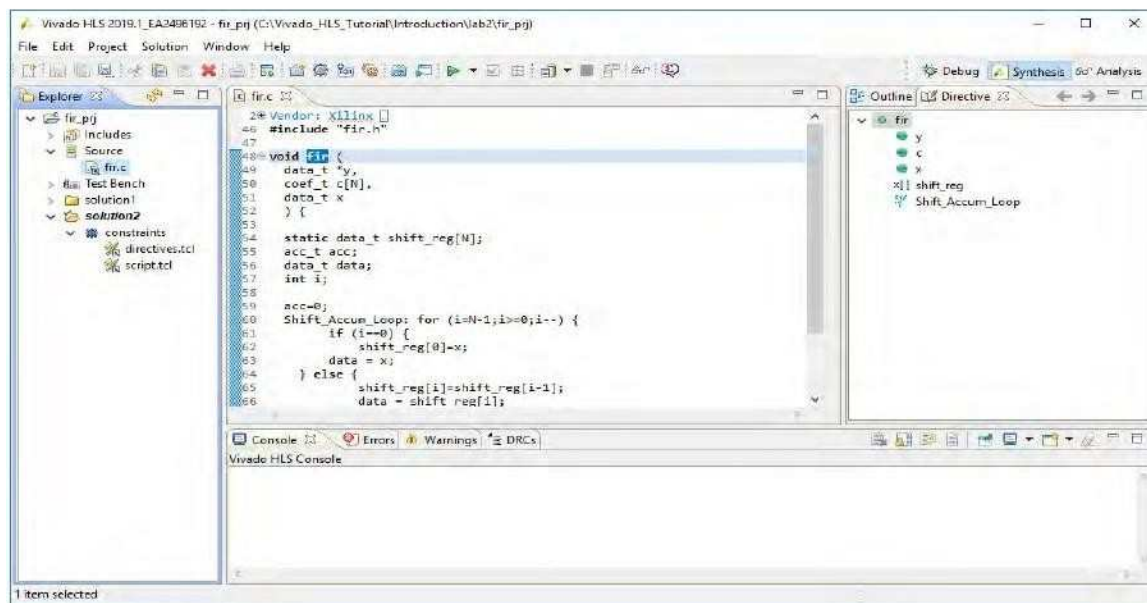


图 2-21: 打开选项卡

在图 2-21 的右侧显示的选项卡列出了设计中可以优化的所有对象。在 **Directive** 选项卡中，你可以向设计中**添加优化指令**。您只能在信息窗格中打开源代码时才能查看选项卡。

将优化指令应用到设计中。

7.在 Directive 选项卡中，选择 **c 参数/端口**(绿点)。

8.右键单击并选择 **Insert 指令**。

9.通过执行以下操作来实现单端口 RAM 接口：

a.从 Directive 下拉菜单中选择 **RESOURCE**。

b.点击 **core** 框。

c.选择 **RAM_1P_BRAM**，如图 2-22 所示。然后选 OK。

[重要]上述步骤指定使用单端口块 RAM 资源实现数组 c。因为数组 c 在函数参数列表中，因此在函数之外，所以会**自动**创建一组数据端口来访问 RTL 实现之外的单端口块 RAM。

因为 I/O 协议不太可能更改，所以可以将这些优化指令作为实用程序添加到源代码中，以确保在设计中嵌入了正确的 I/O 协议。

10.在指令编辑器 (**Directive Editor**) 的 **Destination** 部分，选择源文件 (**Source File**)。

11.要应用该指令，单击 OK。

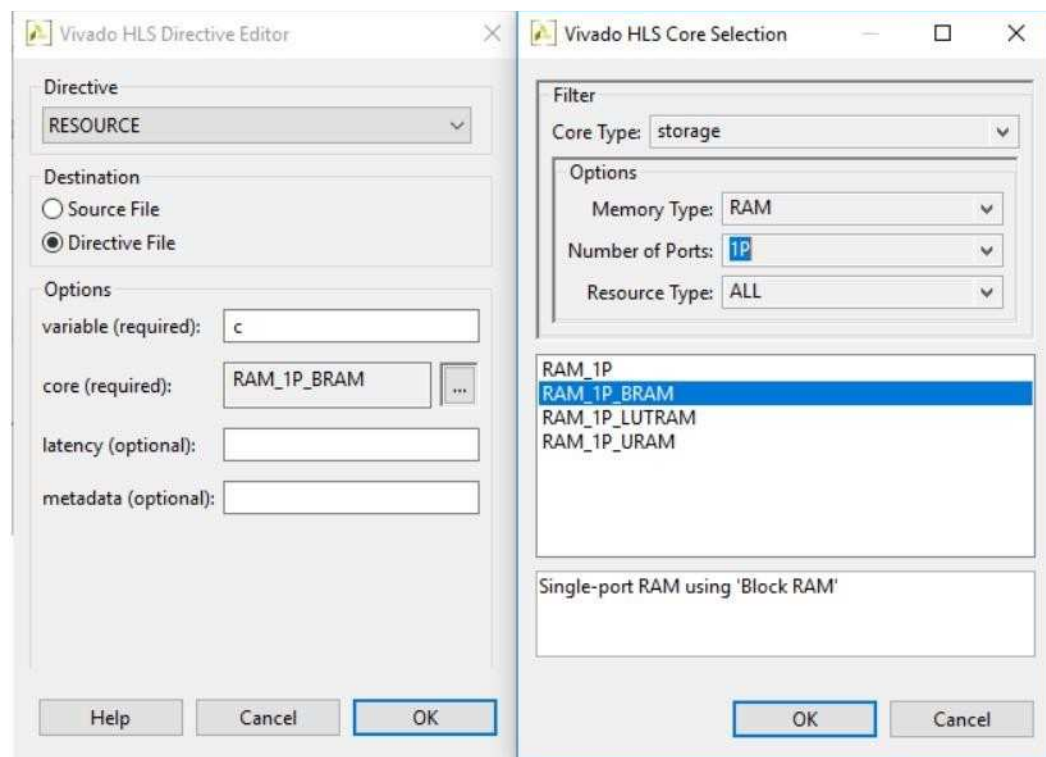


图 2-22: 添加资源指令

提示:如果你想改变任何指令的目标，双击指令标签中的指令并修改目标。



12.接下来，指定**端口 x**具有相关联的有效信号/端口 (**valid signal/port**)。

a) 在 **Directive** 选项卡中，选择 **input port x**(绿点)。

b) 右击并选择 **Insert Directive**。

- c) 从指令下拉菜单中选择 **Interface**。
- d) 从对话框的 “**Destination**” 部分中选择 “**Source File**”。
- e) 选择 **ap_vld** 作为模式。
- f) 单击 **OK** 应用指令。

13.最后，显式地指定端口 y 具有关联的有效信号/端口。

- a) 在 **Directive** 选项卡中，选择 **input port y**(绿点)。
- b) 右击并选择 **Insert Directive**。
- c) 从对话框的 “**Destination**” 部分中选择 “**Source File**”；
- d) 从指令下拉菜单中选择 **Interface**。
- e) 选择 **ap_vld** 模式。
- a) 选择 **ap_vld** 模式。

完成后，验证源代码和 Directive 选项卡是否正确，如图 2-23 所示。右击任何不正确的指令来修改它。

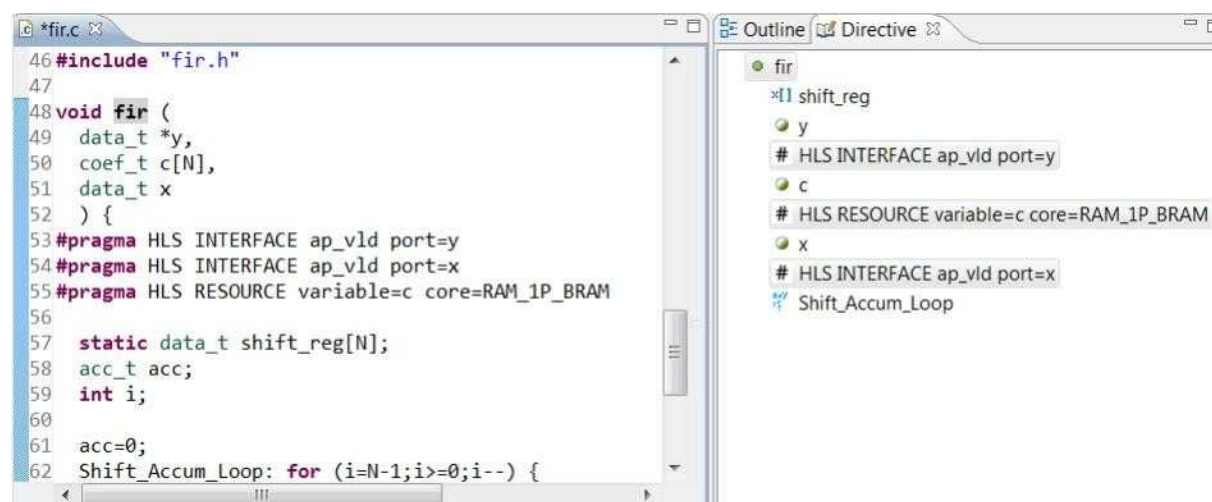
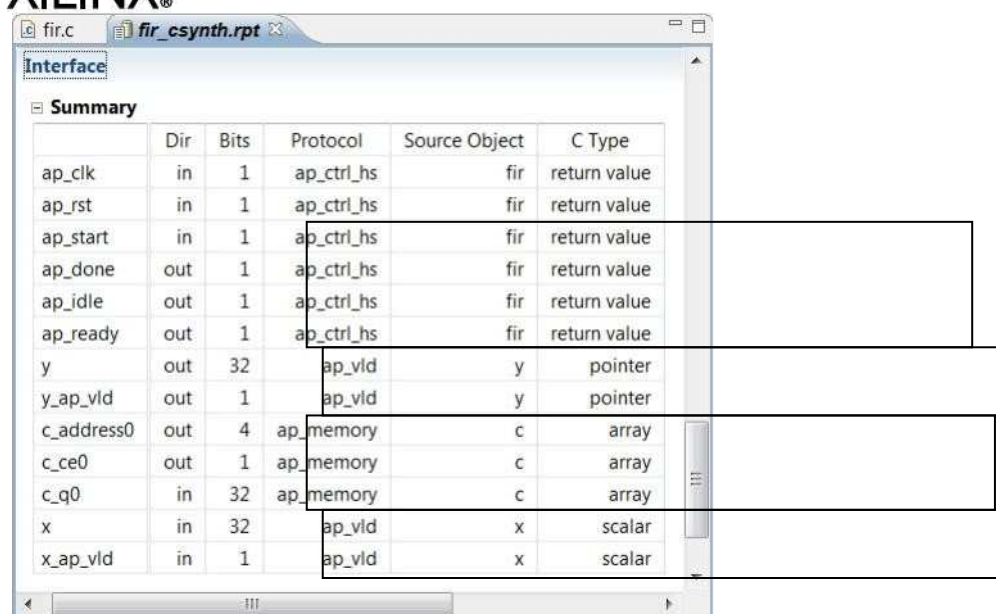


图 2-23:解决方案的 I/O 指令

- 14.单击 “**Run C Synthesis**” 工具栏按钮来合成设计。
- 15.当出现提示时，单击 **Yes** 保存 C 源文件的内容。添加指令作为实用程序修改源代码。合成完成后，报告文件将自动打开。
- 16.单击 **Outline** 选项卡查看界面结果，或者简单地向下滚动到报告文件的底部。

图 2-24 显示端口现在具有正确的 I/O 协议。

第三步:分析结果



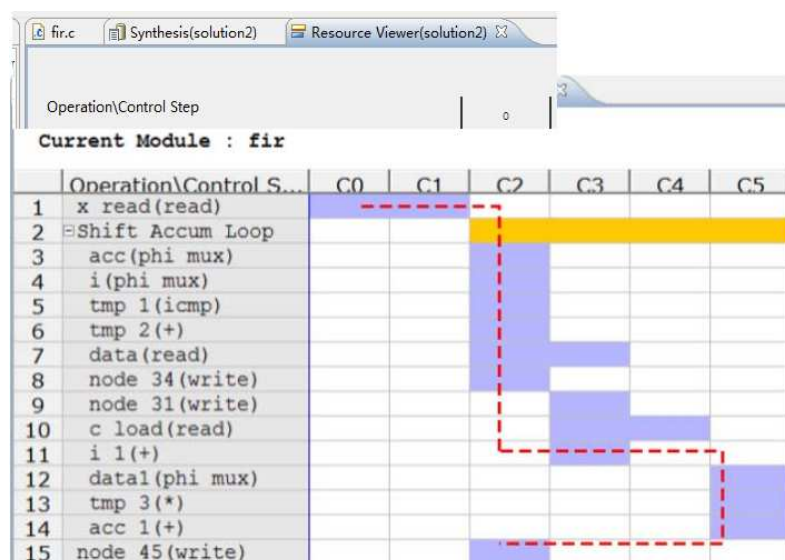
	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	fir	return value
ap_rst	in	1	ap_ctrl_hs	fir	return value
ap_start	in	1	ap_ctrl_hs	fir	return value
ap_done	out	1	ap_ctrl_hs	fir	return value
ap_idle	out	1	ap_ctrl_hs	fir	return value
ap_ready	out	1	ap_ctrl_hs	fir	return value
y	out	32	ap_vld	y	pointer
y_ap_vld	out	1	ap_vld	y	pointer
c_address0	out	4	ap_memory	c	array
c_ce0	out	1	ap_memory	c	array
c_q0	in	32	ap_memory	c	array
x	in	32	ap_vld	x	scalar
x_ap_vld	in	1	ap_vld	x	scalar

图 2-24: 解决方案的 I/O 协议

在优化设计之前，了解当前的设计是很重要的。在实验室 1 中展示了如何使用综合报告 来理解实现。然而，分析视角以交互活动的方式提供了更详细的信息。

按照下面的步骤来显示分析透视图，如图 2-25 所示。

1. 单击 **Analysis perspective** 按钮。
2. 在 “**Performance**” 窗口中单击 **Shift_Accum_Loop** 以将其展开。
 - 第六章，**Design Analysis** 教程提供了对分析视角的更全面的理解，但是下面解释了从这个源代码创建最小和最快的 RTL 设计需要什么。
 - **Performance** pane 视图的左列显示了 RTL 层次结构中此模块中的操作。
 - 第一行列出了设计中的控件状态。控制状态是将操作 **调度(schedule)**到时钟周期的高级综合使用的内部状态。在 RTL **有限状态机(FSM)**中，控制状态与最终状态之间存在密切的相关性，但不存在一一映射。



Operation\Control Step	C0	C1	C2	C3	C4	C5
1 x read(read)						
2 Shift Accum Loop						
3 acc(phi mux)						
4 i(phi mux)						
5 tmp 1(icmp)						
6 tmp 2(+)						
7 data(read)						
8 node 34(write)						
9 node 31(write)						
10 c load(read)						
11 i 1(+)						
12 data1(phi mux)						
13 tmp 3(*)						
14 acc 1(+)						
15 node 45(write)						

图 2-25: 解决方案 2 分析透视图: 性能

这里的一些对象与 C 源代码直接相关。右击该对象以与 C 代码交叉参考：

- 在第一个状态下开始设计，在 **端口 x** 上执行读操作。
- 在下一个状态中，它开始执行 **for 循环** 创建的逻辑 **Shift_Accum_Loop**。循环显示为 **黄色**，您可以展开或折叠它们。将光标停留在该视图的黄色循环主体上可显示循环详细信息：4 个周期，11 次迭代，总延迟为 44。
- 在第一种状态下，循环迭代计数器被检查：添加、比较和潜在的循环退出。
- 从阵列数据综合的 **Block RAM** 需要两个周期的读取操作（一个周期生成地址，一个周期读取数据）。
- c 端口上有内存读取。

- 乘法运算需要 1 个周期才能完成。
 - for 循环执行 11 次。
 - 在最后一次迭代结束时，循环退出控制状态 C1，并向端口 y 进行写操作。
- 您还可以使用 Analysis 透视图来分析设计中使用的资源。

- 3.单击 **Resource** 视图，如图 2-26 所示。
- 4.展开所有资源组。

Current Module : fir							
	Resource\Control Step	C0	C1	C2	C3	C4	C5
1	I/O Ports						
2	x	read					
3	y			write			
4	c(p0)				read		
5	Memory Ports						
6	shift reg(p0)			write	write		
7	c(p0)				read		
8	Expressions						
9	acc phi fu 105			phi_mux			
10	i phi fu 118			phi_mux			
11	grp fu 135			+	+		
12	tmp 1 fu 154			icmp			
13	data1 phi fu 129						phi_mux
14	acc 1 fu 170						+
15	tmp 3 fu 165						*

图 2-26:解决方案 2 分析透视图:资源

图 2-26 显示:

- 有一个读的端口 x 和写端口 y，端口 c 是报告在内存部分。因为这也是一个内存访问(内存在设计的外部)。
- 本设计中使用了一个单独的流水线乘法器。
- 一个加法器正在被共享：在一行有两个加法器实例。

通过分析获得的洞察力，您可以继续优化设计。

在结束分析之前，有必要对需要多个 dsp48 实现的多循环乘法操作进行讨论。源代码使用 int 数据类型。这是一个 32 位的数据类型，可以产生较大的乘数。**一个 DSP48 乘法器是 18 位的，它需要多个 DSP48 来实现大于 18 位的数据宽度的乘法。**

任意精度类型教程展示了如何使用更适合硬件的数据类型创建设计。使用任意精度类型可以定义任意位大小的数据类型(超过标准的 C/ c++ 8-、16-、32-或 64 位类型)。

步骤 4：优化最高吞吐量(最低间隔：**Lowest Interval**)

在本设计中限制吞吐量的两个问题是：

- 1) **循环**。for 默认情况下循环是滚动的：循环体的一个副本被综合并在每次迭代中重复使用（**分时复用**）。这确保循环的每个迭代都是**按顺序执行的**。您可以展开循环，以允许所有操作并行进行。
- 2) **用于 shift_reg 的 Block RAM**。由于变量 shift_reg 是 C 源代码中的数组，因此默认情况下将其实现为 Block RAM。但是，这阻止了其实现为移位寄存器。因此，您应该将此 Block RAM 划分为单独的寄存器。

首先创建一个新的解决方案。

- 1.单击 **Synthesis** 透视图按钮。
- 2.单击 **New Solution** 按钮。
- 3.保持解决方案名称不变。
- 4.单击 **Finish** 创建新解决方案。

5.在“**Project**”菜单中,选择“**Close Inactive Solution Tabs**”,以关闭以前解决方案中的任何现有选项卡。

图2-27中总结的以下步骤解释了如何展开循环。

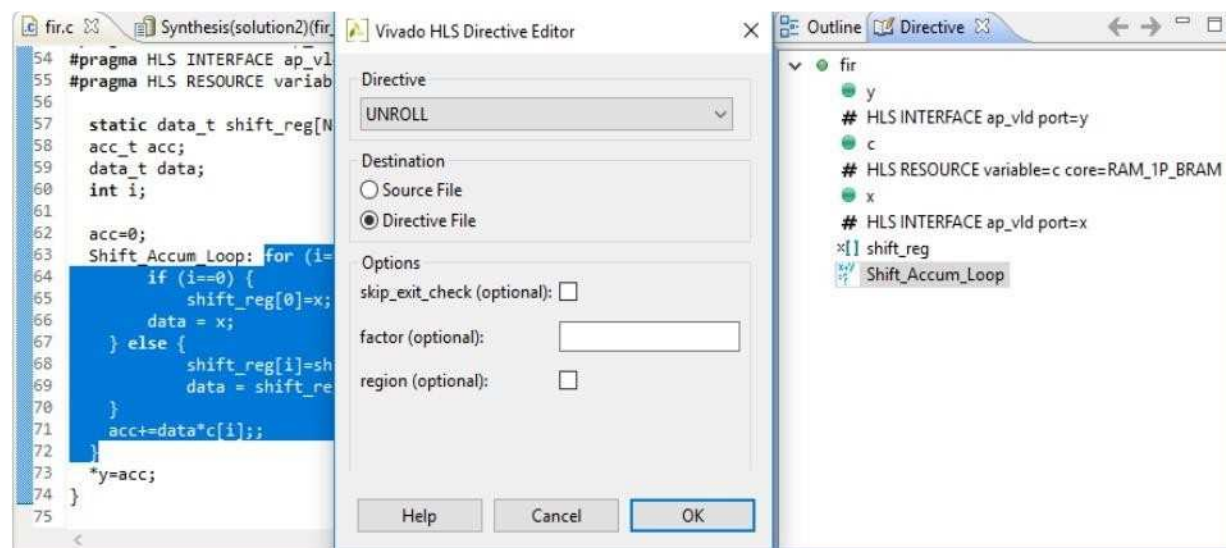


图 2-27:展开FOR 循环

6.单击 **fir.c** 文件,然后在指令选项卡中,选择循环 **shift_Accum_loop**。

重要提示:源代码必须在信息窗格中打开,才能看到指令选项卡中的任何代码对象。



7.右键单击并选择 **Insert** 指令。

8.从指令下拉菜单,选择 **Unroll**。将目标作为指令文件。

在优化设计时,您必须经常执行多次优化迭代,以确定最终的优化应该是什么。添加优化方法到指令文件,可以确保它们不会自动转移到下一个解决方案。在解决方案指令文件中存储优化允许不同的解决方案进行不同的优化。如果您将这些优化作为 **pragmas** 添加到代码中,它们将被自动转到新的解决方案中,您将不得不修改代码以返回并重新运行以前的解决方案。

保留指令窗口中的其他选项为未选中和空白,以确保循环完全展开。

9.单击 **OK** 应用指令。

10.应用指令将数组划分为各个元素。

- 在指令选项卡中,选择数组 **shift_reg**。
- 右击并选择 **Insert** 指令。
- 从指令下拉菜单中选择 **Array_Partition**。
- 指定类型为 **complete**。
- 选择 **OK** 应用指令。

在 solution2 中嵌入刚刚添加的两个新指令,directive 窗格 for 就出现了,如图 2-28 所示。

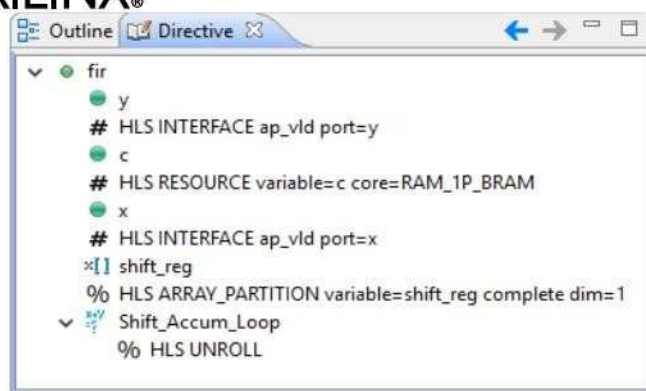


图 2-28: 解决方案 3 指令

在图 2-28 中, 请注意在 solution2 中应用的指令, 因为编译指示与刚刚应用并保存到指令文件 (%HLS) 的指令具有不同的注释 (#HLS)。您可以在 Tcl 文件中查看新添加的指令, 如下所示。

11. 在“资源管理器 (Explorer)”窗格中展开“**Constraint**”文件夹, 如图 2-29 所示。
12. 双击文件 **solution3 directives.tcl** 以在信息窗格中打开它。

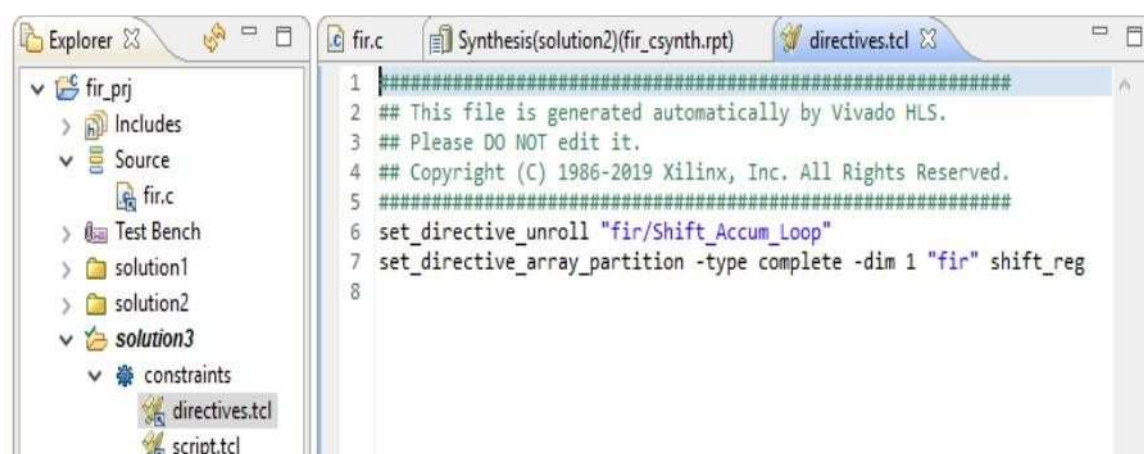


图 2-29: 解决方案 3 指令。tcl 文件

13. 单击 **Synthesis** 工具栏按钮来综合设计。当综合完成时, 综合报告将自动打开。
14. 比较不同解决方案的结果。单击“**Compare Reports**”工具栏按钮。或者, 使用 **Project>Compare Reports**。
15. 在**比较**中加入 solution1, solution2 和 solution3。
16. 单击 OK。

图 2-30 显示了报告的比较。solution3 具有最小的启动间隔, 可以更快地处理数据。由于间隔只有 16, 所以它每 16 个时钟周期开始处理一组新的输入。

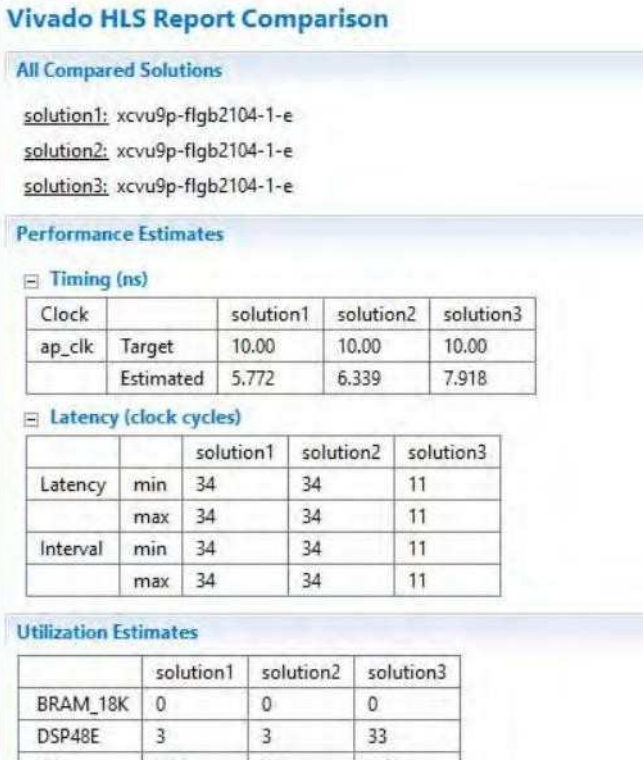


图 2-30: Lab3 解决方案的比较

可以对这种设计执行额外的优化。例如，您可以使用**管道**(*pipelining*)来进一步提高吞吐量(*throughput*)并降低间隔(*interval*)。在第七章“设计优化教程”中，详细介绍了如何使用管道来改进时间间隔。

如前所述，您可以修改代码本身以使用任意精度类型。例如，如果不要求数据类型为 32 位 int 类型，则可以使用任意精确类型(例如，6 位、14 位或 22 位类型)，前提是它们满足所需的精度。有关使用任意精度类型的详细信息，请参阅第 5 章“**任意精度类型教程**”。

结论

在本教程中，您学习了如何：

- 在 GUI 和 Tcl 环境中创建 Vivado 高级综合项目。
- 执行 HLS 设计流程中的主要步骤。
- 创建并使用一个 Tcl 文件来运行 Vivado HLS。
- 创建新的解决方案，添加优化指令，并比较不同的结果 解决方案。

第三章

C 验证

概述

C 算法的**验证(Validation)**是高水平综合(HLS)过程的重要组成部分。确保 C 算法执行正确的操作并创建 C 测试平台所花费的时间，可以确认结果正确，减少了“*by design*”分析不正确的设计所花费的时间，并确保可以自动执行 RTL 验证。

本教程包括三个实验练习。

实验室 1 描述

回顾一个好的 C 测试平台，C 验证和 C 调试器的基本操作。

实验室 2 描述

使用任意精度 (*arbitrary precision*) 的 C 类型验证和调试 C 设计。

实验室 3 描述

使用任意精度的 c++ 类型验证和调试设计。

教程设计描述

您可以从 Xilinx 网站下载教程设计文件。请参阅定位教程设计文件中的信息。本教程使用教程目录中的设计文件

Vivado_HLS_Tutorial \ C_Validation。

本教程中使用的示例设计是一个 *Hamming Window FIR*。这个设计有三个版本：

- 使用本机 C 数据类型。
- 使用 ANSI C 任意精度数据类型。
- 使用 c++ 任意精度数据类型。

本教程解释了使用高级综合的 C 验证的操作和方法。本教程没有设计目标。

实验 1：C 验证与除错

概述

本练习回顾了良好的 C 测试平台的各个方面，并解释了高级综合 C 调试环境的基本操作。



重要提示:本教程中的图和命令假设教程数据目录已解压缩并放置在位置中，如果将教程数据目录解压缩到其他位置，或者在 Linux 系统上，将引用的少数路径名调整到您选择的目录位置。*Vivado_HLS_Tutorial C:\Vivado_HLS_Tutorial.*

Vivado_HLS_Tutorial

步骤 1:创建并打开项目

1.打开 Vivado HLS 命令提示符。

在 Windows 上使用*启动>所有程序>Xilinx 设计工具>Vivado 2019.1 > Vivado HLS > Vivado HLS 2019.1 命令提示符*。

在 Linux 上, 打开一个新的 shell。

2.使用命令提示符窗口(图 3-1), 将目录更改为 C 验证教程 lab1。

3.使用图 3-1 所示的命令执行 Tcl 脚本来设置 Vivado HLS 项目。

```
C:\Uivado_HLS_Tutorial>cd C_Validation
C:\Uivado_HLS_Tutorial\C_Validation>cd lab1
C:\Uivado_HLS_Tutorial\C_Validation\lab1>vivado_hls -f run_hls.tcl
```

图 3-1: 设置教程项目

4.Vivado HLS 完成后, 使用 ***vivado_hls -p hamming_window_prj*** 命令在 VivadoHLS GUI 中打开项目, 如图 3-2 所示。

```
@I [APCC-3] Tmp directory is apcc_db
@I [APCC-1] APCC is done.
@I [LIC-101] Checked in feature [HLS]
Generating csim.exe
Running DUT...done.
Testing DUT results
.....
.....
*** Test Passed ***
@I [SIM-1] CSim done with 0 errors.
@I [LIC-101] Checked in feature [HLS]
C:\Uivado_HLS_Tutorial\C_Validation\lab1>vivado_hls -p hamming_window_prj
```

图3-2:C 验证实验室1 的初始项目

步骤 2:检查测试台并运行 C 仿真

1.双击 ***test bench*** 文件夹, 打开 C 测试工作台以进行检查(图 3-3)。

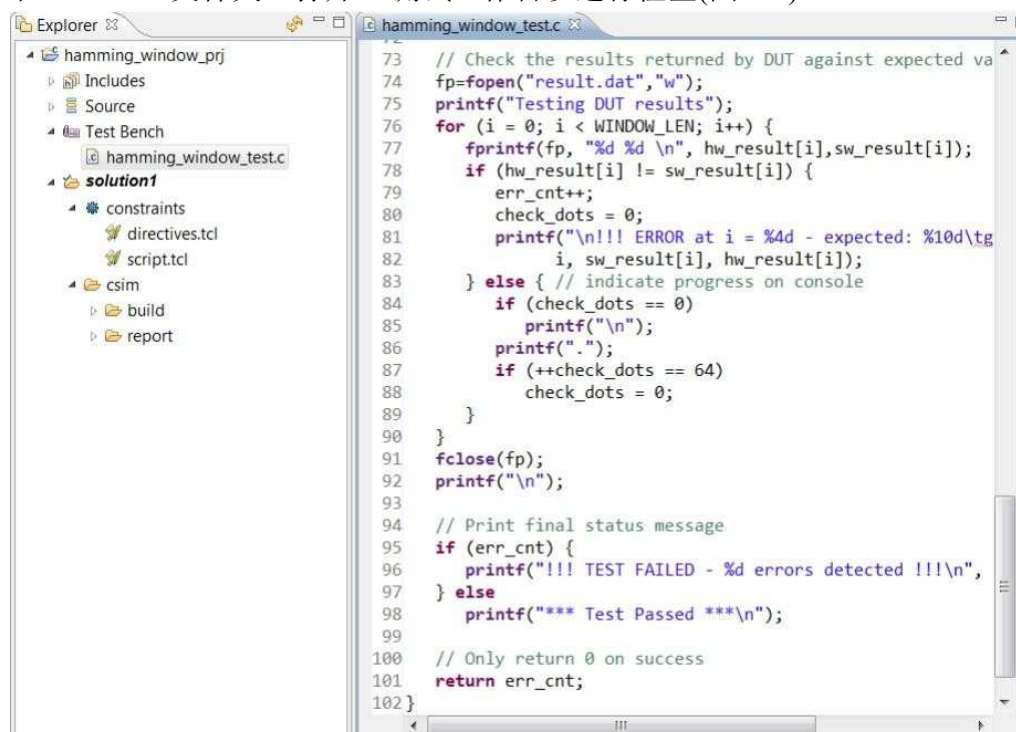


图3-3:C 验证实验室1 的C 测试台

- 测试台(**test bench**):
 - 创建一组预期结果，以确认函数是正确的。
 - 将结果存储在数组 **sw_result** 中。
- 调用待测设计(**Design Under Test ,DUT**)来生成结果，这些结果存储在数组 **hw_result** 中，因为综合的函数使用数组 **hw_result**，正是这个数组在设计流程的后面保留了 RTL 生成的结果。
- **比较实际结果和预期结果**。如果比较失败，则将变量 **err_cnt** 的值设置为非零值。
- 如果比较失败，测试工作台会向控制台发出一条消息。如果返回值为 0，则测试工作台验证结果正确，否则非零。

您可以执行 **C 代码**和**测试平台**以确认代码是否按预期工作。

2.单击 **Run C Simulation** 工具栏按钮，打开 **C Simulation** 对话框，如图 3-4 所示。

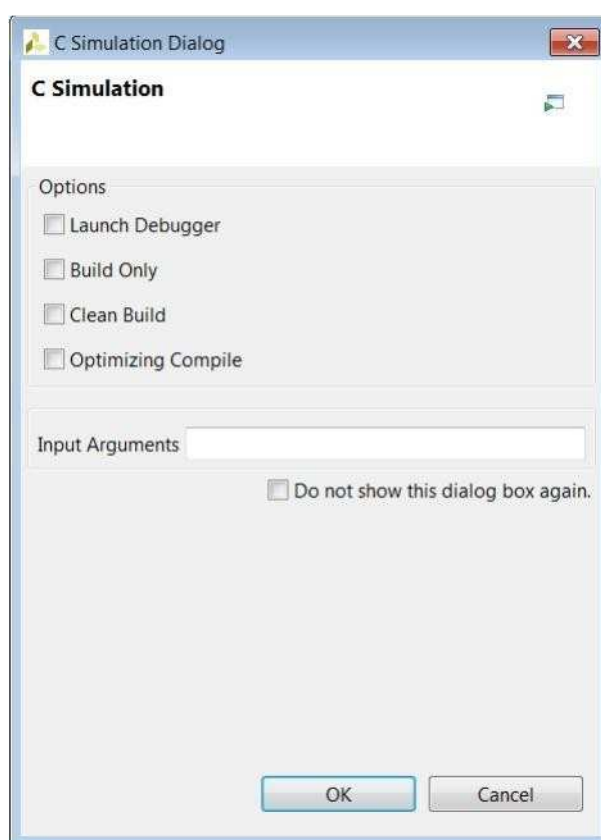


图 3 - 4: 运行 C 仿真对话框

3.选择 **OK** 运行 **C simulation**。

如图 3-5 所示，C 仿真执行时发生以下动作：

- C 代码中的任何打印语句都会控制在控制台窗口中进行回显。此示例显示模拟已正确通过。
- 在 **solution** 子目录中执行 C 仿真，您可以在 **build** 文件夹中找到来自 C 仿真的任何输出，在这个位置您可以看到由图 3-5 中突出显示的命令 **fprintf** 编写的输出文件 **csim.result.dat**。

因为 C 仿真不是在项目目录中执行的，所以必须将任何数据文件作为 C 测试工作台文件添加到项目中(以便在模拟运行时将它们复制到目录 **csim/build** 中)。例如，这些文件将包括测试工作台读取的输入数据。

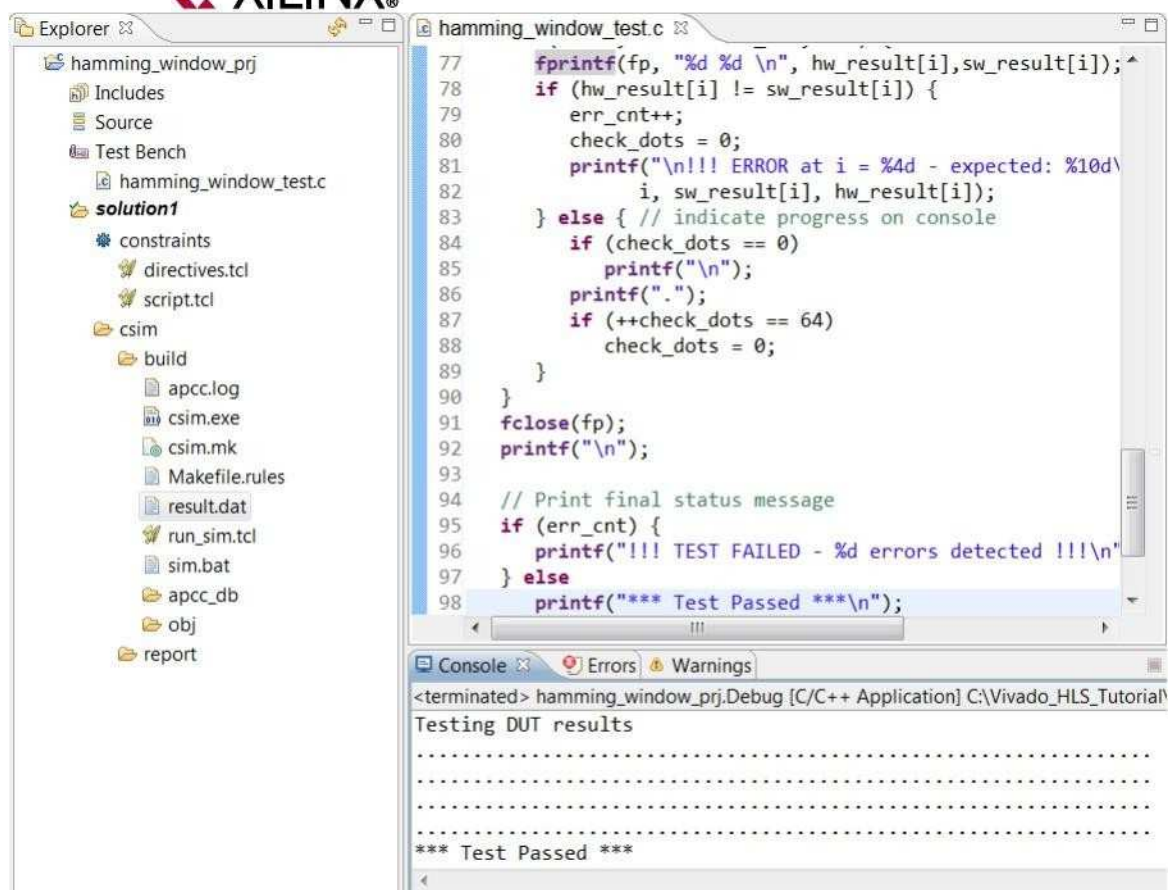


图 3 - 5: C 仿真结果

步骤 3:运行 C 调试器

C 调试器是高级综合的一部分。

- 1.单击“**Run C Simulation**”工具栏按钮,打开“**C 仿真**”对话框。
- 2.选择 **Launch Debugger** 选项,如图 3-6 所示。
- 3.单击 **OK** 运行模拟。

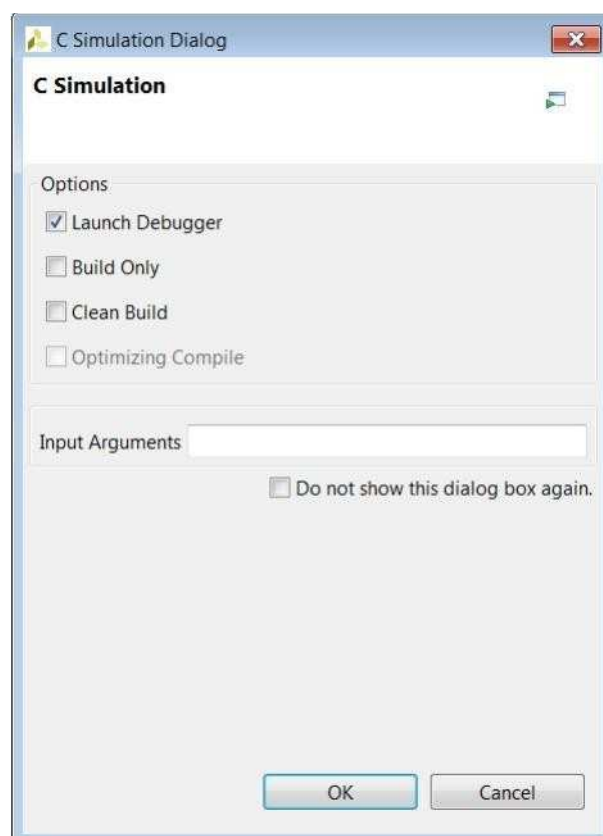


图 3-6:C 仿真对话框

启动调试器选项编译C代码，然后打开调试环境，如图3-7所示。在进行之前，请注意以下几点：

- 在图 3-7 的右上角高亮显示，您可以看到透视图已经从综合法更改为 **Debug**。随时单击透视图按钮返回综合环境。
- 默认情况下，代码以调试模式编译。启动调试器选项在时间 0 时自动打开 debug 透视图，准备开始调试。若要编译没有调试信息的代码，请在 C 仿真对话框中选择 **Optimizing Compile** 选项。
-

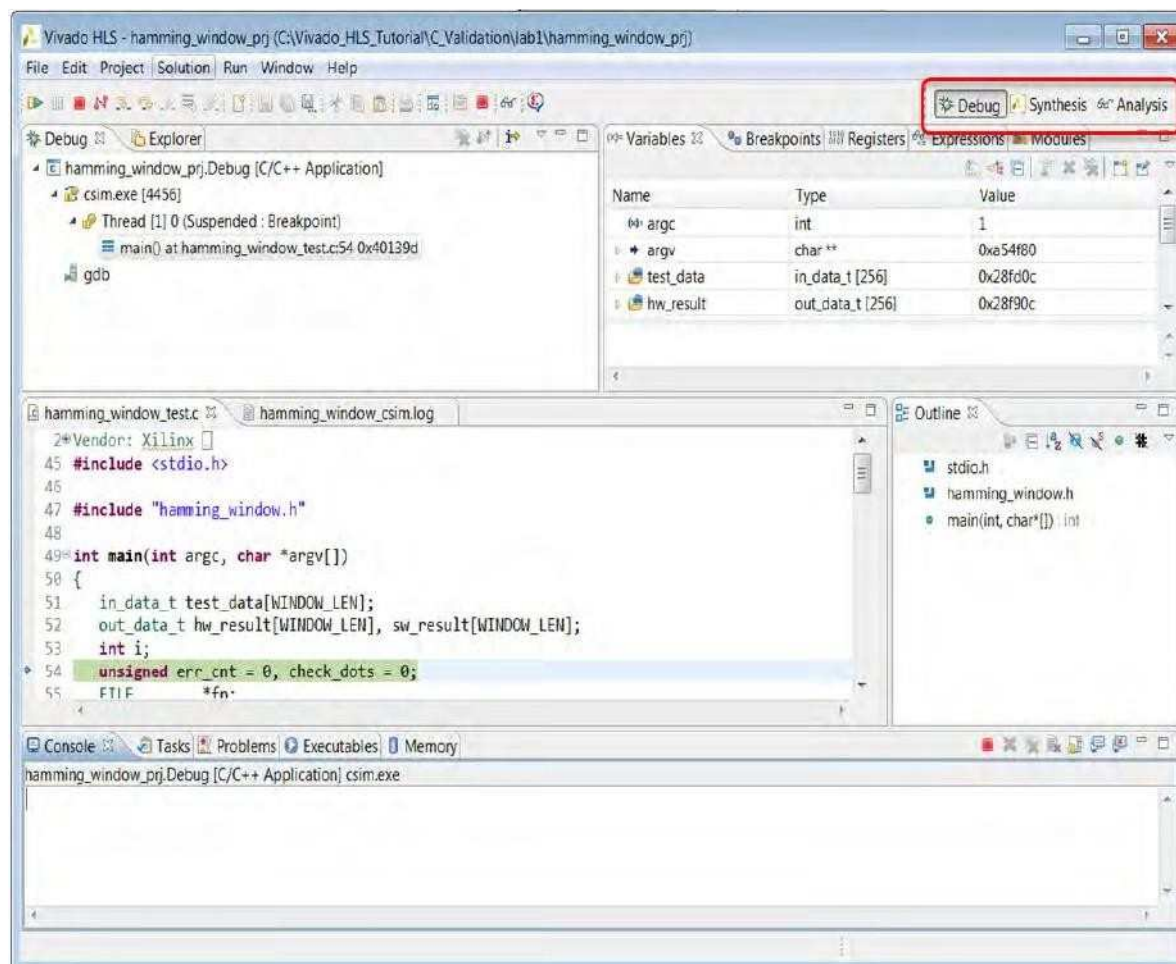


图3-7:HLS 调试透视图

您可以使用 Step Into 按钮(图3-8)逐行检查代码。



图3-8: 调试步骤进入按钮

- 4.展开 **Variables** 窗口以查看 **sw_result** 数组。
- 5.将数组 **sw_result** 展开到图 3-9 所示的视图。
- 6.重复单击 **Step Into** 按钮(或键 **F5**)，直到在 Variables 窗口中看到更新的值。

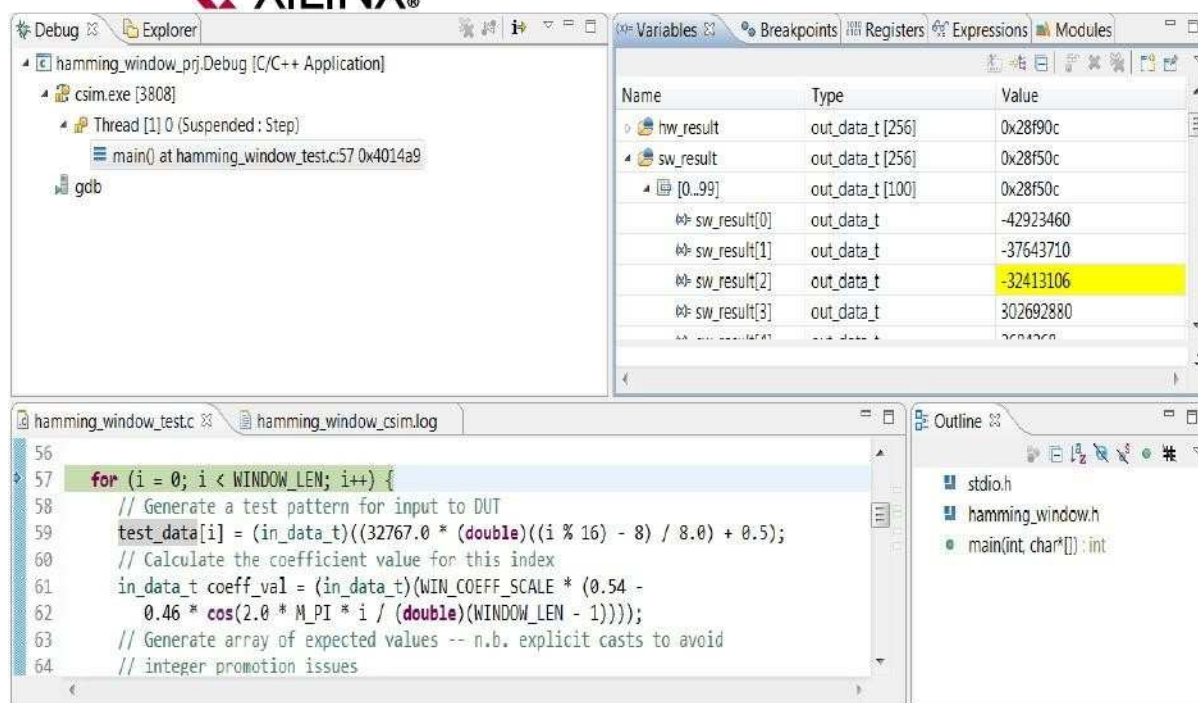


图 3-9:C变量分析

通过这种方式，您可以分析 C 代码并在行为不正确时进行调试。更详细的分析是，在 Step Into 按钮的右边是 Step Over (F6)、Step Return (F7)和 Resume (F8)按钮。

7.滚动到 **hamming_window_test.c**文件 中的第 69 行。

8.将光标放在第69行左边的空白处，用鼠标右键单击并选择 **Toggle Breakpoint**。在页边距处显示一个 **断点(蓝点)**，如图3-10所示。

9.激活如图 3-10 所示的 **Breakpoints 选项卡**，以确认在第 69 行有一个断点设置。

10.单击 **Resume** 按钮(在图 3-10 中突出显示)或 F8 键以执行到断点。

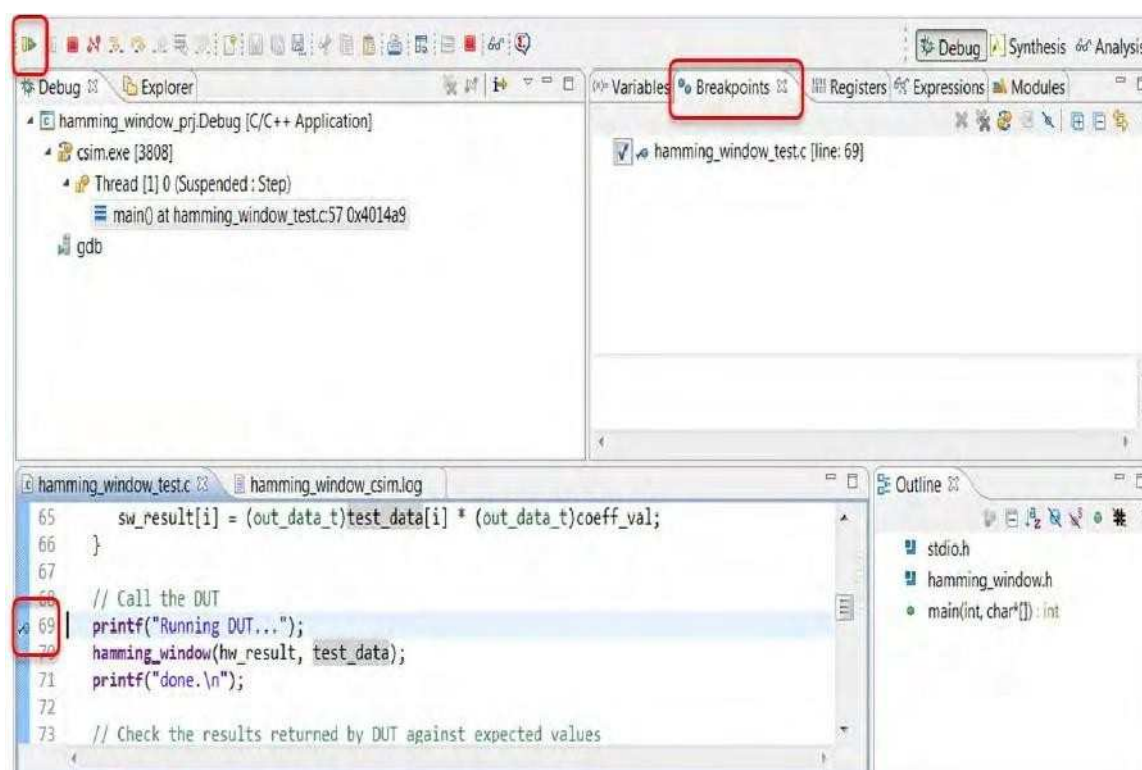


图 3-10: 使用断点

11.多次单击 **Step Into** 按钮(或 F5 键)进入 **hamming_window** 函数。

12.单击 **Step Return** 按钮(或 F7 键)返回主函数。

13.单击红色的 **Terminate** 按钮以结束调试会话。

您可以使用 **Run C simulation** 按钮从 debug 透视图重新启动调试会话。

14.退出 Vivado HLS GUI并返回到命令提示符。

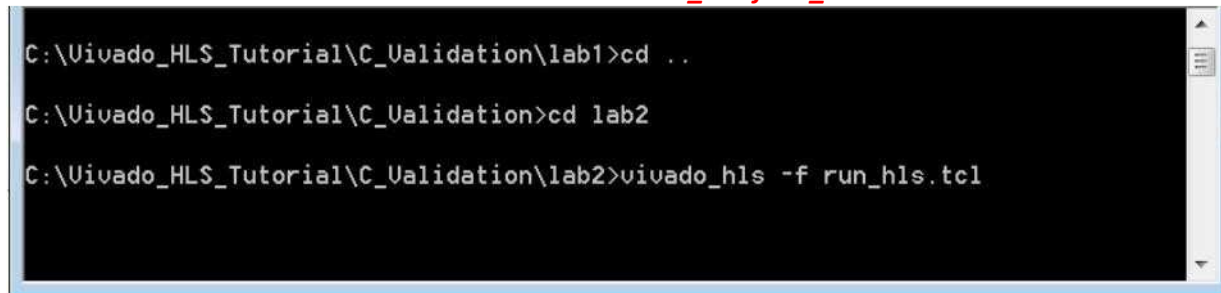
实验2:用ANSIC任意精度类型进行C验证

介绍

本练习使用具有任意精度 C 类型的设计。您将在 GUI 中检查和调试设计。

步骤 1:创建并打开项目

- 1.从实验室 1 中使用的 Vivado HLS 命令提示符，切换到目录 **lab2**，如图 3-11 所示。
- 2.要创建一个新的 Vivado HLS 项目，键入 **vivado_hls -f run_hls.tcl**。



```
C:\Vivado_HLS_Tutorial\C_Validation\lab1>cd ..
C:\Vivado_HLS_Tutorial\C_Validation>cd lab2
C:\Vivado_HLS_Tutorial\C_Validation\lab2>vivado_hls -f run_hls.tcl
```

图3-11:任意精度类型实验室的C验证

- 3.要打开 Vivado HLS GUI 项目，输入 **vivado_hls -p hamming_window_prj**
- 4.在资源管理器窗格中打开源文件夹并双击 **hamming_window.c** 打开代码，如图3-12 所示。

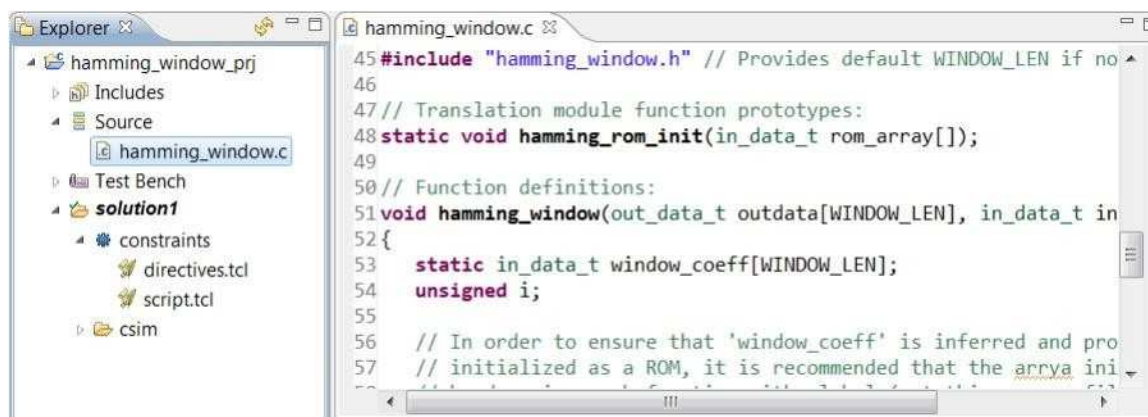


图 3-12:C 验证实验室 2 的 C 代码

- 5.按住 Ctrl 键并单击第 45 行打开 **hamming_window.h** 头文件。
- 6.向下滚动查看类型定义(图 3-13)。

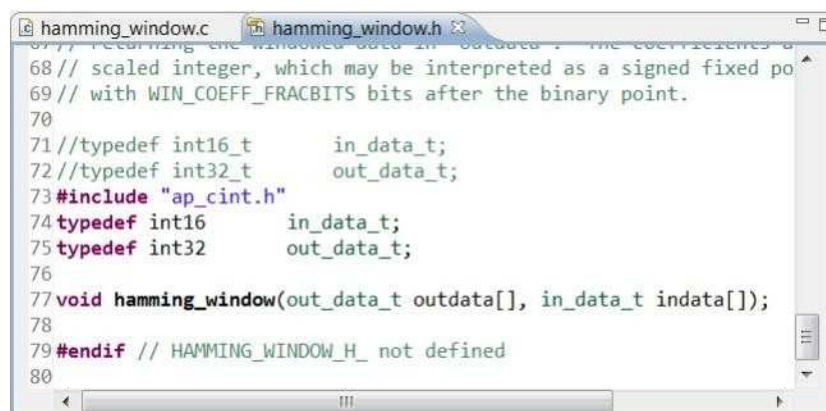


图 3-13:C 验证实验室 2 的类型定义

在这个实验室中，设计与实验室 1 相同，但是，类型已经从标准的 C 数据类型(int16_t 和 int32_t)更新为 Vivado 高级综合提供并在头文件 *ap_cint.h* 中定义的任何精度类型。

关于使用任意精度类型的更多细节将在第 5 章“任意精度类型教程”中讨论。使用任意精度类型的一个例子是将这个文件更改为使用 12 位输入数据类型:标准 C 类型只支持 8 位边界上的数据宽度。

这个练习演示了如何调试这些类型。

步骤 2:运行 C 调试器

- 1.单击“*Run C Simulation*”工具栏按钮，打开“C 仿真”对话框。
- 2.选择 *launch Debugger*选项。
- 3.单击 *OK* 运行模拟。

出现如图 3-14 所示的警告和错误消息。

控制台窗格和日志文件中的消息表明，**您无法调试调试环境中用于 ANSI C 设计的任意精度类型。**

重要提示:在处理任意精度类型时，您能使用的 Vivado HLS 调试环境只能是 c++或 SystemC。

在 ANSI C 中使用任意精度类型时，不能使用调试环境。对于 ANSI C，必须使用 printf 或 fprintf 语句进行调试。

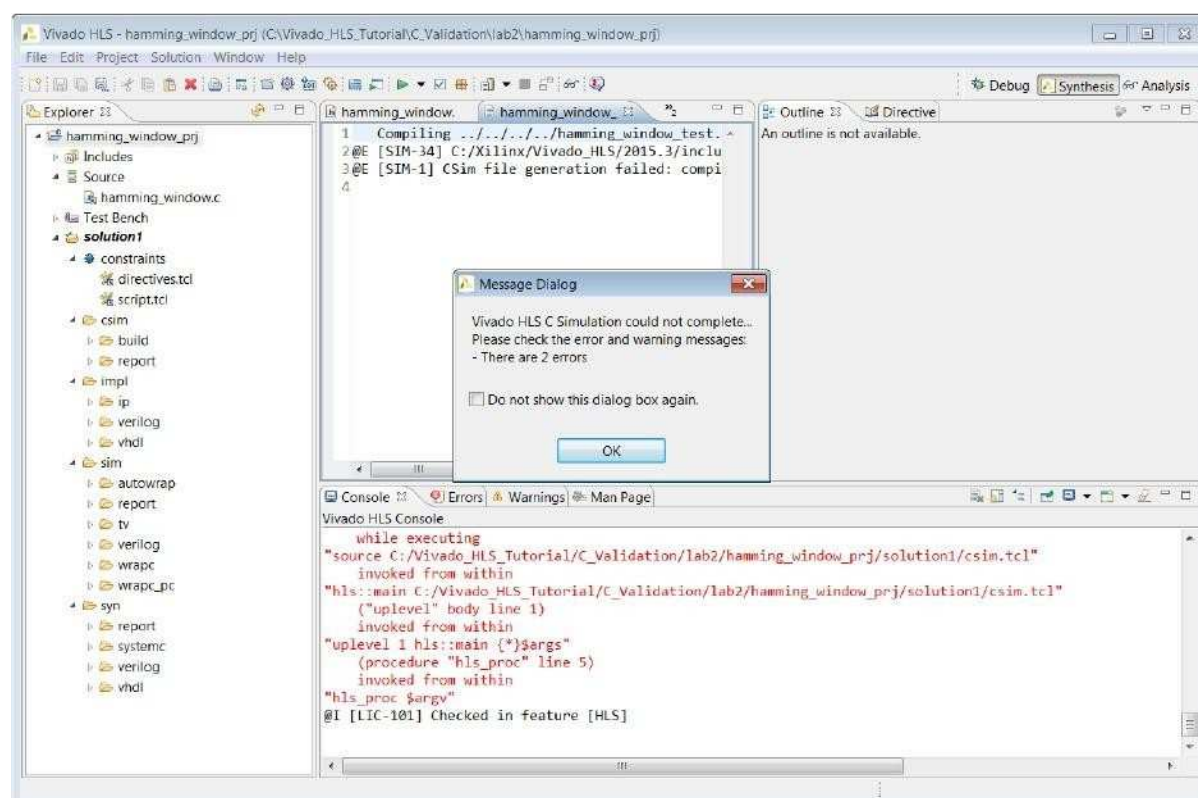


图 3-14:C 仿真对话框

- 4.选择“资源管理器 (*Explorer*)”窗格。
- 5.展开“资源管理器”窗格中的“*Test Bench*”文件夹。
- 6.双击该文件 *hamming_window_test.c*.

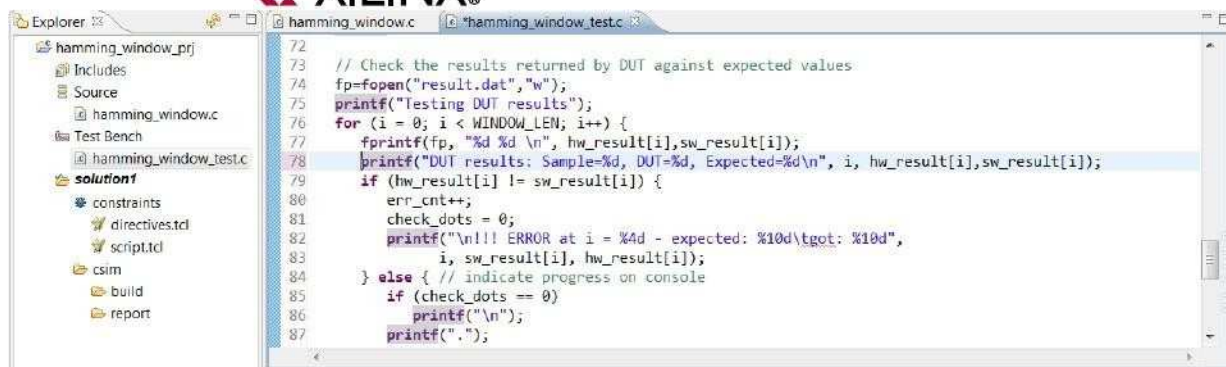


图3-15:启用结果打印

- 7.保存文件。
- 8.选择 **Synthesis**。
- 9.单击 **Run C Simulation** 工具栏按钮或菜单 **Project > Run C Simulation** 打开C 仿真对话框。
- 10.确保没有选择 **Launch Debugger** 选项。
- 11.单击 **OK** 运行模拟。结果显示在 console 窗口中(图 3-16)。

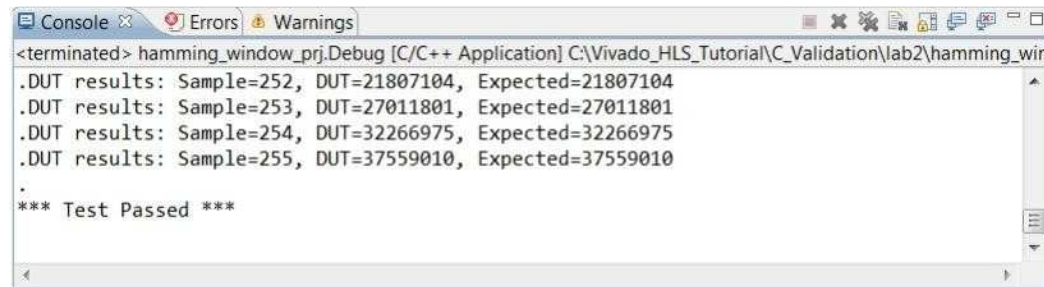


图3-16:C 验证实验室2 结果退出 Vivado

HLS GUI 并返回到命令提示符。

实验 3:使用 c++任意精度类型的 C 验证

概述

本练习使用具有任意精度的 c++类型的设计。您将在 GUI 中检查和调试设计。

步骤 1:创建并打开项目

- 1.从 *lab2*中使用的VivadoHLS 命令提示符，切换到目录 *lab3*。
- 2.通过键入 **vivado_hls -f run_hls.tcl** 创建一个新的 Vivado HLS 项目。
- 3.打开 Vivado HLS GUI 项目，通过 **vivado_hls -p hamming_window_prj**
- 4.在资源管理器窗格中打开源文件夹并双击 **hamming_window.cpp** 打开代码，如图 3-17 所示。

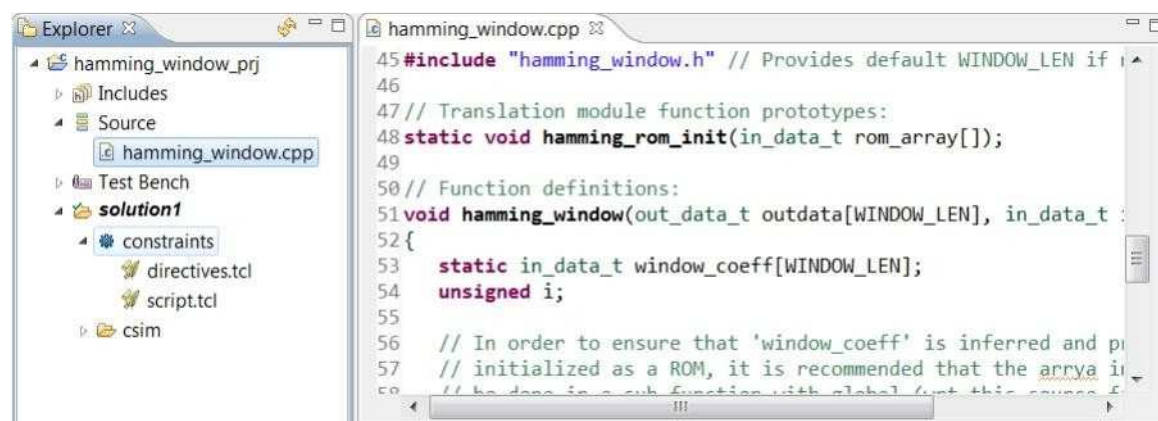


图 3-17:C Validation Lab 3 的 c++ 代码

5. 按住 Ctrl 键并单击第 45 行, 打开这个 **hamming_window.h** 头文件。
6. 向下滚动查看类型定义(图 3-18)。

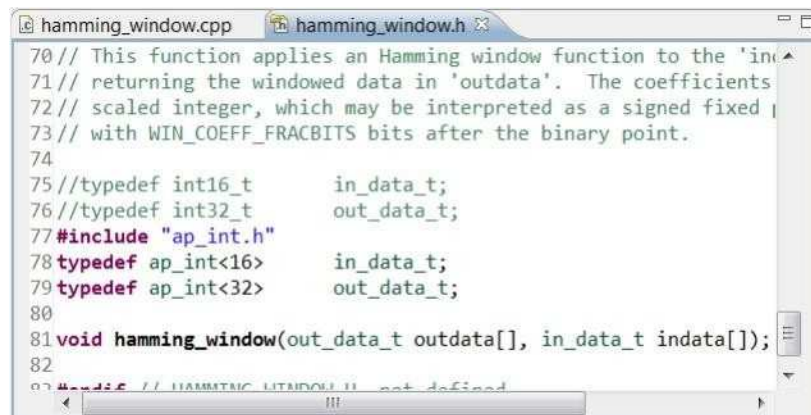


图 3-18:C 验证实验室 3 的类型定义

注:本实验室设计与实验室1、2相同, 但有一个例外。现在的设计是c++, 类型已经更新为使用 c++ 任意精度类型 ap_int<#N>, 由 Vivado HLS 提供, 并在头文件 ap_int.h 中定义。

步骤 2:运行 C 调试器

1. 单击 “**Run C Simulation**” 工具栏按钮, 打开 C 仿真对话框。
2. 选择启动 **Launch Debugger** 选项。
3. 单击 OK。调试环境打开。
4. 选择 **hamming_window.cpp** 选项卡。
5. 在文件 **hamming_window.cpp** 中的第 61 行设置一个断点, 如图 3-19 所示。
6. 单击 **Resume** 按钮(或按 F8 键)执行代码直到断点。

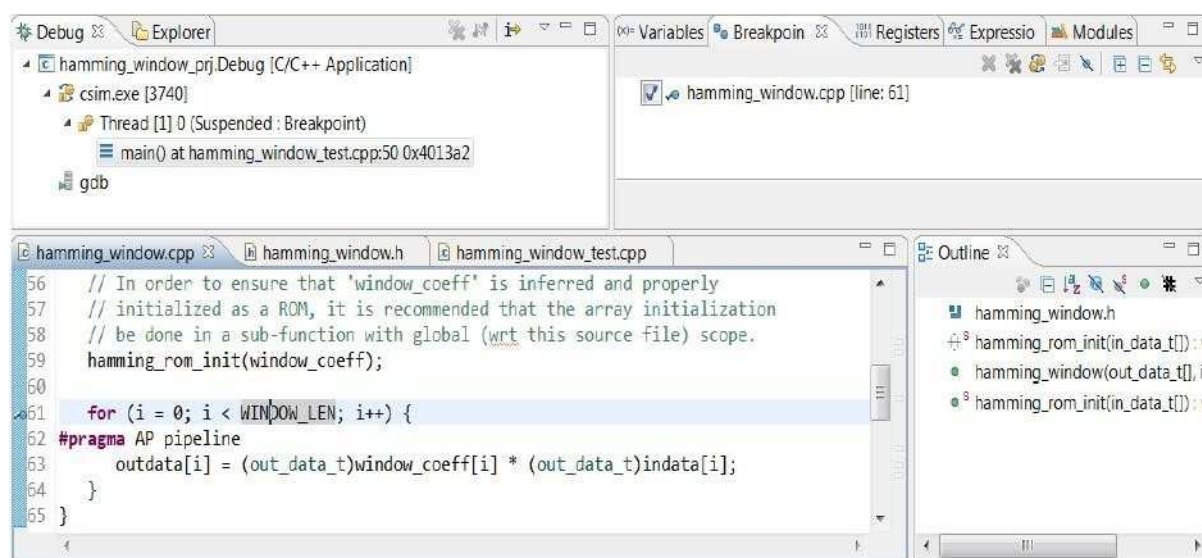


图 3-19:C 验证实验室 3 的调试环境

7. 单击 **Step Into** 按钮(或按 F5 键)两次, 以查看图 3-20 中的视图。设计中的变量现在是 c++ 任意精度类型。这些类型是在头文件 **ap_int.h** 中定义的, 当调试器遇到这些类型时, 它按照定义进入头文件。

当您继续遍历代码时, 您有机会更详细地观察如何计算任意精度类型的结果。

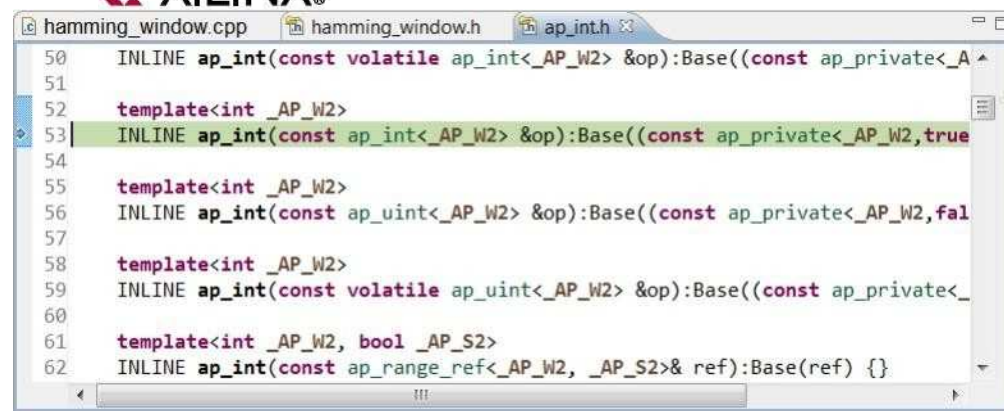


图3-20:任意精度的头文件 更有效的方法是

退出头文件 **ap_int.h** 并返回查看结果。

8. 点击步骤返回按钮(或按 F7 键)返回调用函数。

9. 选择 **Variables** 选项卡并展开 **ap_private** 变量。

10. 展开 **outdata** 变量, 如图 3-21 所示, 以查看 VAL 参数中显示的变量值。

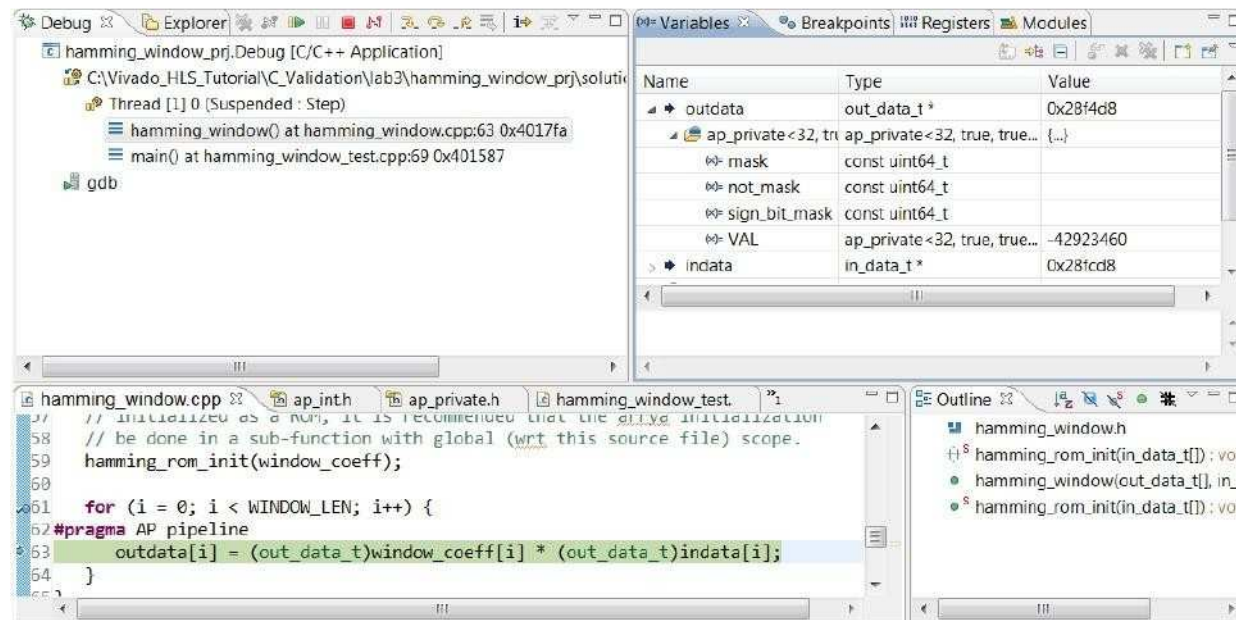


图 3-21:任意精度变量

任意精度类型是创建高性能、位精确硬件设计的强大手段。但是, 在调试环境中, 可以通过逐步遍历头文件定义来降低工作效率。使用断点和步骤返回特性来跳过低级计算并在 variables 选项卡中查看变量的值。

结论

在本教程中, 您学习了:

- **C Test Bench** 在仿真过程中的重要性。
- 如何使用 **C 调试环境**, 设置断点并逐步执行代码。如何调试 C 和 c++任意精度类型。

第四章

接口综合

概述

接口综合是将 RTL 端口添加到 C 设计中的过程。除了将物理端口添加到 RTL 设计之外，接口集成还包括一个相关的 I/O 协议，允许通过端口进行的数据传输自动地与内部逻辑进行最佳同步。

本教程由 4 个实验练习组成，涵盖了接口综合的主要特性和功能。

实验室 1 描述

检查函数返回 (*function return*) 和块级协议 (*block-level protocols*)。

实验室 2 描述

了解端口的默认 I/O 协议，并学习如何选择 I/O 协议。

实验室 3 描述

回顾如何实现数组端口，以及如何对其进行分区 (*partition*)。

实验室 4 描述

创建设计的优化实现并添加 AXI4 接口。

教程设计描述

从 Xilinx 网站下载教程设计文件。参见查找教程设计文件。本教程使用教程目录中的设计文件

Vivado_HLS_Tutorial \ Interface_Synthesis。

关于实验室的

- 本教程前两个实验室中使用的示例设计很简单，这有助于将重点放在接口上。
- 最后两个实验练习使用一个**多通道累加器** (*multichannel accumulator*)。
- 本教程解释如何使用高级综合实现 I/O 端口和协议。
- 在 *lab4* 中，您创建了 *Lab3* 中使用的设计的最佳实现。

实验 1: 块级 I/O 协议

概述

本实验室解释了什么是块级 I/O 协议以及如何控制它们。

重点:本教程中的图形和命令使用教程数据目录



将 *Vivado_HLS_Tutorial* 解压缩并放置在位置 *C:\Vivado_HLS_Tutorial* 中。如果将教程数据目录解压缩到不同的位置, 或者在 Linux 系统上, 请将引用的少数路径名调整到您选择的位置, 以放置 *Vivado_HLS_Tutorial* 目录。

步骤 1: 创建并打开项目

1. 打开 Vivado HLS 命令提示符。

在 Windows 上使用 **Start > All Programs > Xilinx Design Tools > Vivado 2018.3 > Vivado HLS > Vivado HLS 2018.3 Command Prompt**

在 Linux 中, 打开一个新的 shell。

2. 使用命令提示窗口(图 4-1), 将目录更改为接口综合教程 *lab1*。

3. 使用图 4-1 所示的命令, ***vivado_hls -f run_hls.tcl*** 执行 Tcl 脚本来设置 Vivado HLS 项目。

```
C:\Uivado_HLS_Tutorial>cd Interface_Synthesis
C:\Uivado_HLS_Tutorial\Interface_Synthesis>cd lab1
C:\Uivado_HLS_Tutorial\Interface_Synthesis\lab1>vivado_hls -f run_hls.tcl
```

图 4-1: 设置教程项目

4. Vivado HLS 完成后, 使用图 4-2 所示的命令 ***vivado_hls -p adders_prj*** 在 Vivado HLS GUI 中打开项目。

```
@I [LIC-101] Checked in feature [HLS]
Generating csim.exe
10+20+30=60
20+30+40=90
30+40+50=120
40+50+60=150
50+60+70=180
-----Pass!-----
@I [SIM-1] CSim done with 0 errors.
@I [LIC-101] Checked in feature [HLS]
C:\Uivado_HLS_Tutorial\Interface_Synthesis\lab1>vivado_hls -p adders_prj
```

图 4-2: 界面合成实验室 1 的初始项目

步骤 2: 创建并检查默认块级 I/O 协议

1. 双击 **Source** 文件夹以打开源代码 *adders.c* 以进行检查(图 4-3)。

本例使用一个简单的设计来关注 I/O 实现(而不是设计中的逻辑)。**本代码的要点如下:**

- 指令以 **pragmas** 的形式被添加到源代码中, 以防止任何 I/O 协议被任何数据端口(inA、inB 和 inC)合成。I/O 端口协议将在下一次实验室练习中进行介绍。
- 函数有一个返回值, 也是函数的唯一输出。正如在后面的练习中看到的, 并不是所有函数都返回一个值。在这个实验练习中讨论了为函数返回创建端口。

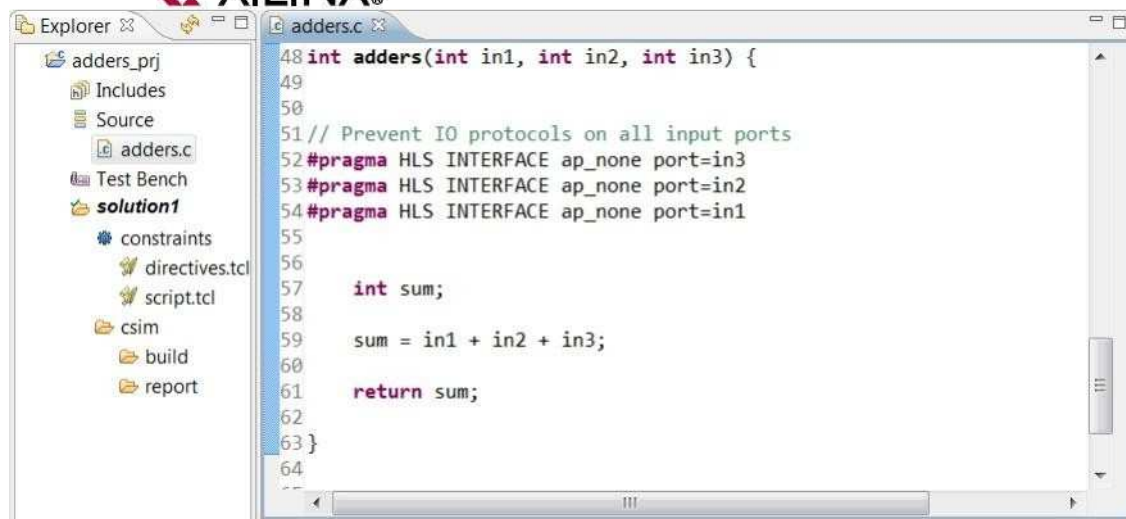


图4-3: 接口合成实验室1的C代码

2. 使用专用工具栏按钮或 **Solution** 菜单执行 **Run C Synthesis** 命令。当综合完成时，合成报告将自动打开。

3. 要查看 RTL 接口，请滚动到综合报告末尾的 **Interface Summary**。

界面摘要和大纲选项卡如图4-4所示。

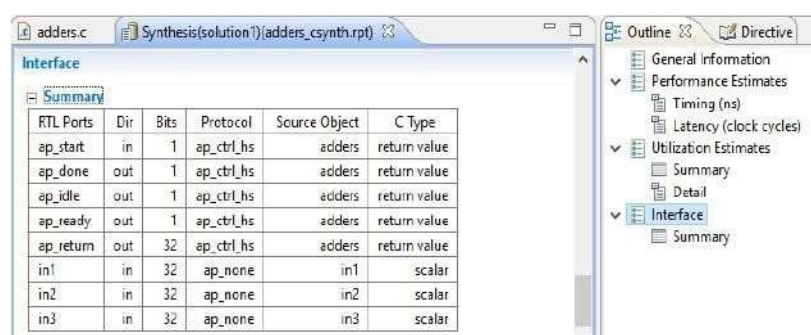


图4-4: 界面摘要

有四种类型的端口需要检查:

- 设计需要一个以上的时钟周期来完成，所以在设计中添加了一个**时钟和复位**：**ap_clk** 和 **ap_rst**，都是标量输入。
- 增加了块级 I/O 协议来控制 RTL 设计：**ap_start, ap_done, ap_idle, ap_ready**，端口和这些端口将很快进行讨论。
- 该设计有四个数据端口。in1、in2 和 in3 是 32 位输入，具有 I/O 协议 **ap_none** (由图 4-4 中的指令指定)。
- 该设计还为函数 返回提供了 32 位的输出端口，**ap_return**。

块级 I/O 协议允许 RTL 设计由独立于数据 I/O 端口的其他端口控制。这个 I/O 协议与函数本身相关联，而不是与任何数据端口相关联。默认的块级 I/O 协议称为 **ap_ctrl_hs**。图 4-5 显示了该协议与函数返回值相关联(即使函数没有在代码中指定返回值，也是如此)。

表 4-1 总结了块级 I/O 协议的 **ap_ctrl_hs**

注意: 这里的解释使用了术语 **“transaction”**。在高级综合的上下文中，事物相当于 C 函数的一次执行(或综合的 RTL 设计中的等效操作)。

表 4-1: 块级 I/O 协议 *ap_ctrl_hs*

信号	描述
ap_start	该信号控制块的执行，必须将其置为逻辑 1 才能使设计开始工作。 它应保持为逻辑 1，直到相关联的输出握手 ap_ready 被声明为止。当 ap_ready 变高时，可以决定是否保持 ap_start 断言并执行另一个事务，或者将 ap_start 设置为逻辑 0 并允许设计在当前事务结束时暂停。 如果在 ap_ready 为高电平之前将 ap_start 断言为低电平，则该设计可能没有读取所有输入端口，并且可能在下一次输入读取时停止操作。
ap_ready	该输出信号指示设计何时准备好用于新的输入。 当设计准备好接受新的输入时，将 ap_ready 信号设置为逻辑 1，表示此事务的所有输入读取均已完成。 如果设计没有流水线操作，则在下一个事务开始之前不会执行新的读取。 该信号用于决定何时将新值应用于输入端口，以及是否应该使用 ap_start 输入信号来启动新事务。 如果 ap_start 信号未断言为高，则此符号
ap_done	该信号指示设计何时完成了当前事务中的所有操作。 此输出上的逻辑 1 表示设计已完成此事务中的所有操作。 因为这是事务的结束，所以此信号上的逻辑 1 也表示 ap_return 端口上的数据有效。 并非所有函数都具有函数返回参数，因此并非所有 RTL 设计都具有 ap_return 端口。
ap_idle	该信号指示设计是正在运行还是处于空闲状态（不运行）。 空闲状态由该输出端口上的逻辑 1 指示。 设计开始运行后，此信号被置为低电平。 当设计完成操作且不再执行任何操作时，此信号被置为高电平。

通过查看 *RTL CoSimulation* 生成的跟踪文件，可以观察这些信号的行为。这将在第 8 章 RTL 验证教程中讨论，但是图 4-5 显示了当前合成结果的波形。

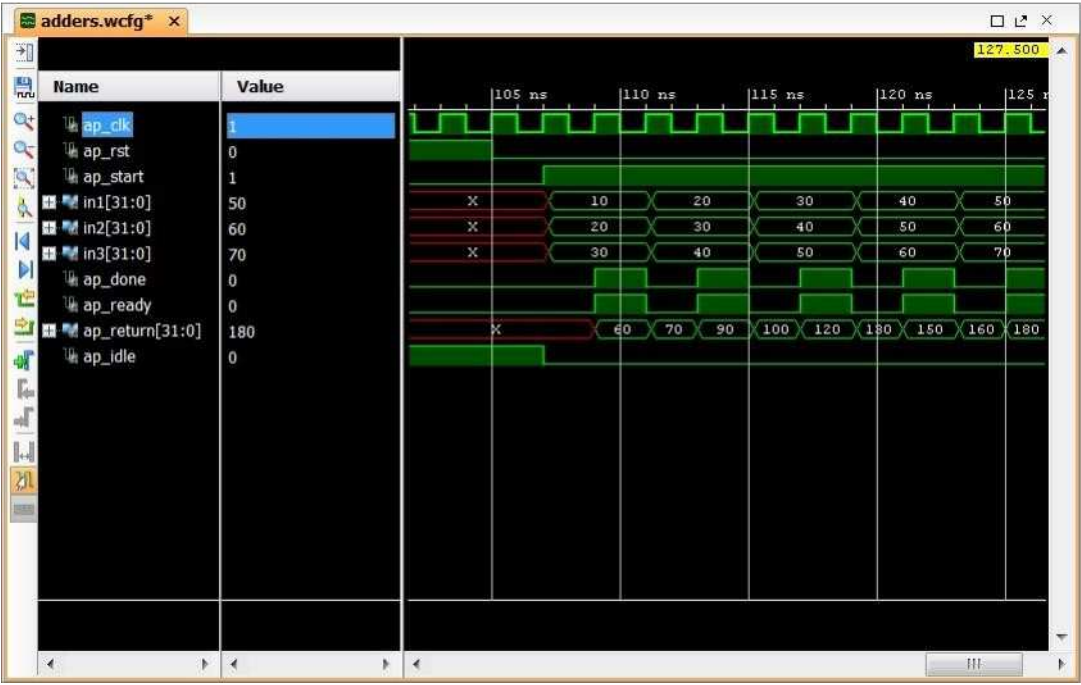


图 4-5: 块协议信号的 RTL 波形

图 4-5 中的波形显示了块级 I/O 信号的行为。

- 在设置 **ap_start** 为逻辑 1 之前，设计不会启动操作。
- 通过设置 **ap_idle** 端口低，设计表明不再空闲。
- 显示 5 个事务。前三个输入值(10、20 和 30)分别应用于输入端口 In1、In2 和 In3。
- 输出信号 **ap_ready** 变高，表明设计已经为下一个时钟的新输入做好了准备。
- 输出信号 **ap_done** 表明当设计完成时，输出端口 **ap_return** 上的值是有效的(第一个输出值 60，是所有三个输入的总和)。
- 因为 **ap_start** 为高，下一个事务开始于下一个时钟周期。

注意: 在 RTL 协仿真中，所有的设计和端口输入控制信号都是启用的。例如，在图 4-5 中，信号 **ap_start** 总是高的。

在第二个事务中，注意端口 **ap_return**，第一个输出的值是 70。此端口上的结果在信号 **ap_done** 拉高之后有效。

步骤3: 修改块级 I/O 协议

默认的块级 I/O 协议是 **ap_ctrl_hs** 协议(控制握手协议)。在此步骤中，您将创建一个新的解决方案并修改此协议。

1. 从工具栏或项目菜单中选择 “**New Solution**” 以创建新解决方案。
2. 将 “**New Solution**” 对话框中的所有设置保留为默认设置，然后单击 “**Finish**”。在信息窗格中选择 C 源代码选项卡(如果已关闭，则重新打开 C 源代码)。
4. 激活选项卡并选择顶级函数 **adders**，如图 4-6 所示：

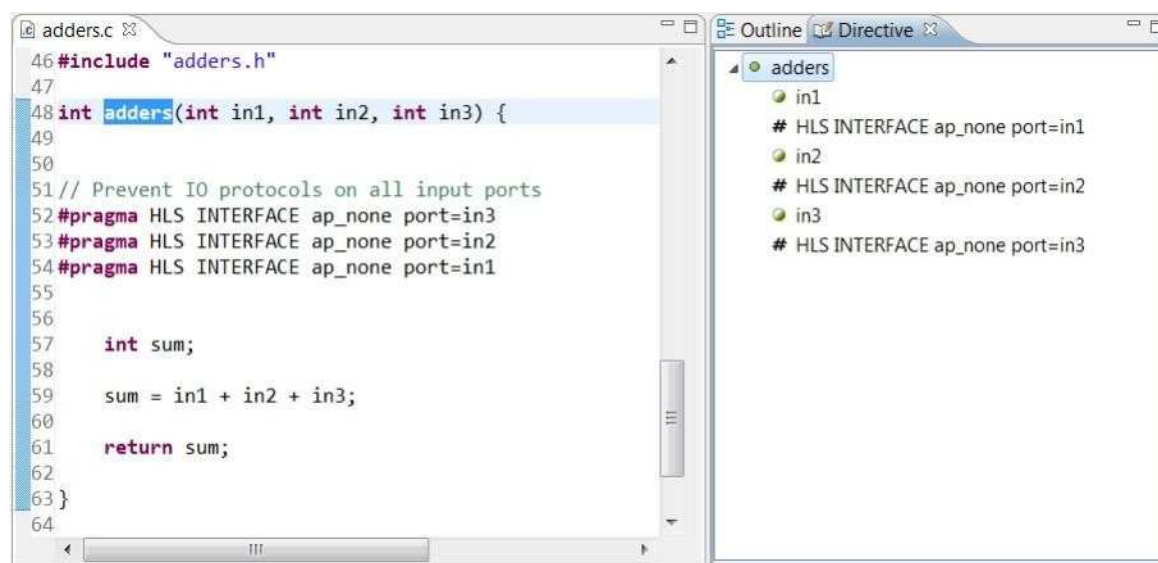


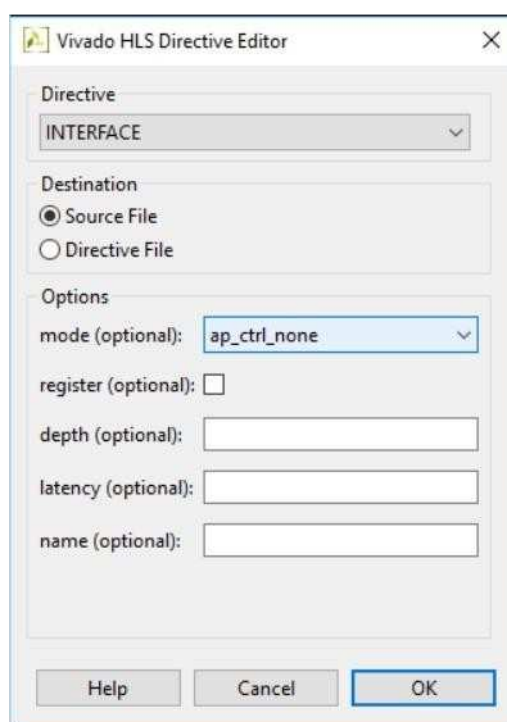
图 4-6: 选择的顶级函数

因为块级 I/O 协议与函数相关，所以必须通过选择顶级函数来指定它们。

5. 在 **Directive** 选项卡中，将鼠标右键放在顶层函数 **adders** 上，选择 **Insert Directive**。

“指令编辑器”对话框打开。从指令下拉列表中选择 **INTERFACE** 选项。

图 4-7 显示了激活界面模式的下拉菜单对话框。

图 4-7: `ap_ctrl_none` 的指令对话框

下拉菜单显示块级接口协议有四个选项:

- **`ap_ctrl_none`**: 没有块级 I/O 控制协议。
- **`ap_ctrl_hs`**: 我们已经审查了块级 I/O 控制握手协议。
- **`ap_ctrl_chain`**: 用于控制链接的块级 I/O 协议。这个 I/O 协议主要用于将管道块链接在一起。
- **`s_axilite`**: 可以应用于 `ap_ctrl_hs` 或 `ap_ctrl_chain` 之外, 将块级 I/O 协议实现为一个 AXI 从接口, 代替单独的离散 I/O 端口。

本教程不讨论块级 I/O 协议。**`ap_ctrl_chain`** 这个协议与协议 **`ap_ctrl_hs`** 类似, 但是有一个额外的输入信号 **`ap_continue`**, 当断言下一个事务继续进行时, 该信号必须是高的。

6. 在“指令编辑器”对话框的“目标”部分, 选择“**`Source File`**”。

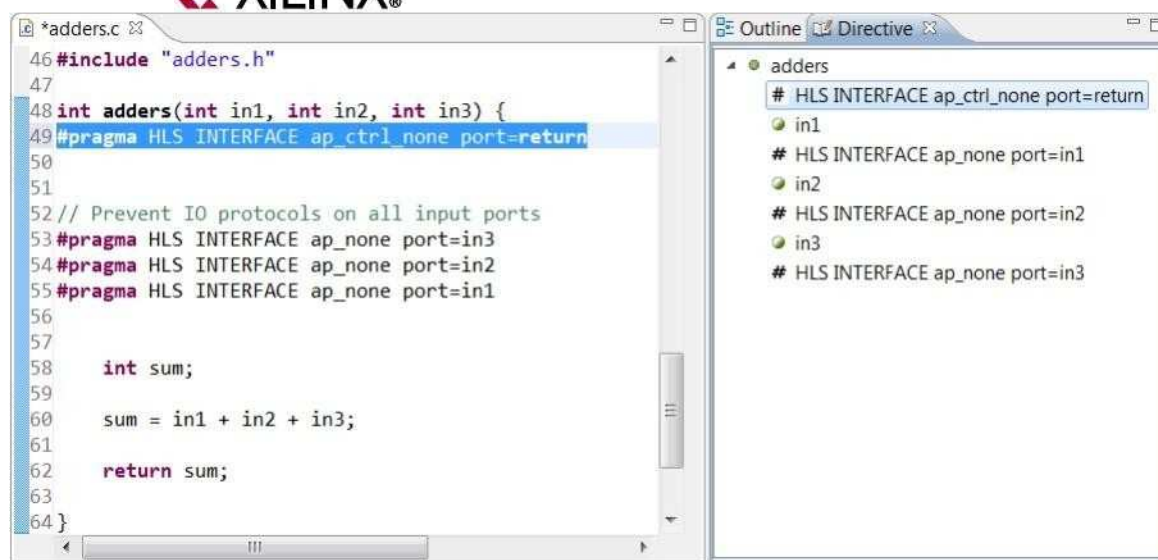
默认情况下, 指令放在 **`directives.tcl`** 文件中。在本例中, 指令与现有的 I/O 指令一起放在源文件中。

7. 从 **`Mode`** 选项中, 从下拉菜单中选择 **`ap_ctrl_none`**。

8. 单击 **`OK`**。

源文件现在有了一个新的指令, 在图 4-8 的源代码和指令选项卡中高亮显示。

新指令显示了关联的函数自变量/端口, 称为 **`return`**。所有接口指令都附加到函数参数上。对于块级 I/O 协议, **`return`** 参数用于指定块级接口。即使函数在源代码中没有返回参数, 也是如此。

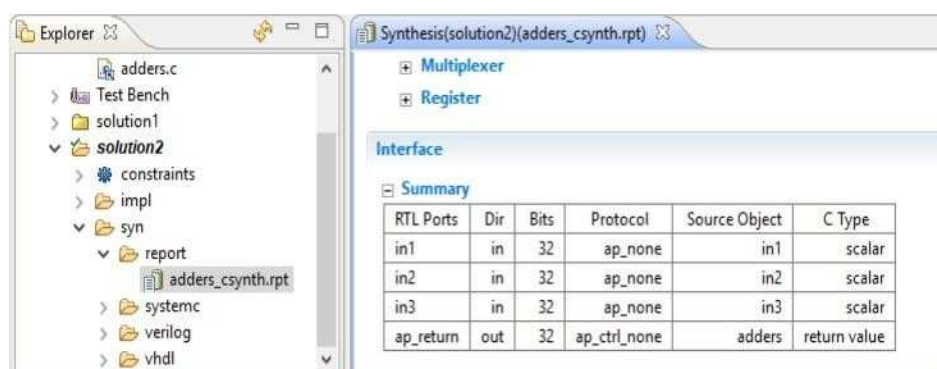
图 4-8: 块级接口指令 `ap_ctrl_none`

9. 单击运行 **Run C Synthesis** 工具栏按钮或使用菜单 **Solution > Run C Synthesis** 设计。

将指令添加到源文件中会修改源文件。图 4-8 显示源文件名 **adders.c*，星号表示文件已修改但未保存。

10. 单击 Yes 接受对源文件的更改。

当报告打开时，界面摘要会出现，如图 4-9 所示。

图 4-9: `ap_ctrl_none` 的接口摘要

在使用接口协议 **`ap_ctrl_none`** 时，不会向设计中添加任何块级 I/O 协议。唯一的端口是时钟、复位和数据端口。

请注意，如果没有 **`ap_done`** 信号，则从 **`ap_return`** 端口接受数据的使用者块现在没有指示数据何时有效的指示！

此外，RTL 协同仿真功能需要块级 I/O 协议来对测试平台和 RTL 设计进行排序，以自动进行协同仿真。任何尝试使用 RTL 协同仿真的操作都会导致以下错误消息以及 RTL 协同仿真的暂停：

Cosim 只支持以下“`ap_ctrl_none`”设计：(1) 组合设计；(2) 流水线设计，任务间隔为 1；(3) 使用数组流或 `hls_stream` 端口进行设计。
@E [SIM-4] *** C/RTL 协同仿真完成:FAIL ***

退出 Vivado HLS GUI 并返回到命令提示符。

实验 2: 端口 I/O 协议

概述

这个练习解释了如何指定端口 I/O 协议。

步骤 1:创建并打开项目

- 1.从 lab1 中使用的 Vivado HLS 命令提示符，切换到如图 4-10 所示的目录 lab2。
- 2.键入 `vivado_hls -f run_hls.tcl`以创建一个新的 Vivado HLS 项目。

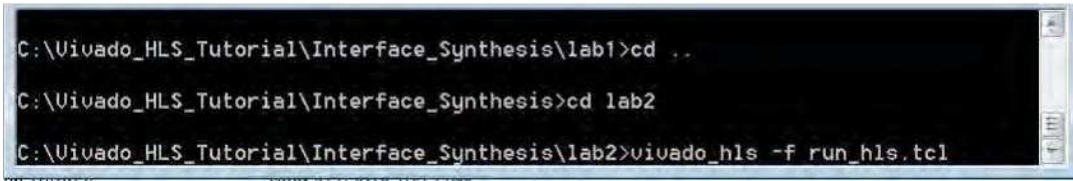


图 4-10: 界面合成实验室 2 的设置

- 3.键入 `vivado_hls -p adders_io_prj`以打开 Vivado HLS GUI 项目。
- 4.打开源代码，如图4-11 所示。

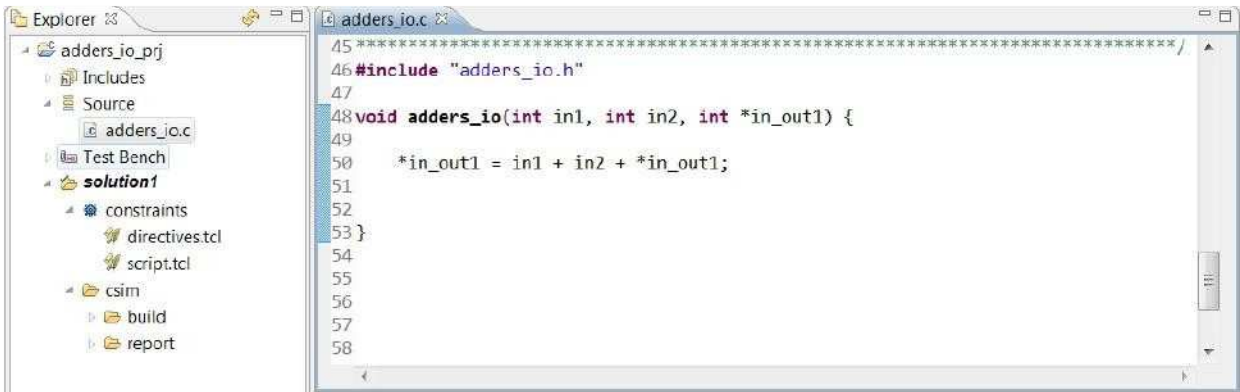


图 4-11: 接口 Sythesis lab2 的 C 代码

这个练习的源代码(`adders_io.c`)类似于 lab1 中使用的简单代码。出于类似的原因，它有助于关注接口行为，而不是核心逻辑。

这一次，代码没有函数返回，而是通过**指针参数 `*in_out1`**传递函数的输出。这也为探索**双向**(输入和输出)端口的接口选项提供了机会。

可以通过接口综合添加到 C 函数参数的 I/O 协议的类型取决于**参数类型**。这些选项在《**Vivado 设计套件用户指南：高级综合 (UG902) [参考 2]**》中有完整描述。

本例中的指针参数既是函数的输入又是函数的输出。在 RTL 设计中，这个参数被实现为**单独**的输入和输出端口。

对于图 4-11 所示的代码，表 4-2 描述了每个函数参数的可能选项。

表 4-2: 实验室 2 的端口级别 I/O 协议选项

函数参数	I / O 协议选项
In1 和 In2	方法实现的 按值传递参数 I / O 协议: •ap_none:无 I/O 协议。这是输入的默认值。 •ap_stable:没有 I/O 协议。 •ap_ack:通过关联的输出确认端口实现。 •ap_vld:使用关联的输入有效端口实现。 - ap_hs:通过输入有效端口和输出确认端口实现。
in_out1	实现的按 引用传递 输出 I / O 协议: •ap_none:无 I/O 协议。这是输入的默认值。 •ap_stable:没有 I/O 协议。 •ap_ack:使用关联的输入确认端口实现。 •ap_vld:使用关联的输出有效端口实现。这是默认的输出。

- ap_ovld: 使用关联的输出有效端口（对于任何 inout 端口的输入部分均没有有效端口）实现。
- ap_hs:通过输入有效端口和输出确认来实现端口
- ap_fifo:一个 FIFO 接口与相关的输出写和输入 FIFO 满端口。
- ap_bus:一个 Vivado HLS 总线接口协议。

注意: 实际上没有必要在 lab1 中应用端口指令，因为 **ap_none** 是这些 C 参数的默认 I/O 协议。提供这些指令是为了避免解决该练习中的任何 I/O 端口协议行为，无论是否存在默认行为。

步骤 2:指定端口的 I/O 协议

- 1.确保您可以在信息窗格中看到 C 源代码。
- 2.激活选项卡并选择 input **in1**，如图 4-12 所示。

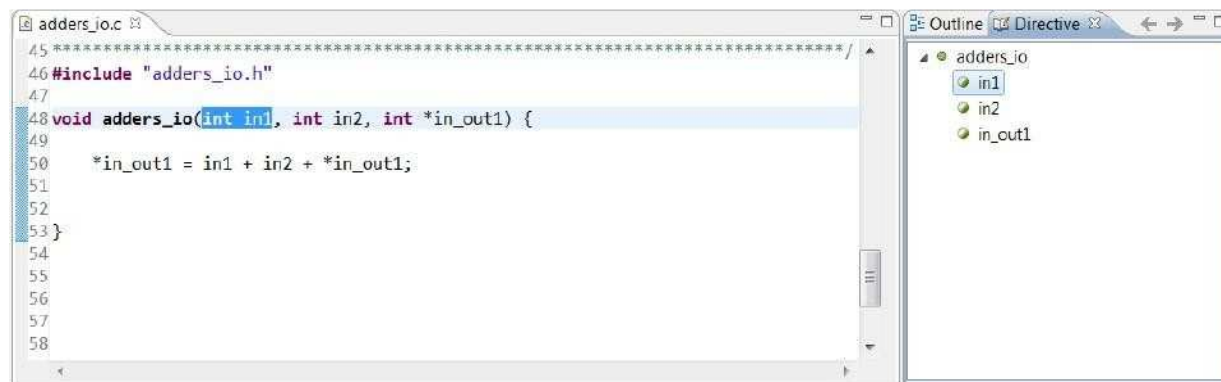


图 4-12:添加端口 I/O 协议

- 3.右键单击并选择 **Insert Directive**。
- 4.当指令编辑器打开时，将指令下拉菜单作为接口。
 - a.保持默认值。这一次，指令存储在 **directives.tcl** 文件指令中。
 - b.从 **mode** 下拉菜单中选择 **ap_vld**
 - c.单击 OK。
- 5.选择参数 **in2** 并添加一个接口指令 **ap_ack** 来指定 I/O 协议。
- 6.选择参数 **in_out1** 并添加一个接口指令 **ap_hs** 来指定 I/O 协议。
- 7.在资源管理器窗格中，展开 **Constraints** 文件夹，双击 **directives.tcl** 文件打开它，如图 4-13 所示。

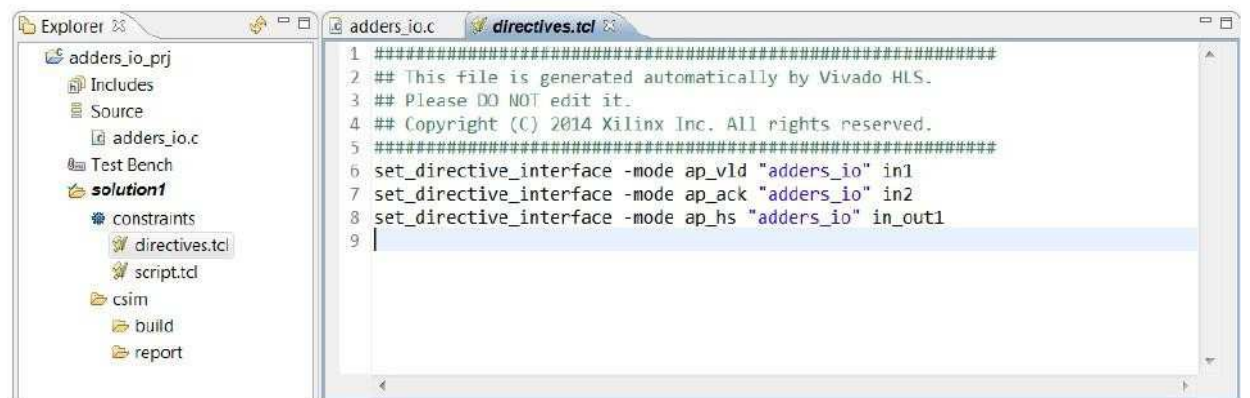
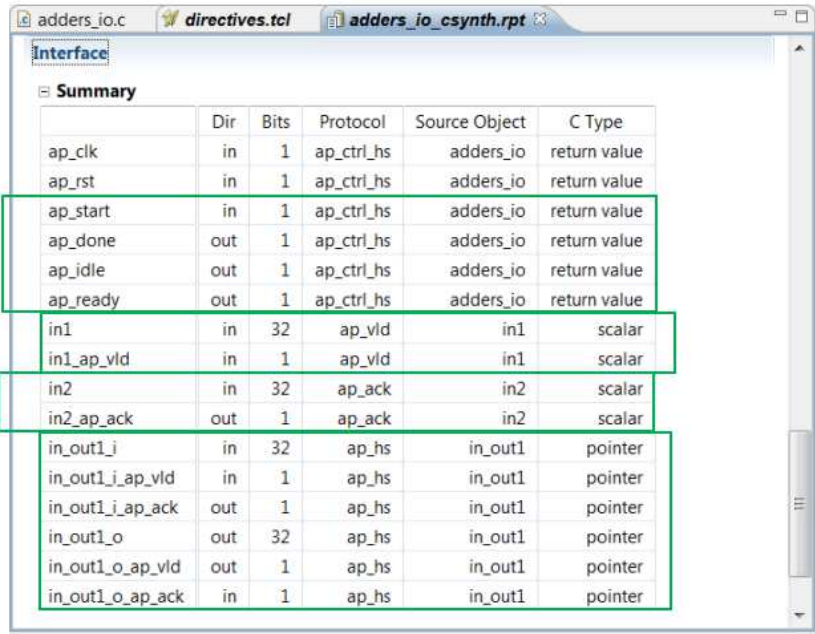


图 4-13 实验室 2 的指令

- 8.综合设计。
- 9.当报告文件打开时，查看接口摘要(图 4-14)。



	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	adders_io	return value
ap_rst	in	1	ap_ctrl_hs	adders_io	return value
ap_start	in	1	ap_ctrl_hs	adders_io	return value
ap_done	out	1	ap_ctrl_hs	adders_io	return value
ap_idle	out	1	ap_ctrl_hs	adders_io	return value
ap_ready	out	1	ap_ctrl_hs	adders_io	return value
in1	in	32	ap_vld	in1	scalar
in1_ap_vld	in	1	ap_vld	in1	scalar
in2	in	32	ap_ack	in2	scalar
in2_ap_ack	out	1	ap_ack	in2	scalar
in_out1_i	in	32	ap_hs	in_out1	pointer
in_out1_i_ap_vld	in	1	ap_hs	in_out1	pointer
in_out1_i_ap_ack	out	1	ap_hs	in_out1	pointer
in_out1_o	out	32	ap_hs	in_out1	pointer
in_out1_o_ap_vld	out	1	ap_hs	in_out1	pointer
in_out1_o_ap_ack	in	1	ap_hs	in_out1	pointer

图 4-14: 实验室 2 的界面摘要

- 该设计有一个时钟和复位。
- 默认块级 I/O 协议信号出现。
- 端口 *in1* 是用一个数据端口和一个相关的输入有效信号来实现的。
- *in1* 端口上的数据只有在 *in1_ap_vld* 端口处于高时才读取。
- *in2* 端口由一个数据端口和一个相关的输出确认信号来实现。
- 当读取 *in2* 数据端口时，*in2_ap_ack* 端口必须处于状态。
- *inout_i* 标识参数 *inout1* 的输入部分。它具有关联的输入有效端口 *inout1_i_ap_vld* 和输出确认端口 *inout1_i_ap_ack*。
- 参数 *inout1* 的输出部分被标识为 *inout_o*。这具有关联的输出有效端口 *inout1_o_ap_vld* 和输入确认端口 *inout1_o_ap_ack*

10.退出 Vivado HLS
GUI 并返回到命令提示符。

实验 3:将数组实现为 RTL 接口

介绍

本练习展示了如何将顶级函数接口上的数组参数实现为许多不同类型的 RTL 端口。

步骤 1:创建并打开项目

- 1.从先前实验中使用的 Vivado HLS 命令提示窗口，切换到目录 *lab3*。
- 2.通过键入 *vivado_hls -f run_hls.tcl*，创建一个新的 Vivado HLS 项目。
3. 输入 *vivado_hls -p array_io_prj* 打开 Vivado HLS GUI 项目。
4. 打开源代码(*array_io.c*)，如图 4-15 所示。

此设计具有一个输入数组和一个输出数组。C 源代码中的注释说明了如何将输入数组中的数据作为通道排序以及如何累积通道。要了解设计，您还可以查看测试台以及文件 *result.golden.dat* 中的输入和输出数据。

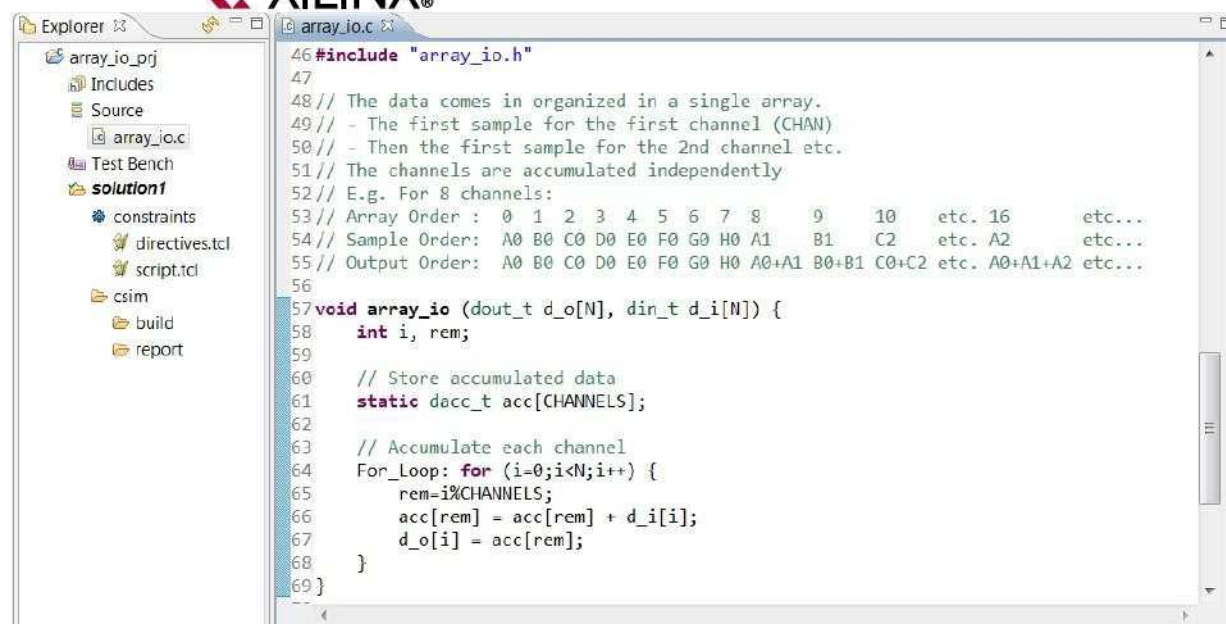


图4-15: 接口合成实验室3的C代码

步骤 2: 将数组函数参数综合到 RAM 端口

在这一步中，您将回顾如何将数组端口综合为 RAM 端口。

1. 在工具栏中运行 **C Synthesis** 按钮，并在报表打开时查看界面摘要(图 4-16)。

接口摘要显示了默认情况下如何将 C 源代码中的数组参数合成为 RTL RAM 端口。

- 该设计具有时钟，复位和默认的块级 I/O 协议 **ap_ctrl_hs**（在报告中的时钟上注明）。
- **d_o 参数** 已综合到 RAM 端口（I/O 协议 **ap_memory**）。
- 一个数据端口 (**d_o_d0**)。
- 一个地址端口 (**d_o_address0**)。
- 控制芯片启用端口 (**d_o_ce0**) 和写启用端口 (**do_we0**)。
- **d_i 参数** 已经综合到类似的 RAM 接口，但是具有输入数据端口 (**d_i_q0**)，并且没有写使能端口，因为该接口仅读取数据。

在这两种情况下，数据端口都是 C 源中数据值的宽度（在这种情况下为 16 位整数），并且地址端口的宽度已自动调整大小以匹配必须访问的地址数（5 位，用于 32 个地址）。

	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	array_io	return value
ap_rst	in	1	ap_ctrl_hs	array_io	return value
ap_start	in	1	ap_ctrl_hs	array_io	return value
ap_done	out	1	ap_ctrl_hs	array_io	return value
ap_idle	out	1	ap_ctrl_hs	array_io	return value
ap_ready	out	1	ap_ctrl_hs	array_io	return value
d_o_address0	out	5	ap_memory	d_o	array
d_o_ce0	out	1	ap_memory	d_o	array
d_o_we0	out	1	ap_memory	d_o	array
d_o_d0	out	16	ap_memory	d_o	array
d_i_address0	out	5	ap_memory	d_i	array
d_i_ce0	out	1	ap_memory	d_i	array
d_i_q0	in	16	ap_memory	d_i	array

图4-16: 实验室3 初步设计的界面摘要

默认情况下，将数组参数综合到 RAM 端口。您可以使用许多其他选项控制如何实现这些端口。实验 3 中的其余步骤演示了这些选项：

- 使用单端口或双端口 RAM 接口。

- 使用 FIFO 接口。
- 划分成离散的端口 (*partitioning into discrete ports*)。

步骤 3: 使用双端口 RAM 和 FIFO 接口

高级综合允许您将 RAM 接口指定为单端口或双端口。如果不进行这样的选择，Vivado HLS 将自动分析设计并选择端口的数量，以最大限度地提高数据速率。

第 2 步使用了单端口 RAM 接口，因为默认情况下左滚源代码中 for 循环：循环的每个迭代依次执行：

- 1) 读取输入端口。
- 2) 从内部 RAM 读取累积的结果。
- 3) 对累积的数据和新数据求和，然后写入内部 RAM。
- 4) 将结果写入输出端口。
- 5) 在循环的下一迭代中重复。

这样可确保仅需要一次输入读取和输出写入。即使提供了多个输入和输出，内部逻辑也无法利用任何其他端口。

注意: 如果您指定了一个双端口 RAM，并且 Vivado HLS 可以确定只需要一个端口，那么它将使 用一个单端口并覆盖双端口规范。

在这个设计中,如果你想使用多个 RTL 端口实现数组参数,你必须做的第一件事就是展开 for 循环,让所有内部操作发生在平行, 否则在多个端口没有好处: 滚循环限制只有一个数据样本可以读(或写)。

1. 从工具栏或项目菜单中选择 “*New Solution*” 以创建新解决方案。
2. 接受默认设置，然后单击 Finish。
3. 确保 C 源代码在信息窗格中可见。
4. 在 *Directive* 选项卡中选择 *For_Loop*，右键点击并选择 Insert Directive，打开 Directive Editor 对话框。
 - a) 在指令编辑器对话框中激活顶部的指令下拉菜单并选择展开 (*UNROLL*)。
 - b) 使用如图 4-17 所示的指令编辑器，点击 OK。

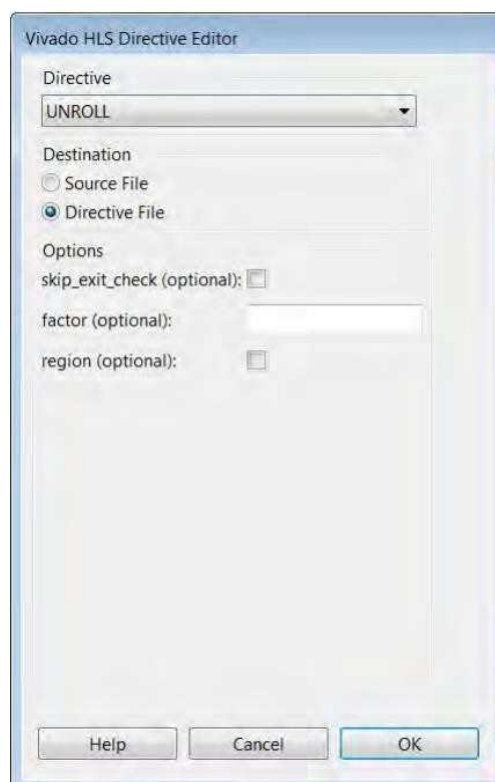


图 4-17: 展开 For_Loop 的指令编辑器

5. 接下来, 为输入读取指定一个双端口 RAM。Resource 指令指示连接到接口的 RAM 类型。

- 在 **Directive** 选项卡中, 选择 **port d_i**, 右键点击并选择 **Insert Directive**, 打开 Directive Editor 对话框。
- 在指令编辑器中激活顶部的指令下拉菜单并选择资源 **RESOURCE**。
- 点击 **core** 框, 选择 **RAM_2P_BRAM**。
- 确认如图 4-18 所示的 directive Editor 对话框中的设置, 然后点击 OK。

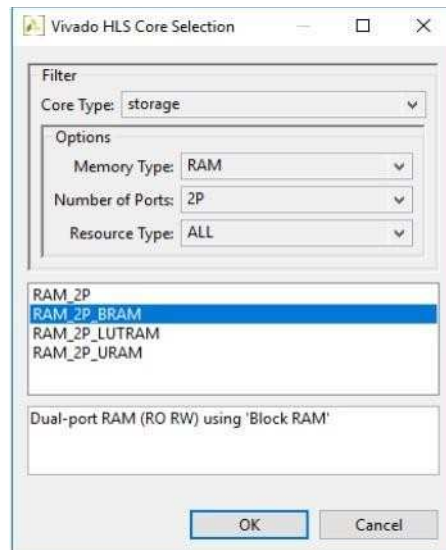


图 4-18: 用于指定双端口 RAM 6 的指令编辑器。

6. 使用 FIFO 接口实现输出端口:

- 在 **Directive** 选项卡中, 选择端口 **d_o**, 右键点击并选择 **Insert Directive**, 打开 **Directive Editor** 对话框。
- 在指令编辑器中, 确保指令是 **Interface**。
- 从 **Mode** 下拉 mode 菜单中选择 **ap_fifo**。
- 单击 **OK**。

Directive 选项卡显示了现在应用于设计的指令(图4-19)。

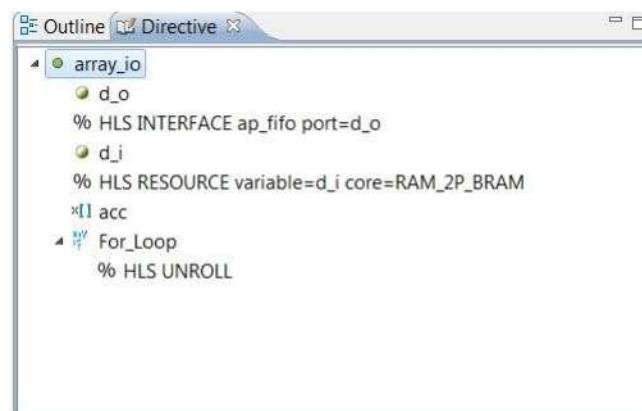
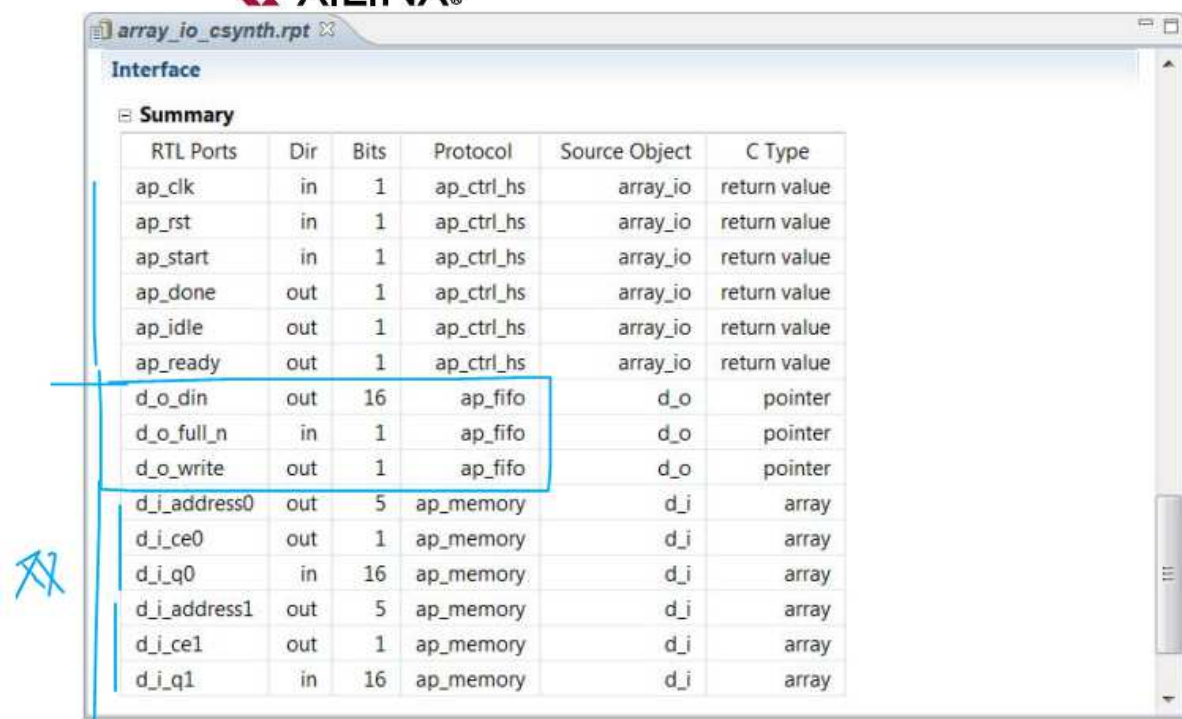


图4-19: 实验2 解决方案7 的指令摘要。

7. 从工具栏运行 **C Synthesis** 来综合设计

当报告在信息窗格中打开时, 界面摘要如图 4-20 所示。

- 该设计具有标准的时钟、复位和块级 I/O 端口。
- 数组参数 **d_o** 已实现为具有 16 位数据端口 (**d_o_din**) 以及关联的输出写 (**d_o_write**) 和输入 FIFO 已满 (**d_o_full_n**) 端口的 FIFO 接口。
- 参数 **d_i** 已实现为双端口 RAM 接口。



RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	array_io	return value
ap_rst	in	1	ap_ctrl_hs	array_io	return value
ap_start	in	1	ap_ctrl_hs	array_io	return value
ap_done	out	1	ap_ctrl_hs	array_io	return value
ap_idle	out	1	ap_ctrl_hs	array_io	return value
ap_ready	out	1	ap_ctrl_hs	array_io	return value
d_o_din	out	16	ap_fifo	d_o	pointer
d_o_full_n	in	1	ap_fifo	d_o	pointer
d_o_write	out	1	ap_fifo	d_o	pointer
d_i_address0	out	5	ap_memory	d_i	array
d_i_ce0	out	1	ap_memory	d_i	array
d_i_q0	in	16	ap_memory	d_i	array
d_i_address1	out	5	ap_memory	d_i	array
d_i_ce1	out	1	ap_memory	d_i	array
d_i_q1	in	16	ap_memory	d_i	array

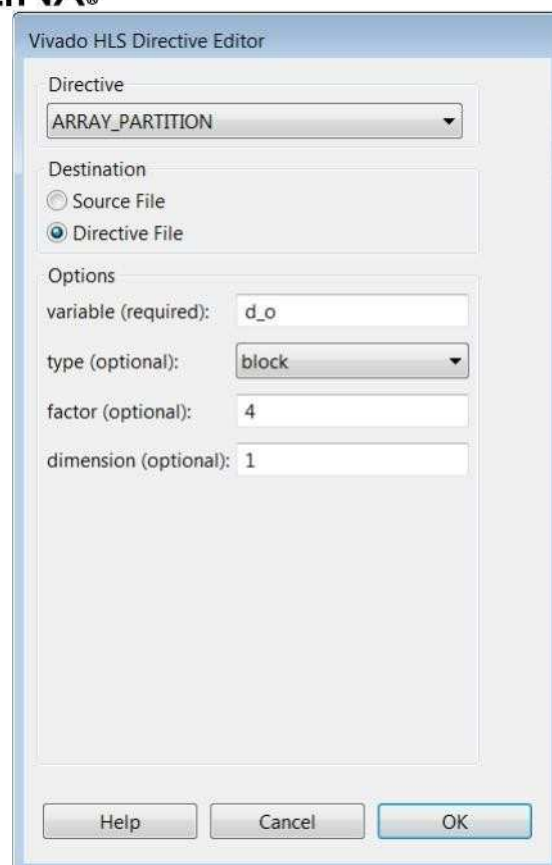
图 4-20: 双端口 BRAM 和 FIFO 接口

通过使用双端口 RAM 接口，该设计可以以以前设计两倍的速率接受输入数据。因为 for 循环已展开，所以循环中的逻辑能够以此速率使用数据。默认情况下，每个循环迭代依次执行。此实现代码将逻辑限制为在每次迭代中对 d_i 进行一次读取。**展开循环可以执行更多读取（但会创建 N 个逻辑副本）。但是，在输出上使用单端口 FIFO 接口，输出数据速率与之前相同。**

步骤 4: 分区 RAM 和 FIFO 数组接口

在此步骤中，您将了解如何将数组接口划分(Partition)为任意数量的端口。

1. 从工具栏或项目菜单中 选择新解决方案并创建新解决方案。
2. 接受默认设置，然后单击 Finish。这包括复制 solution2 中的现有指令。
3. 确保 C 源代码在信息窗格中可见。
4. 在 **Directive** 选项卡中，选择 **d_o**，右键点击并选择 **Insert Directive**，打开 Directive Editor 对话框。
 - a. 在指令编辑器对话框中激活顶部的指令下拉菜单并选择 **ARRAY_PARTITION**。
 - b. 激活类型下拉，将数组划分为块。设置类型为 **block**。
 - c. 在 Vivado HLS 指令编辑器 对话框中，将 **factor(可选)** 设置为 **4**，分成 4 份。
 - d. 使用 Vivado HLS 指令编辑器(如图 4-21 所示)，单击 OK。

图 4-21: 用于划分数组 d_o 的指令编辑器

现在，将输入数组划分为两个块(而不是四个)。

5. 在 Directive 选项卡中，选择 d_i 并重复前面的步骤，但是这次将端口分区为 **2**。选项卡显示了现在应用于设计的指令(图 4-22)。

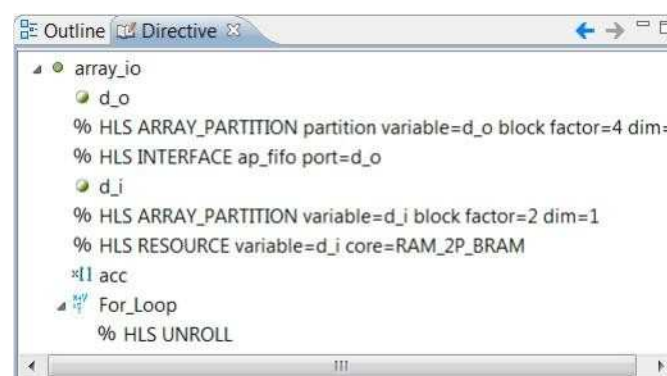


图 4-22: 实验 2 解决方案 3 的概要

6. 从工具栏运行 C 综合法来综合设计。当报告在信息窗格中打开时，界面摘要如图 4-23 所示。

- 该设计具有标准的时钟、复位和块级 I/O 端口。
- 数组参数 d_o 被实现为四个独立的 FIFO 接口。
- 参数 d_i 被实现为两个独立的 RAM 接口，每个 RAM 接口使用一个双端口接口。(如果您看到四个独立的 RAM 接口，请确认分区因子 factor=2 而不是 4)。

Interface

Summary

RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	array_io	return value
ap_rst	in	1	ap_ctrl_hs	array_io	return value
ap_start	in	1	ap_ctrl_hs	array_io	return value
ap_done	out	1	ap_ctrl_hs	array_io	return value
ap_idle	out	1	ap_ctrl_hs	array_io	return value
ap_ready	out	1	ap_ctrl_hs	array_io	return value
d_o_0_din	out	16	ap_fifo	d_o_0	pointer
d_o_0_full_n	in	1	ap_fifo	d_o_0	pointer
d_o_0_write	out	1	ap_fifo	d_o_0	pointer
d_o_1_din	out	16	ap_fifo	d_o_1	pointer
d_o_1_full_n	in	1	ap_fifo	d_o_1	pointer
d_o_1_write	out	1	ap_fifo	d_o_1	pointer
d_o_2_din	out	16	ap_fifo	d_o_2	pointer
d_o_2_full_n	in	1	ap_fifo	d_o_2	pointer
d_o_2_write	out	1	ap_fifo	d_o_2	pointer
d_o_3_din	out	16	ap_fifo	d_o_3	pointer
d_o_3_full_n	in	1	ap_fifo	d_o_3	pointer
d_o_3_write	out	1	ap_fifo	d_o_3	pointer
d_i_0_address0	out	4	ap_memory	d_i_0	array
d_i_0_ce0	out	1	ap_memory	d_i_0	array
d_i_0_q0	in	16	ap_memory	d_i_0	array
d_i_0_address1	out	4	ap_memory	d_i_0	array
d_i_0_ce1	out	1	ap_memory	d_i_0	array
d_i_0_q1	in	16	ap_memory	d_i_0	array
d_i_1_address0	out	4	ap_memory	d_i_1	array
d_i_1_ce0	out	1	ap_memory	d_i_1	array
d_i_1_q0	in	16	ap_memory	d_i_1	array
d_i_1_address1	out	4	ap_memory	d_i_1	array
d_i_1_ce1	out	1	ap_memory	d_i_1	array
d_i_1_q1	in	16	ap_memory	d_i_1	array

4 FIFO

双端口

2

双端口

图 4-23: 分区接口的接口报告

如果将输入端口 d_i 划分为四个，则每个端口仅需要一个单端口 RAM 接口。因为输出端口一次只能输出四个值，所以一次读取八个输入不会有任何好处。

本教程的最后一步是对数组进行完全分区。

步骤 5: 完全分区数组接口

这一步向您展示了如何将数组接口划分为各个端口。

1. 从工具栏中选择 **New Solution** 并创建一个新解决方案。
2. 单击 **Finish** 并接受默认设置。这包括复制 solution3 的现有指令。
3. 确保 C 源代码在信息窗格中可见。
4. 在 Directive 选项卡中，选择现有 d_o 的 **partition Directive**，如图 4-24 所示。
5. 右键单击并选择 **Modify** 指令。

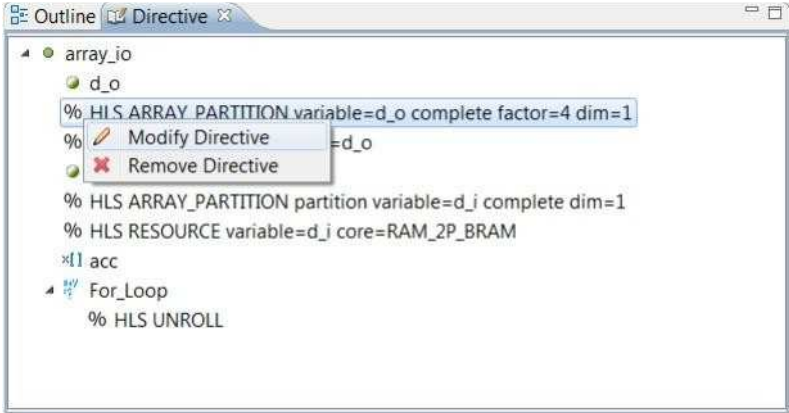


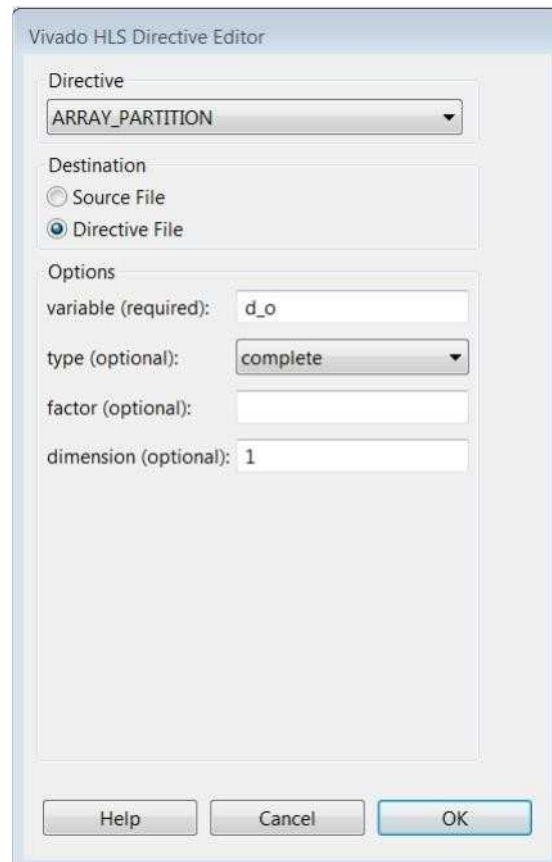
图 4-24: 修改 d_o 指令

6. 在 Vivado HLS 指令编辑器对话框:

在 Vivado HLS 指令编辑器对话框中, 删除值 4。由于该数组将被完全划分到寄存器中, 因此分区因子不再相关。(如果您将它放在那里, 它将被忽略)。

b. 激活类型(可选)下拉菜单, 修改分区类型 *Complete*。

使用如图 4-25 所示的指令编辑器, 点击 OK。

图 4-25: 用于划分数组 d_o 的指令编辑器

7. 在 Directive 选项卡 d_i 中, 选择并重复前面的步骤来完全划分数组 d_i 。

8. 在 Directive 选项卡中 d_i , 用鼠标右键点击选择 *RESOURCE* Directive, 然后选择 *Remove Directive*。因为如果数组被分割成单独的元素, 则不能将其分配给块 RAM。 d_o 同理。

选项卡显示了现在应用于设计的指令(图 4-26)。

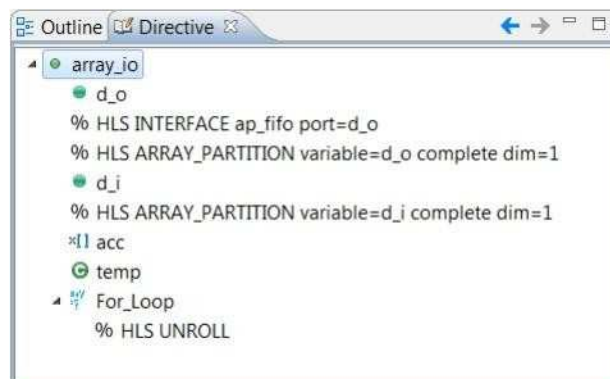


图 4-26: 实验 2 解决方案的概要

9. 从工具栏 *Run C Synthesis* 来综合设计。

10. 当报告在信息窗格中打开时, 请查看接口摘要。请注意以下几点:

- 该设计具有标准的时钟、复位和块级 I/O 端口。
- 数组参数 d_o 被实现为 32 个独立的 FIFO 接口。
- 参数 d_i 被实现为 32 个独立的标量端口。因为输入标量的默认接口不在 I/O 协议中, 所以它们有 I/O 协议 *ap_none*。

虽然本教程只关注 I/O 接口，但是在这一点上，值得研究所有四种解决方案的性能差异。

- 11.从工具栏或项目菜单中选择 *Compare Reports*，以打开解决方案的比较。
- 12.在“解决方案选择”对话框中，将四个解决方案中的每一个都添加到“选择解决方案”窗格中(图 4-27)。
- 13.单击 OK。

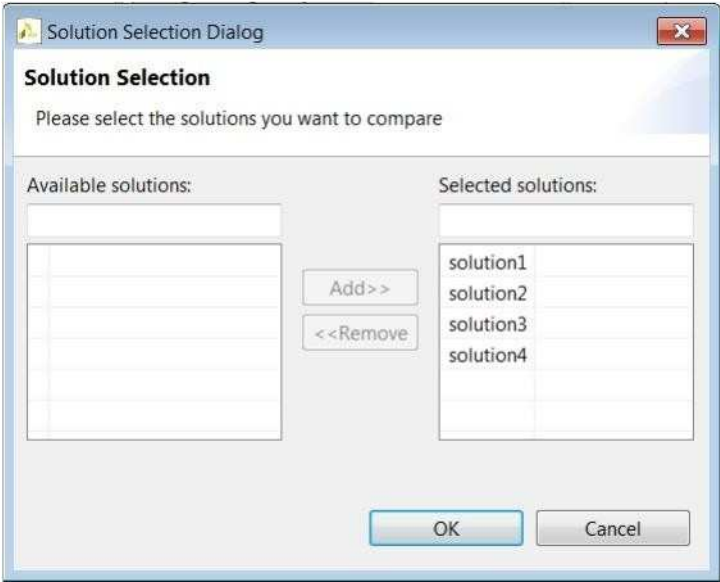


图4-27: 比较实验室 3 的所有解决方案

当打开解决方案比较报告(图 4-28)时，它显示 solution4(为每个数组元素使用唯一的端口) 比以前的解决方案快得多。内部逻辑可以在需要的时候访问数据。(没有端口访问造成的性能瓶颈)

Performance Estimates

Timing (ns)

Clock		solution1	solution2	solution3	solution4
ap_clk	Target	4.00	4.00	4.00	4.00
	Estimated	2.602	3.150	3.363	3.363

Latency (clock cycles)

		solution1	solution2	solution3	solution4
Latency	min	65	33	10	1
	max	65	33	10	1
Interval	min	65	33	10	1
	max	65	33	10	1

图 4-28: 所有实验室 3 解决方案的性能比较

向下滚动比较报告(图 4-29)，注意，具有更多 I/O 端口的解决方案(解决方案 2、3 和 4)允许更多的并行处理，但也会使用更多的资源。

Utilization Estimates					
		solution1	solution2	solution3	solution4
BRAM_18K		0	0	0	0
DSP48E		0	0	0	0
FF		88	1202	731	642
LUT		137	2125	2083	1945
URAM		0	0	0	0

图 4-29: 所有实验室 3 解决方案的资源比较

在下一个练习中，您将使用端口和资源之间的最佳平衡实现相同的设计。除了这个更优化的实现之外，下一个练习将展示如何将 AXI4 接口添加到设计中。

14.退出 Vivado HLS GUI并返回到命令提示符。

实验 4: 实现 AXI4 接口

介绍

这个练习解释了如何为 I/O 端口指定 AXI4 总线接口。除了添加 AXI4 接口之外，本练习还展示了如何通过同时使用接口和逻辑指令来创建最佳设计。

步骤 1: 创建并打开项目

- 1.从先前实验中使用的 Vivado HLS 命令提示窗口，切换到 *lab4* 目录。
- 2.通过键入 *vivado_hls -f run_hls.tcl* 创建一个新的 Vivado HLS 项目。
- 3 输入 *vivado_hls -p axi_interfaces_prj*，打开 VivadoHLS GUI 项目。
- 4.打开源代码，如图 4-30 所示。

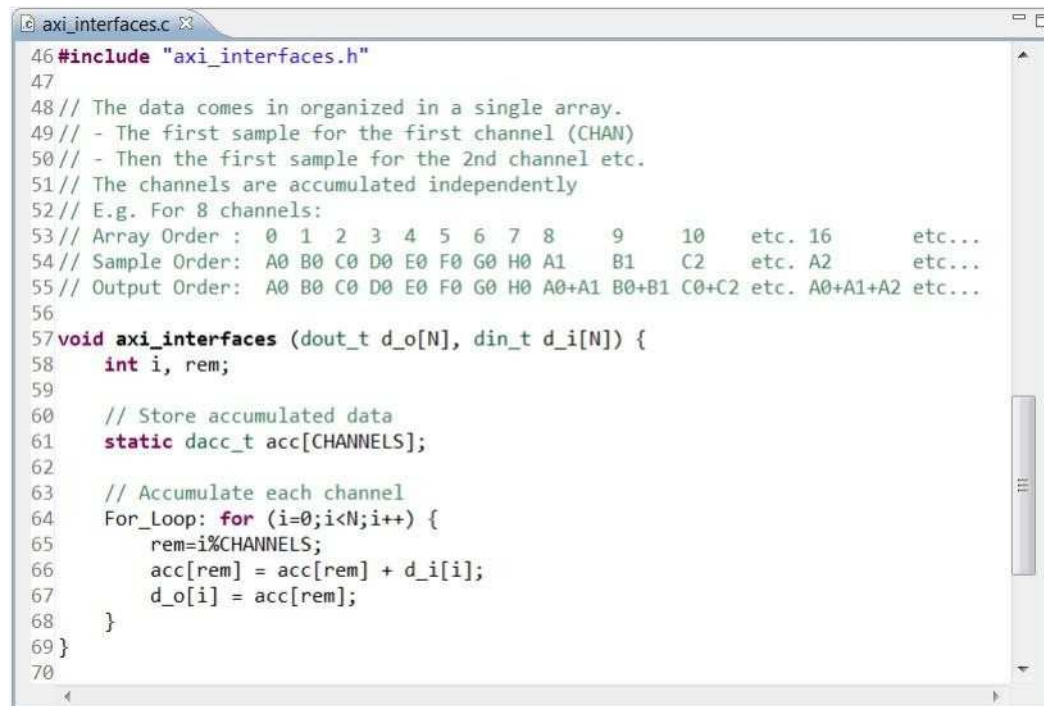


图 4-30: 实验 4 的源代码

此设计使用与 *lab3* 类似的源代码 C：将设计重命名为 *axi_interfaces*。

步骤 2: 创建带有 AXI4-Stream 接口的优化设计

为了达到本设计的最佳性能实现，每个通道的数据都是并行处理的，每个通道都有专用的硬件。

理解如何最好地执行这种优化的关键是认识到输入和输出数组中的通道可以进行循环分区。

《Vivado Design Suite User Guide: High-Level Synthesis (UG902)》中对 *循环分区(cyclic partitioning)* 进行了详细的解释，但这基本上意味着每个数组元素依次被划分到不同的分区中。

在本练习中，您将指定数组参数要作为 AXI4-Stream 接口实现。如果将数组划分为通道，则可以并行地设计将每个通道的样本流式传输。

最后，如果将 I/O 端口配置为提供和使用通道数据的各个流，则 *for-loop* 的部分展开可以确保每个通道都有专用的硬件处理。

首先，对数组进行分区 (*partition*)：

1. 确保 C 源代码在信息窗格中可见。
 2. 在 Directive 选项卡中, 选择 **d_o**, 右键单击并选择 **Insert Directive** 来打开 type 对话框。
 - a. 选择顶部的 **Directive** 下拉菜单, 选择 **ARRAY_PARTITION**。
 - b. 点击 **type (optional)** 下拉菜单, 指定循环分区。在 **factor(可选)** 框中, 输入值 **8**, 创建 **8** 个单独的分区。(结果是 **8** 个端口)
- 在如图 4-31 所示的 directive Editor 对话框中, 点击 OK。

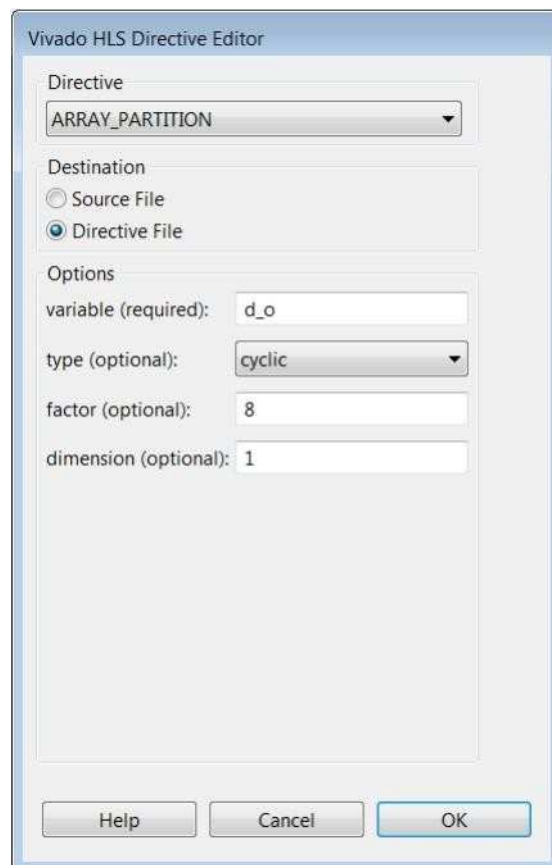


图 4-31: 循环分区的指令编辑器

3. 在 Directive 选项卡中, 再次选择 **d_o**, 右键单击并选择 **Insert Directive**, 打开 **Insert Directive Editor** 对话框。
 - a. 激活顶部的指令下拉菜单, 选择 **INTERFACE**。
 - b. 点击模式下拉菜单, 指定轴 (**axis**) **interface**。
 - c. 单击 **OK**。
4. 在 Directive 选项卡中, 选择 **d_i** 并重复上面的步骤 2 和 3。
 - a. 应用 **ARRAY_PARTITION**。
 - b. 循环系数为 **8**。
 - c. 应用 **Interface**。
 - d. 应用 **axis** 接口。
5. 接下来, 部分展开 (**partially unroll**) 并流水化 (**pipeline**) for 循环:
 - a. 在 Directive 选项卡中, 选择 **For_Loop**, 右键单击, 选择 **Insert Directive**, 打开 **Vivado HLS Directive Editor** 对话框。
 - b. 选择顶部的 Directive 下拉菜单, 选择 **UNROLL**。

选择一个因子 8 来部分地展开 for 循环。这相当于重写 C 代码, 在循环的每次迭代中执行 8 个环体副本(其中新循环总共只执行 4 次迭代, 而不是 32 次)。

单击 **OK**。

- c. 在 Directive 选项卡中，再次选择 **For_Loop**，右键选择 Insert Directive，打开 Vivado HLS Directive Editor 对话框。

激活顶部的指令下拉菜单并选择 **PIPELINE**。将 **interval (II)** 保留为空，并让它默认为 **1**。

- d. 选择 **enable loop rewinding**。
e. 单击 **OK**。

当设计的顶层是循环时，可以使用**管道倒带(loop rewinding)**选项。这通知 Vivado HLS，在 RTL 中实现时，该循环连续运行（没有功能终止和功能重启周期）。执行上述步骤后，选项卡应该如图 4-32 所示。请务必检查所有选项是否正确应用。如果没有，双击指令重新打开指令编辑器。

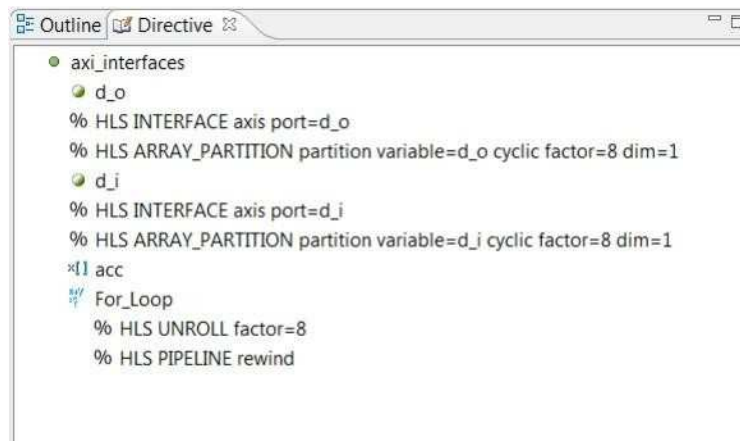


图 4-32: 实验室 4 解决方案 1 的选项卡

6. 从工具栏 **Run C Synthesis**。

当报告在信息窗格中打开时，确认 **d_i** 和 **d_o** 都被实现为八个单独的 **axi4** 流端口。

7. 在报告的性能部分，确认 for 循环每一个时钟周期处理一个样本(启动间隔 1)，延迟为 4，并且该设计的面积小于 **lab3** 中的解决方案 2、3 或 4(图 4-29)。

数组接口的循环分区和 for 循环的部分展开允许将此 C 代码实现为硬件中的八个**独立通道**。流水线 for 循环允许每个通道中的逻辑为每个时钟处理一个样本。改变分区和循环展开允许您创建一个最佳的面积和性能平衡的设计，以满足您的特定需求。

步骤 3：实现 AXI4-Lite 接口

在本练习中，将块级 I/O 协议端口分组到单个 AXI4-Lite 接口中，该接口允许从 CPU 控制和访问这些块级控制信号。

1. 从工具栏或项目菜单中选择 “**New Solution**” 以创建新解决方案。
2. 接受默认设置并单击 **Finish**。这包括复制 **solution1** 中的现有指令。
3. 确保 C 源代码在信息窗格中可见。
4. 在 Directive 选项卡中，选择**顶层函数** **axi_interfaces**，右键单击并选择 Insert Directive，以打开 Insert Directive Editor 对话框。
 - a. 选择顶部的 Directive 下拉菜单，选择 **INTERFACE**。
 - b. 选择 **mode** 下拉菜单，选择 **s_axilite**。这指定与函数返回(块级 I/O 端口)关联的端口被实现为一个 **AXI4-Lite** 接口。因为函数返回的默认模式是 **ap_ctrl_hs**，所以不需要指定这个 I/O 协议。
 - c. 单击 OK。

将出现 instructions 选项卡，如图 4-33 所示。

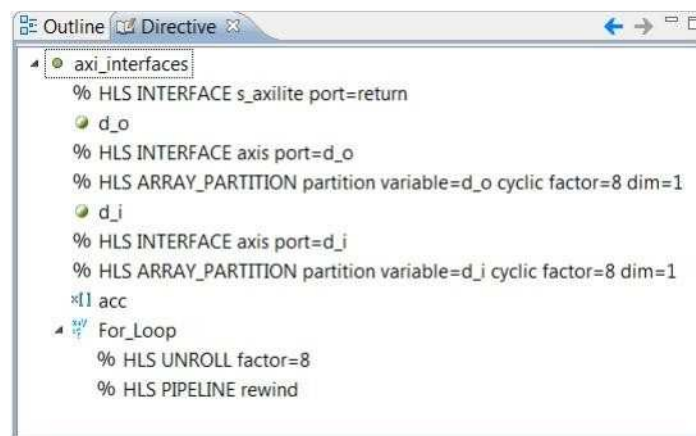


图 4-33: 指定 AXI4-Lite 接口的指令

5. 从工具栏 **Run C Synthesis**。

当报告打开时，检查接口摘要以确认块级 I/O 协议端口(**ap_start**, **ap_done** 等)已被 AXI4Lite 接口替换，并且**输出中断信号**已添加到设计中，可以通过 AXI-Lite 接口选择中断源。

6. 从工具栏或解决方案菜单中选择 **Export RTL** 来创建一个 IP 包。

7. 将格式选择保留为 **IP Catalog**，然后单击 OK。

您可以在 **solution2/impl** 文件夹中看到 IP 包。因为使用了 Vivado IP Catalog 格式，所以包在 IP 文件夹中。

该 **IP** 文件夹包括子 **Drivers** 文件夹，如图 4-34 所示。

在设计中添加 AXI4-Lite 接口时，IP 打包过程还创建软件驱动程序文件，以启用外部块(通常是 CPU)来控制这个块(启动、停止、设置端口值、查看中断状态)。

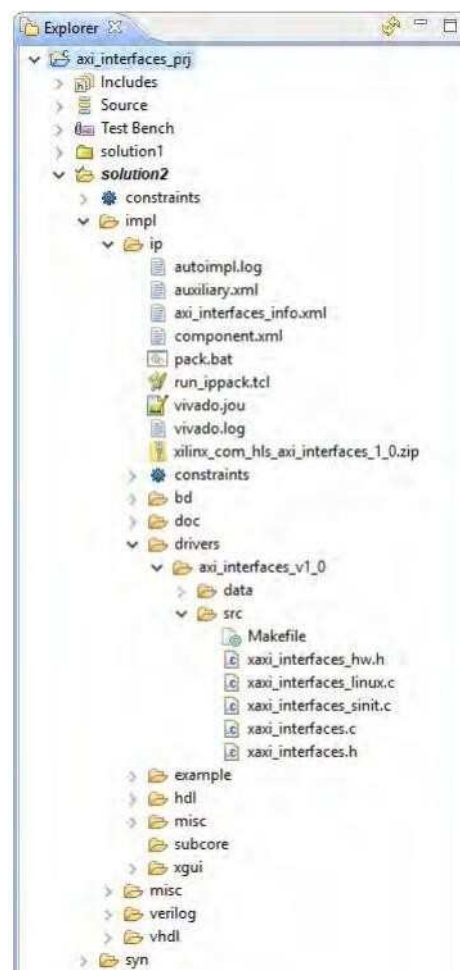


图 4-34: 带有 AXI4 接口的 IP 包

8. 双击文件 `xaxi_interfaces_hw.h` 以在信息窗格中打开它。

该图显示了访问和控制块级接口信号的地址。例如，将控制寄存器 `0x0` 的 `0` 位设置为值 `1` 将启用 `ap_start` 端口，或者设置第 `7` 位将启用自动重启，设计将在每个事务结束时自动重新启动。

其余的 C 驱动程序文件用于将 AXI4 从属 Lite 接口的控制集成到运行在 CPU 或微控制器上的代码中，并包含在打包的 IP 中。

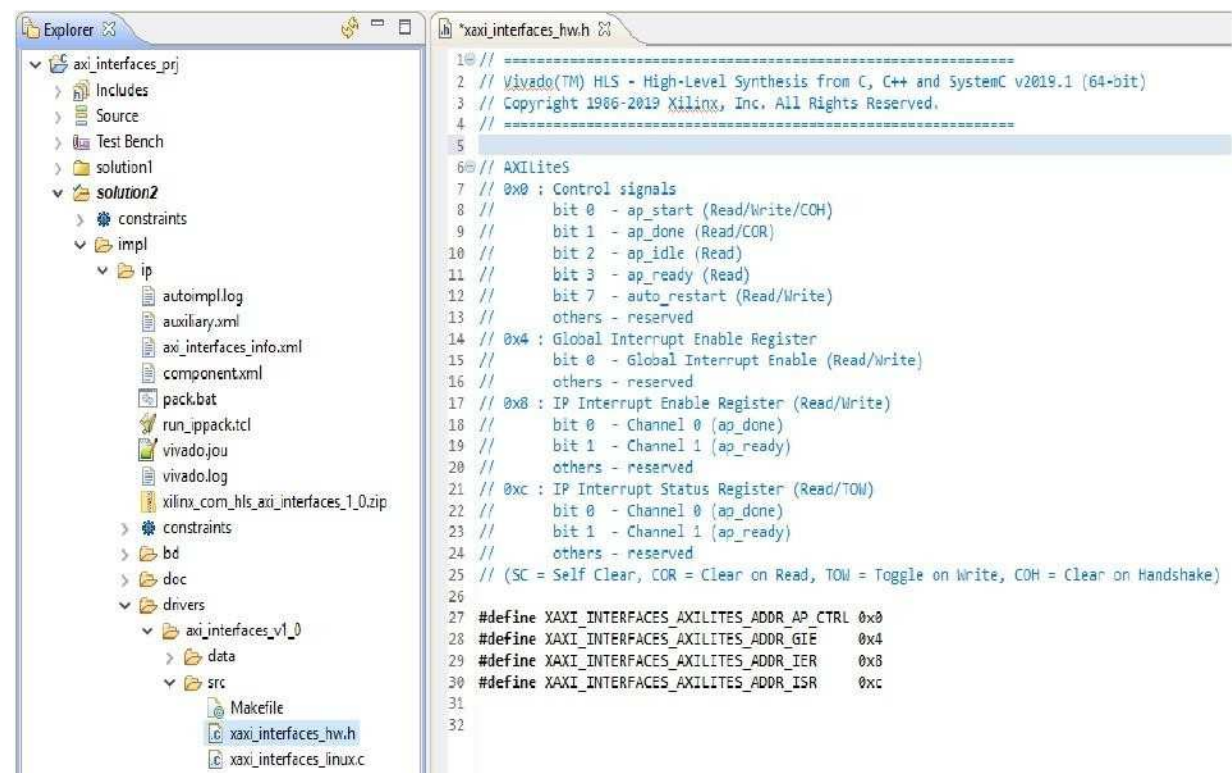


图 4-35: IP 软件驱动文件

结论

在本教程中，您学习了：

- 什么是块级 I/O 协议，以及如何控制它们。
- 如何指定和应用端口级 I/O 协议。
- 如何指定数组端口作为 RAM 和 FIFO 接口。
- 如何将 RAM 和 FIFO 接口划分为子端口。
- 如何使用 I/O 指令和优化指令创建带有 AXI4 接口的优化设计。

任意精度类型

概述

C/C++提供的数据类型固定为 8 位边界:

- char(8 位)
- 短(16 位)
- int(32 位)
- long long(64 位)
- 浮动(32 位)
- 双(64 位)
- 精确宽度整数类型, 如 *int16_t*(16 位)和 *int32_t*(32 位)

在创建硬件时, 通常需要更精确的位宽。例如, 考虑这样一种情况: 过滤器的输入是 12 位的, 而结果的累积只需要 27 位的最大范围。在硬件设计中使用标准 C 数据类型会导致不必要的硬件成本。操作可以使用比所需精度更多的 LUTs 和寄存器, 延迟甚至可能超过时钟周期, 需要更多的周期来计算结果。

Vivado 高级综合(HLS)提供了许多位精度或任意精度的数据类型, 允许您使用任意宽度对变量进行建模。
本教程包括两个实验练习:

实验室 1 描述

使用浮点类型综合设计并检查结果。该设计使用标准的 c++浮点类型。

实验室 2 描述

使用任意精度定点类型综合 *lab1* 中使用的相同函数, 强调了准确性和结果方面的好处。这个练习展示了相同的设计可以转换为 *Vivado HLS ap_fixed* 类型, 保留所需的精度, 但创建更优化的硬件实现。

教程设计描述

从 Xilinx 网站下载教程设计文件。请参阅定位教程设计文件中的信息。本教程使用教程目录中的设计文件

Vivado_HLS_Tutorial \ Arbitrary_Precision

实验一: 任意精度

任意精度实验 1: 使用标准的 C/ c++ 类型检查设计。

在这个实验室中, 您使用标准的 C 类型来合成一个设计。您可以使用此设计作为使用任意精度类型的设计的参考, 这是 *Lab2* 的基础。重要提示: 本教程中的图和命令假设教程数据目录

Vivado_HLS_Tutorial 被解压缩并放置在位置 C:\Vivado_HLS_Tutorial 中。如果将教程数据目录解压缩到不同的位置, 或者在 Linux 系统上, 请将引用的少数路径名调整到您选择的位置, 以放置



步骤 1:创建并打开项目

1.打开 Vivado HLS 命令提示符。

- 在 Windows 上使用 **Start > All Programs > Xilinx Design Tools > Vivado 2018.3 > Vivado HLS > Vivado HLS 2018.3 Command Prompt**
- 在 Linux 上, 打开一个新的 shell。

在命令提示符窗口中(图 5-1), 将目录更改为任意精度教程 **lab1**。

2.执行 Tcl 脚本来设置 Vivado HLS 项目, 使用如图 5-1 所示的命令:

vivado_hls -f run_hls.tcl

```
C:\Uivado_HLS_Tutorial>cd Arbitrary_Precision
C:\Uivado_HLS_Tutorial\Arbitrary_Precision>cd lab1
C:\Uivado_HLS_Tutorial\Arbitrary_Precision\lab1>vivado_hls -f run_hls.tcl
```

图 5-1: 设置教程项目

3.Vivado HLS 完成后, 使用图 5-2 所示的命令在 Vivado HLS GUI 中打开项目。

vivado_hls -p window_fn_prj

```
i = 23 hw_result = 38.24289 sw_result = 38.24289
i = 24 hw_result = 32.00000 sw_result = 32.00000
i = 25 hw_result = 25.75711 sw_result = 25.75711
i = 26 hw_result = 19.75413 sw_result = 19.75413
i = 27 hw_result = 14.22175 sw_result = 14.22175
i = 28 hw_result = 9.37258 sw_result = 9.37258
i = 29 hw_result = 5.39297 sw_result = 5.39297
i = 30 hw_result = 2.43585 sw_result = 2.43585
i = 31 hw_result = 0.61487 sw_result = 0.61487

Test Passed
@I [SIM-1] CSim done with 0 errors.
@I [LIC-101] Checked in feature [HLS]
C:\Uivado_HLS_Tutorial\Arbitrary_Precision\lab1>vivado_hls -p window_fn_prj
```

图 5-2: 任意精度 Lab1 的初始项目

步骤 2:检查测试台并运行 C 仿真

1.在资源管理器窗格中打开 **Source** 文件夹, 双击打开代码 **window_fn_top.cpp**, 如图 5-3 所示。



图5-3:Lab 1 中任意精度类型的C代码

2. 按住 **Ctrl** 键并单击第 45 行打开这个 **window_fn_top.h** 头文件。
3. 向下滚动查看类型定义(图 5-4)。

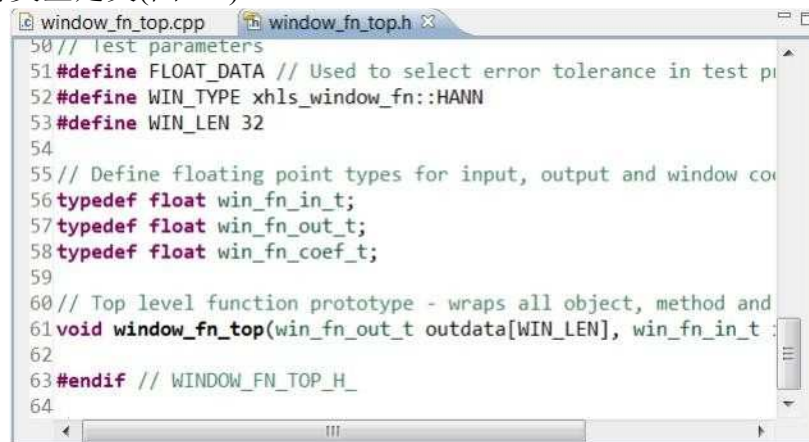


图5-4:实验室1 中任意精度类型的类型定义

本设计对所有数据操作使用标准的 C/ c++浮点类型。Vivado 如果操作是标准的算术操作(+、-、*、%)，那么高级综合可以将浮点类型直接合成到硬件中。

当使用数学函数时 **math.h** 或 **cmath.h**。参见 Vivado Design Suite User Guide: High-LevelSynthesis (UG902) [Ref 2]，了解哪些数学函数支持进行综合。

4. 单击 “**Run C Simulation**” 工具栏按钮，打开 “C 仿真” 对话框。
5. 接受默认设置(没有 选择任何选项)并单击 OK。

控制台窗格显示设计与预期结果进行了模拟。

第三步:综合设计和评审结果

1. 单击 **Run C Synthesis** 工具栏按钮，将设计合成为 RTL。当合成完成时，合成报告将自动打开。图 5-5 显示了综合报告。

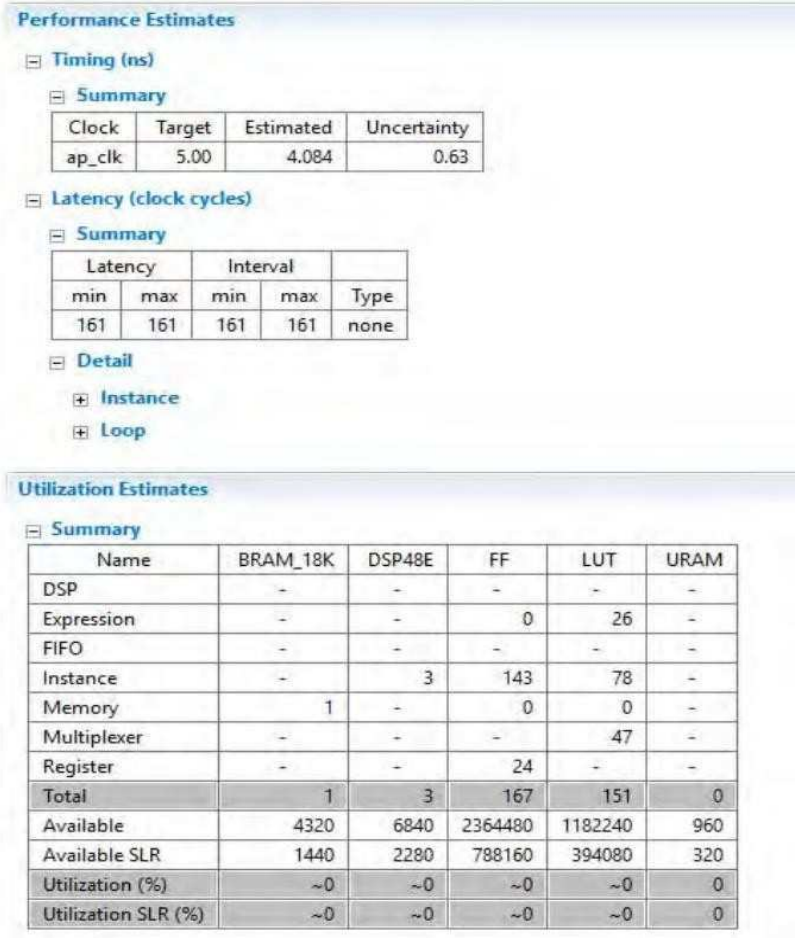


图 5-5: 浮点设计的综合报告

顶层设计中的实例占所使用的大部分区域。

2. 向下滚动报告并在**利用率估计**的详细部分展开实例(图 5-6)。

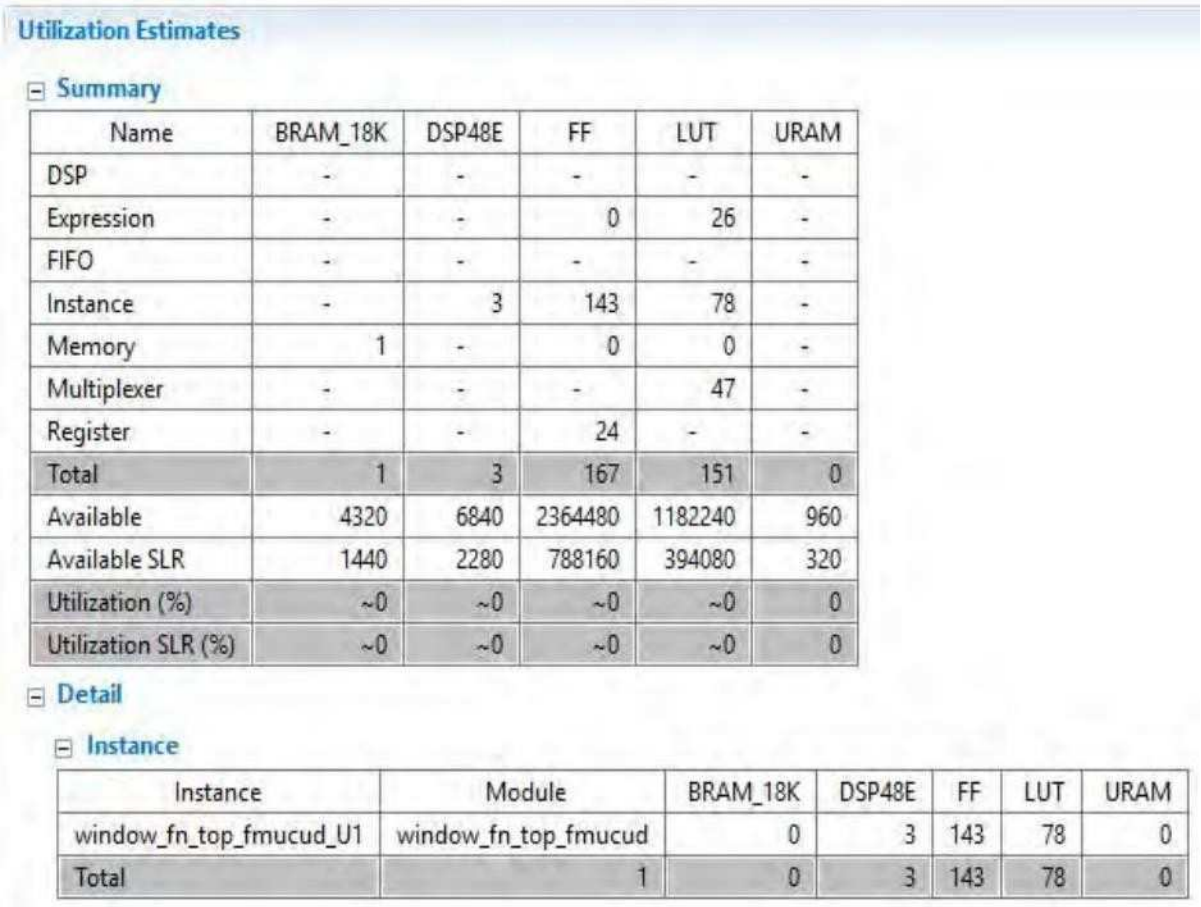


图 5-6:浮点设计的区域细节

详细说明这是一个**浮点乘法器(fmul)**。浮点运算在面积和时钟周期方面代价高昂。分析透视图(图 5-7)显示这个操作符也占用大部分时钟周期(它需要五种状态中的三种来执行循环 *winfn_loop* 创建的逻辑)。

有关使用分析透视图的更多细节，请参阅第 6 章“设计” [分析教程](#)。为了理解这种设计，第一个状态中的两个操作是单周期从内存中读取操作，而最后一个状态中的操作是写内存操作。

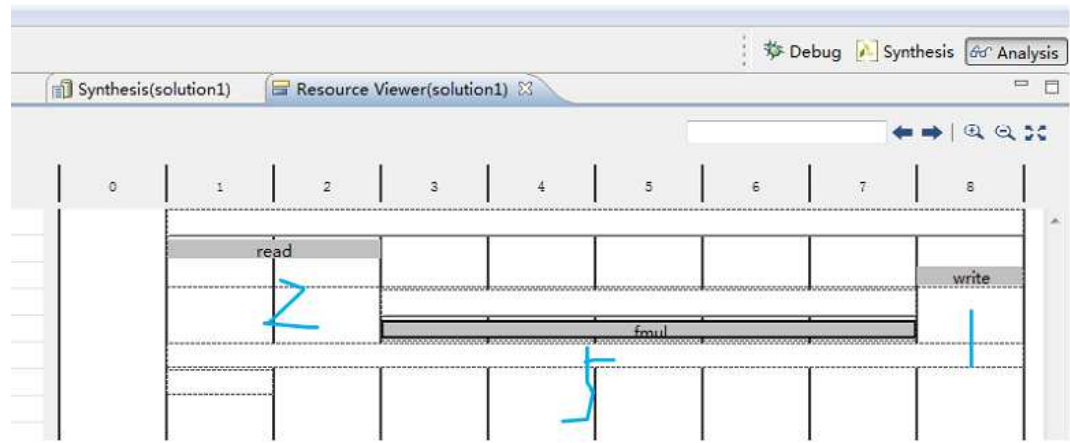


图 5-7:浮点设计的性能细节退出

Vivado HLS GUI 并返回到命令提示符

实验二:任意精度

使用任意精度类型检查设计。

介绍

这个实验练习使用与 *lab1* 相同的设计，但是数据类型现在是任意精度类型。您首先检查设计，然后检查合成结果。

步骤 1:创建并模拟项目

1.从 *lab1* 中使用的 Vivado HLS 命令提示符，切换到 *lab2* 目录，如图 5-8 所示。

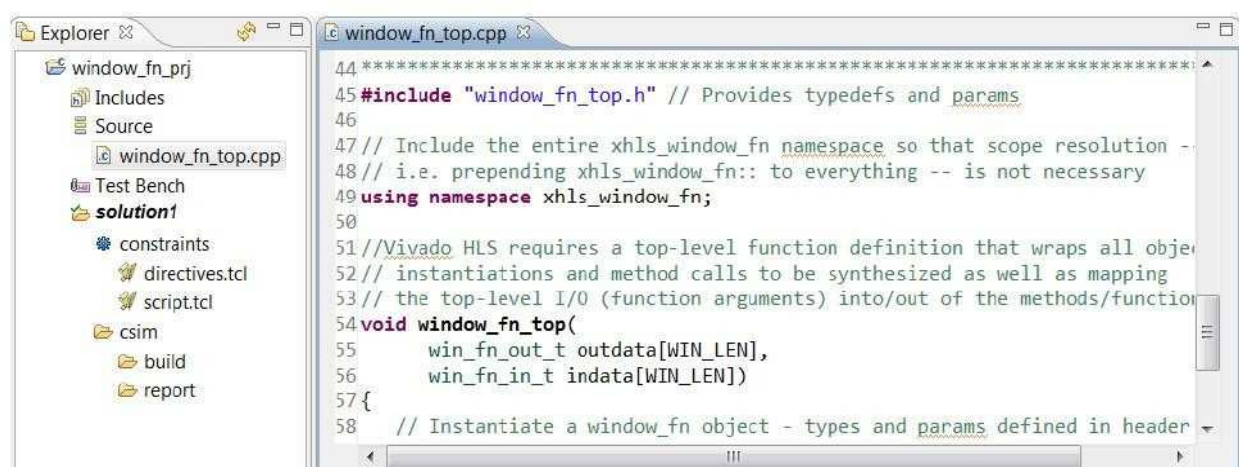
2.通过键入 ***vivado_hls -f run_hls.tcl*** 创建一个新的 Vivado HLS 项目

```
C:\Uivado_HLS_Tutorial>cd Arbitrary_Precision\lab1
C:\Uivado_HLS_Tutorial\Arbitrary_Precision\lab1>cd ..
C:\Uivado_HLS_Tutorial\Arbitrary_Precision>cd lab2
C:\Uivado_HLS_Tutorial\Arbitrary_Precision\lab2>vivado_hls -f run_hls.tcl
```

图5-8:任意精度实验室2的设置

3.通过键入 ***vivado_hls -p window_fn_prj*** 打开 Vivado HLS GUI 项目

4.在资源管理器窗格中打开源文件夹并双击 ***window_fn_top.cpp*** 打开如图 5-9 所示的代码。



```
44 *****
45 #include "window_fn_top.h" // Provides typedefs and params
46
47 // Include the entire xhls_window_fn namespace so that scope resolution -
48 // i.e. prepending xhls_window_fn:: to everything -- is not necessary
49 using namespace xhls_window_fn;
50
51 //Vivado HLS requires a top-level function definition that wraps all object
52 // instantiations and method calls to be synthesized as well as mapping
53 // the top-level I/O (function arguments) into/out of the methods/function
54 void window_fn_top(
55     win_fn_out_t outdata[WIN_LEN],
56     win_fn_in_t indata[WIN_LEN])
57 {
58     // Instantiate a window_fn object - types and params defined in header
```

图5-9:任意精度实验室2的C代码

5.按住 Control 键并单击打开此头文件 ***window_fn_top.h***

6.向下滚动查看类型定义(图 5-10)。

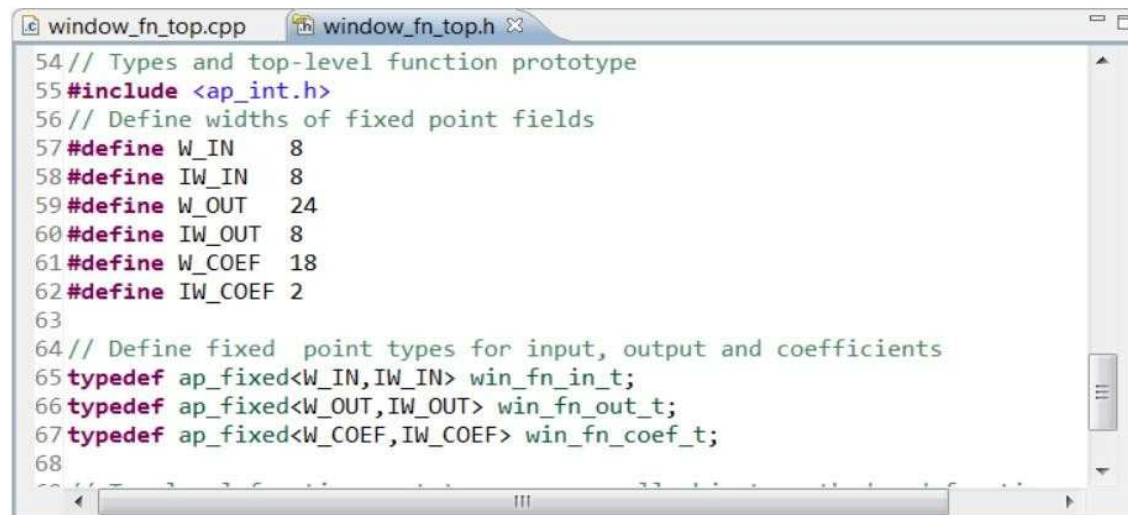


图5-10:任意精度实验室2的类型定义

这个头文件是唯一不同于 *lab1* 的文件 *window_fn_top.h*。数据类型已更改为 *ap_fixed point* 类型，这与 *float* 和 *double* 类型类似，因为它们支持整数和小数位表示。这些数据类型定义在头文件 *ap_fixed.h*。定义在头文件中定义数据大小的类型：

- *W_IN* 定义了总字长度。
- *IW_IN* 定义整数位的数量。因此，小数位数是第一项减去第二项。

当您修改 C 代码以使用任意精度类型而不是标准 C 类型时，您必须做的最常见的更改之一是**减少数据类型的大小**。在本例中，将设计改为使用 8 位、24 位和 18 位的字长，而不

是 32 位的浮点类型。这将导致更小的操作符、更小的区域和更少的时钟周期。当您更改更常见的 C 类型(如 *int*、*short* 和 *char*)时，类似的优化会有所帮助。例如，将只需要 18 位的数据类型从 *int*(32 位)更改为确保只需要单个 DSP48 就可以执行任何乘法。

在这两种情况下，您必须确认设计仍然执行正确的操作，并且具有所需的准确性。Vivado 高级综合提供的任意精度类型的好处是，您可以模拟更新的 C 代码来确认它的功能和准确性。

7.在资源管理器窗格中打开测试工作台文件夹并双击 *window_fn_test.cp*,打开代码。

8.向下滚动查看图 5-11 所示的视图。

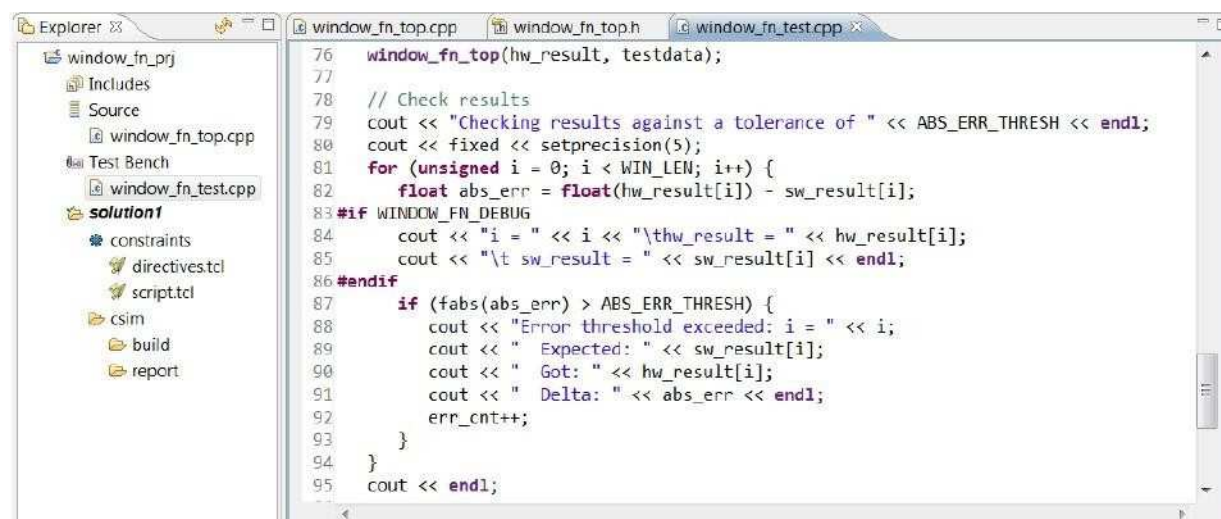


图5-11:任意精度实验室2的测试台

此设计的测试平台包含用于检查结果准确性的代码。预期的结果仍然使用浮点类型生成。由于定点操作和浮点操作之间的精度差异，结果检查将验证结果是否在指定的精度范围内(在本例中，在预期结果的 0.001 范围内)。

这使得更新后的设计可以在 C 语言中快速有效地进行验证，并具有快速的编译和运行时间。

- 9.单击“*Run C Simulation*”工具栏按钮，打开“C 仿真”对话框。
- 10.接受默认设置(没有选择任何选项)并单击 OK。

控制台窗格显示了 C 模拟的结果。使用更新的数据类型，结果不再与预期的结果相同。然而，他们是在容忍范围内的。

发送反馈

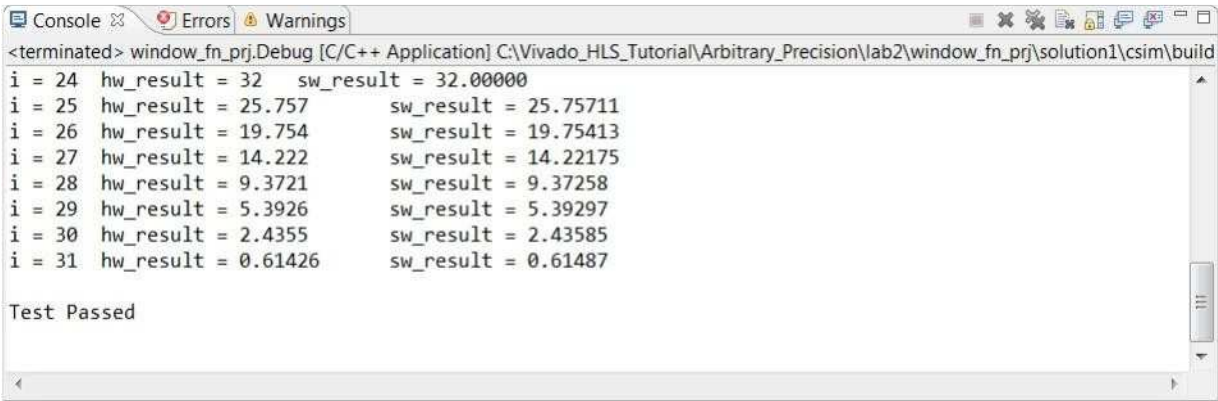


图5-12:固定点类型的C 仿真结果

第二步:综合设计和评审结果

- 1.单击 *Run C Synthesis* 工具栏按钮，将设计合成为 RTL。当合成完成时，合成报告将自动打开。图 5-13 显示了综合报告。

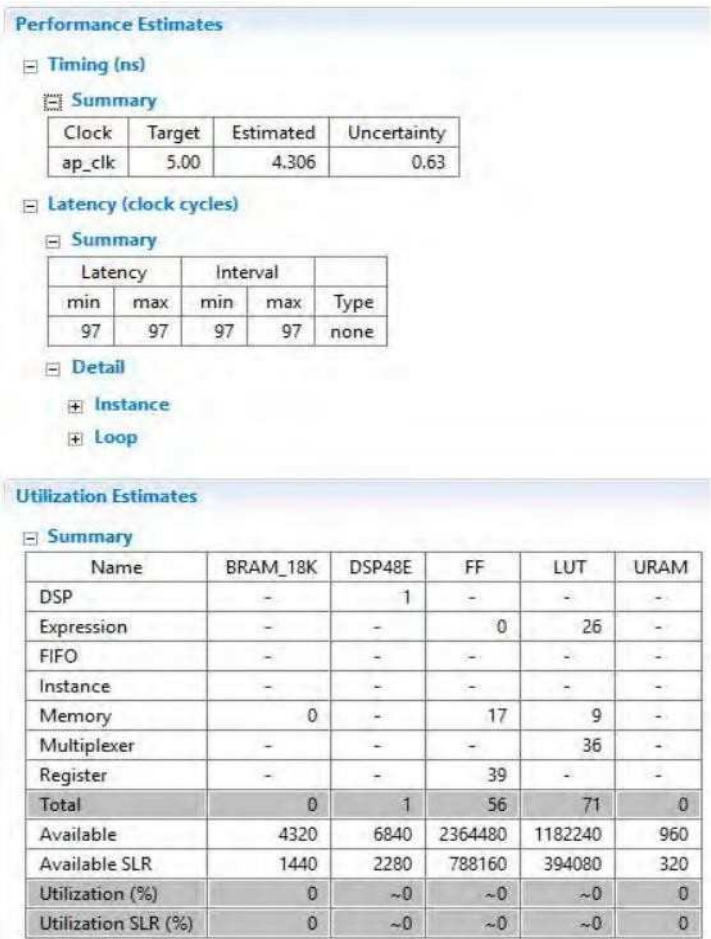


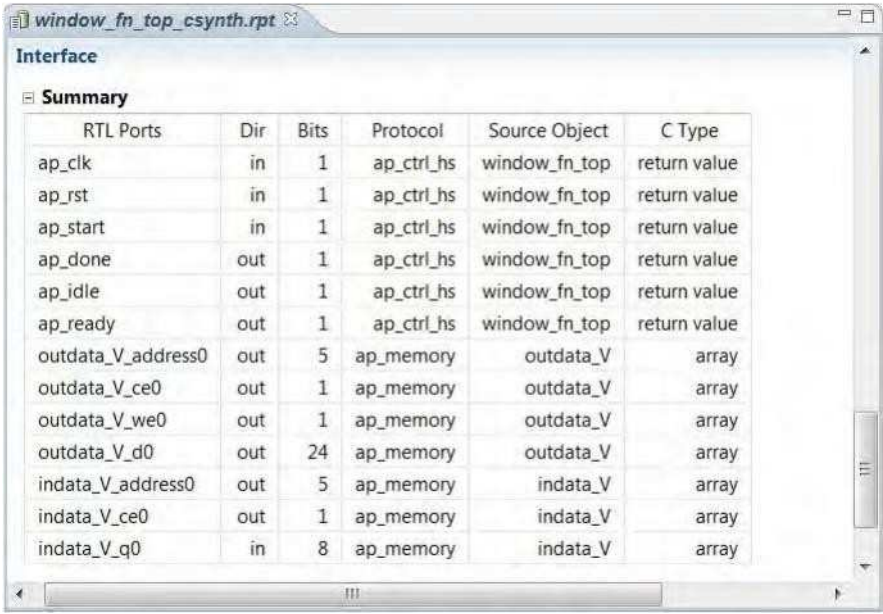
图5-13 固定点设计综合报告

注意，通过使用任意精度类型，可以减少延迟和面积(分别减少 40%和 50%)，并且 RTL 硬

件中的操作不会超出必要的范围。由于内存中位的总数现在小于 **1024** 位，所以它现在是用 LUTs 和 FFs 自动实现的，而不是用块 RAM。

2.向下滚动报告到接口摘要(图 5-14)。

图 5-14 显示数据端口现在是 8 位和 24 位。



The screenshot shows the 'Interface' window in Vivado HLS, displaying a summary table of RTL ports. The table lists various ports, their directions, widths in bits, protocols, source objects, and C types.

RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	window_fn_top	return value
ap_rst	in	1	ap_ctrl_hs	window_fn_top	return value
ap_start	in	1	ap_ctrl_hs	window_fn_top	return value
ap_done	out	1	ap_ctrl_hs	window_fn_top	return value
ap_idle	out	1	ap_ctrl_hs	window_fn_top	return value
ap_ready	out	1	ap_ctrl_hs	window_fn_top	return value
outdata_V_address0	out	5	ap_memory	outdata_V	array
outdata_V_ce0	out	1	ap_memory	outdata_V	array
outdata_V_we0	out	1	ap_memory	outdata_V	array
outdata_V_d0	out	24	ap_memory	outdata_V	array
indata_V_address0	out	5	ap_memory	indata_V	array
indata_V_ce0	out	1	ap_memory	indata_V	array
indata_V_q0	in	8	ap_memory	indata_V	array

图5-14: 定点接口

退出 Vivado HLS GUI 并返回到命令提示符

结论

在本教程中，您学习了：

- 如何更新现有的标准 C 类型 Vivado 高级合成任意精度类型。
- 在硬件性能和使用比特精确数据类型方面的优势。

第六章

设计分析

概述

从 C、c++或 SystemC 创建 RTL实现的一般设计方法包括以下任务:

- 综合设计。
- 回顾最初实施的结果。
- 应用优化指令 (*optimization directives*) 来提高性能。

您可以重复上述步骤，直到达到所需的性能。随后，您可以重新审视设计以改进。

这个过程的关键部分是对结果的分析。本教程解释了如何使用**报表和 GUI 分析透视图**来**分析设计并确定应用哪些优化**。

本教程包含一个实验练习:

- ◆ 演示 HLS 交互式分析特性。
- ◆ 带您通过一个设计，从最初的实现，通过六个步骤和多次优化，以产生最终的优化设计。

正如本教程所演示的，在单个项目中执行这些步骤可以比较不同的解决方案。

实验室 1 描述

综合和分析 DCT 设计。利用设计分析的洞察力来应用优化并判断优化的有效性。

教程设计描述

您可以从 Xilinx 网站下载教程设计文件。请参阅定位教程设计文件中的信息。本教程使用教程目录中的设计文件

Vivado_HLS_Tutorial \ Design_Analysis。

实验中使用的样本设计是一个二维 DCT 函数。为了突出设计分析特性，您的**目标是让该设计以 125 或更少的间隔运行。设计应该能够至少每 125 个时钟周期处理一组新的输入数据。**

实验一:设计最佳化

本练习解释了 GUI 分析透视图的基本操作，以及如何使用它来驱动设计优化。

重点:本教程中的图形和命令使用教程数据目录



Vivado_HLS_Tutorial 如果将教程数据目录解压缩到不同的位置，或者是在 Linux 系统上，请将引用的少数路径名调整到放置该目录的位置。C:\Vivado_HLS_Tutorial. *Vivado_HLS_Tutorial*

步骤 1:创建并打开项目

- 1.打开 Vivado HLS 命令提示符。
在 Windows 上点击 *Start > All Programs > Xilinx Design Tools > Vivado 2018.3 > Vivado HLS > Vivado HLS 2018.3 Command Prompt*
在 Linux 上，打开一个新的 shell。
- 2.使用命令提示窗口(图 6-1)，将目录更改为设计分析教程 *lab1*。
- 3.使用图 6-1 所示的命令 *vivado_hls -f run_hls.tcl*，执行 Tcl 脚本来设置 Vivado HLS 项目。

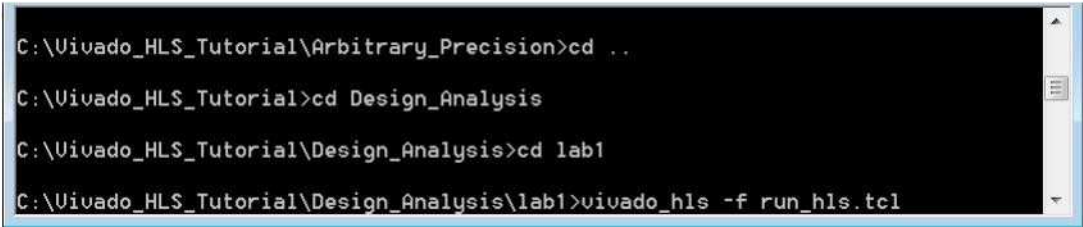


图 6 - 1: 设置设计分析教程项目

- 4.Vivado HLS 完成后，使用图 6-2 所示的命令在 Vivado HLS GUI 中打开项目。
vivado_hls -p dct_prj

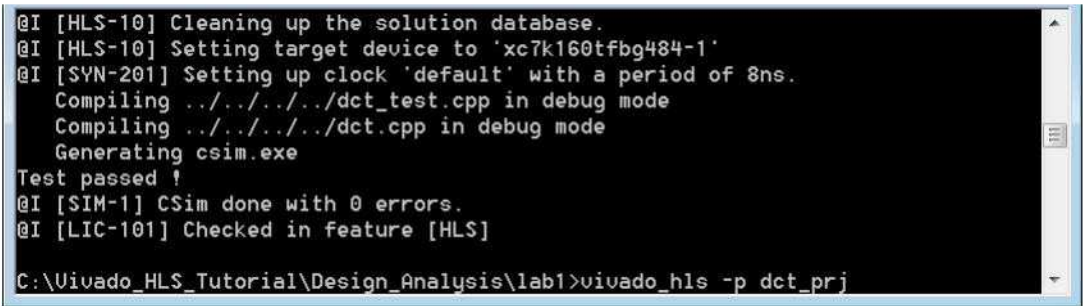


图6-2:实验室1 的开放设计分析项目

步骤 2:检查源代码并创建初始设计

- 1.双击源文件夹中的文件 *dct.cpp* 以打开源代码以进行检查。本例使用 *DCT* 函数。图 6-3 显示了这段代码的概述。

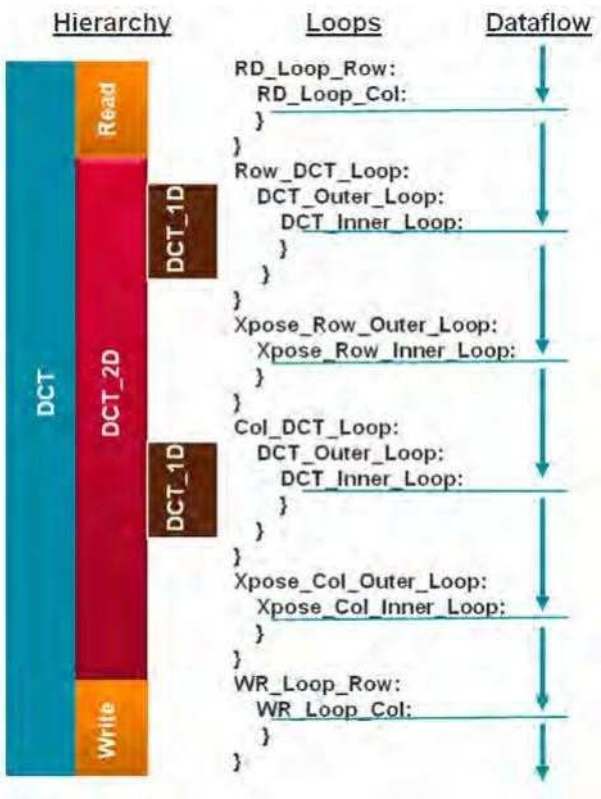


图 6-3:DCT 设计概述

- 图 6-3 的左侧显示了代码层次结构。
 - dct 有三个子函数:*read_data*、*dct_2d* 和 *write_data*。dct_2d 有一个单独的子函数 *dct_1d*。
 - 图 6-3 的**中心**显示了每个函数内部的循环。
 - 图 6-3 的**右侧**显示了如何通过函数和循环处理数据。
- 执行 *read_data* 函数，通过循环 *RD_Loop_Row* 处理数据，该循环有一个子循环 *RD_Loop_Col*。
- 在 *read_data* 函数完成后，执行 *dct_2d* 函数。在函数 *dct_2d* 中，*Row_DCT_Loop* 处理数据。*Row_DCT_Loop* 内部有两个嵌套循环：*DCT_output_loop* 和 *DCT_inner_loop*。调用 *DCT_inner_loop* 函数 *dct_1d*。
- 以此类推，直到函数 *write_data* 处理数据为止。
- 点击 *Run C Synthesis* 工具栏按钮，将设计综合为 RTL。

步骤 3:使用综合报告检查性能

当综合完成时，综合报告将自动打开。图 6-4 显示了报告的性能部分。

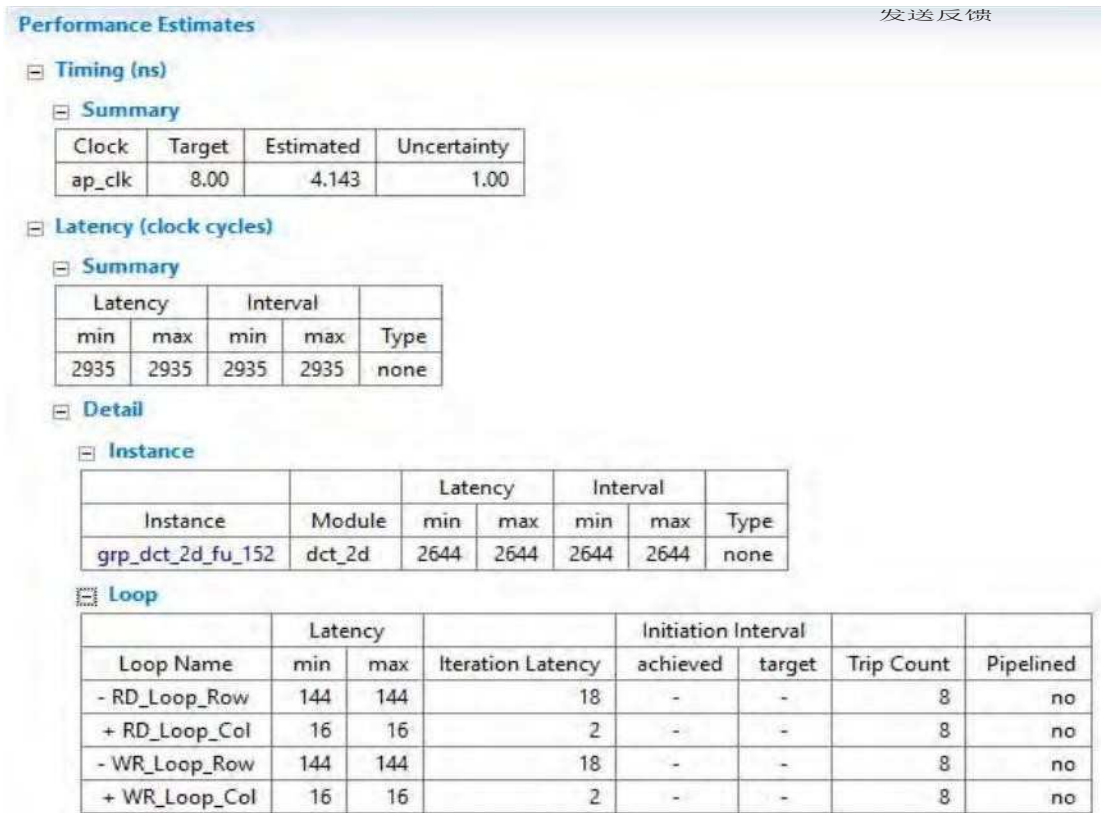


图 6-4: 初始 DCT 设计报告

- 图 6-4 突出显示了以下信息。
- ◆ 已满足 *8ns* 的时钟频率
 - ◆ 顶层设计需要 2935 个时钟周期来编写所有输出。
 - ◆ 您可以在 2935 个时钟周期后应用新的输入。这立即表明设计不是 *pipeline*，但是报告中也指出了这一事实：*pipeline=no*。
 - ◆ 顶层有一个实例，延迟和启动间隔为 2644。这个块也没有流水线，占了大部分的时钟周期。
- 请注意**，这里并没有将函数 *read_data* 和 *write_data* 标记为顶层的实例。

图 6-5 显示，在综合过程中，这些块被 *自动内联*(层次被移除)。

- ◆ **高级综合可以自动内联小函数来提高结果的质量(QoR)。** 可以通过将带有选项的内联指令 *-off* 添加到任何自动内联的函数来防止这种情况。

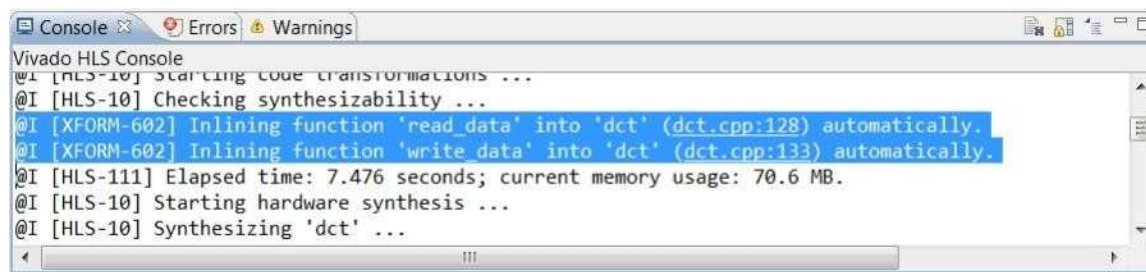


图 6-5: 自动优化报告

- **read_data** 和 **write_data** 函数中的循环因此在顶层实现，并在顶层函数中报告为循环(图 6-4)。
- 每个循环有 **144** 个时钟周期的延迟。(因为循环不是流水线，所以没有启动间隔)
- 以 **RD_Loop_Row** 为例，您可以看到为什么循环延迟是 144:

子循环 RD_Loop_Col 的延迟: 为循环的每次迭代 2 个周期(迭代延迟)，循环次数为 8: $2 * 8 = 16$ 个时钟周期。

从 **RD_Loop_Row** 开始，进入循环 RD_Loop_Col 需要 1 个时钟，返回到 RD_Loop_Row 需要 1 个时钟周期。

所以，**RD_Loop_Row** 的迭代延迟为 $(1 + 16 + 1) = 18$ 个时钟周期。

循环次数为 8，因此总循环延迟为 $8 * 18 = 144$ 个时钟周期。

因此，**dct** 块 2935 个周期的总延迟为:

RD_Loop_Row 块的 **144** 时钟。

dct_2d 块的时钟周期为 **+2644**。

加上 144 个时钟周期的 **WR_Loop_Row**。

加上一个时钟周期，以进入这三个块中的每一个。

要查看实例化的子块 dct_2d 和 dct_1d 的详细信息，请从 Explorer 窗格中 solution1 下的 **syn/report** 文件夹中打开它们各自的报告。

您还可以使用 design analysis 透视图以更具交互性的方式查看这些细节。

步骤 4: 使用分析视角检查性能

在综合完成后随时调用 (*invoke*) 分析透视图。

1. 单击 Analysis 透视图按钮(图 6-6)，开始交互设计分析。

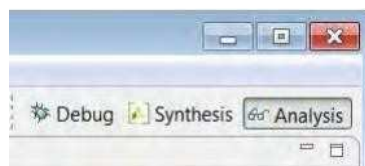


图 6-6: 打开分析透视图

分析透视图由 **五个窗格** 组成，图 6-7 中突出显示了每个窗格。在本教程中，您将使用所有这些方法。模块和循环层次结构显示为展开的(默认情况下，它们显示为折叠的)。

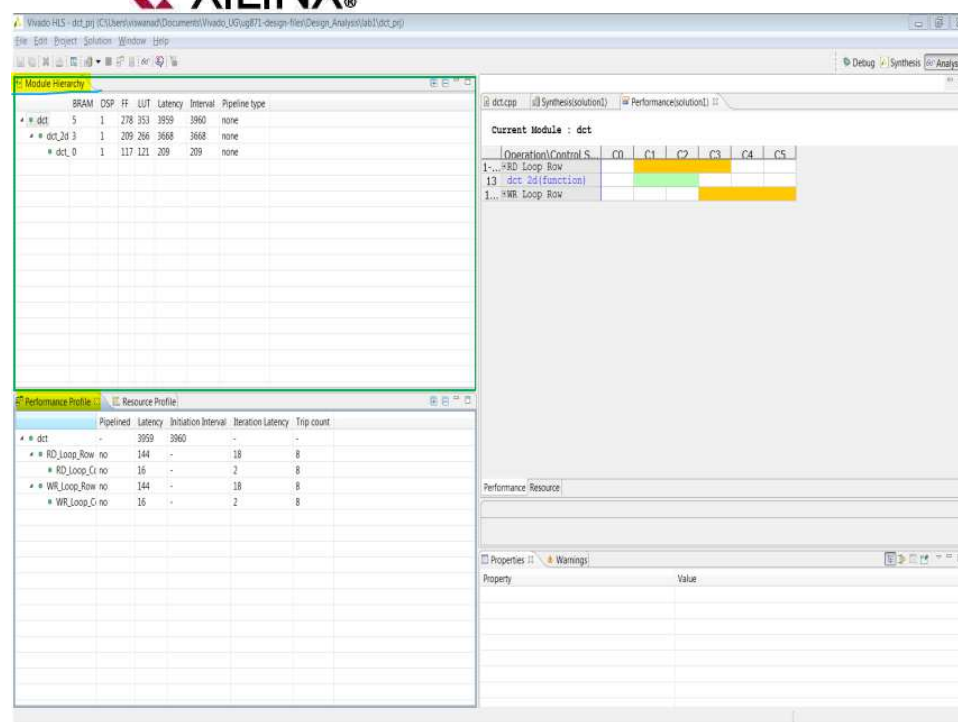


图 6-7: 分析视图的概述

使用“**模块层次结构 (Module Hierarchy pane)**”窗格在层次结构中导航。模块层次结构窗格显示了整个设计的性能和区域信息。“**性能概要 (Performance Profile)**”窗格显示此层次结构级别的性能详细信息。这两个窗格中的信息与您在前面的报告查看的信息(用于顶级 dct 块)类似。

还显示了 **Performance** 视图(在图 6-8 的右侧)。此视图显示如何将此特定块中的操作调度到时钟周期中。

- 左侧列出了资源。
 - ◆ 子块以蓝色文本显示。
 - ◆ 源代码中循环产生的操作为黄色。
 - ◆ 标准操作是显示为紫色。
- 注意 **dct** 有三个主要块:
 - ◆ 一个称为 **RD_Loop_Row** 的循环。**加号(+)**表示循环具有层次结构，您可以展开循环查看它。
 - ◆ 一个名为 **dct_2d** 的子块。
 - ◆ 一个名为 **WR_Loop_Row** 的循环。

第一行列出了设计中的**控件状态**。控制状态是将操作调度到时钟周期的高级综合使用的内部状态。在 RTL 有限状态机(FSM)中，控制状态与最终状态之间存在密切相关性，但不存在一一映射。

2. 单击 **循环 RD_Loop_Row** 和 **子循环 RD_Loop_Col** 以完全展开循环层次结构(图 6-8)。

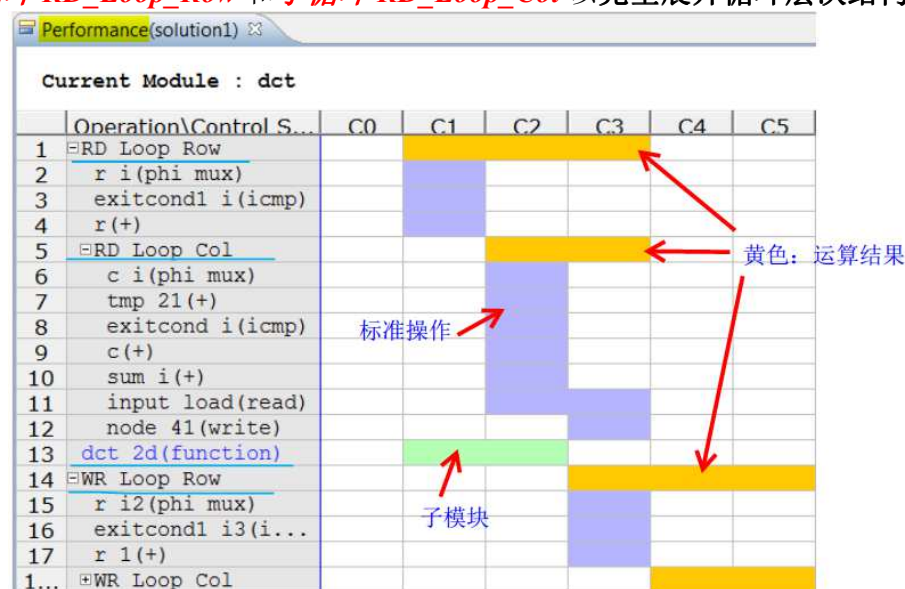


图 6-8: RD_Loop_Row 的扩展视图

从这里可以看到，在 RD_Loop_Row 的第一个状态(C1)中，检查了循环退出条件并执行了添加操作。这个添加可能是循环迭代的计数器，我们可以确认这一点。

3. 在状态 **C1** 中选择加法器，右键单击并选择 **Go to Source**(图 6-9)。当对话框打开时，按 OK 键选择项目 0。

这将打开 C 源代码以突出显示创建这个加法器的 C 源代码中的操作。从屏幕上的细节(也显示在图 6-9 中)，您可以确定它确实是循环计数器。它是这一行中唯一的加法，变量名为“**r**”。

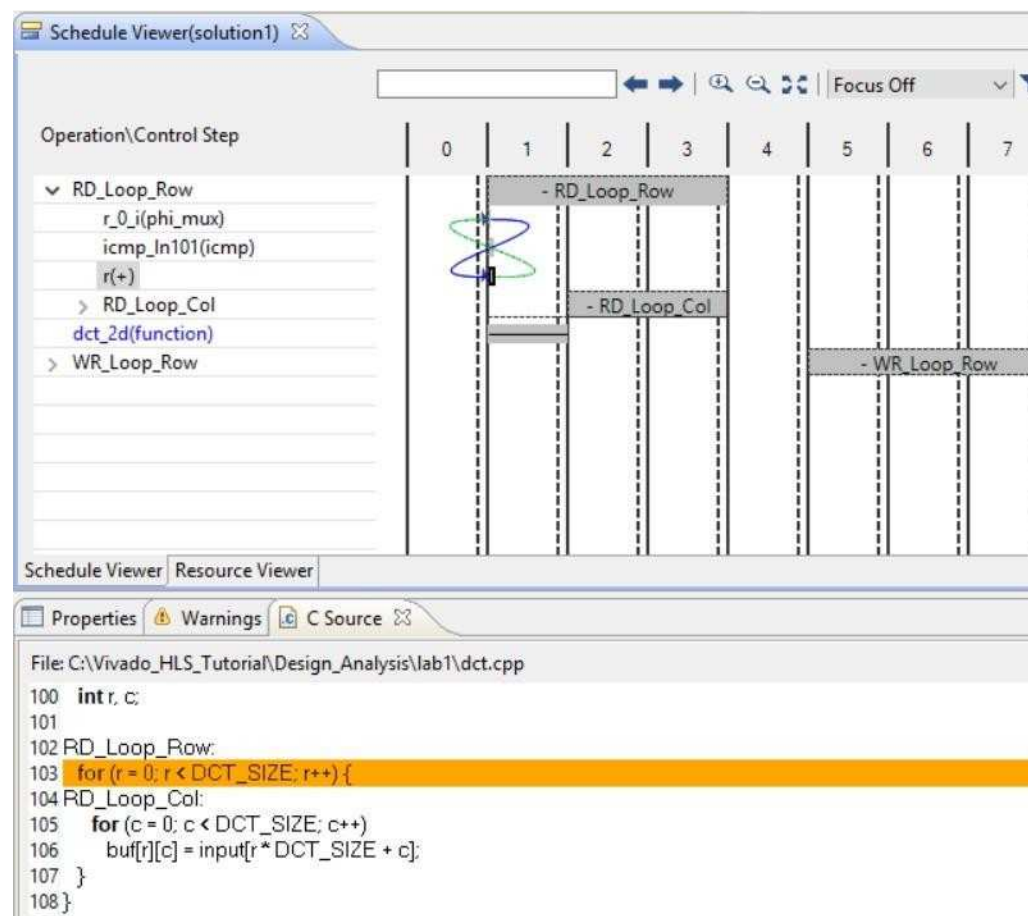


图 6-9: C 源代码视图

在循环 **RD_Loop_Row** 的下一个状态(**C2 状态**)中，循环 **RD_Loop_Col** 开始执行。

4. 单击 **RD_Loop_Col** 中的任何操作以查看源代码高亮显示更新。

这有助于确认您对 RTL 中如何实现 C 源代码中的操作的理解。

- 检查循环退出条件。
- 这是一个循环计数变量“**c**”的加法器。
- 从 RAM 中读取数据(一个周期生成地址，一个周期读取数据)。
- 对 RAM 执行写操作。

注意：性能视图中的循环意味着设计围绕这些状态进行多次迭代。迭代的次数被记录为循环次数，并显示在性能配置文件中。

发送反馈

为了提高性能，这些循环应该是流水线的。您可以查看其他性能优化机会的其余设计。

5. 单击 **C Source** 窗格选项卡中的 X 关闭此窗口。

6. 在模块层次结构窗格中，单击函数 **dct_2d** 以导航到该函数的视图中(图 6-10)。

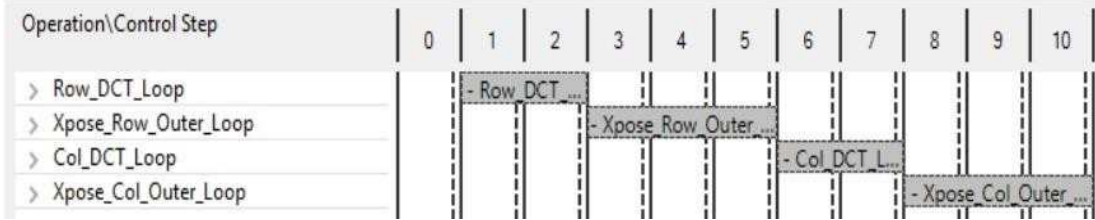


图 6-10:DCT_2D 性能视图

同样，您可以看到许多循环(如图 6-11 所示)。循环确保设计的区域很小，但是设计将需要多个迭代状态来完成：循环的每个迭代将在下一次迭代开始之前完成。

您可以对循环进行流水线操作以提高性能。性能概要 (*Performance Profile*) 中的细节显示，大部分延迟是由 *Row_DCT_Loop* 和 *Col_DCT_Loop* 循环造成的。

7.在 performance viewer 中单击 dct_2d 块的Row_DCT_Loop 和 Col_DCT_Loop 以完全展开它们，如图 6-11 所示。

在 **Performance** 视图中展开这些循环可以看到两个循环都调用了函数 *dct_dct_1d2*。除非这个函数本身是流水线的，否则流水线循环没有任何好处。**module** 层次结构显示 *dct_1d2* 的间隔是 210 个时钟周期，这意味着它每隔 210 个时钟周期只能接受一个新的输入。

8.在**模块层次结构**中，单击函数 *dct_1d2*以导航到该函数的视图。

9.展开 *Performance Profile* 和 *Performance* 视图中的循环以查看视图 如图 6-11 所示。

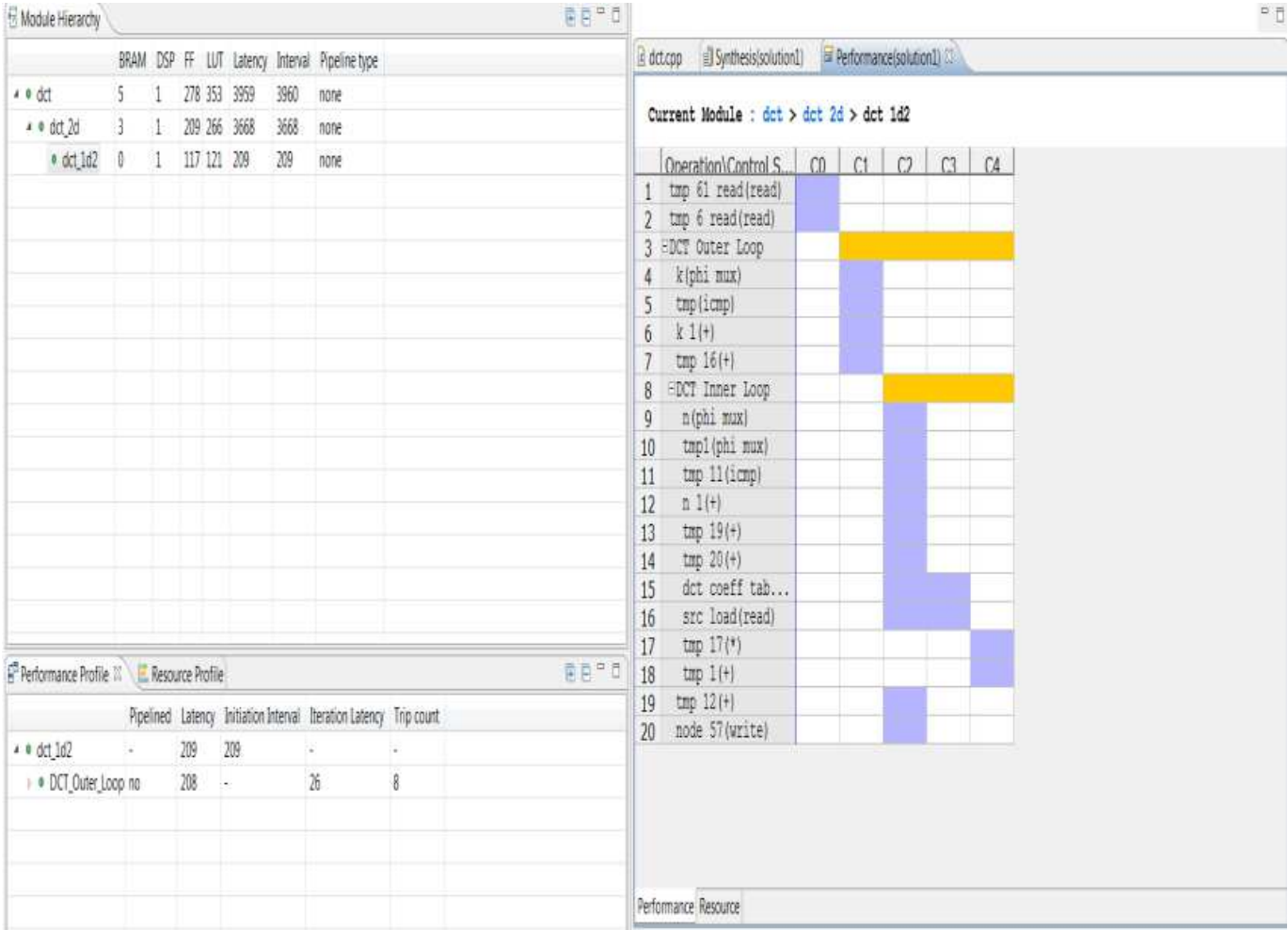


图 6-11:DCT_1D 性能视图

在图 6-11 中，您可以看到一系列可以被流水线化的嵌套循环(*Row_DCT_loop*、*Col_DCT_Loop*)。 你可以选择做以下其中之一：



- ◆ 你可以先对函数进行流水线操作，然后再对调用它的循环进行流水线操作。(因为函数是流水线的，循环可以利用流水线部分)
- ◆ 您可以在这个函数中流水线循环，并简单地使这个函数执行得更快。

Pipelining 函数展开其中的所有循环，从而大大增加了面积。如果目标是在不考虑面积的情况下获得尽可能高的性能，那么这可能是执行的最佳优化。

你可以在第七章“设计”中找到更多关于**流水线(pipelining)**循环和函数的细节[优化教程](#)。对于这种情况，方法是优化循环并将面积保持在最小。

10.单击综合法透视图按钮返回主综合法视图。



图 6-12: 重新打开合成透视图

步骤 5: 应用循环流水 (Loop Pipelining) 并检查循环优化

当对嵌套循环进行流水线处理时，通常最好对**最内层的循环**进行**流水线**处理。通常，“高级综合”通常可以自动平整循环嵌套（允许外循环简单地填充内循环）。有关为什么最好执行某些循环优化而不是其他循环优化的更多信息，请参见第 7 章，设计优化教程。

1. 选择“**New Solution**”工具栏按钮或使用菜单项目“>新解决方案”创建新解决方案。
 2. 单击 Finish 并接受默认设置。
 3. 确保您可以在信息窗格中看到 C 源代码(**dc.cpp**)。
 4. 在 Directive 选项卡中，在 **dct_1d** 函数的 **DCT_Inner_Loop 循环**中添加一个管道指令。
在指令面板中右键单击 **DCT_Inner_Loop** 并选择 **Insert** 指令。
在指令编辑器对话框中激活顶部的指令下拉菜单并选择 **PIPELINE**。
c. 单击 **OK** 以选择**默认的最大管道速率(H=1)**。
 5. 对以下循环重复步骤 4:
 - a. 在函数 **dct_2d** 循环 **Xpose_Row_Inner_Loop**
 - b. 在函数 **dct_2d** 循环中 **Xpose_Col_Inner_Loop**c.
 - c. 在函数 **read_data** 循环 **RD_Loop_Col**
 - d. 在函数 **write_data** 中循环 **WR_Loop_Col**
- 指令窗格显示了以下(高亮显示)应用的优化指令。

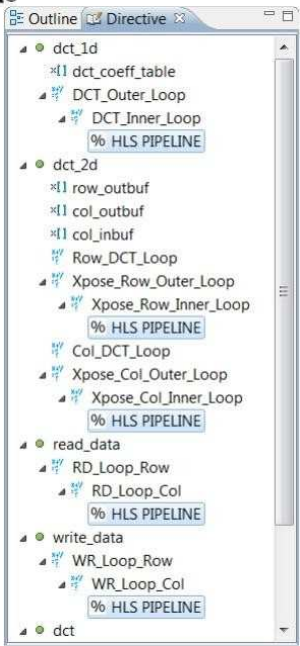


图 6-13:DCT 回路管道优化指令

- 6.单击 **Run C Synthesis** 工具栏按钮，将设计综合为 RTL。如果文件被修改，请选择 **YES**。
- 7.综合完成后，使用 **Compare Reports** 工具栏按钮或菜单 **Project > Compare Reports** 来比较解决方案 1 和 2。

图 6-14 显示了 solution1 和 solution2 的对比结果。**管道循环**提高了设计的延迟，解决方案 2 减少了近 50%。

Performance Estimates				
Timing (ns)				
Clock		solution1	solution2	
	ap_clk	Target	8.00	8.00
		Estimated	4.143	4.143
Latency (clock cycles)				
Latency	min	solution1	solution2	
	max	2935	1723	
Interval	min	2935	1723	
	max	2935	1723	

图 6-14:DCT Solution1 与 Solution2 的对比

接下来，您再次打开分析透视图，分析结果，并确定是否有更多的机会进行优化。

- 8. 单击 Analysis 透视图按钮开始交互设计分析。
- 当分析透视图打开时，您可以**看到大部分延迟仍然是由于阻塞 dct_2d 造成的**。在进一步分析之前，您可以回顾一下在这个级别上循环是如何优化的。

性能概要(图 6-15)显示，两个循环的延迟已经从解决方案 1 中的 **144** 个时钟周期减少到只有 **64** 个时钟周期。

Performance Profile		Resource Profile				
		Pipelined	Latency	Iteration Latency	Initiation Interval	Trip count
▼	dct	-	1723	-	1724	-
	RD_Loop_Row_RD_Loop_Col	yes	64	2	1	64
	WR_Loop_Row_WR_Loop_Col	yes	64	2	1	64

图 6-15: 顶级循环的 DCT 解决方案 2

流水线循环转换延迟：

延迟= 迭代延迟 * (tripcount * interval)

来

$$\text{延迟} = \text{迭代延迟} + (\text{tripcount} * \text{interval})$$

Vivado HLS 还通过**自动执行循环扁平化 (Flatten)**(不再有任何循环嵌套结构)实现了这一点。您可以通过查看 Console 窗格或日志文件来查看解决方案 2。
图 6-16 显示了自动优化的循环。

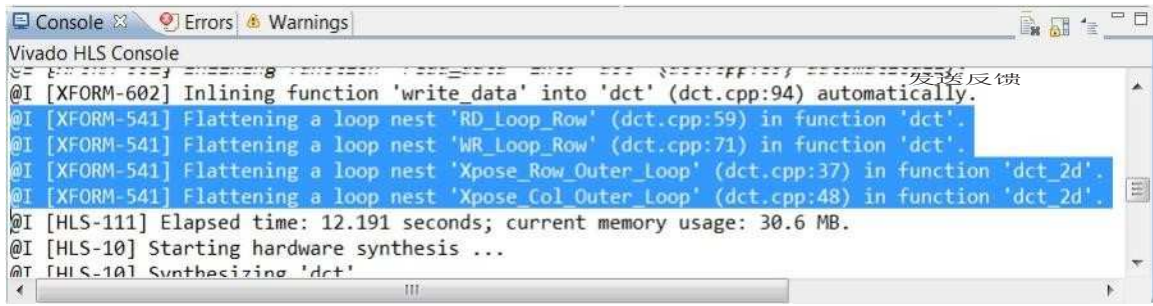


图 6-16:DCT Solution2 Loop 展平

9.在**模块层次 (Module Hierarchy)**结构中, 单击函数 **dct_2d**以导航到此函数的视图

在**性能配置 (Performance Profile)**文件中, 您可以看到所有循环的延迟已经大大减少(与图 6-10 中的早期报告相比, Row DCT Loop 和 Col DCT loop 的延迟大约减少了一半)。但是, 大部分延迟仍然是由于这两个循环造成的, 每个循环都调用 dct_1d 块。

10.在模块层次结构中, 单击函数 **dct_1d2**以导航到此函数的视图。

性能概要(图 6-17)显示循环延迟已经减少, 但是这里仍然有一个循环层次结构。(仍然有 DCT_Outer_Loop 循环, 如图 6-17 所示, 所以没有发生循环扁平化)。

	Pipelined	Latency	Iteration Latency	Initiation Interval	Trip count
dct_1d2	-	89	-	89	-
DCT_Outer_Loop	no	88	11	-	8
DCT_Inner_Loop	yes	8	2	1	8

图 6-17:dct_1d 循环的 DCT 解决方案 2 性能

在 Performance 视图中查看这些循环可以看出为什么这个循环没有进一步优化。

- 11.在 Performance 视图中, 单击循环 **DCT_Outer_Loop**和 **DCT_Inner_Loop**并查看循环层次结构(图 6-18)。
- 12.选择状态 **C3** 中的写操作。
- 13.右键单击并选择 **Go to Source**。

图 6-18 显示这个循环没有被压扁, 因为在防止循环压扁的级别之外还有额外的操作。下面的图 6-18 突出显示了防止循环扁平化的操作之一。

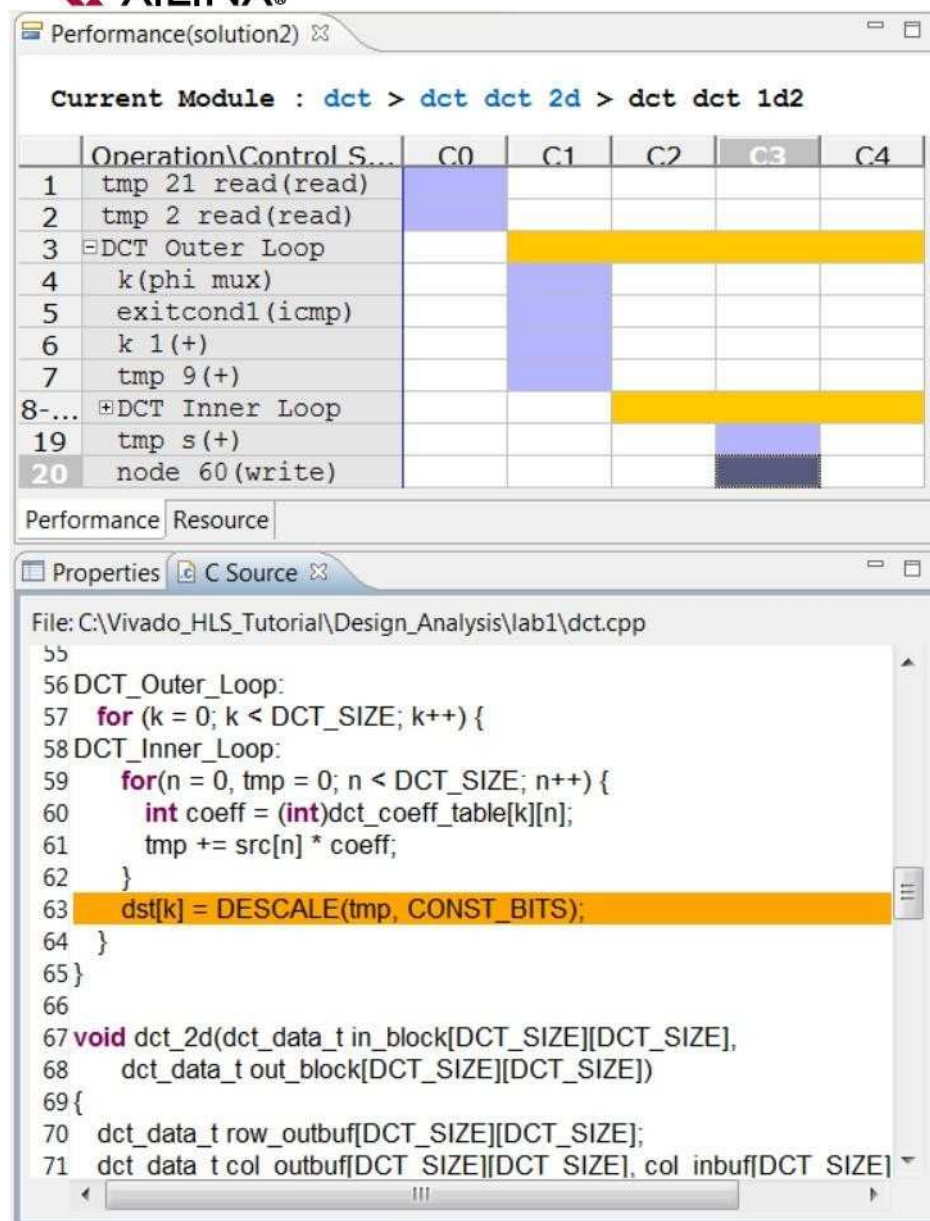


图 6-18:DCT Solution2 dct_1d 性能视图

无法将写入数组的内容展开到内部循环中。要在 DCT 外循环上达到间隔 1，您将需要对输出循环进行流水线处理-简单地对内循环本身进行流水处理没有任何好处

您应该改为管道外循环。这将导致内部循环完全展开。会导致面积增加，但是您仍然没有达到 125 的吞吐量目标，并且还没有准备好流水线化整个功能（并且看到更大的面积增加，因为外部循环也已完全展开）。

14.单击综合法透视图按钮返回主综合法视图。

步骤 6:应用循环优化并检查瓶颈

- 1.选择 **New Solution** 工具栏按钮或使用菜单 **Project > New Solution** 创建新解决方案。
- 2.单击 **Finish** 并接受默认设置以创建解决方案 3。
- 3.确保 C 源代码在信息窗格中可见。
- 4.在指令选项卡中
 - 1) 在函数 **dct_1d** 中，选择循环 **DCT_Inner_Loop** 上的 **PIPELINE** 指令。
 - 2) 右键单击并选择 **Remove** 指令。
 - 3) 仍然在函数 **dct_1d** 中，选择 loop **DCT_Outer_Loop**。
 - 4) 右键单击并选择 **Insert** 指令。
 - 5) 在指令编辑器对话框中激活顶部的指令下拉菜单并选择 **PIPELINE**。
 - 6) 单击 **OK** 以选择默认的**最大管道速率(II=1)**。

指令窗格应该显示以下(高亮显示)应用的优化指令。

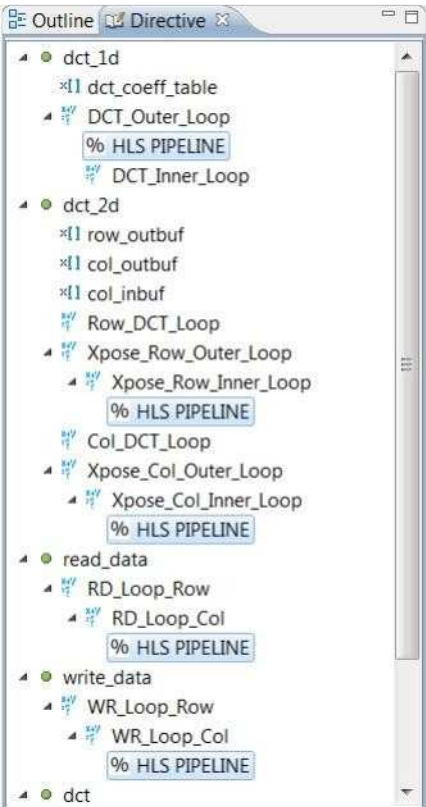


图 6-19: 更新的 DCT 循环管道优化指令

- 5.单击 **Run C Synthesis** 工具栏按钮，将设计合成为 RTL。
- 6.综合完成后，单击 **Compare Reports** 工具栏按钮来比较解决方案 2 和 3。

图 6-20 显示了 solution2 和 solution3 的对比结果。

[发送反馈](#)



外部循环的流水线实际上提高 了性能和面积。

由于设计中的多个循环多次调用 `dct_1d` 函数，因此可以获得显著的延迟优势。在此块中节省延迟将成倍增加，因为此函数在许多循环中使用。

Performance Estimates

[-] Timing (ns)

Clock		solution2	solution3
ap_clk	Target	8.00	8.00
	Estimated	4.143	6.351

[-] Latency (clock cycles)

		solution2	solution3
Latency	min	1723	843
	max	1723	843
Interval	min	1723	843
	max	1723	843

Utilization Estimates

	solution2	solution3
BRAM_18K	5	5
DSP48E	1	8
FF	223	546
LUT	1211	1356
URAM	0	0

图 6-20:DCT Solution2 与 Solution3 的对比

在这种情况下，报告指出解决方案的时钟周期较大，但仍然可以实现。Vivado HLS 有时会创建一个设计，其中估计的时钟周期无法满足所需的时钟周期。通常情况下，设计将在 RTL 综合之后满足时间要求—在这种情况下，您可以通过使用 **Export RTL** 特性并选择 **Evaluate** 来确认这一点。如果遇到 RTL 综合后设计未能满足计时要求的情况，请结合 C 代码中的区域使用 **LATENCY** 指令强制 Vivado HLS 在失败的 RTL 路径上注册中间点。

既然所有的循环都是流水线的，就有必要检查一下设计，看看是否存在性能限制的“瓶颈”。瓶颈是数据流中的限制，它可以阻止逻辑块以最大数据速率工作。

数据流中的这种限制可能来自许多来源，例如，作为块 RAM 实现的 I/O 端口和数组。在这两种情况下，有限数量的端口(在 I/O 或块 RAM 上)限制了数据的读写速度。

瓶颈的另一个来源是原始源代码中的数据依赖性。在某些情况下，这些数据依赖关系在算法的操作方式中是固有的，比如在之前的计算完成之后才能执行计算。然而，有时使用优化指令或对 C 代码做一些小的修改就可以删除它们。

第一个任务是在 RTL 设计中识别这些问题。你可以采取以下几种方法:

- 从模块层次结构报告中最大的延迟或间隔开始并导航沿着层次结构向下查找任何大延迟或间隔的源。
- 单击资源配置文件来检查 I/O 和内存使用情况。
- 利用图形查看器的强大功能，在性能视图中查找模式表示数据流中的限制。

在这种情况下，您将使用后一种方法。您可以使用 **Analysis 透视图**来快速识别设计中的这些位置。

- 7.单击 **Analysis 透视图**按钮开始交互设计分析。
- 8.在模块层次结构中，确保选择了模块 **dct**。
- 9.在 Performance 视图中，展开设计中的第一个循环，如图 6-21 所示(这些循环被消除，名称现在是两个循环的连接 **RD_Loop_Row_RD_Loop_Col**)。

此循环以两种状态实现。图 6-21 中的红色箭头显示了从循环开始到循环结束的路径：箭头几乎是垂直的(所有事情都在两个时钟周期内发生)，并且这个循环在延迟方面实现得很好。

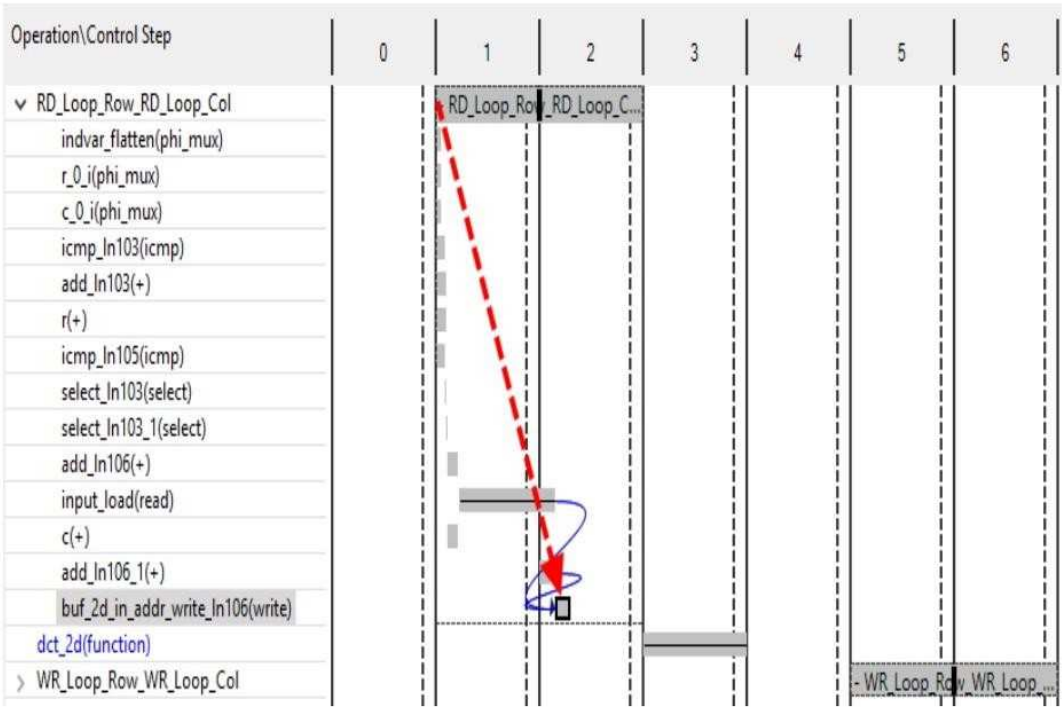


图 6-21:DCT RD_Loop_Row 分析

10.在 **Performance** 视图中,展开 **WR_Loop_Row_WR_Loop_Col** 并执行类似的分析。它在延迟方面也进行了很好的优化。

11.双击函数 **dct_2d** 并导航到该函数。

您可以在整个层次结构中使用相同的分析过程。如果执行此分析,您将发现所有函数块和循环都具有类似的最佳(几个循环)实现,直到检查 **dct_1d** 块为止。

12.在 **Performance** 视图中,双击函数 **dct_1d2** 并导航到该函数。

13.展开 **DCT_Outer_Loop** 以查看图 6-22 所示的视图。

图 6-22 显示了与早期循环调度(只有几个延迟周期)非常不同的视图。进度表显示了从输入到输出的长时间偏移(如红色箭头所示)。图 6-22 为分析结果 **dct_1d RD_Loop_Row**。

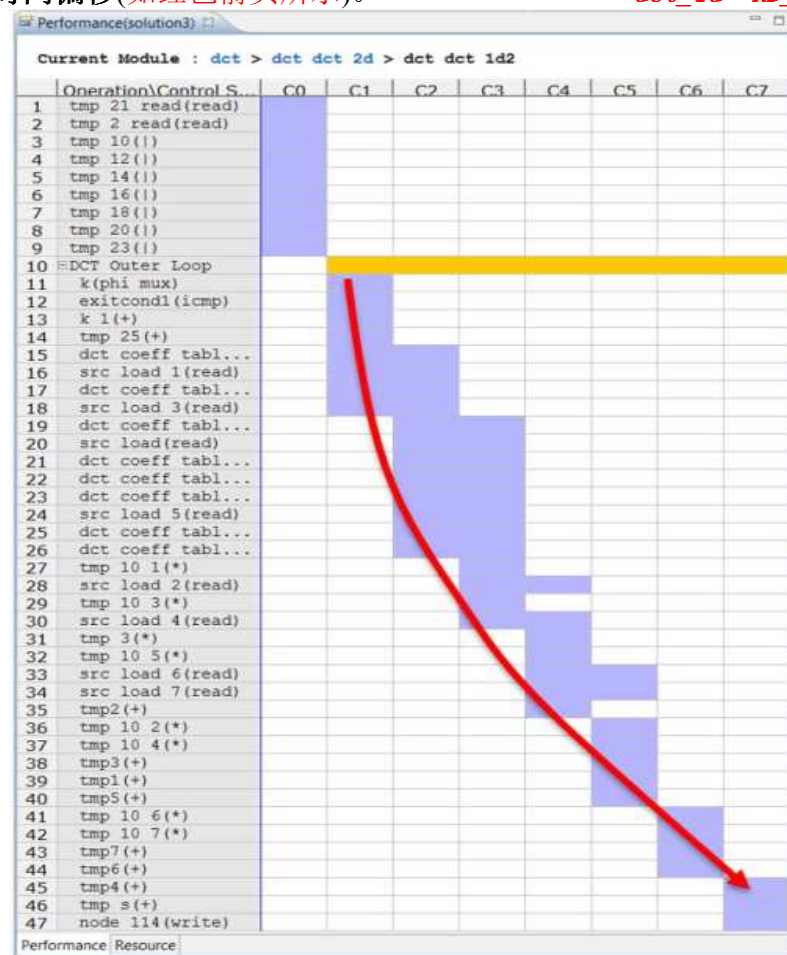


图 6-22:dct_1d RD_Loop_Row 的分析

通常有两件事情会导致这种类型的调度:源代码中的数据依赖关系和 I/O 或块 RAM 的限制。现在您将检查此块中的资源共享。

14.在 **Performance** 视图中,单击窗口底部的 **Resource Viewer** 选项卡。

15.展开内存端口,如图 6-23 所示。

Resource\Control Step		C0	C1	C2	C3	C4	C5	C6	C7	
1	I/O Ports									
2	i 21	read								
3	i 2	read								
4	src(p0)		read	read	read	read				
5	src(p1)		read	read	read	read				
6	dst(p0)								write	
7	Memory Ports									
8	dct coeff tabl...		read							
9	src(p0)		read	read	read	read				
10	src(p1)		read	read	read	read				
11	dct coeff tabl...		read							
12	dct coeff tabl...			read						
13	dct coeff tabl...			read						
14	dct coeff tabl...				read					
15	dct coeff tabl...				read					
16	dct coeff tabl...				read					
17	dct coeff tabl...				read					
18	dst(p0)								write	
1...	Expressions									

图 6-23: DCT_1d 中内存端口的资源共享

资源视图显示了在不同的控制状态下如何使用设计中的资源。这些行列出了设计中的资源。在图 6-23 中，扩展了内存资源。

这些列显示了资源使用的控制状态。如果某个资源处于多个状态，则该资源将在不同的时钟周期中被重用。

图 6-23 显示在每个时钟周期中，块 RAM 上的内存访问被使用到最大。(一个块 RAM 最多可以有两个端口，并且两个端口都在使用)。这很好地说明了设计可能受到内存资源的带宽限制。要确定是否真的是这样，您可以进一步检查。

- 16.为 *src* 块 RAM 选择一个读操作。
- 17.右键单击并选择 *Goto Source* 以查看图 6-24 所示的视图。

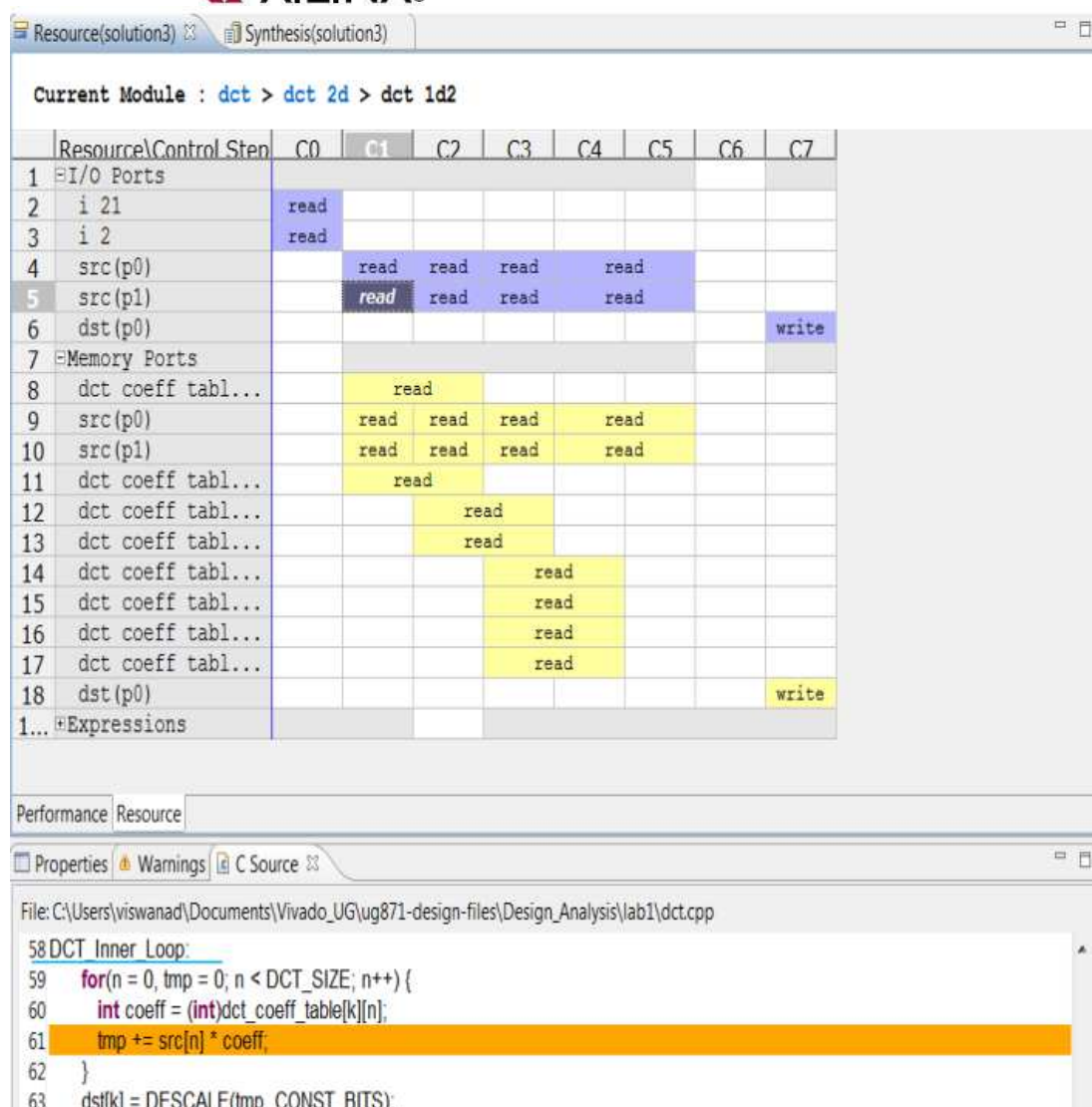


图 6-24: 内存资源 SRC 和源代码

图 6-25 显示了对 **src** 变量的读取来自循环 **DCT_Inner_Loop** 内部的读取操作。当对 **DCT_Outer_Loop** 进行管道传输时，将自动展开此循环，并且该循环中的所有操作都可以并行发生（如果允许数据依赖）。

由于阵列 **src** 在 RTL 中被实现为 **Block RAM**，而 Block RAM 在任何一个时钟周期内只能允许两次读取（**最大**），因此必须在多个周期内进行八次读取。

在图 6-25 中，读取操作需要 2 个时钟周期：一个为 Block RAM 生成地址的周期，另一个为读取数据的周期。

仅显示启动（地址生成周期），因为它与下一个时钟周期中的操作重叠。

您可以使用优化指令对 Block RAM 进行优化，以对 Block RAM 进行**分区**。函数 **dct_1d** 访问的数组被定义为函数的输入参数，因此位于该块之外。

- **dct_1d** 的第一个实例的输入数组是函数 **dct** 中的 **buf_2d_in**。
- **dct_1d** 的第二个实例的输入数组是函数 **dct_2d** 中的 **col_inbuf**。

在这两种情况下，数组都是二维的，大小为 **DCT_SIZE * DCT_SIZE (8x8)**。默认情况下，这将导致具有 64 个元素的单个 Block RAM。因为数组是在代码中以逐列的形式配置的，所以我们可以划分二维并创建八个单独的 Block RAM：**每个行一个 RAM，允许并行访问行数据**。

18. 单击综合法透视图按钮返回主综合法视图。

步骤 7: 分区块 RAMs 并分析并发性 (Partition Block RAMs and Analyze Concurrency)

1. 选择 “**New Solution**” 工具栏按钮或使用菜单 **Project > New Solution** 来创建新解决方案，解决方案 4”。
2. 单击 **Finish** 并接受默认设置以创建解决方案 4。
3. 确保 C 源代码在信息窗格中可见。

4. 在指令标签:

- 在函数 **dct** 选择数组 **buf_2d_in**。
- 右击并选择 **Insert** 指令。在指令编辑器对话框中，激活顶部的指令下拉菜单并选择 **ARRAY_PARTITION**。
- 设置类型为 **Complete**。
- 将维度 (**dimension**) 设置更改为 2，以便沿着第二个维度对数组进行分区。
- 单击 OK。

5. 对函数 **col_inbuf** 中的数组 **dct_2d** 重复此过程。

指令窗格显示优化指令，如图 6-25 所示(突出显示了两个新指令)。

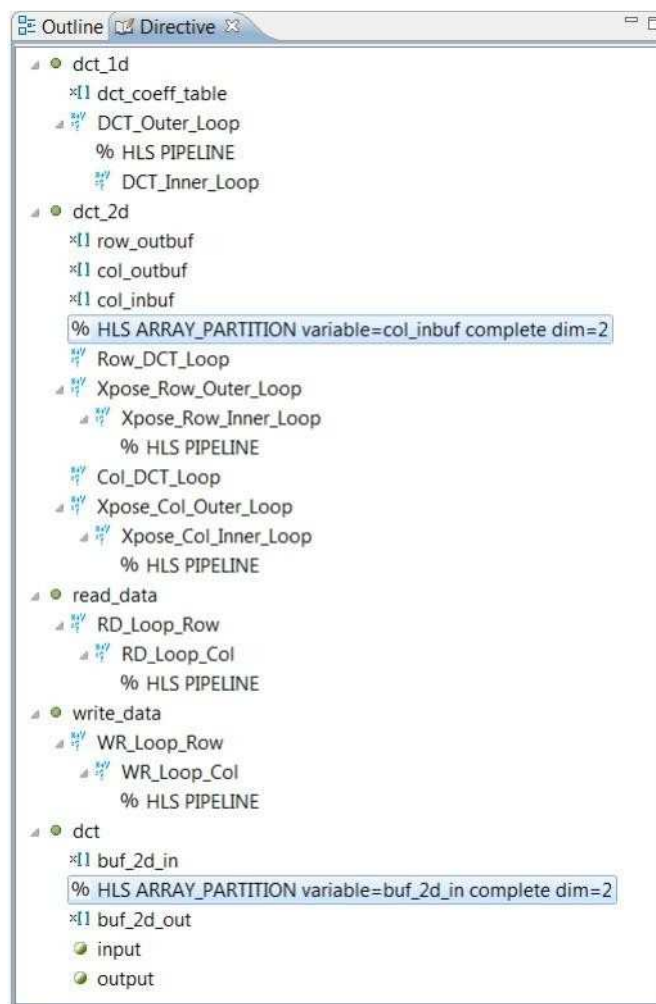


图6-25: 数组分区的优化指令

- 单击 **Run C Synthesis** 工具栏按钮，将设计综合为 RTL。
- 综合完成后，使用 **Compare Reports** 工具栏按钮进行比较 solution3 和 4。

图 6-26 显示了 solution3 和 solution4 的对比结果。改善对 **dct_1d** 块中 **src** 块 RAM 中的数据的访问已经改善了整体性能，因为 **dct_1d** 块执行频繁。

Performance Estimates			
Timing (ns)			
Clock		solution3	solution4
ap_clk	Target	8.00	8.00
	Estimated	6.351	6.904
Latency (clock cycles)			
		solution3	solution4
Latency	min	843	477
	max	843	477
Interval	min	843	477
	max	843	477

图 6-26:DCT Solution3 与 Solution4 的对比

您可以查看分区指令对设备资源的影响。

- 8.单击 *Analysis* 透视图按钮开始交互设计分析。
- 9.在模块层次 (*Module Hierarchy*) 结构中，确保选择了模块 *dct*。
- 10.选择左下方的 *Resource Profile* 选项卡。
- 11.展开内存 (*memories*) 和表达式 (*Expressions*) 参见图 6-27 中的视图。

	BRAM	DSP	FF	LUT	Bits P0	Bits P1	Bits P2	Banks/Depth	Words	W*Bits*Banks
dct	3	8	1003	1879						
> I/O Ports(2)					32					
> Instances(2)	2	8	711	1320						
> Memories(9)	1		256	16	144			9	128	2048
> buf_2d_out_U	1		0	0	16			1	64	1024
> buf_2d_in_6_U	0		32	2	16			1	8	128
> buf_2d_in_5_U	0		32	2	16			1	8	128
> buf_2d_in_4_U	0		32	2	16			1	8	128
> buf_2d_in_3_U	0		32	2	16			1	8	128
> buf_2d_in_7_U	0		32	2	16			1	8	128
> buf_2d_in_2_U	0		32	2	16			1	8	128
> buf_2d_in_1_U	0		32	2	16			1	8	128
> buf_2d_in_0_U	0		32	2	16			1	8	128
> Expressions(11)	0	0	0	103	39	44	8			
> +	0	0	0	69	23	23	0			
> icmp	0	0	0	22	11	13	0			
> select	0	0	0	8	2	5	8			
> xor	0	0	0	4	3	3	0			

图 6-27:DCT 资源配置文件

资源配置文件显示当前层次结构级别上正在使用的资源(在模块层次结构窗格中选择的块)。图 6-27 显示:

- 这个块有两个 I/O 端口。
- 大部分区域归因于该块内的实例（子块）
- 有九个存储器 (*memories*)，其中八个是分区的 *buf_2d_in* 块 *RAM*。由于它们少于 *1024* 位，因此它们会自动实现为 *LUTRAM*。
- 在此层次结构级别上的大多数逻辑（表达式）是由于**加法器**，有些是由于**比较器**和**选择器**。

前面优化的重点是，您可以看到现在由于数组分区优化而增加了内存。

您仍然有一个目标，以确保设计可以每 125 个时钟周期接受一组新的样本。**然而，综合报告显示，每 477 个时钟只能接受新数据**，优于原来的设计(约 2600 个时钟周期)，但是需要进一步的优化。

注意: 到目前为止, 您主要关注于改进设计中**每个单独循环和函数的延迟和间隔**。现在必须应用数据流优化, 它使各个循环和函数能够并行执行, 从而改进总体设计间隔。

12. 单击综合法透视图按钮返回主综合法视图。

步骤 8: 分区块 ram 并应用数据流 (DATAFLOW) 优化

1. 选择 “*New Solution*” 工具栏按钮或使用菜单 *Project > New Solution* 创建新解决方案 “解决方案 5”。
2. 单击 *Finish* 并接受默认设置以创建解决方案 5。
3. 确保 C 源代码在信息窗格中可见。
4. 在**指令**标签:
 - a. 选择顶层函数 *dct*。
 - b. 右击并选择 **Insert** 指令。
 - c. 在指令编辑器对话框中激活指令下拉菜单并选择 *DATAFLOW*。
 - d. 单击 *OK*。

指令窗格现在显示以下优化指令(突出显示新指令)。



图 6-28: DCT 设计的数据流优化

5. 单击 *Run C Synthesis* 工具栏按钮, 将设计综合为 RTL。
6. 合成完成后, 使用 *Compare Reports* 工具栏按钮或菜单 *Project > Compare Reports*, 以比较解决方案 4 和 5。

图 6-29 显示了 solution4 和 solution5 的对比结果, 可以看出**时间间隔**得到了改善。该设计需要 **476** 个时钟周期来产生输出, 但现在每 **343** 个时钟可以接受新的输入。

Performance Estimates

Timing (ns)

Clock		solution4	solution5
ap_clk	Target	8.00	8.00
	Estimated	6.904	6.904

Latency (clock cycles)

		solution4	solution5
Latency	min	477	476
	max	477	476
Interval	min	477	343
	max	477	343

图 6-29:DCT Solution4 与 Solution5 的对比

这仍然大于所需的 125 个周期，因此必须分析当前性能。

- 7.单击 *Analysis* 透视图按钮开始交互设计分析。
- 8.在模块层次结构中，您可以看到 *dct_dct_2d* 占大部分时间间隔。
确保选中模块 *dct_2d* 以查看图 6-30 中的视图。

Module Hierarchy

	Negative Slack	BRAM	DSP	FF	LUT	Latency	Interval	Pipeline type
dct	-	3	8	1009	1654	476	343	dataflow
dct_2d	-	2	8	684	1171	342	342	none
dct_1d	-	0	8	350	200	11	11	none
write_data	-	0	0	32	186	66	66	none
read_data	-	0	0	29	171	66	66	none

Performance Profile

Resource Profile

	Pipelined	Latency	Iteration Latency	Initiation Interval	Trip count
dct_2d	-	342	-	342	-
Row_DCT_Loop	no	104	13	-	8
Xpose_Row_Outer_Loop_Xpose_Row_Inner_Loop	yes	64	2	1	64
Col_DCT_Loop	no	104	13	-	8
Xpose_Col_Outer_Loop_Xpose_Col_Inner_Loop	yes	64	2	1	64

图 6-30: 数据流优化后的 DCT 分析图

在这里，你可以看到两件事：

- 块 *dct* 的间隔小于单个延迟 (*read_data*, *dct_2d* 和 *write_data*) 的总和, 即意味着块是并行操作的。
- dct* 的间隔与子块 *dct_2d* 的间隔相同。因此, *dct_2d* 块是限制因素。

因为在“模块层次结构”中选择了 *dct_2d* 块，并且“性能概要文件”显示了该块的详细信息。图 6-32 显示了间隔 (*interval*) 与等待时间 (*latency*) 相同，因此这些块都不是并行运行的。

使 *dct_2d* 中的块并行运行的一种方法是对整个函数进行流水线处理。但是，这将展开所有循环，有时可能导致大面积增加。一种替代方法是在函数 *dct_2d* 上使用数据流优化。另一种选择是使用不太明显的技术：将这些循环提升到层次结构的顶层，在此将它们包含在已经应用于顶层的数据流优化中。这可以通过使用优化指令删除 *dct_2d* 层次结构来实现：内联 *dct_2d* 函数。

在执行此优化之前，请查看由于使用数据流优化而导致的面积增加。

- 9.在模块层次结构中，确保选择了模块 *dct_2d*
- 10.激活资源配置文件 (*Resource Profile*) 视图。
- 11.展开内存以查看图 6-31 中的视图。

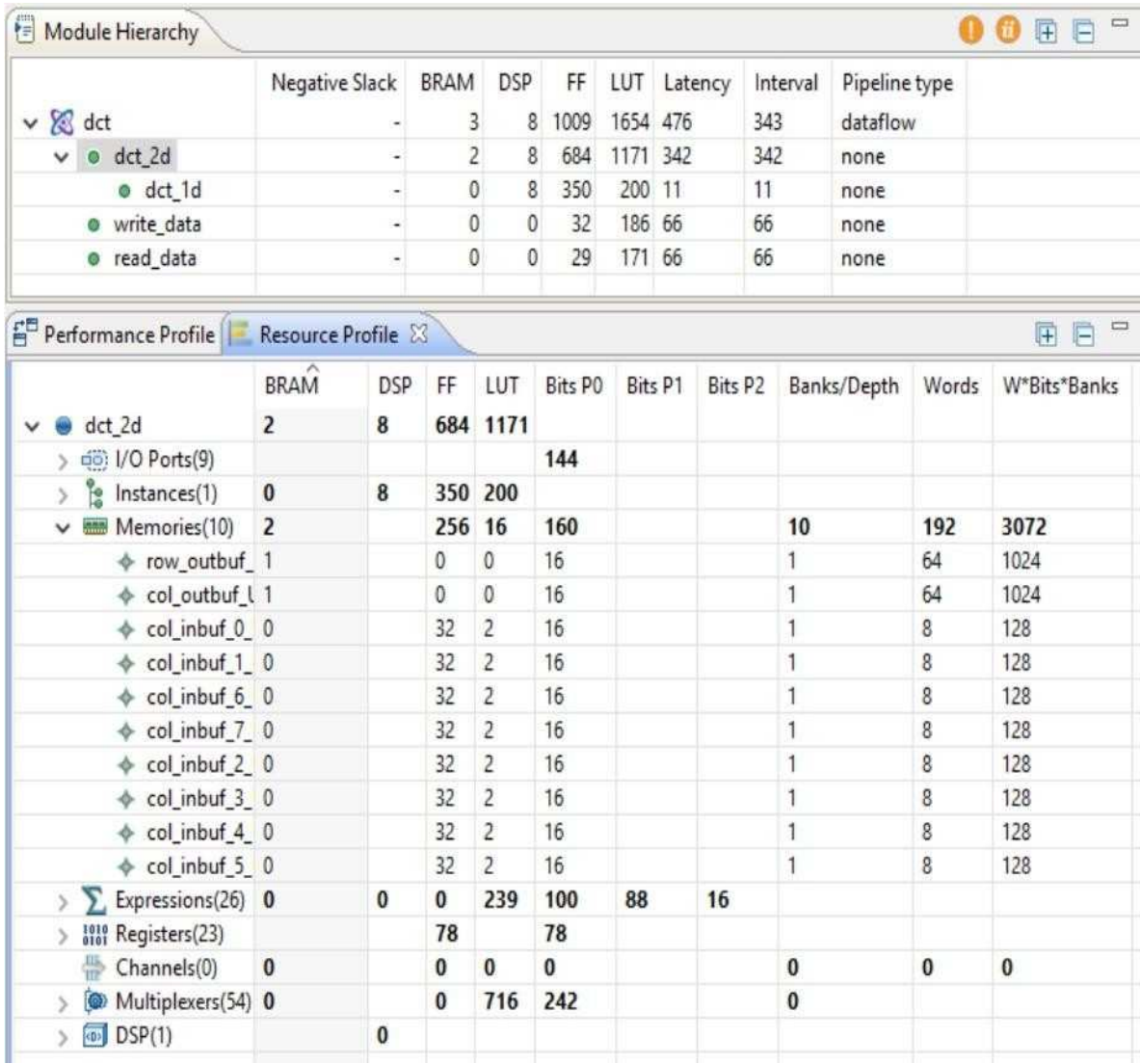


图 6-31:DCT 资源配置文件

12.单击综合法透视图按钮返回主综合法视图。

步骤 9:优化数据流的层次结构

- 1.选择 “*New Solution*” 工具栏按钮创建一个新解决方案 “解决方案 6” 。
- 2.单击 Finish 并接受默认设置以创建解决方案 6。
- 3.确保 C 源代码在信息窗格中可见。
- 4.在指令标签:
 - a. 选择函数 *dct_2d*.
 - b.右击并选择 *Insert* 指令。
 - c. 在指令编辑器对话框中激活顶部的指令下拉菜单并选择 *Inline*。
 - d. 单击 *OK*。 指令窗格现在显示以下优化指令(突出显示新指令)

发送反馈

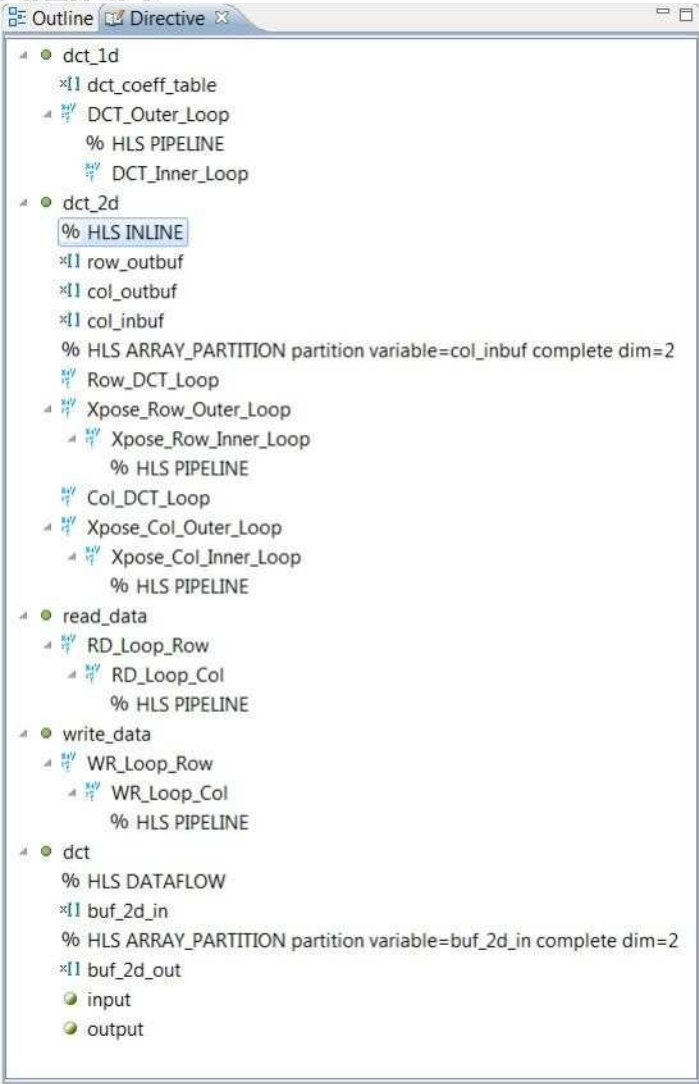


图6-32:DCT 设计的数据流优化

5. 单击 **Run C Synthesis** 工具栏按钮，将设计合成到RTL。

6.综合完成后，使用 **Compare Reports** 工具栏按钮或菜单 **Project > Compare Reports** 来比较解决方案 5 和 6。

图 6-33 显示了 solution5 和 solution6 的对比结果。你可以看到间隔已经有了很大的改善。

Performance Estimates				
Timing (ns)				
Clock		solution5	solution6	
	ap_clk	Target	8.00	8.00
		Estimated	6.904	6.904
Latency (clock cycles)				
Latency	min	476	463	
	max	476	463	
Interval	min	343	98	
	max	343	98	

图 6-33:DCT Solution5 与 Solution6 的对比

这个时间间隔(interval)现在低于 125 的目标时间。这种设计可以每 98 个时钟周期接受一组新的输入数据。

结论

在本教程中，您学习了：

- 如何使用分析视角分析设计。
- 如何使用 C 代码交叉链接视图中的操作。
- 如何应用和判断优化。
- 获取初始设计结果并创建满足设计目标的实现的方法。

第七章

优化设计

概述

使用高级综合创建高质量 RTL 设计的一个关键部分是能够对 C 代码应用优化。高级综合总是试图**最小化循环和函数的延迟**。为了实现这一点，在循环和函数中，它尝试并行执行尽可能多的操作。在函数级别，高级合成总是试图并行执行函数。

除了这些**自动优化**，指令还用于：

- **并行执行多个任务**，例如同一个函数的多次执行或同一个循环的多次迭代。这是**流水线**。
- **重组数组(块 ram)、函数、循环和端口的物理实现**，以**提高数据的可用性，帮助数据更快地通过设计**。
- **提供关于数据依赖项或缺乏数据依赖项的信息**，以**便执行更多的优化**。

最后一种**优化技术是修改 C 源代码**，以删除代码中可能限制硬件性能的非预期依赖项。本教程由两个实验练习组成。您可以在这些实验练习中使用分析透视图执行分析。本教程的先决条件是完成第 6 章“设计分析教程”。

实验室 1 描述

对比使用循环和功能流水线来创建一个可以处理每个时钟一个样本的设计。这个实验包括一些例子，让您有机会**分析导致设计未能满足性能需求的两个最常见原因：循环依赖(loop dependencies) 和 数据流限制或瓶颈(data flow limitations or bottlenecks)**。

实验室 2 描述

这个实验展示了如何修改来自实验 1 的代码，以帮助克服代码中一些固有的、但并非故意的性能限制。

教程设计描述

您可以从 Xilinx 网站下载教程设计文件。请参阅定位教程设计文件中的信息。对于本教程，您将使用教程目录中的设计文件

Vivado_HLS_Tutorial \ Design_Optimization。

在实验练习中使用的示例设计是一个矩阵乘子函数。设计目标是在每个时钟周期处理一个新样本，并将接口实现为流数据接口。

实验 1: 优化矩阵乘法器 (*Matrix Multiplier*)

本练习使用矩阵乘法器设计来展示如何充分优化基于循环的设计。设计目标是使用 FIFO 接口在每个时钟周期中读取一个样本，同时最小化该区域。

分析包括在循环级别优化的方法与在功能级别优化的方法的比较。

重点:本教程中的图形和命令使用教程数据目录



将 *Vivado HLS Tutorial* 解压缩并放置在位置 *C:\Vivado HLS Tutorial* 中。如果将教程数据目录解压缩到不同的位置, 或者在 Linux 系统上, 请将引用的少数路径名调整到您选择的位置, 以放置 *Vivado HLS Tutorial* 目录。

步骤 1: 创建并打开项目

1. 打开 Vivado HLS 命令提示符。

在 Windows 上使用 **Start > All Programs > Xilinx Design Tools > Vivado 2018.3 > Vivado HLS > Vivado HLS 2018.3 Command Prompt**

在 Linux 上, 打开一个新的 shell。

2. 使用命令提示窗口(图 7-1), 将目录更改为 ***Design Optimization tutorial/lab1***。

3. 使用命令 ***vivado_hls -f run_hls.tcl*** 执行 Tcl 脚本来设置 Vivado HLS 项目, 如图 7-1 所示。

```
C:\Uivado_HLS_Tutorial>cd Design_Optimization
C:\Uivado_HLS_Tutorial\Design_Optimization>cd lab1
C:\Uivado_HLS_Tutorial\Design_Optimization\lab1>vivado_hls -f run_hls.tcl
```

图7-1: 设置 *Design Optimization Tutorial Project*

4. Vivado HLS 完成后, 使用图 7-2 所示的命令在 Vivado HLS GUI 中打开项目。
vivado_hls -p matrixmul_prj

```
@I [HLS-10] Creating and opening solution 'C:/Uivado_HLS_Tutorial/Design_Optimization/lab1/matrixmul_prj/solution1'.
@I [HLS-10] Cleaning up the solution database.
@I [HLS-10] Setting target device to 'xc7k160tfbg484-1'
@I [SYN-201] Setting up clock 'default' with a period of 13.3333ns.
Compiling ../../../../matrixmul_test.cpp in debug mode
Compiling ../../../../matrixmul.cpp in debug mode
Generating csim.exe
Test passes.
@I [SIM-1] CSim done with 0 errors.
@I [LIC-101] Checked in feature [HLS]
C:\Uivado_HLS_Tutorial\Design_Optimization\lab1>vivado_hls -p matrixmul_prj
```

图7-2: 实验室 1 的开放式设计优化项目

5. 展开“资源管理器”窗格中的“源代码”文件夹, 双击 ***matrixmul.cpp*** 以查看源代码(图 7-3)。

向下滚动文件, 可以看到源代码有两个**输入数组 *a* 和 *b***, 以及**输出数组 *res***。将鼠标放在宏上(如图 7-3 所示), 可以看到每个都是 3*3, 总共有 9 个元素。

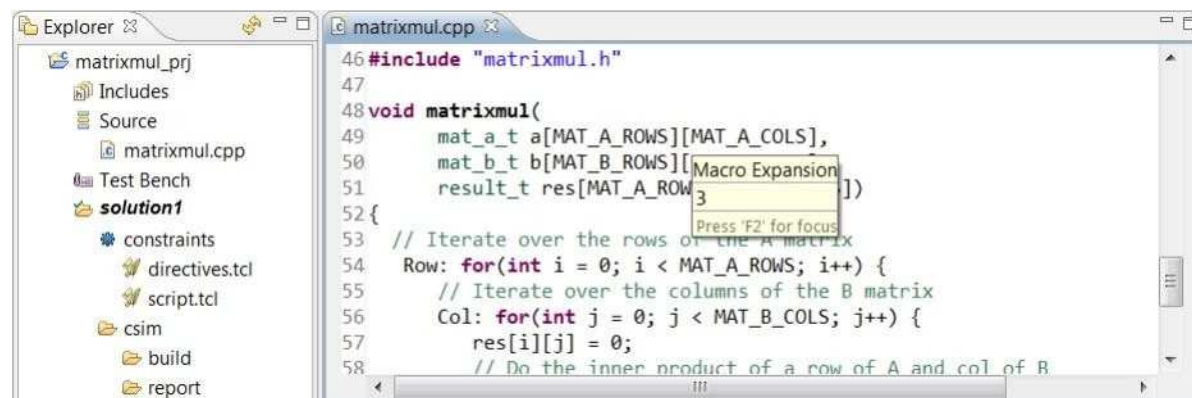


图 7-3: 矩阵乘法器的源代码

第二步: 综合分析设计

1. 单击 **Run C Synthesis** 工具栏按钮，将设计合成为 RTL。
当综合完成时，将打开综合报告(图 7-4)，并出现性能评估:

- 间隔是 80 个时钟周期。因为每个输入数组中有 9 个元素，所以设计每个输入读大约需要 9 个周期。
- 间隔(*interval*)时间比延迟(*latency*)时间长一个周期，因此此时硬件中不存在并行性。
- 延迟/间隔是由于嵌套循环。
 - ◆ 内部循环称为 **Product**:
 - 有 2 个时钟周期的延迟。
 - 所有迭代共有 6 个时钟周期。
 - ◆ **Col 循环**:
 - 需要 1 个时钟进入循环 Product，1 个时钟退出。
 - 每次迭代需要 8 个时钟周期 (1+6+1)。
 - 有 24 个周期的所有迭代完成。
 - ◆ **顶级循环的延迟**为每次迭代 26 个时钟周期，对于循环的所有迭代总共有 78 个时钟周期。

发送反馈

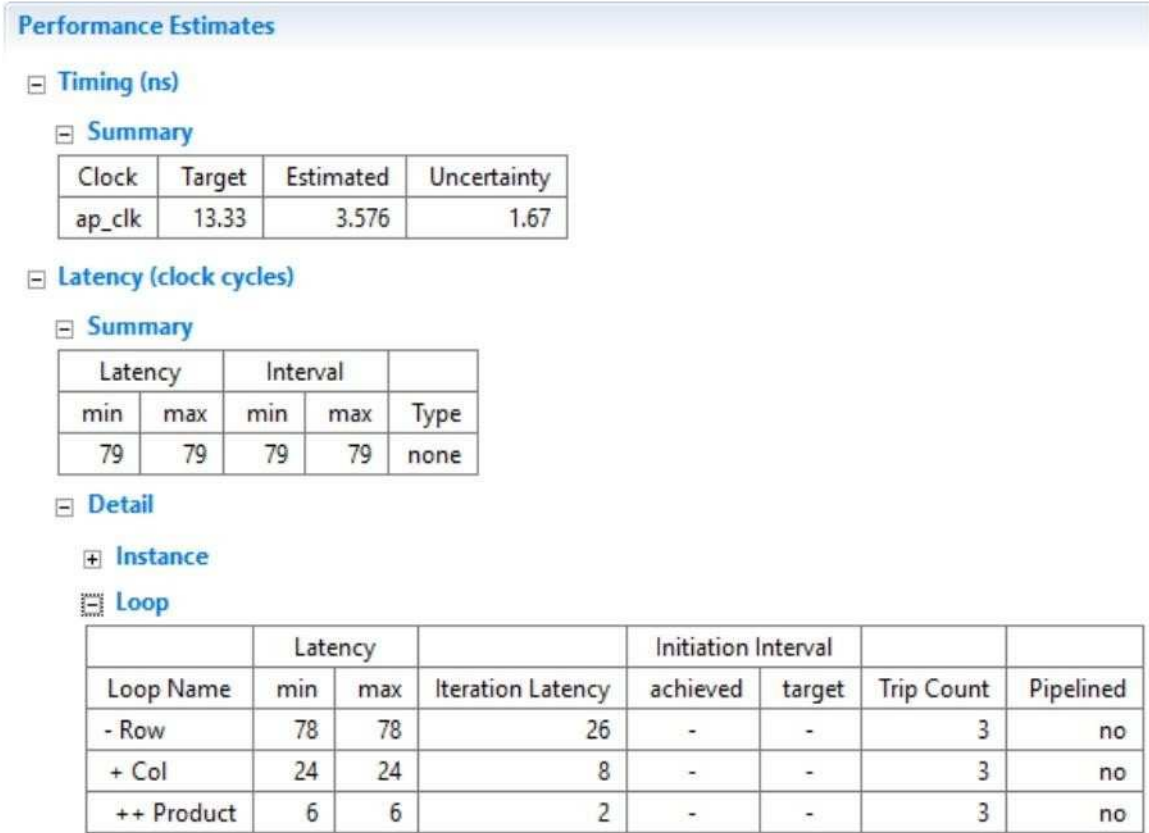


图 7-4: 矩阵乘法器的合成报告

可以通过以下两种方法之一来提高启动时间间隔：**对循环进行流水线操作，或者对整个函数进行流水线操作**。您可以首先对循环进行流水线操作，然后将这些结果与整个函数的流水线操作进行比较。

在流水线循环时，循环的启动间隔(*initiation interval*)是监控的重要指标。从本练习中可以看出，即使当设计达到循环可以在每个时钟周期处理一个样本的阶段时，该函数的启动间隔仍会报告为该函数中包含的循环完成所有处理所花费的时间。

步骤 3: 对 Product 循环进行流水线操作

1. 选择 “**New Solution**” 工具栏按钮或使用菜单 **Project > New Solution** 创建新解决方案，*solution2*。
2. 单击 **Finish** 并接受要创建的默认值

3.确保 C 源代码在信息窗格中可见。

当流水化嵌套循环时，最大的好处是流水化最内部的循环，它处理一个数据样本。高级综合 **自动**应用**循环扁平化、折叠嵌套循环、删除循环转换(本质上是创建一个迭代更多但总体时钟周期更少的循环)**。

4.在指令标签:

- a.选择 loop **Product**。
- b.右击并选择 **Insert** 指令。
- c. 在“指令编辑器”对话框中，激活顶部的“指令”下拉菜单并选择“**PIPELINE**”。
- d. 单击 **OK**。使用默认选项，**初始化间隔(II)**为 1(每个时钟一个新的循环迭代)将是默认值。

发送反馈



指令窗格应该显示以下优化指令。(新指令被高亮显示)

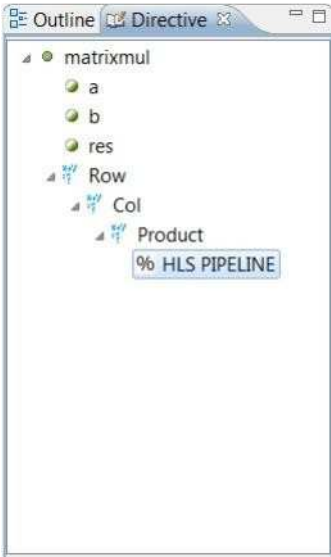


图 7-5: 初始流水线指令

5.单击 **Run C Synthesis** 工具栏按钮，将设计合成为 RTL。

在综合期间，控制台窗格中报告的信息显示在循环 **Row** 上执行了**循环压扁 (loop flattening)**，并且由于依赖项，在**循环积 (loop product)** 上无法实现默认的初始化内部目标 1。

信息:[XFORM 203-541]在函数'matrixmul'中平展一个嵌套'Row' (matrixmulp .cpp:54:37)。

信息:[SCHED 204-61]管道循环“产品”。
警告:[SCHED 204-68]无法强制附带依赖约束(II = 1，距离=1)
变量“tmp_8”的“存储”操作(matrixmul.cpp:60)， matrixmul。数组'res'上的
的 cpp:60 和数组'res'上的'load'操作('res_load', matrixmul.cpp:60)。信
息:[SCHED204-61]流水线结果:目标 II: 1，最终 II: 2，深度:2。

综合报告(图 7-6)显示，**虽然 Product 循环是流水线的，间隔为 2，但是顶层循环的间隔不是流水线的。**

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	13.33	4.306	1.67

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
82	82	82	82	none

Detail

Instance

Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- Row_Col	81	81	9	-	-	9	no
+ Product	6	6	2	2	1	3	yes

图 7-6:Matrixmul 初始管线报告

顶层循环没有流水线的**原因**是循环扁平化只发生在循环 **Row_Col** 上。在 **Product** 循环中没有出现循环压扁现象。要理解为什么循环扁平化不能使所有嵌套循环扁平化，请使用分析透视图。

- 6.打开分析透视图。
- 7.在 Performance 视图中，展开循环 **Row_Col** 和 **Product**。
- 8.在状态 **C1** 中选择**写操作**。
- 9.右键单击并选择 **Goto Source** 查看图 7-7 中的视图。

状态 C1 中的写操作是由于在 **Product** 循环之前将 **res** 设置为 **0** 的代码造成的。因为 res 是一个顶层函数参数，所以它是对 RTL 中的一个端口的写操作：这个操作必须在执行**循环 Product** 操作之前发生。因为它不是一个内部操作，但是对 I/O 行为有影响，所以不能移动或优化这个操作。这阻止 Product 循环被压扁到 Row_Col 循环中。

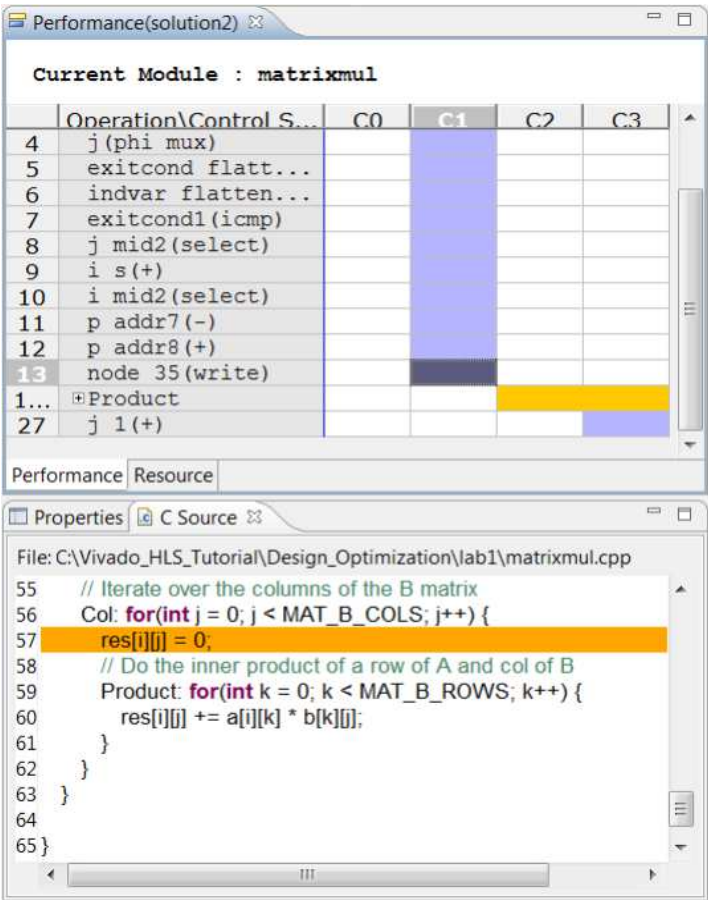


图 7-7:Matrixmul 初始性能视图

更重要的是，值得指出的是，为什么 **Product** 循环只有 2 的启动间隔(II)是可能的(如图 7-6 所示)。

控制台窗格中的消息 SCHED-68(并告诉您:file vivado_hls.log)

警告:[SCHED 204-68]无法强制附带依赖约束(II = 1，距离=1)

变量“tmp_8”的“存储”操作(matrixmul.cpp:60)，matrixmul。数组'res'上的cpp:60和数组'res'上的'load'操作('res_load'，matrixmull.cpp:60)。

- ◆ 问题是携带的依赖关系。这是循环的一次迭代中的操作与同一循环的不同迭代中的操作之间的依赖关系。例如，当 $k = 1$ 且 $k = 2$ 时的操作（其中 k 是循环索引）。
- ◆ 第一个操作是第 60 行数组 **res** 的加载(内存读取操作)。
- ◆ 第二个操作是第 60 行数组 **res** 上的存储(内存写操作)。

从图 7-8 中可以看到，第 60 行是从数组 **res** 读取（由于 += 运算符）和对数组 **res** 的写入。默认情况下，阵列映射到 **Block RAM**，并且“性能视图”中的详细信息可以显示发生此冲突的原因。

Performance 视图显示了调度操作的状态。图 7-8 显示其中两个操作要为 II 违规负责。这些操作在循环迭代之间具有相关性。分析视图提供了将分析视图过滤到导致 II 冲突的操作的功能。要使用此功能，请在筛选器下拉组合框中选择 II 违例。

循环的第一次迭代显示操作发生的状态。状态 2 和 3 的读，状态 3 的写。下一个迭代中的操作必须在此之后开始一个循环，因为第 2 次读操作必须等到第 1 次写操作完成后才能进行:循环的每个迭代中的操作针对的是不同的地址，同时只能应用一个地址。

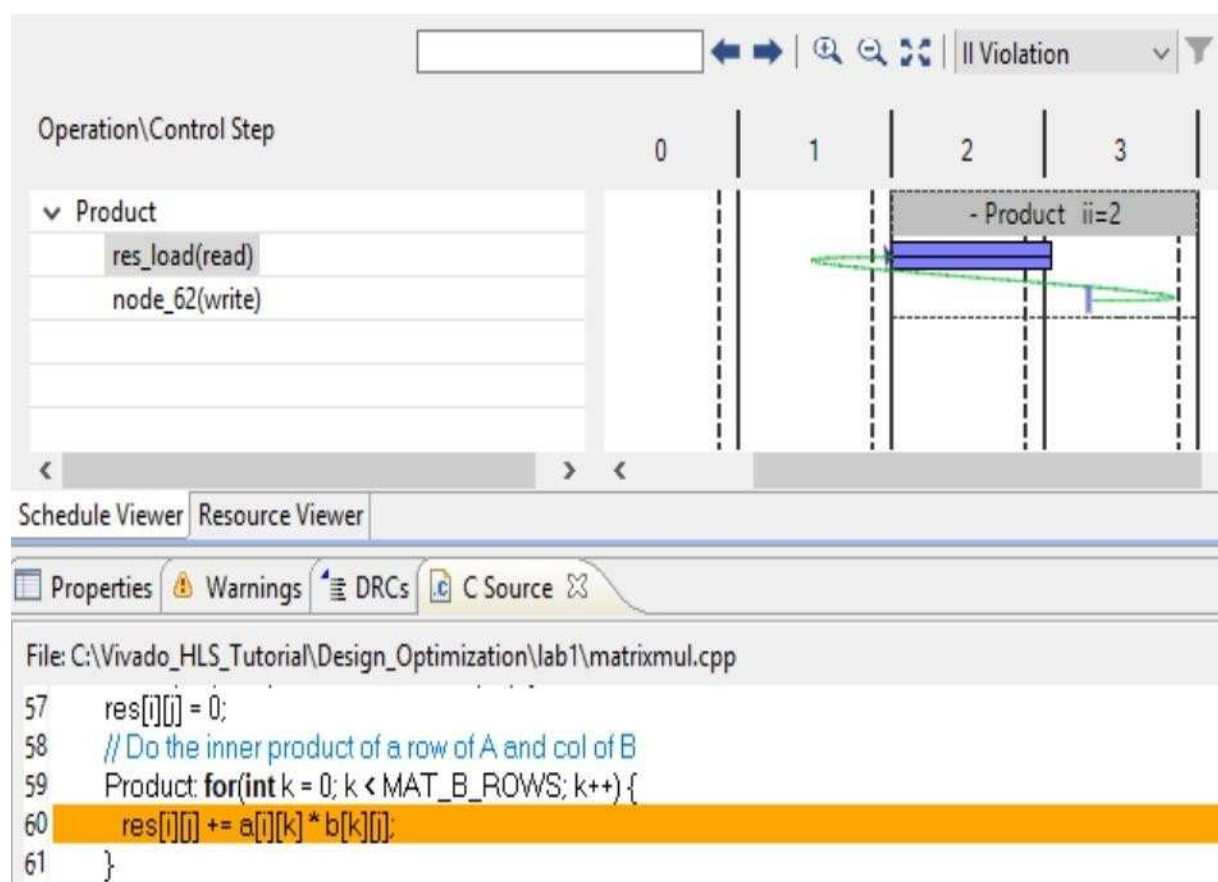


图 7-8: 进行依赖分析

您不能将 **Product** 循环的启动间隔设置为 1。下一个实验练习展示了如何重写代码来消除这个限制。在这个实验练习中，您将继续优化代码。

下一步是用 pipeline 输送上面的循环，**Col** 循环。这将自动展开 **Product** 循环，并创建更多的操作符，从而产生更多的硬件资源，但是这将确保 **Product** 循环的不同迭代之间不存在依赖关系。

10. 返回到综合透视图。

步骤 4: 对 Col 循环进行流水线操作

1. 选择 “**New Solution**” 工具栏按钮创建一个新解决方案 “解决方案 3”。
2. 因为 **solution2** 已经添加了一个指令，所以使用下拉菜单进行选择 **solution1** 作为现有指令和约束的来源(**solution1** 没有)。
3. 单击 **Finish** 并接受默认的 解决方案名称 **solution3**。
4. 打开 C 源代码 **matrixmul.cpp**，使其在信息窗格中可见。
5. 在指令标签:
 - a. 选择 loop **Col**。
 - b. 右击并选择 **Insert** 指令。
 - c. 在 “指令编辑器” 对话框中，激活顶部的 “指令” 下拉菜单并选择 “**PIPELINE**”。
 - d. 单击 **OK**。使用默认选项，初始化间隔(II)为 1(每个时钟一个新的循环迭代)将成为默认值。

如下所示，指令窗格显示了以下优化指令(突出显示了新指令)。

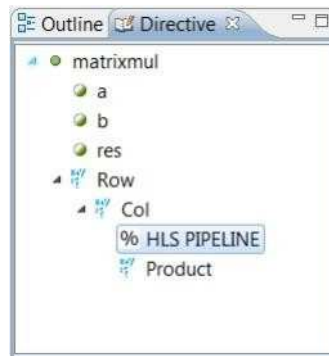


图 7-9: Col 管线指示

6. 单击 **Run C Synthesis** 工具栏按钮，将设计综合为 RTL。

在综合过程中，控制台窗格中报告的信息显示，循环 **Product** 被展开，循环压扁在循环 **Row** 上执行，并且由于 **数组 a** 的内存资源限制，在循环 **Row_Col** 上无法实现默认的初始化内部目标 1。

```

信息:[XFORM 203-502]在函数'matrixmul'中展开所有子循环的内部循环
'Col' (matrixmul.cpp:56)进行管道操作。
信息:[XFORM 203-501]在函数 “matrixmul” 中完全展开循环
“Product” (matrixmul.cpp:59)。
信息:[XFORM 203-541]在函数'matrixmul'中平展一个嵌套'Row' (matrixmul.cpp:54:37)。
---
---
信息:[SCHED 204-61]流水线循环'Row_Col'。
警告:[SCHED 204-69]由于内存端口有限，无法在数组 a 上调度'load'操作
('a_load_1', matrixmul.cpp:60)。

信息:[SCHED 204-61]流水线结果:目标 II: 1，最终 II: 2，深度:3。
  
```

如前所述，查看综合报告可以发现，循环 **Row_Col** 的时间间隔只有两个：**目标是每个循环处理一个样本**。同样，您可以使用 **Analysis** 透视图来强调为什么没有实现启动目标。

7. 打开分析透视图。

8. 在 Performance 视图中，展开循环 **Row_Col**

图 7-10 突出显示了对 **数组 a** 的操作(在上面的 SCHED-69 消息中提到)。**数组 a** 上有三个读操作，从 C1 到 C3 的每个状态都有一个操作。

数组被实现为块 RAM，作为函数参数的数组被实现为块 RAM 端口。在这两种情况下，

一个块 RAM 最多只能有两个端口(对于双端口块 RAM)。通过一个块 RAM 接口访问数组 a，没有足够的端口能够在一个时钟周期内读取所有三个值。

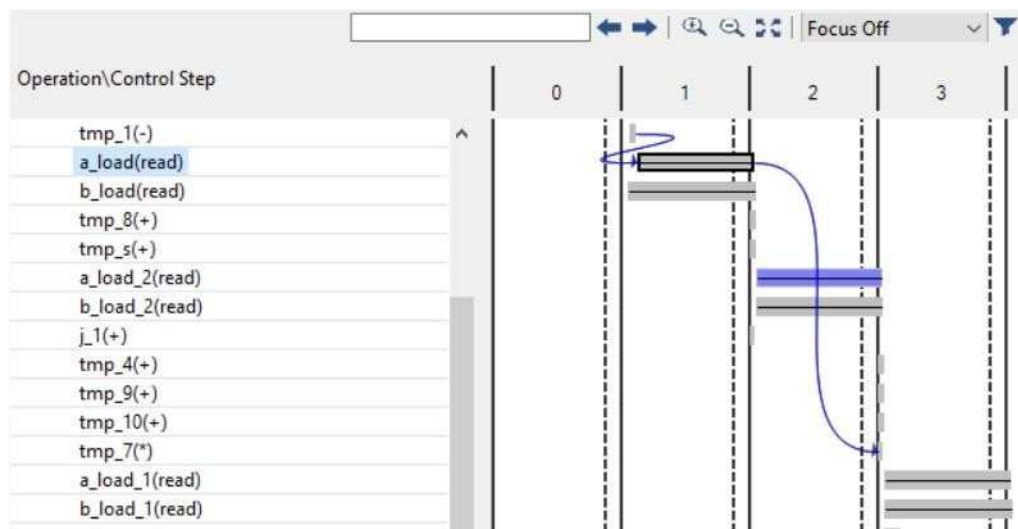


图7-10 查看此资源限制的另一种方法是使用 resource 窗格。

9. 单击 **Resource** 选项卡。

10. 展开内存端口 (**memory ports**) 以查看图 7-11 所示的视图。

在图 7-11 中，状态 C1 中的两周期读取操作与状态 C2 中开始的两周期读取操作重叠，因此只有一个周期可见：但是，很明显，这个资源在多个状态中使用。

在查看这个视图时，很明显，即使解决了端口 a 的问题，端口 b 也会出现同样的问题：它还必须执行 3 次读取。

高级综合一次只能报告一个进度错误或警告，因为一旦第一个问题出现，创建可实现的进度的操作将使任何其他不可行的进度失效。

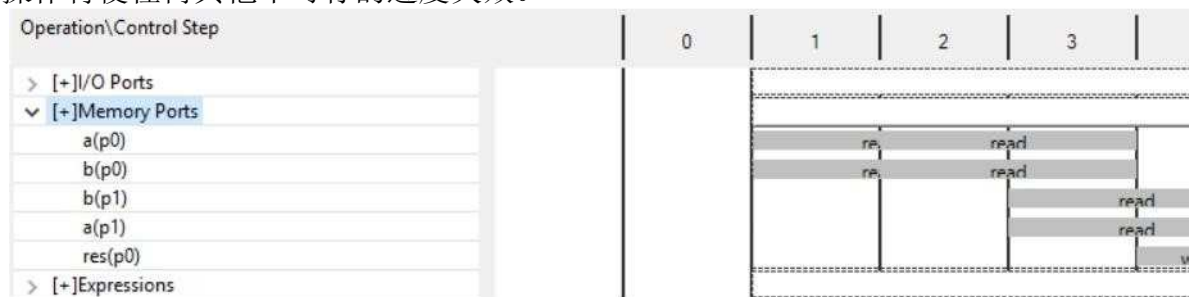


图 7-11: 矩阵 mul 管道 Col 资源视图

高级综合允许对数组进行分区 (partition)、映射 (map) 和重新塑造 (reshape)。这些技术允许在不更改源代码的情况下修改对数组的访问。

11. 返回到综合透视图。

步骤 5: 重新排列 (Reshape) 数组

1. 选择 “**New Solution**” 工具栏按钮或使用 **Project > New Solution** 创建新解决方案，**solution4**。

2. 单击 Finish 并接受默认的解决方案名称 **solution4**。

因为 **Product 循环** 的循环索引是 **k**，所以两个数组应该按照它们各自的 **k** 维进行分区：设计需要在每个时钟周期中访问两个以上的 **k** 值。

对于数组 a，这是 **维度 2**，因为其访问模式为 **a[i][k]**；对于数组 b，这是 **维度 1**，因为其访问模式为 **b[k][j]**。

对这些数组进行分区将创建 **MAT_A_COLS** 数组—在本例中为 MAT_A_COLS 数字端口。或者，我们可以使用 **重新塑造来代替分区**，允许创建一个宽数组(端口)来代替 **k** 端口。

在这个转换之后，这个块之外的块 RAM 中的数据必须以相同的方式进行重塑形：如果这个过程不是由 HLS 完成的，那么数据必须按照如下方式排列：

- 对于数组 a: *MAT_A_ROWS* 元素，每个元素的宽度 *data_word_size* 乘以 *MAT_A_COLS*。
- 对于数组 b: *MAT_B_COLS* 元素，每个元素的宽度 *data_word_size* 乘以 *MAT_B_COLS*。

3. 打开 C 源代码 *matrixmul.cpp*，使其在信息窗格中可见。

发送反馈



4.在指令标签，做如下：

- a.选择变量 *a*。
- b.右击并选择 *Insert* 指令。
- c.在指令编辑器对话框中激活指令的下拉菜单 顶部并选择 *ARRAY_RESHAPE*。
- d.设置 *维度为2*。
- e. 单击 *OK*。

5.对变量 *b* 重复此过程，但将维度设置为 1。

指令窗格应该显示以下优化指令。

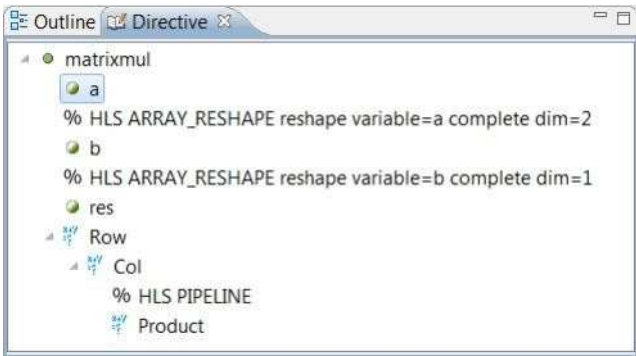


图 7-12: 阵列整形指令

6.单击 *Run C Synthesis* 工具栏按钮，将设计合成为 RTL。综合报告显示，顶层循环*Row_Col* 现在正在每个时钟周期处理一个样本的数据(图 7-13)。

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	13.33	7.566	1.67

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
11	11	11	11	none

Detail

Instance

Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- Row_Col	9	9	2	1	1	9	yes

图 7-13: 优化的循环处理报告

- 顶层模块需要 11 个时钟周期才能完成。

- *Row_Col* 循环在 2 个周期(迭代延迟)后输出一个样本。
- 然后每循环读取 1 个样本(起始时间间隔)。
- 经过 9 次迭代/采样(行程计数)后, 它完成所有的采样。
- $2 + 9 = 11$ 个时钟周期。

然后该函数可以完成并返回开始处理下一组数据。

现在, 将块 RAM 接口更改为 FIFO 接口, 以允许流数据(streaming data)。

步骤 6:应用 FIFO 接口

- 1.选择 “*New Solution*” 工具栏按钮以创建新解决方案。
- 2.单击 Finish 并接受默认的解决方案名称 *solution5*。
- 3.打开 C 源代码 *matrixmul.cpp*, 使其在信息窗格中可见。
- 4.在指令标签, 做如下:
 - a.选择变量 *a*。
 - b.右击并选择 Insert 指令。在指令编辑器对话框中激活顶部的指令下拉菜单并选择 *INTERFACE*。
 - c.点击 *mode* 下拉菜单进行选择 *ap_fifo*。
 - d.单击 *OK*。
- 5.对变量 *b* 和变量 *res* 重复此过程

指令窗格显示以下优化指令。(新指令被突出显示)。

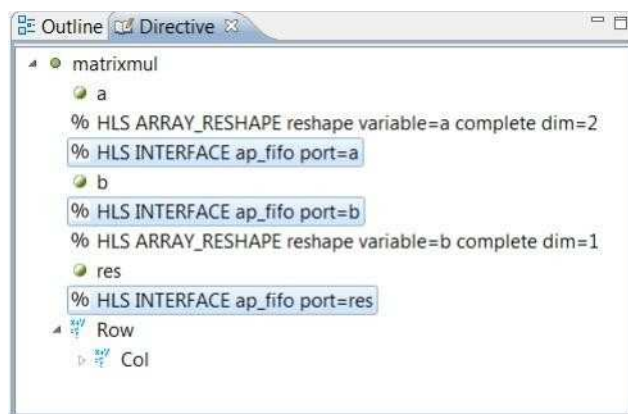


图 7-14:Matrixmul FIFO 指令

- 6.单击 *Run C Synthesis* 工具栏按钮, 将设计合成到 RTL。图 7-15 显示了综合运行后的控制台显示。

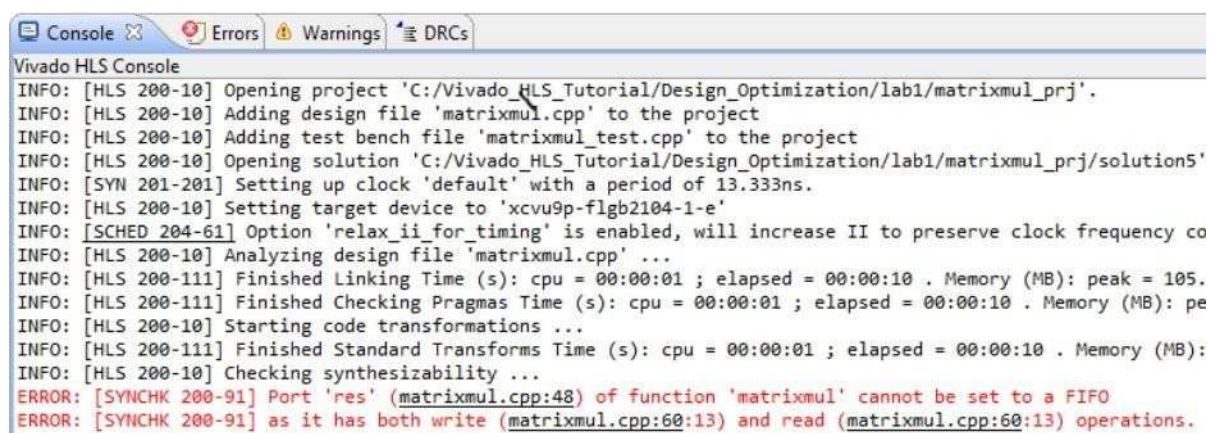


图 7-15:FIFO 合成警告

从图 7-16 所示的代码中, 数组 *res* 按如下顺序执行写操作(*MAT_B_COLS=MAT_B_ROWS=3*)

- 在第 57 行上写入[0][0]。
- 然后在第 60 行写[0][0]。
- 然后在第 60 行写[0][0]。
- 然后在第 60 行写[0][0]。
- 在第 57 行写到[0][1](在索引 J 增量之后)。
- 然后在第 60 行写[0][1]。

发送反馈



连续四次写地址[0][0]不构成流访问模式；这是随机存取。

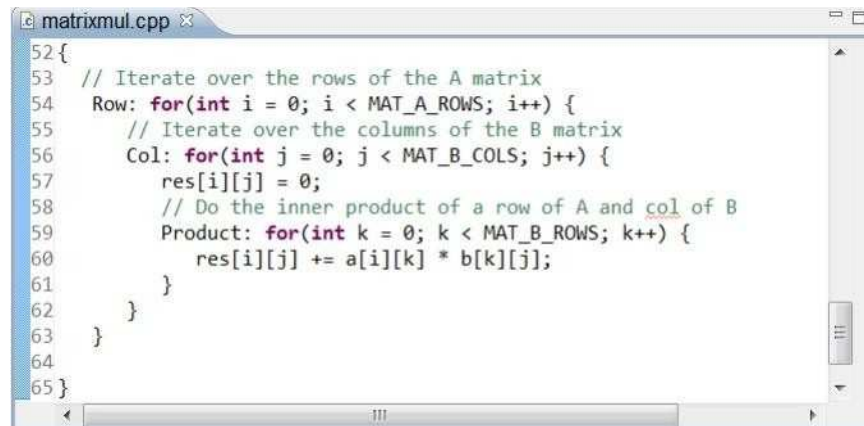


图 7-16: Matrixmul 代码

检查图 7-16 中的代码可以发现, 读取数组 *a* 和 *b* 存在类似的问题。不能使用 FIFO 接口通过已编写的代码进行数据访问。要使用 FIFO 接口, Vivado 高级综合中可用的优化指令是不充分的, 因为该代码当前强制执行一定的读取和写入顺序。进一步的优化需要重新编写代码, 您可以在实验 2 中完成该代码。

然而, 在修改代码之前, 值得对函数进行流水线操作, 而不是对循环进行流水线操作, 以对比这两种方法的不同之处。

步骤 7: 流水线 (Pipeline) 函数

1. 选择 “*New Solution*” 工具栏按钮创建一个新解决方案 “解决方案 6”。

重点: 在这个步骤中, 复制 *solution4* 中的指令, 因为这个解决方案没有指定 FIFO 接口。



2. 从 “选项” 部分的下拉菜单中选择 “解决方案 4”。解决方案向导如图 7-17 所示。

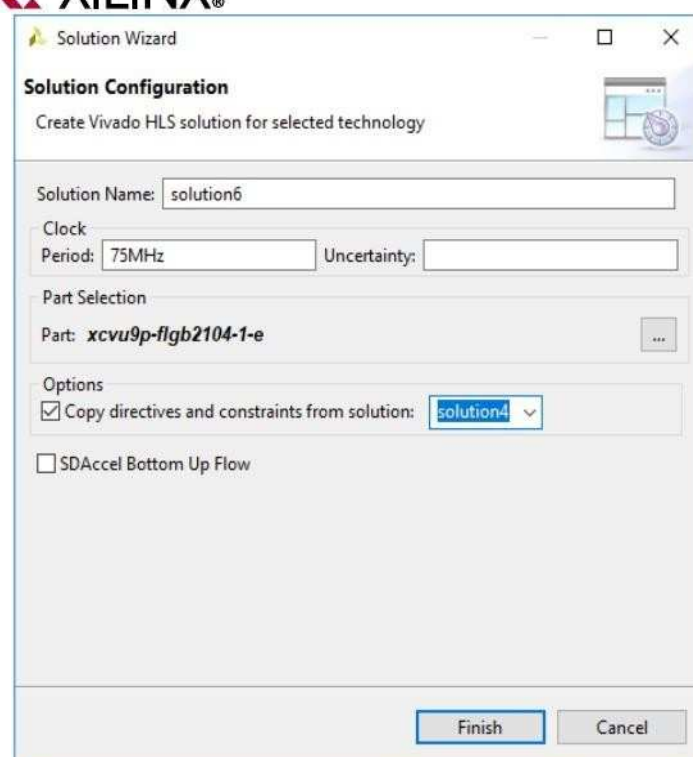


图 7-17: 基于 Solution4 directive 3 的新解决方案

3. 单击 **Finish** 并接受默认的解决方案名称，**solution6**。
4. 打开 C 源代码 **matrixmul.cpp**，使其在信息窗格中可见。
5. 在指令标签：
 - a. 选择 loop **Col** 上的 **pipeline** directive。
 - b. 右键单击并选择 **Remove** 指令。
 - c. 选择顶级函数 **matrixmul**。
 - d. 右键单击并选择 **Insert** 指令。
 - e. 在指令编辑器对话框中激活指令下拉菜单顶部并选择 **pipeline**。
 - f. 单击 **OK**。

选项卡应该如图 7-18 所示。

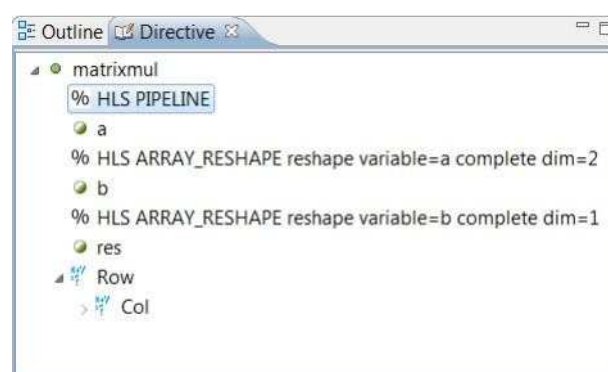


图 7-18: 解决方案的指示

6. 单击 **Run C Synthesis** 工具栏按钮，将设计合成为 RTL。
7. 单击 “**Compare Reports**” 工具栏按钮。
 - a. 添加 solution4。
 - b. 添加 solution6。
 - c. 单击 OK。

方案 4 和方案 6 的比较如图 7-19 所示。

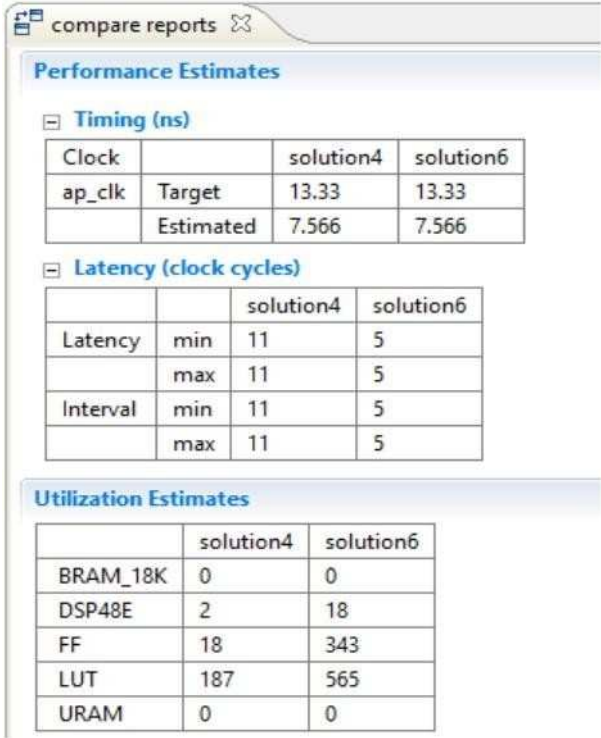


图 7-19: 循环与功能流水线

现在设计完成在更少的时钟，可以开始一个新的事务每 5 个时钟周期。**然而，面积和资源已大幅增加，因为所有的循环在设计中展开。**

信息:[XFORM 203-502]在函数 “matrixmul” (matrixmul.cpp:49)中展开所有管道循环。信息:[HLS 200-489]展开循环'行' (matrixmul.cpp:54)在函数'matrixmul'完全与 3 的因素。

INFO: [HLS 200-489]在函数 “matrixmul” 中完全展开循环 “Col” (matrixmul.cpp:56)，系数为 3。

[HLS 200-489]在函数 “matrixmul” 中完全展开循环 “Product” (matrixmul.cpp:59)，系数为 3。

流水线循环允许循环保持滚动，因此提供了控制区域的好方法。当流水线操作一个函数时，函数中包含的所有循环都被展开，这是流水线操作的要求。**流水线功能设计可以每 5 个时钟周期处理一组新的 9 个样本。**这超出了每个时钟 1 个样本的要求，因为高级综合的默认行为是生成性能最高的设计。

流水线功能实现了最佳性能。但是，如果它超过了所需的性能，可能需要多个额外的指令来降低设计速度。

Pipelining loops 为您提供了一种控制资源的简单方法，并提供了部分展开设计以满足性能的选项。

实验 2：为 I/O 存取优化的 C 代码

在 *lab1* 中，您无法使用流接口。**C 代码的性质**（指定对同一地址的多次访问，阻止了流接口的应用）

- 在**流接口**中，必须**按顺序**访问值。
- 在代码中，访问也是**端口访问，高级综合无法移动和优化端口访问**。指定的 C 代码在每个 **Product 循环**开始时将零值写入**端口 res**。这可能是预期行为的一部分。HLS 无法简单地决定更改算法的规范。

该代码直观地捕获了矩阵乘法的行为，但它阻止了硬件中必需的行为：**流访问**。本实验练习使用您在实验 1 中使用的 C 代码的更新版本。以下说明了如何更新 C 代码。下面解释如何更新 C 代码。

图 7-20 显示了 *lab1* 中的代码的 I/O 访问模式。图 7-21 显示了 *lab1* 中代码的 I/O 访问模式。

地址值不必用小字体显示。

当变量 *i*, *j* 和 *k* 从 0 迭代到 3 时, 图 7-20 的下部显示了生成的用于读取 *a*, *b* 和写入 *res* 的地址。另外, 在每个 *Product* 循环的开始, *res* 都设置为零值。

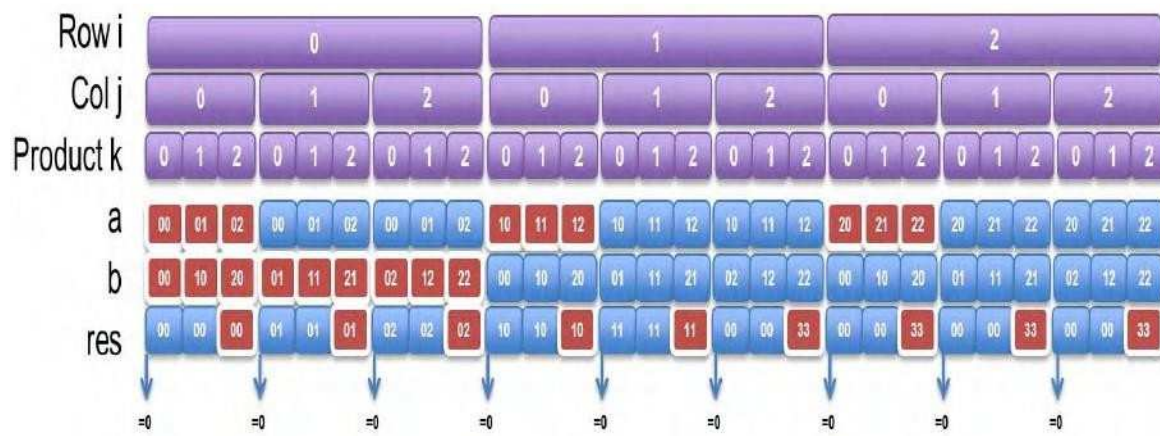


图 7-20: 矩阵乘法器地址访问

要进行具有顺序流访问的硬件设计, 端口访问只能是以红色突出显示的端口。对于读取端口, 必须在内部缓存数据, 以确保设计不必重新读取端口。对于写入端口 *res*, 必须将数据保存到一个临时变量中, 并且仅在红色显示的周期内将其写入端口。

这个实验室的 C 代码反映了这种行为。

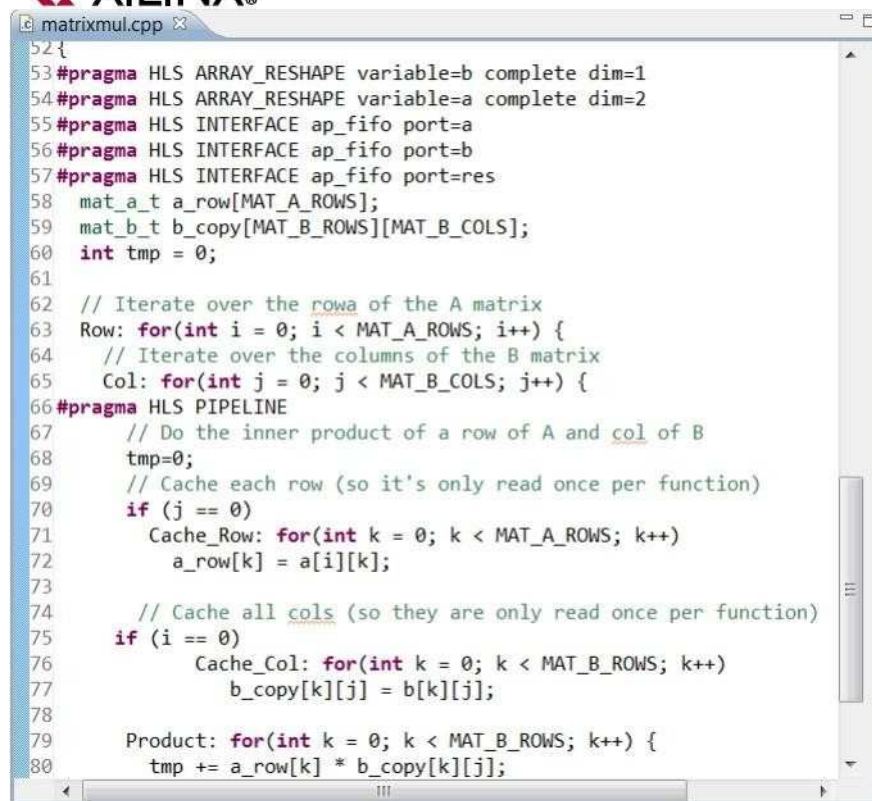
步骤 1: 创建并打开项目

1. 从 *lab1* 中使用的 Vivado HLS 命令提示符, 切换到 *lab2* 目录, 如图 7-21 所示。
2. 通过键入 `vivado_hls -f run_hls.tcl` 创建一个新的 Vivado HLS 项目。

```
C:\Uivado_HLS_Tutorial\Design_Optimization\lab1>cd ..
C:\Uivado_HLS_Tutorial\Design_Optimization>cd lab2
C:\Uivado_HLS_Tutorial\Design_Optimization\lab2>vivado_hls -f run_hls.tcl
```

图 7-21: 界面合成实验室 2 的设置

3. 输入 `vivado_hls -p matrixmul_prj` 打开 Vivado HLS GUI 项目。
4. 在资源管理器窗格中打开源文件夹并双击打开 `matrixmul.cpp` 代码, 如图 7-22 所示。



```
52{
53 #pragma HLS ARRAY_RESHAPE variable=b complete dim=1
54 #pragma HLS ARRAY_RESHAPE variable=a complete dim=2
55 #pragma HLS INTERFACE ap_fifo port=a
56 #pragma HLS INTERFACE ap_fifo port=b
57 #pragma HLS INTERFACE ap_fifo port=res
58 mat_a_t a_row[MAT_A_ROWS];
59 mat_b_t b_copy[MAT_B_ROWS][MAT_B_COLS];
60 int tmp = 0;
61
62 // Iterate over the rows of the A matrix
63 Row: for(int i = 0; i < MAT_A_ROWS; i++) {
64     // Iterate over the columns of the B matrix
65     Col: for(int j = 0; j < MAT_B_COLS; j++) {
66 #pragma HLS PIPELINE
67         // Do the inner product of a row of A and col of B
68         tmp=0;
69         // Cache each row (so it's only read once per function)
70         if (j == 0)
71             Cache_Row: for(int k = 0; k < MAT_A_ROWS; k++)
72                 a_row[k] = a[i][k];
73
74         // Cache all cols (so they are only read once per function)
75         if (i == 0)
76             Cache_Col: for(int k = 0; k < MAT_B_ROWS; k++)
77                 b_copy[k][j] = b[k][j];
78
79         Product: for(int k = 0; k < MAT_B_ROWS; k++) {
80             tmp += a_row[k] * b_copy[k][j];
81         }
82     }
83 }
84 }
```

图 7-22: C 代码与更新的 I/O 访问

查看代码并确认以下内容:

- 来自 **lab1** 的指令, 包括 **FIFO** 接口, 在代码中被指定为 **pragmas**。
- **for 循环** 已被添加来缓存行和列的读取。
- **临时变量** 用于累加, 仅在为每个值计算最终结果时才写入端口 **res**。
- 由于用于缓存行和列的 for 循环将需要多个周期来执行读取, 因此 **pipeline** 指令已应用于 **Col** for 循环, 以确保**自动展开**这些缓存的 for 循环。

发送反馈

综合设计并使用联合仿真对 RTL 进行验证。

- 5.单击 **Run C Synthesis** 工具栏按钮, 将设计合成为 RTL。
- 6.综合完成后, 使用 **RunC/RTL CoSimulation** 工具栏按钮启动 **CoSimulation** 对话框。
- 7.单击 **OK** 启动 RTL 验证。设计现在已经完全综合, 读取一个样本每个时钟周期使用流式先进先出接口。

结论

在本教程中, 您学习了:

- ◆ 如何分析 **Pipeline 循环**, 并准确地理解哪些限制阻碍了优化目标的实现。
- ◆ 函数与循环 **pipeline** 的优缺点。
- ◆ 代码中的意外依赖关系如何阻止实现硬件设计目标, 以及如何通过对源代码进行修改来克服它们。

第八章

RTL 验证

概述

高级综合工具自动化了 RTL 验证过程，并允许您使用 RTL 验证来生成显示 RTL 设计中波形活动的跟踪文件。您可以使用这些波形来分析和理解 RTL 输出。**本教程涵盖了 RTL 验证过程的所有方面。**

要执行 RTL 验证，可以使用**高级综合的 RTL 输出**(*Verilog*、*VHDL* 或 *SystemC*)和 C 测试平台。RTL 验证通常称为 *CoSimulation* 或 *C/RTL CoSimulation*，因为验证中同时使用了 C 和 RTL。本教程包括三个实验练习。

实验室 1 描述

执行 RTL 验证步骤，并理解 C 测试平台在验证 RTL 中的重要性。

实验室 2 描述

创建 RTL 跟踪文件，并使用 Vivado 设计套件(*Suite*)分析它们。

实验室 3 描述

创建 RTL 跟踪文件，并使用第三方 RTL 模拟器分析它们。本实验室需要 **Mentor Graphics ModelSim** 模拟器的许可证。(您可以使用另一种第三方模拟器，只需对步骤进行少量修改)。

教程设计描述

您可以从 Xilinx 网站下载教程设计文件。请参阅定位教程设计文件中的信息。

本教程使用教程目录

Vivado_HLS_Tutorial\RTL_Verification 中的设计文件。

实验中使用的样本设计是一个 DUC(数字上变频器)功能。这个实验的目的是演示和解释 RTL 验证的特性。这些实验练习没有设计目标。

实验一:RTL 验证与 C 实验台

这个练习解释了 RTL 验证的基本操作，并强调了 C 测试平台的重要性。



重点:本教程中的图形和命令使用教程数据目录

将 Vivado_HLS_Tutorial 解压缩并放置在位置 C:\Vivado_HLS_Tutorial 中。如果将教程数据目录解压缩到不同的位置，或者在 Linux 系统上，请将引用的少数路径名调整到您选择的位置，以放置 Vivado_HLS_Tutorial 目录。

步骤 1:创建并打开项目

1.打开 Vivado HLS 命令提示符。

在 Windows 上使用 **Start > All Programs > Xilinx Design Tools > Vivado 2018.3 > Vivado HLS > Vivado HLS 2018.3 Command Prompt**

在 Linux 上，打开一个新的 shell。

2.使用命令提示窗口(图 8-1)，将目录更改为 **RTL_Verification /lab1**

3.使用图 8-1 所示的命令执行 Tcl 脚本来设置 Vivado HLS 项目。**vivado_hls -f run_hls.tcl**

```
C:\Uivado_HLS_Tutorial>cd RTL_Verification
C:\Uivado_HLS_Tutorial\RTL_Verification>cd lab1
C:\Uivado_HLS_Tutorial\RTL_Verification\lab1>vivado_hls -f run_hls.tcl
```

图8-1: 设置RTL验证教程项目

4.Vivado HLS 完成后，使用图 8-2 所示的命令在 Vivado HLS GUI 中打开项目。

vivado_hls -p duc_prj

```
@I [LIC-101] Checked in feature [HLS]
Generating csim.exe

*** DUC hardware test PASSED ! ***

@I [SIM-1] CSim done with 0 errors.
@I [LIC-101] Checked in feature [HLS]
C:\Uivado_HLS_Tutorial\RTL_Verification\lab1>vivado_hls -p duc_prj
```

图 8-2: 实验室 1 的开放 RTL 验证项目

步骤 2: 执行 RTL 验证

1.单击 **Run C Synthesis** 工具栏按钮，将设计综合为 RTL。

2.综合完成后，使用**Run C/RTL CoSimulation** 工具栏按钮(图 8-3)启动 CoSimulation 对话框。



图8-3: 运行C/RTL 协同仿真工具栏按钮

将打开 **CoSimulation** 对话框，如图 8-4 所示。

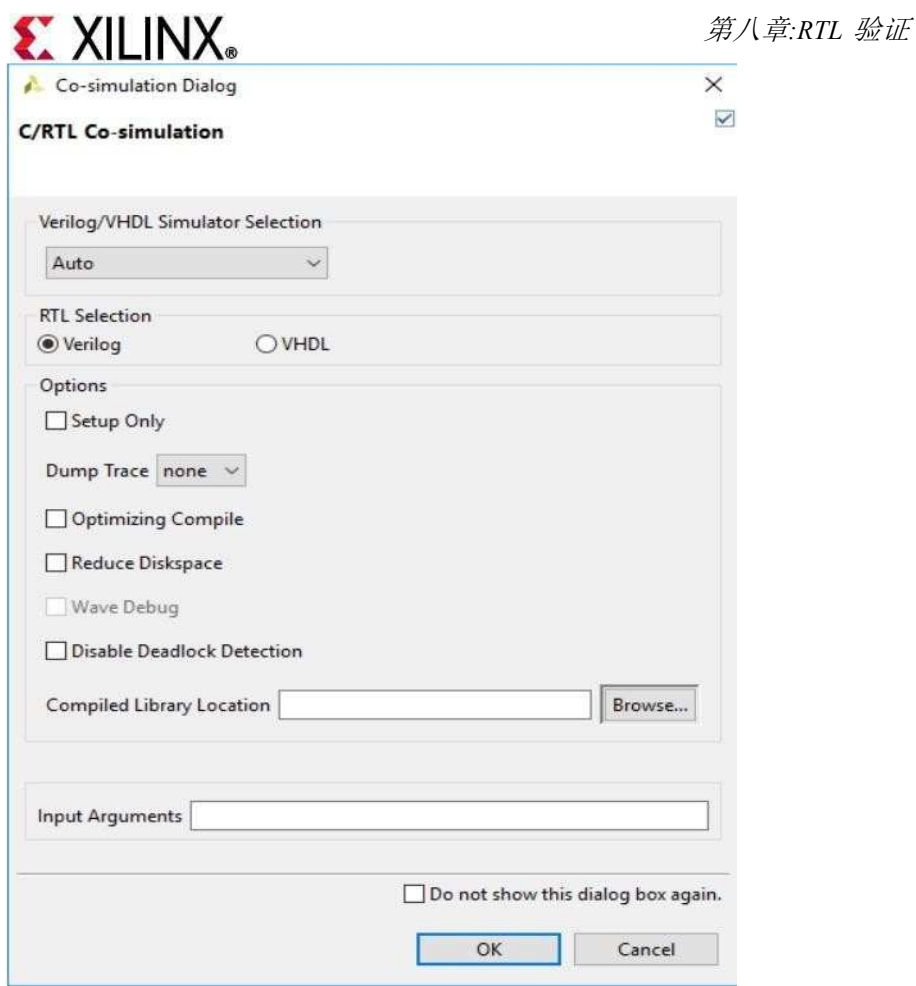


图 8-4:CoSimulation 对话框

下拉菜单允许您为 **HDL** 模拟选择 **RTL 模拟器**。对于本练习，使用 **Verilog RTL** 的默认自动选择(自动选择 Vivado 模拟器)进行联合模拟。

3.单击 **OK** 启动 RTL 验证。

当 RTL 验证完成时，模拟报告将自动打开(图 8-5)。报告指出模拟是否通过或失败。此外，报告指出了测量的延迟和间隔。

Cosimulation Report for 'duc'							
Result							
		Latency			Interval		
RTL	Status	min	avg	max	min	avg	max
VHDL	NA	NA	NA	NA	NA	NA	NA
Verilog	Pass	36	37	44	37	38	45

图 8-5:共仿真报告

RTL 模拟分三步完成。为了更好地理解 RTL 验证过程是如何执行的，在控制台窗口中向上滚动以确认下面描述的消息已经发出。

首先，执行 C 测试工作台为 RTL 设计生成输入刺激。

信息:[COSIM 212-14]仪器 C 测试台...< C 模拟生成输入刺激>

在这个阶段的最后，仿真显示了由 C 测试平台生成的任何消息。在这个阶段，C 函数的输出不会在 C 测试台中使用，但是可以在控制台中看到测试台中输出的任何消息。

信息:[COSIM 212-302]开始 C-TB 测试...

*** DUC 硬件测试通过!***

建立了一个具有新生成的输入刺激的 RTL 测试平台，并进行了 RTL 仿真。

信息:[COSIM 212-333]生成 C post check 测试台...信息:[COSIM 212-12]生成 RTL 测试台...

最后，将 RTL 仿真的输出重新应用到 C 测试台上以检查结果。同样，您可以在控制台中看到 C 测试工作台的任何消息输出。最后，如果 RTL 验证通过，RTL 验证将发出消息 SIM-1000。

信息:[COSIM 212-316]开始 C 后检查...*** DUC 硬件测试通过! * * *

信息:[COSIM 212-1000] *** C/RTL 共仿真完成:PASS ***

为了完全理解为什么 C 测试工作台应该检查结果，以及消息 SIM-1000 是如何生成的，您将修改 C 测试工作台。

步骤3:修改C测试工作台

- 1.在 Explorer 窗格中展开 *Test Bench* 文件夹(图 8-6)。
- 2.双击打开信息窗格中的 C 测试 *duc_test.c* 工作台。

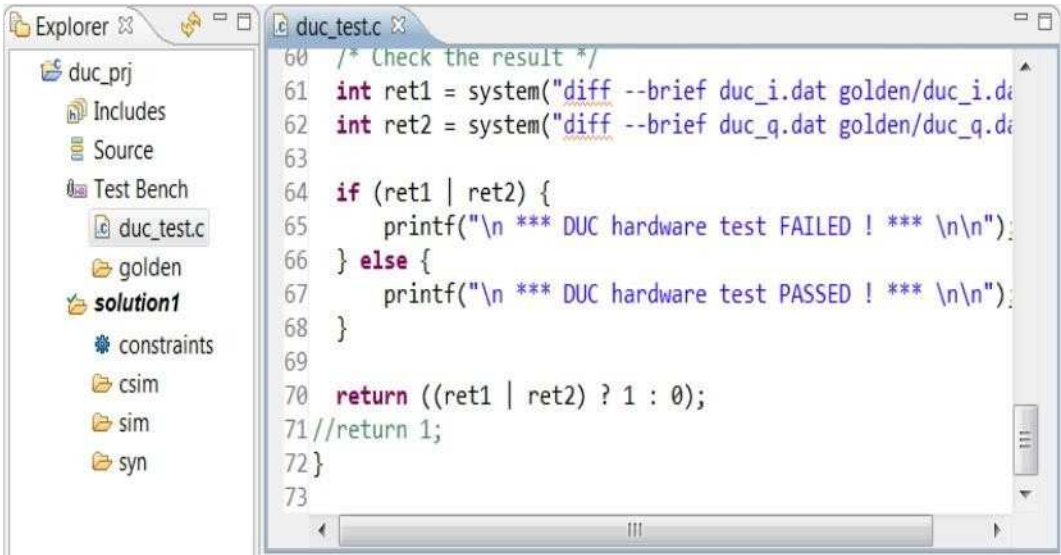


图 8-6:RTL 测试台

- 3.滚动到文件的末尾，查看如图 8-7 所示的代码。
- 4.编辑 *return* 语句以匹配图 8-7，并确保测试工作台始终返回值 1。

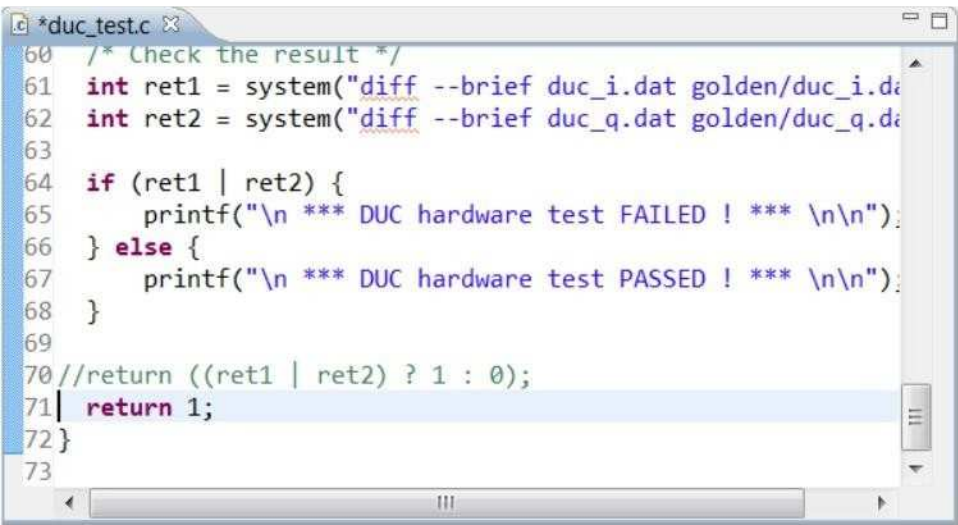


图 8-7:修改后的 RTL 测试台

- 5.保存文件。
- 6.单击 *Run C Synthesis* 工具栏按钮，将设计综合为 RTL。
- 7.单击 *Run C/RTL CoSimulation* 工具栏按钮，启动 *CoSimulation* 对话框。

8.将 **CoSimulation** 选项保留为默认值，然后单击 **OK** 以执行 **RTL CoSimulation**。

当 **RTL CoSimulation** 完成时，**CoSimulation** 报告打开并显示验证失败(图 8-8)。

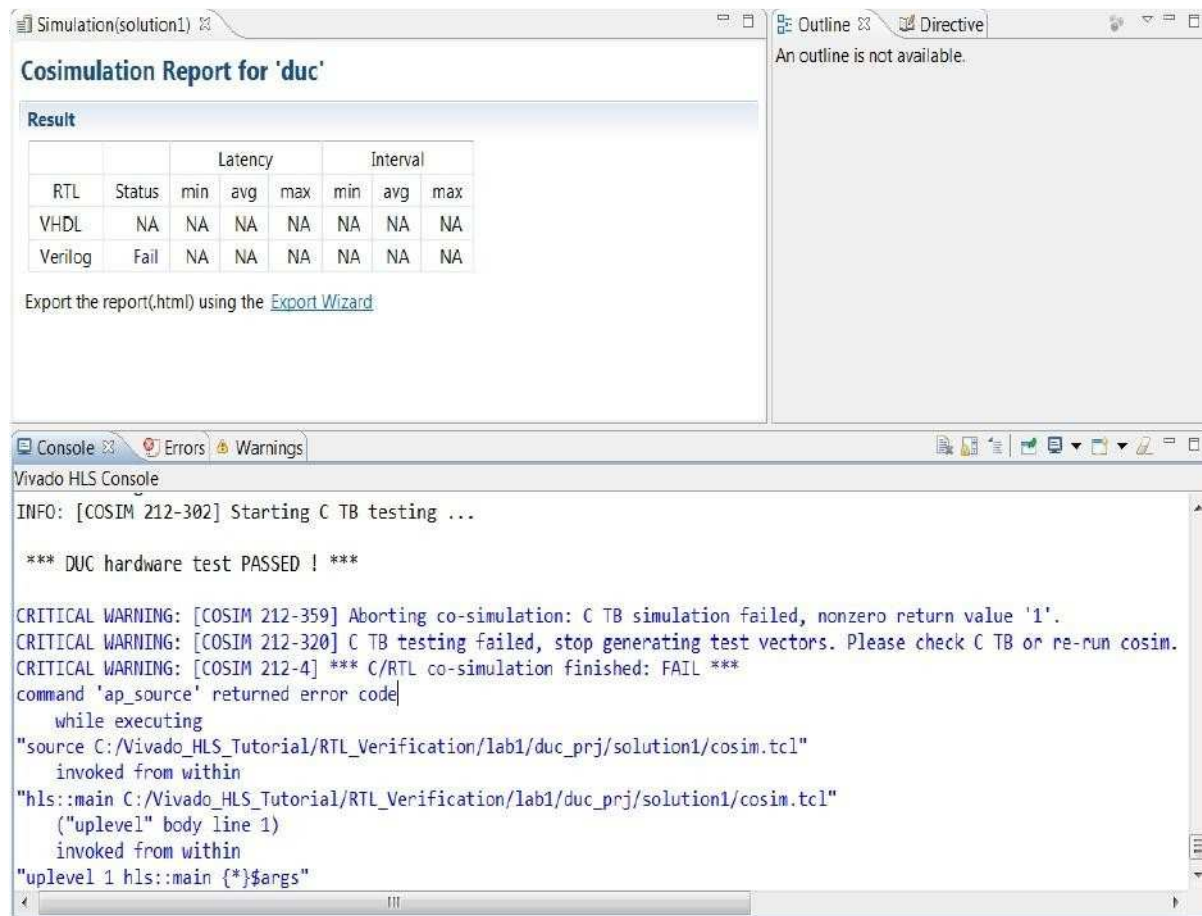


图 8-8:共仿真报告失败

在图 8-8 中，您可以从打印到控制台的消息（DUC 硬件测试已通过）中看到结果是正确的，但是，验证报告指出 RTL 验证失败。

如果需要，您可以确认结果正确。为此，将 RTL 模拟创建的**输出文件**与**黄金结果**进行比较。RTL 模拟是在解决方案目录中的模拟目录包装中执行的。图 8-10 显示了解决方案目录，突出显示了输出文件。图 8-9 显示了解决方案目录，并突出显示了输出文件。

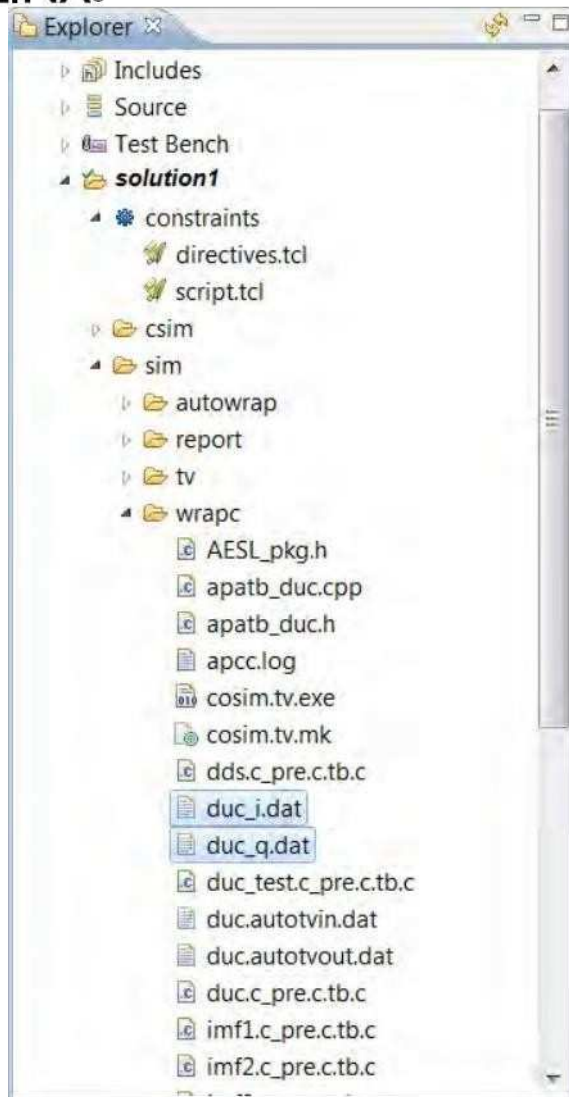


图 8-9: 共仿真输出文件

RTL CoSimulation 只在测试台返回 0(0)值时报告验证成功。修改测试工作台以返回一个非零值可以确保 RTL 验证(如果执行了 C 模拟)总是报告失败。

为了确保自动验证 RTL 结果: C 测试工作台必须始终检查要综合的 C 函数的输出, 如果结果正确, 则返回 0;如果不正确, 则返回任何其他值。

当执行 RTL 验证时, 测试台上也会进行相同的测试, 并且会**自动检查** RTL 块的输出。这就是为什么对 C 测试工作台来说, 检查结果并仅在结果正确时返回零值(或者在结果不正确时返回非零值)是很重要的。

9.退出 Vivado HLS GUI 并返回到命令提示符。

实验 2:在 Vivado 中查看跟踪文件

本练习解释如何生成 **RTL 跟踪文件**, 以及如何使用 Vivado 设计套件工具查看它们。

步骤 1:使用 Vivado 模拟器创建 RTL 跟踪文件

- 1.从您在 *lab1* 中使用的 Vivado HLS 命令提示符, 切换到 *lab2* 目录, 如图 8-10 所示。
- 2.通过键入 **vivado_hls -f run_hls.tcl** 创建一个新的 Vivado HLS 项目。

```
c:\Vivado_HLS_Tutorial\RTL_Verification\lab2>ls
dds.c      duc.c      golden     imf2.c     imf3_coef.h  run_hls.tcl
dds.dat    duc.h      imf1.c     imf2_coef.h  mac.c        srrc.c
dds_table.h duc_test.c imf1_coef.h imf3.c      mixer.c      srrc_coef.h
c:\Vivado_HLS_Tutorial\RTL_Verification\lab2>vivado_hls -f run_hls.tcl
```

图 8-10:RTL 验证实验室 2 的设置

3. 输入 ***vivado_hls -p duc_prj*** 打开 Vivado HLS GUI 项目。
4. 单击 ***Run C Synthesis*** 工具栏按钮，将设计合成为 RTL。
5. 单击 ***Run C/RTL CoSimulation*** 工具栏按钮，启动 ***CoSimulation*** 对话框盒子。

在这种情况下，您将生成一个可以使用 Vivado 模拟器打开的跟踪文件。在协同仿真对话框中：

- a. 保留默认的自动选择(使用 ***Vivado 模拟器***和 ***Verilog***)。
- b. 激活 ***Dump Trace*** 下拉菜单 并选择 ***all*** 选项，得到如图 8-11 所示的选项。
- c. 点击 ***OK*** 执行 RTL CoSimulation。

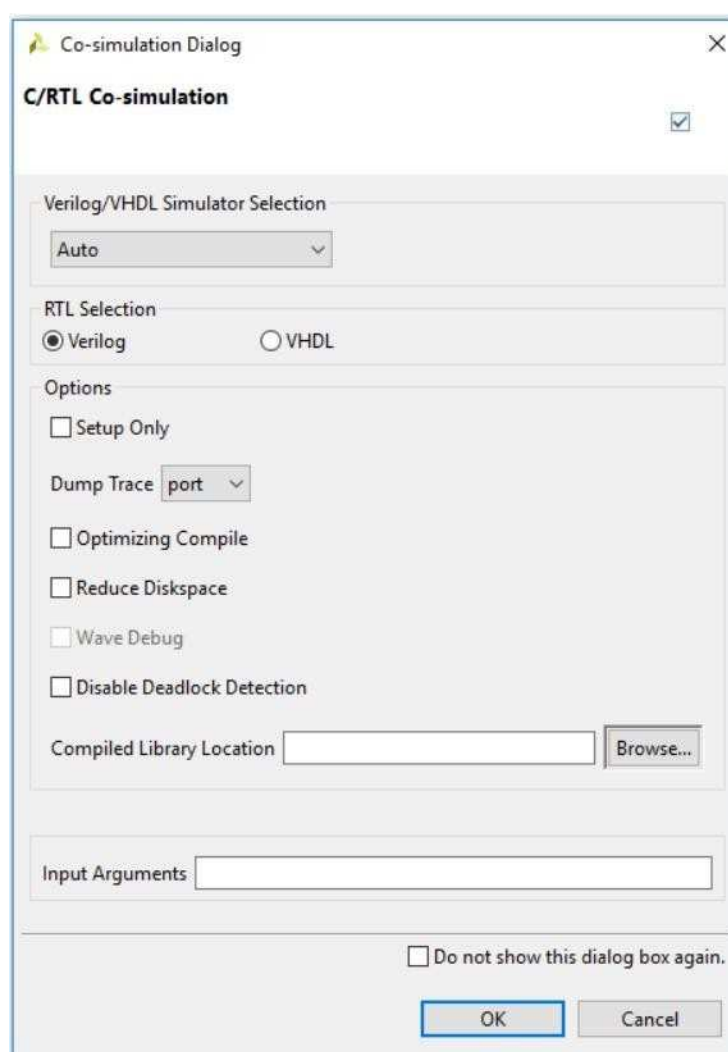


图 8-11:实验 2 的共仿真对话框

当 RTL 验证完成时，CoSimulation 报告将**自动打开**。报告显示 Verilog 模拟已经通过(以及测量的延迟和间隔)。此外，由于 **Dump 跟踪选项**与 **Vivado Simulator 模拟器选项**一起使用，并且由于选择了 **Verilog**，所以 **Verilog** 模拟目录中现在有两个跟踪文件。这些在图 8-12 中突出显示。

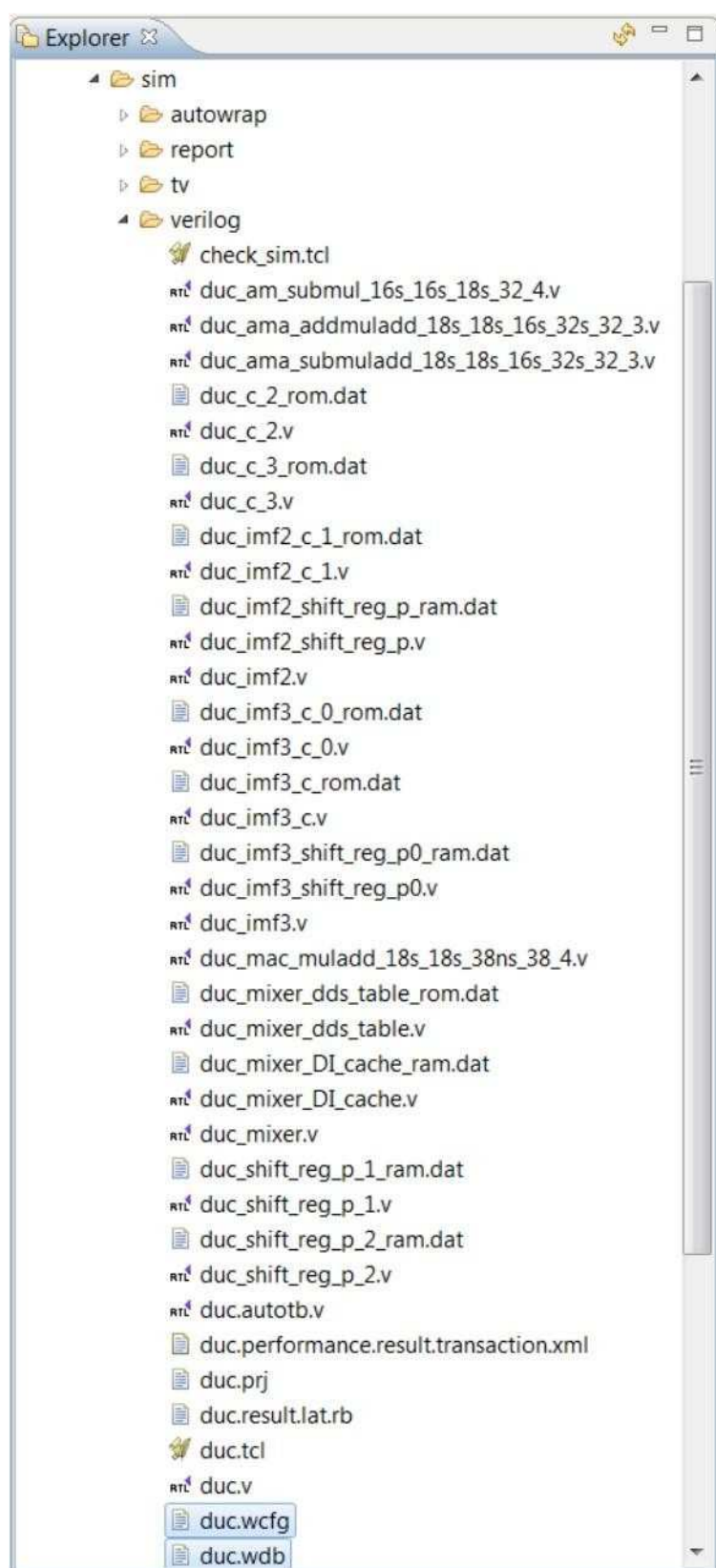


图 8 - 12: Verilog Vivado 仿真器协同仿真结果

下一步是查看 **Vivado** 设计套件中的跟踪文件。

因为已经为 **Vivado** 模拟器生成了波形跟踪数据，所以 **Open Wave Viewer** 工具栏按钮现在突出显示，如图 8-13 所示。

注意: 只有当 Vivado 模拟器被选为 Verilog/VHDL 模拟器时，才能使用 Open Wave Viewer 工具栏

按钮。



图8-13: 在 Vivado 中打开跟踪文件

7.单击 **Open Wave Viewer** 工具栏按钮，打开带有 RTL 波形跟踪的 **Vivado IDE**。

注意:*Vivado IDE 通过这个动作提供的唯一功能是查看和分析 RTL 波形。*

然后可以在波形查看器中查看波形。图 8-14 显示了放大后的波形，其中扩展了输出数据端口及其相关的 I/O 协议信号(输出有效信号)以查看。

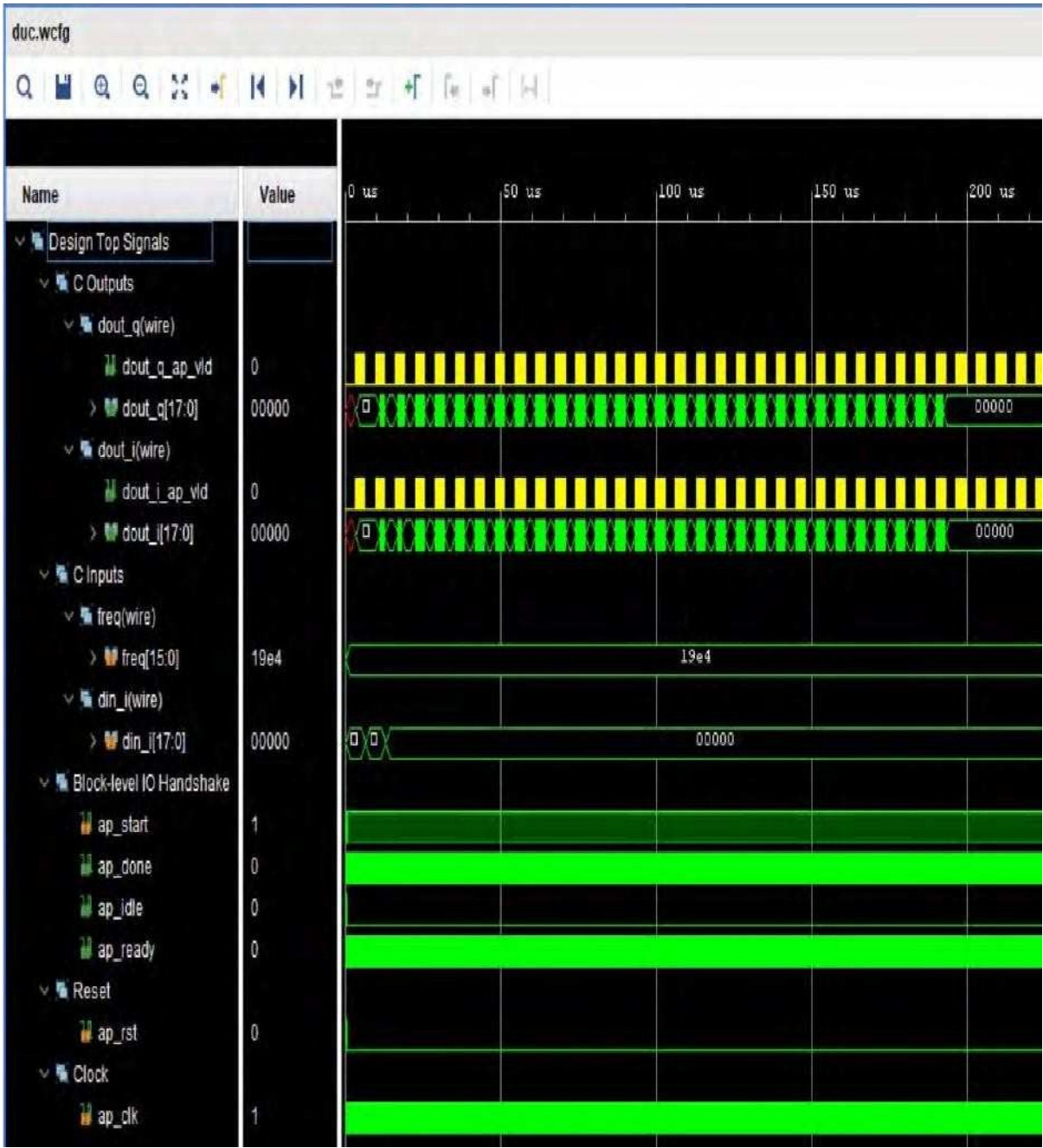


图 8-14: 分析 RTL 跟踪文件

8.退出 Vivado IDE。

9.退出并关闭 Vivado HLS GUI。

实验3:在 ModelSim 中查看跟踪文件

本练习解释了如何生成和查看 RTL 跟踪文件，以及如何使用 Mentor Graphics ModelSim RTL 模拟器。还支持其他第三方模拟器，如果选择另一个 RTL 模拟器，也可以使用类似的过程。



谨慎! 这个实验练习需要 ModelSim 的可执行文件在系统搜索路径中定义，并且系统上有执行 HDL 模拟所需的许可证。

步骤1:使用 ModelSim 创建一个 RTL 跟踪文件

1. 从 *lab2* 中使用的 Vivado HLS 命令提示符，切换到目录 *lab3*
2. 通过键入 `vivado_hls -f run_hls.tcl` 创建一个新的 Vivado HLS 项目。
3. 输入 `vivado_hls -p duc_prj` 打开 VivadoHLS GUI 项目。
4. 单击 **Run C Synthesis** 工具栏按钮，将设计合成为 RTL。
5. 单击 **Run C/RTL CoSimulation** 工具栏按钮，启动 **CoSimulation** 对话框。
本练习使用 **Mentor Graphics ModelSim RTL 模拟器**。模拟器可执行文件的路径必须在系统搜索路径中设置。
6. 在协同仿真对话框中:
 - a. 从 **Verilog/VHDL 模拟器** 选择器中选择 **ModelSim**。
 - b. 选择硬件描述语言 (**VHDL**)。
 - c. 激活 **Dump Trace(转储跟踪)** 下拉菜单并选择 **all** 选项，得到如图 8-15 所示的选项。
 - d. 单击 **OK** 执行 RTL CoSimulation。

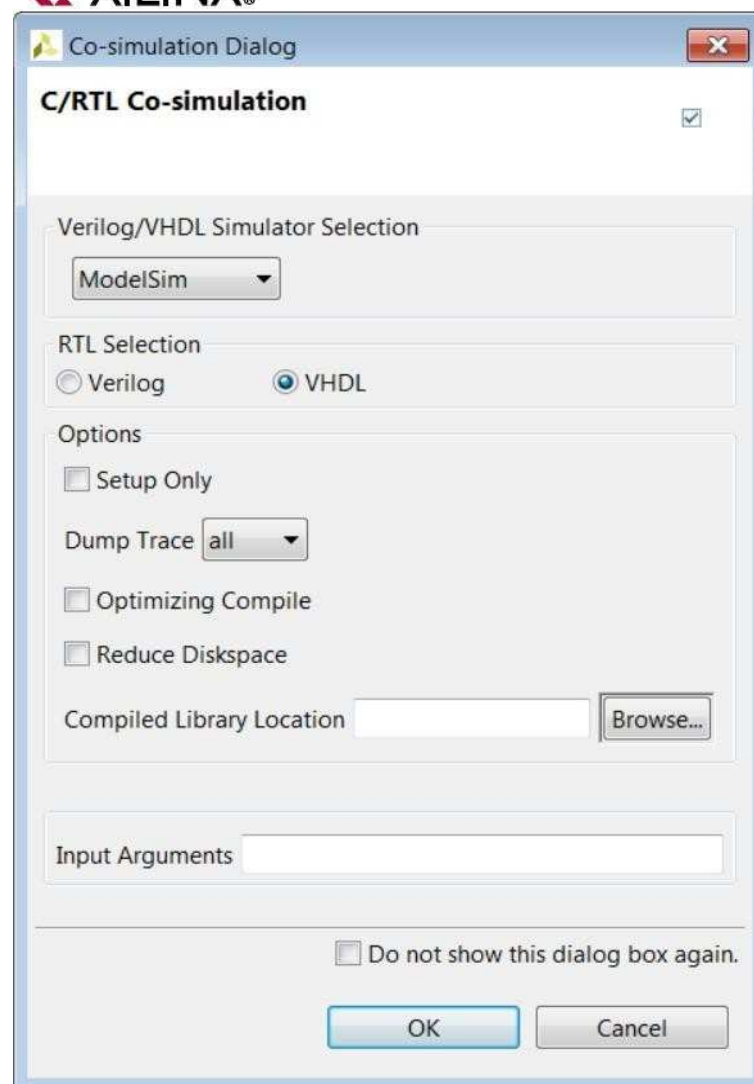


图 8-15: 实验 3 的共仿真对话框

RTL 验证完成后，协同仿真报告将自动打开。该报告显示，Verilog 仿真已通过（以及测得的延迟和间隔）。此外，由于“转储跟踪(*Dump Trace*)”选项与“*Vivado Simulator*”模拟器选项一起使用，并且由于选择了“*VHDL*”，因此，VHDL 模拟目录中现在存在两个跟踪文件。这些在图 8-16 中突出显示。

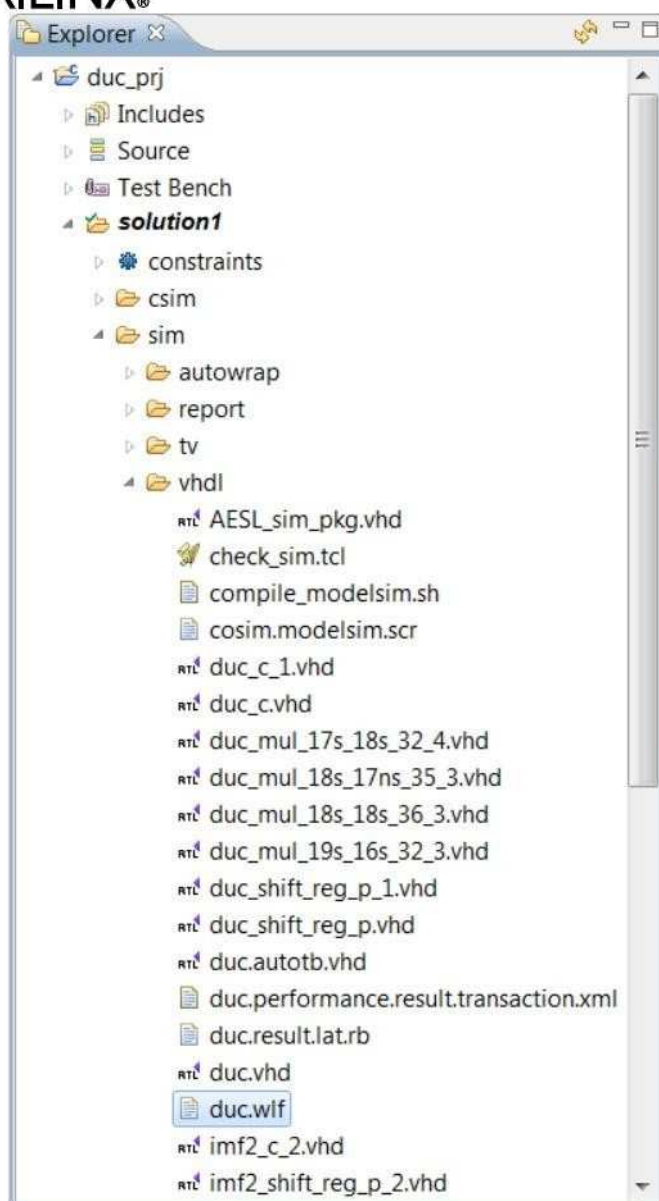


图 8-16:VHDL ModelSim

跟踪文件下一步是查看 ModelSim 内部的跟踪文件。

7. 退出 Vivado HLS GUI 并返回到命令提示符。

步骤2:在 ModelSim 中查看 RTL 跟踪文件

1. 启动 Mentor Graphics ModelSim RTL 模拟器。点击菜单 文件>打开。

3. 选择日志文件作为文件类型(图 8-17)。

4. 导航到 VHDL 仿真目录并选择 5. duc.wlf. 单击 Open。

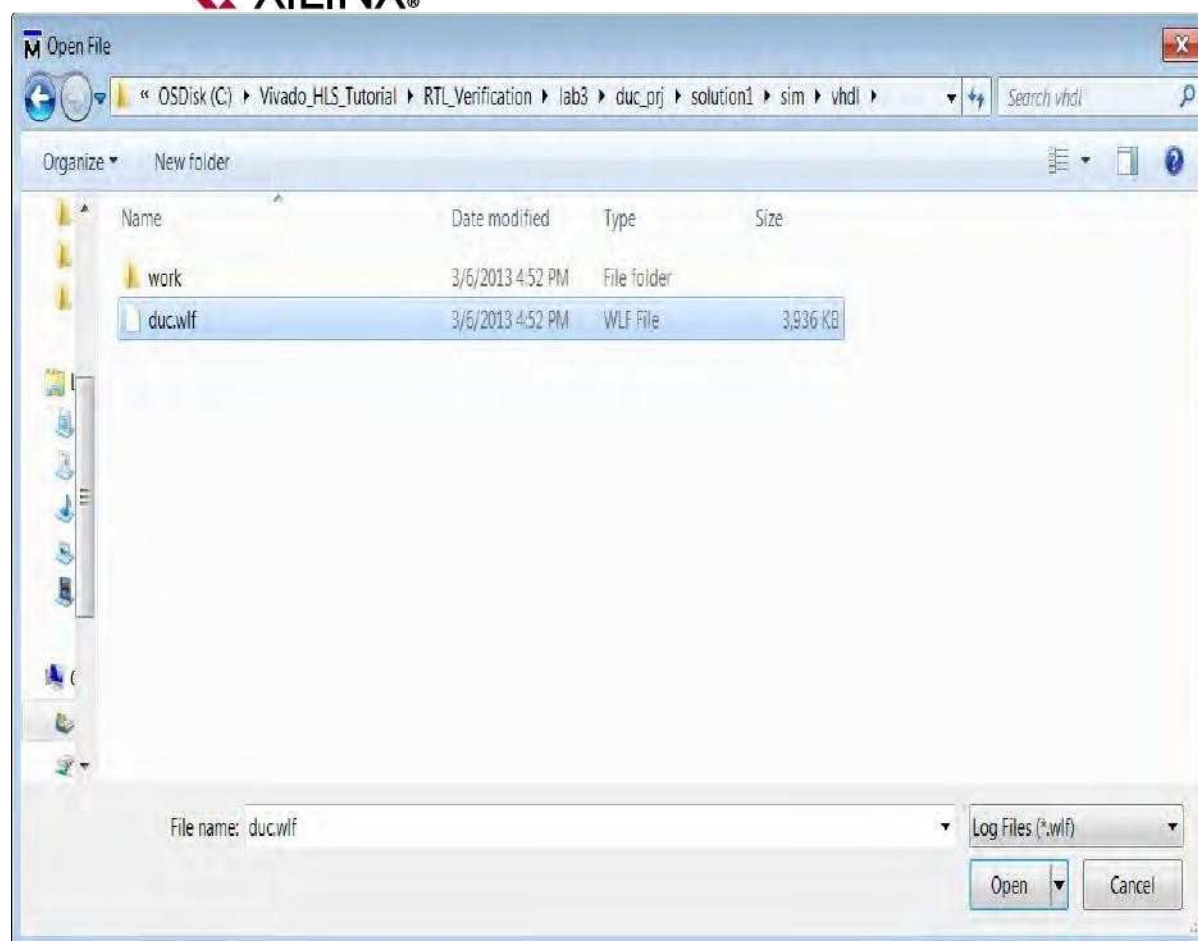


图8-17:ModelSim 打开文件 WLF

6.将信号添加到跟踪窗口并进行调整，以查看类似于图 8-18 的视图。

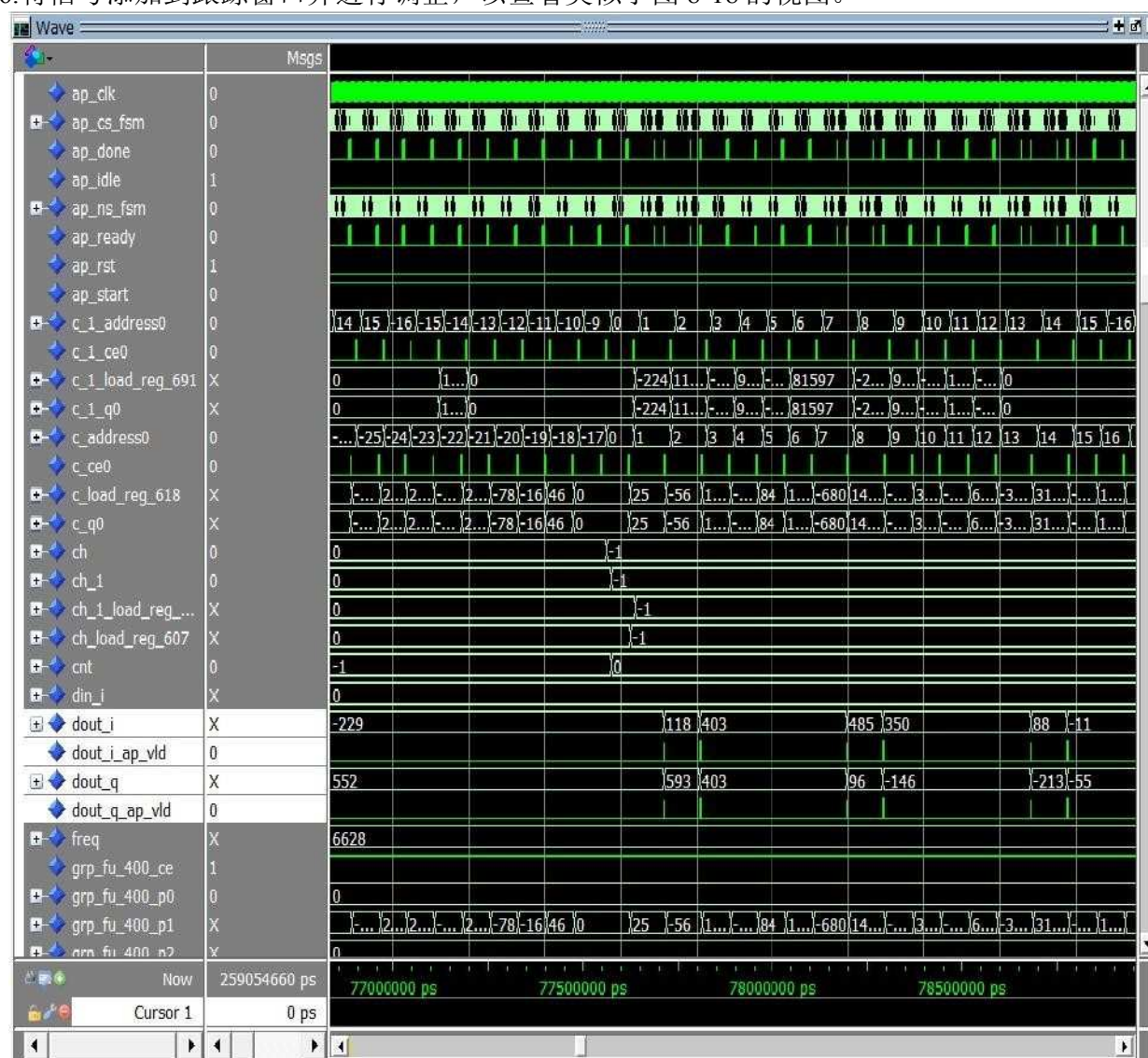


图8-18:在 ModelSim 中查看跟踪文件

7.退出并关闭 ModelSim RTL 模拟器。

结论

在本教程中，您学习了如何：

- 对 C 合成的设计进行 RTL 验证，并说明测试台在此过程中的重要性。
- 使用 Vivado 设计套件创建和打开波形跟踪文件。
- 使用第三方 HDL 模拟器(ModelSim)创建和打开波形跟踪文件，并查看 RTL 验证创建的跟踪文件。

在 IP 集成器中使用 HLS IP

概述

您可以从高级综合中打包 RTL 并在 IP 集成器中使用它。本教程演示了如何获取 HLS IP 并将其作为一个更大的设计的一部分在 IP Integrator 中使用。本教程由一个实验练习组成。

实验室 1 描述

完成为 IP 目录生成两个 HLS 块的步骤，并在使用 Xilinx IP (FFT)的设计中使用它们。您可以使用 RTL 测试平台来验证和验证最终的设计。

教程设计描述

您可以从 Xilinx 网站下载教程设计文件。请参阅定位教程设计文件中的信息。 本

教程使用教程目录中的设计文件，使用的是 **Vivado_HLS_Tutorial\uing_ip_with_ipi**。

本教程中的设计块处理复杂 FFT 的数据。

Xilinx FFT IP 块只对复杂数据进行操作。尽管您可以在所有虚构组件都设置为 0 的复杂数据集上执行 FFT，但是通过预处理数据可以更有效地执行 FFT。

- 本实验室的前端 HLS 块对 1024 (N)个真实数据样本应用汉明开窗 (**Hamming windowing**) 函数，将偶数/奇数对作为复杂数据发送给 N/2 点 XFFT。
- 后端 HLS 块采用位反序数据，将其按自然顺序排列，并对 FFT 输出应用 O(N)变换来 提取
注意，第一个输出对将第 0 个和第 512 个(纯实)光谱数据点分别装入实部和虚部。
- 该设计是**完全流水线**，**流设计高吞吐量**；用于数据的连续处理，但具有节流功能（如果输入停止，则停止）。**AXI4 流接口用于连接 IP 集成器中的所有块。**

实验 1:将 HLSIP 与 Xilinx IP 块集成

这个实验演示了如何在 IP Integrator 中将两个 HLS IP 块与一个 Xilinx IP FFT 相结合，并 在 Vivado 设计套件中验证了设计。

重点:本教程中的图形和命令使用教程数据目录



将 Vivado_HLS_Tutorial 解压缩并放置在位置 C:\Vivado_HLS_Tutorial 中。如果将教程数据目录解压缩到不同的位置，或者在 Linux 系统上，请将引用的少数路径名调整到您选择的位置，以放置 Vivado_HLS_Tutorial 目录。

步骤 1:创建 Vivado HLS IP 块

使用提供的 Tcl 脚本为 Vivado IP 目录创建两个 HLS 块。脚本运行 **HLS C-synthesis**、**RTL co-simulation**，并将两个 HLS 设计(**hls_real2xfft** 和 **hls_xfft2real**)。

1. 打开 Vivado HLS 命令提示符。

在 Windows 上使用 **Start > All Programs > Xilinx Design Tools > Vivado 2018.3 > Vivado HLS > Vivado HLS 2018.3 Command Prompt**

在 Linux 上，打开一个新的 shell。

2. 使用命令提示符窗口，将目录更改为

Vivado_HLS_Tutorial \ Using_IP_with_IPI \ lab1 \ hls_designs(图 9)。

3. 键入 **vivado_hls -f run_hls.tcl** 以创建 HLS IP(图 9-1)。

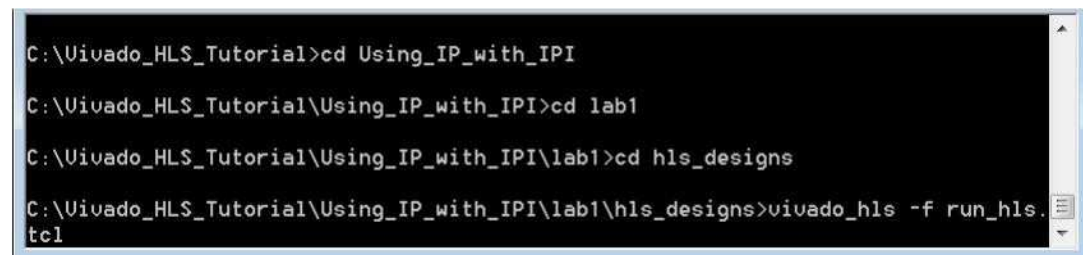


图 9-1: 创建 IP 集成器的 HLS 设计

当脚本完成时，有两个 Vivado HLS 项目目录 **fe_vhls_prj** 和 **be_vhls_prj**，其中包含 HLS IP，包括 Vivado 设计中使用的 Vivado IP 目录归档。

- “前端”IP 存档位于 **fe_vhls_prj/IPXACTExport/impl/IP/**

- “后端”IP 存档位于 **be_vhls_prj/IPXACTExport/impl/ip/**

本教程展示了如何将 Vivado HLS IP 块集成到设计中(在 IP Integrator 中)并进行验证。

步骤 2: 创建一个 Vivado 设计套件项目

1. 启动 Vivado 设计套件(不是 Vivado HLS):

在 Windows 上使用 **Start > All Programs > Xilinx Design Tools > Vivado 2018.3 > Vivado HLS > Vivado HLS 2018.3 Command Prompt**

在 Linux 上，在 **shell** 中键入 **vivado**。

2. 在欢迎屏幕中，单击 **Create Project**(图 9-2)。

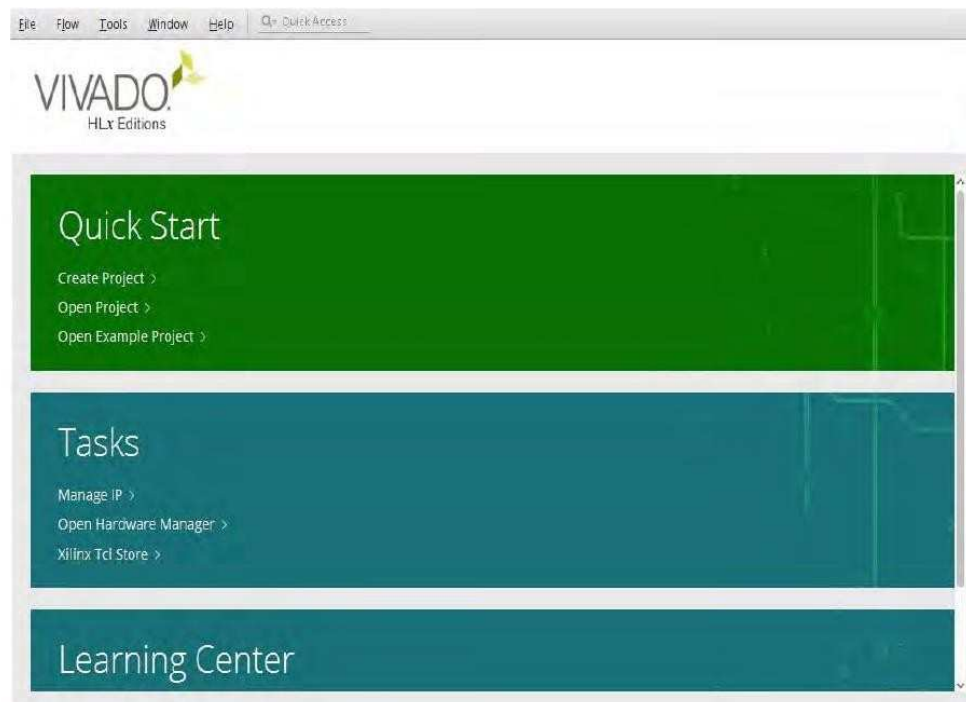


图 9-2: 创建一个 Vivado 项目

3. 单击“新建 Vivado 项目向导”的第一页上的“下一步”。

4.单击项目位置文本输入框右侧的省略号按钮，浏览并选择教程目录(图 9-3)。

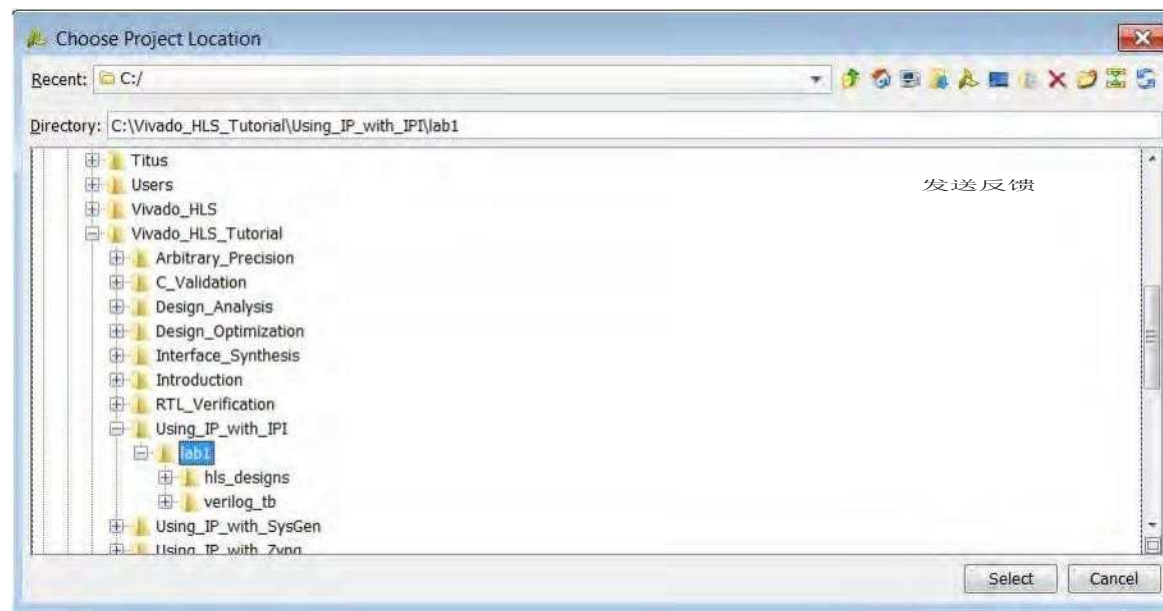


图9-3: Vivado Design Suite 项目 5 的路径。

5. 单击 Next 以移动到向导的项 目类型页面。

- a. 选择 RTL 项目。
- b. 选择此时不要指定来源(如果不是默认)。
- c. 单击 Next。

6. 在默认的 Part 页面，在 specific 下，点击 Boards，选择 ZYNQ-7 ZC702 Evaluation Board，如图 9-4 所示，然后按 Next。

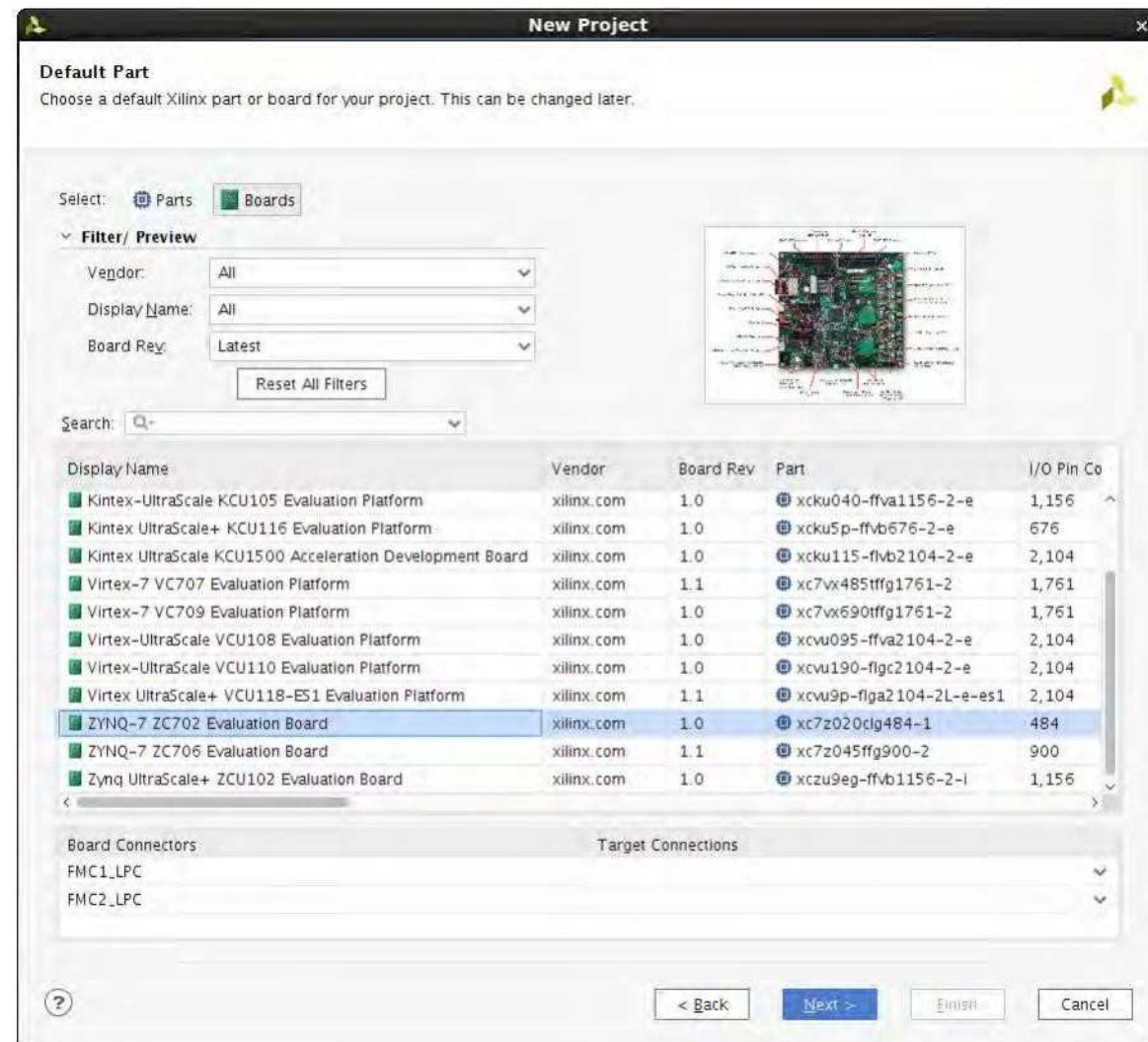


图 9-4: Vivado 项目规范

7. 在新项目摘要页面上，单击 **Finish** 完成新项目设置。Vivado 工作区填充并显示，如图 9-5 所示。

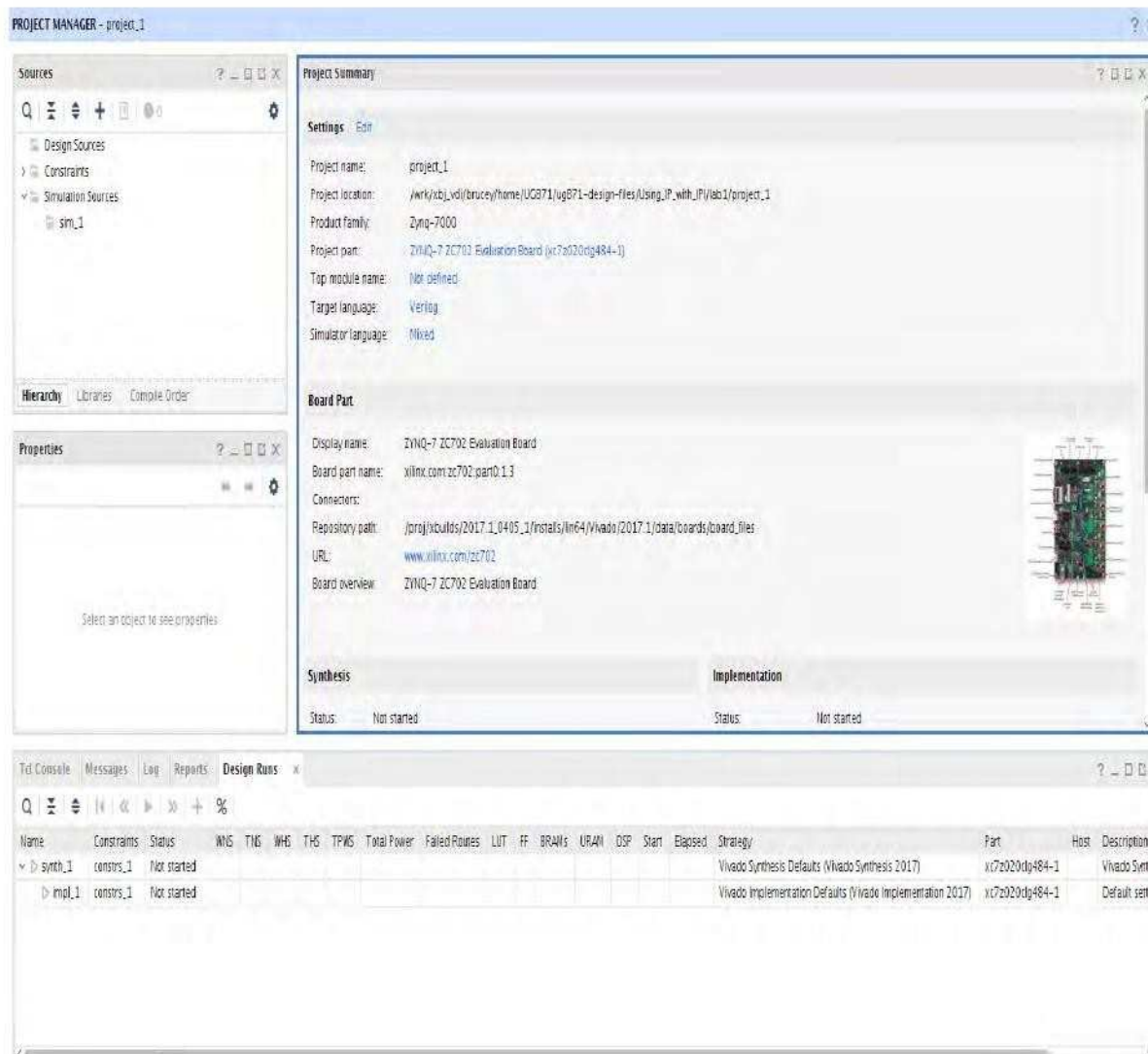


图 9-5: Vivado 项目

步骤 3: 将 HLS IP 添加到 IP 存储库

1. 在 **Flow Navigator** 窗格的 **Project Manager** 区域，单击 **IP Catalog**。

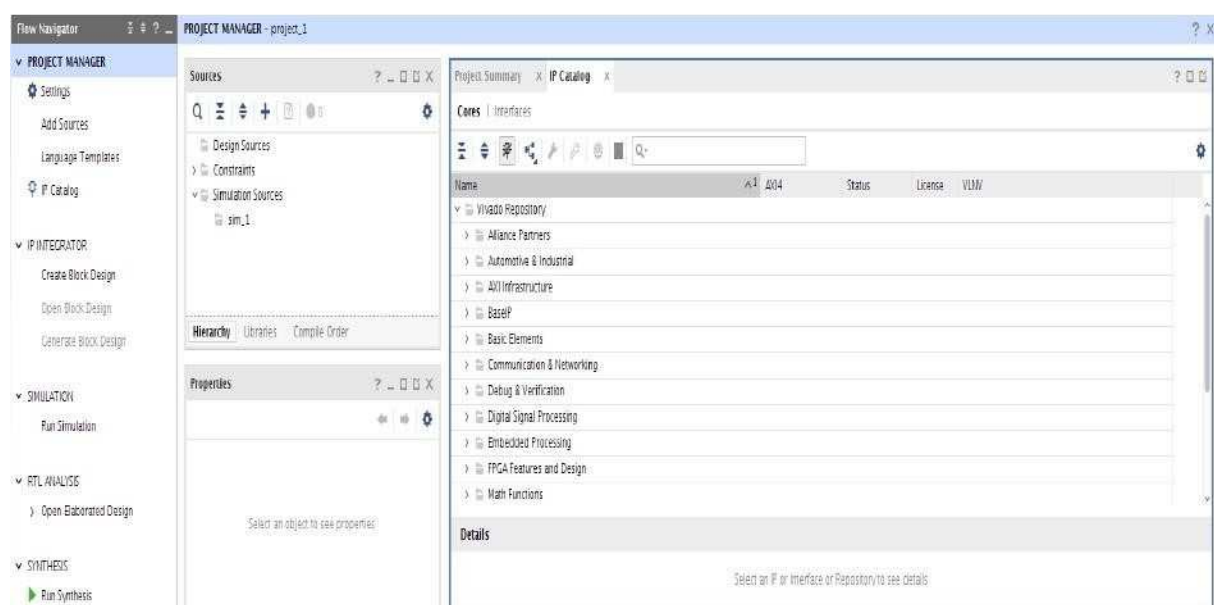


图 9-6: 打开 IP 目录

2. IP 目录出现在工作区的主窗格中。单击 IP 设置图标。

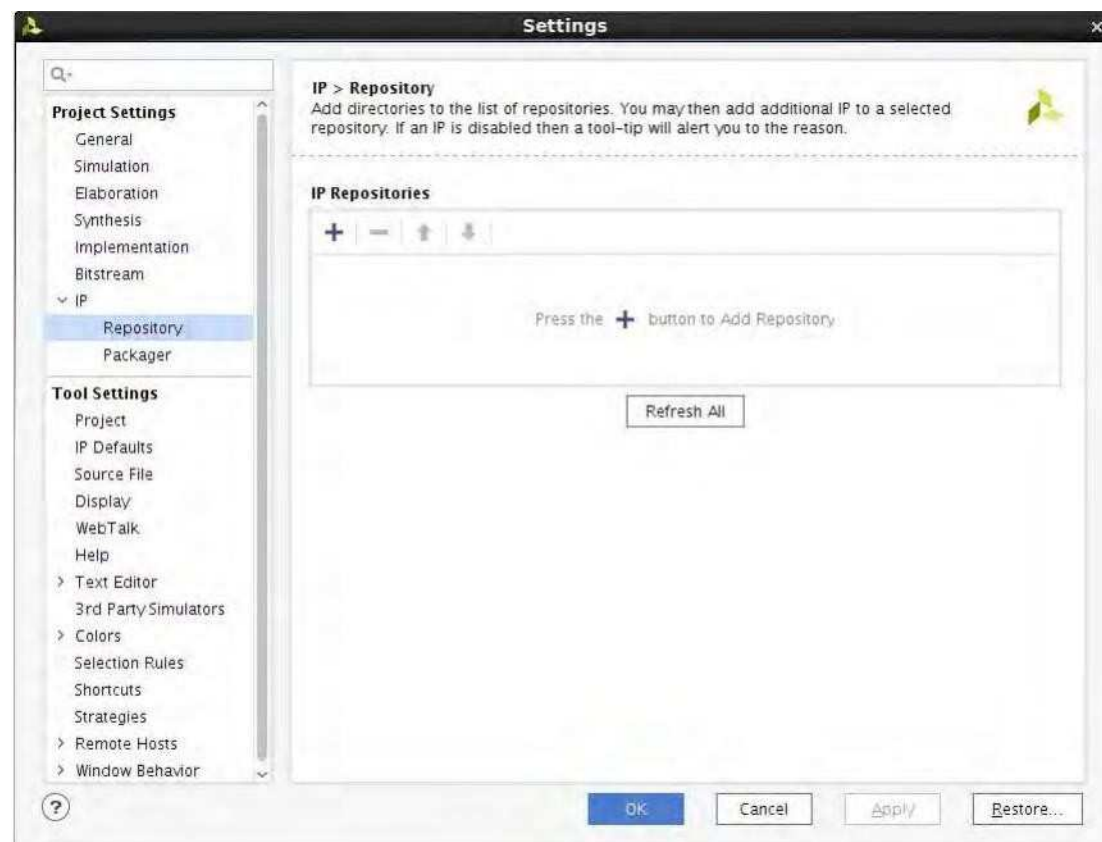


图9-7: 打开 IP 目录设置

3. 右键单击并选择 **Add Repository**。
4. 在 IP 知识库对话框:
 - a. 浏览到教程目录，
使用 `ip_ip_with_ipi \lab1\hls_designs\fe_vhls_prj\IPXACTExport\impl\ip`，如图 9-8 所示。
 - b. 单击 **Select** 关闭 IP 知识库窗口。

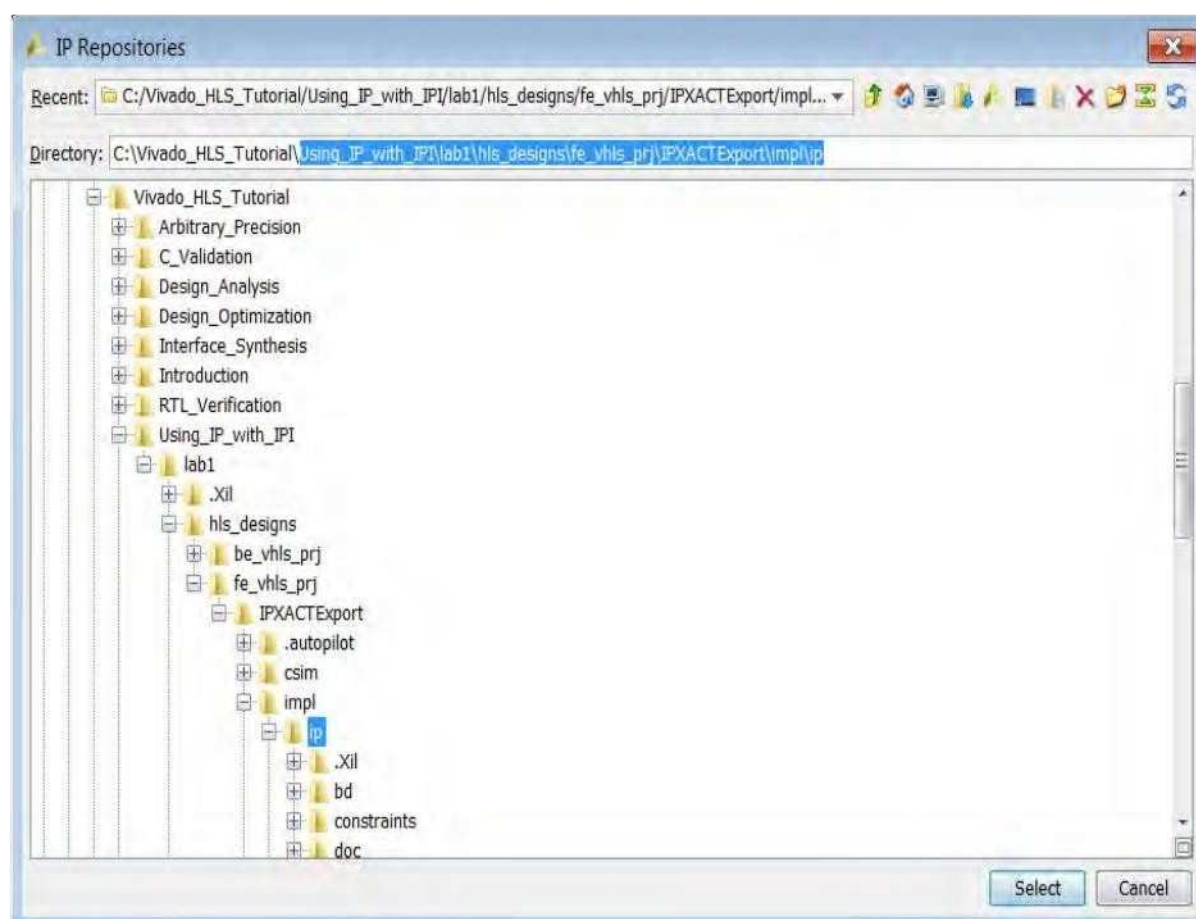


图9-8: 创建一个新的 IP 存储库

- 5.按 **Select** 接受新存储库，然后在添加的存储库上选择 OK。
- 6.按照相同的步骤添加第二个 HLS IP 包:
lab1 / hls_designs / be_vhls_prj / IPXACTExport / impl / ip /。
- 7.单击 OK 退出对话框。

Vivado HLS IP 类别现在在 IP 目录中显示为 HLS IP(图 9-9)。

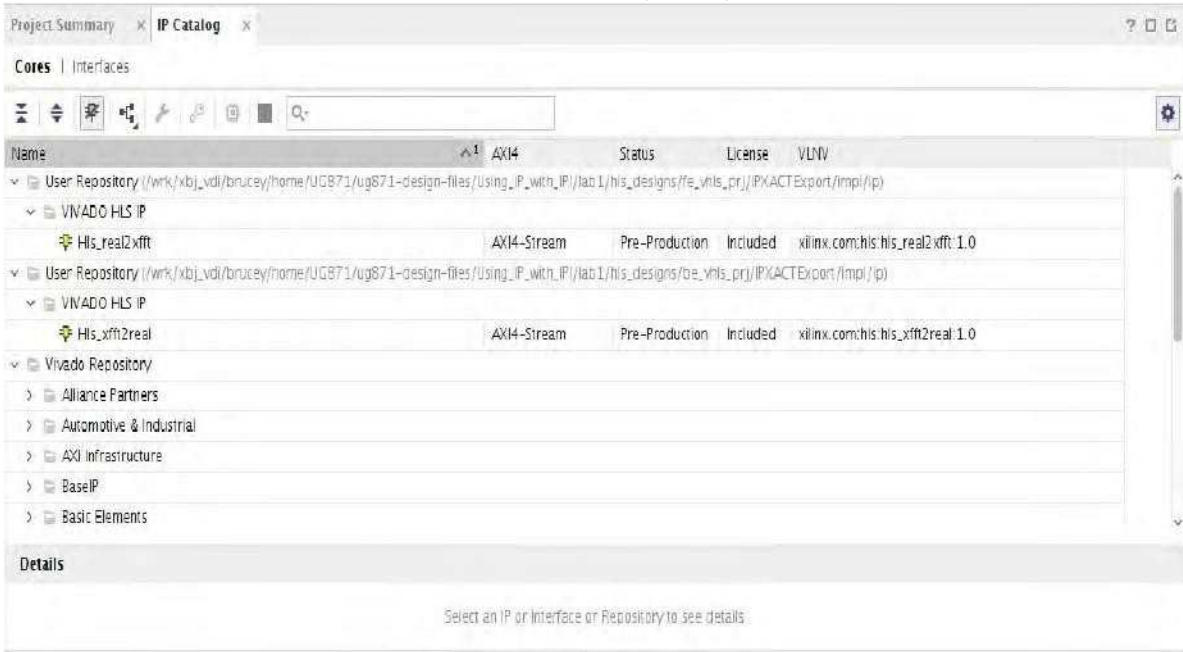


图9-9: 带有 HLS IP 的 IP 目录

步骤 4: 为 Real FFT 创建块设计

- 1.在流导航器中，单击 IP Integrator 下的 **Create Block Design**。
 - a.在结果对话框中，将设计命名为 **Real1FFT**。
 - b.单击 **OK**。

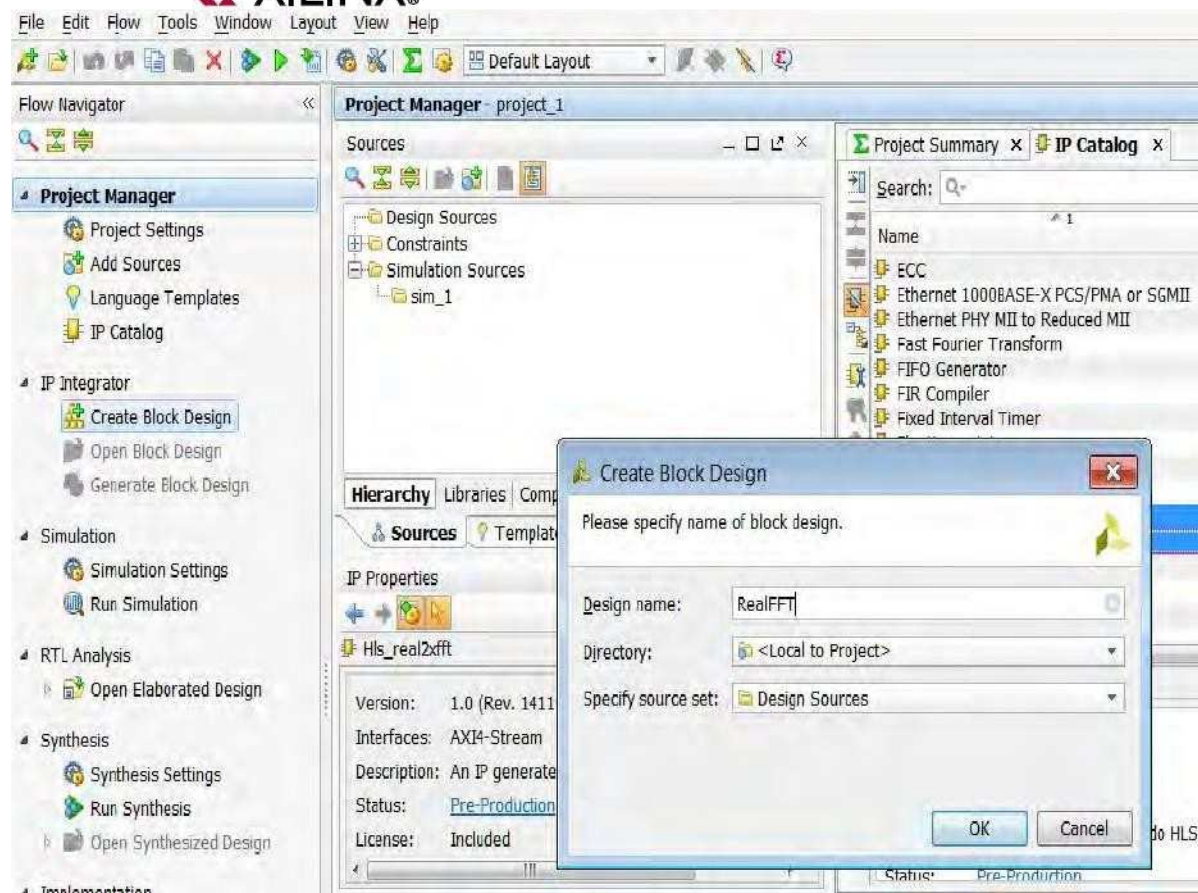


图9-10: 创建块设计

右上角窗格现在有一个 Diagram 选项卡。在设计中添加一个 Xilinx FFT IP 块并进行自定义。

2. 在 Diagram 选项卡中单击 Add IP 链接(图 9-11)。

- a. 在搜索框中键入 *fourier*.
- b. 选择 *Fast Fourier Transform*.
- c. Press *Enter*.

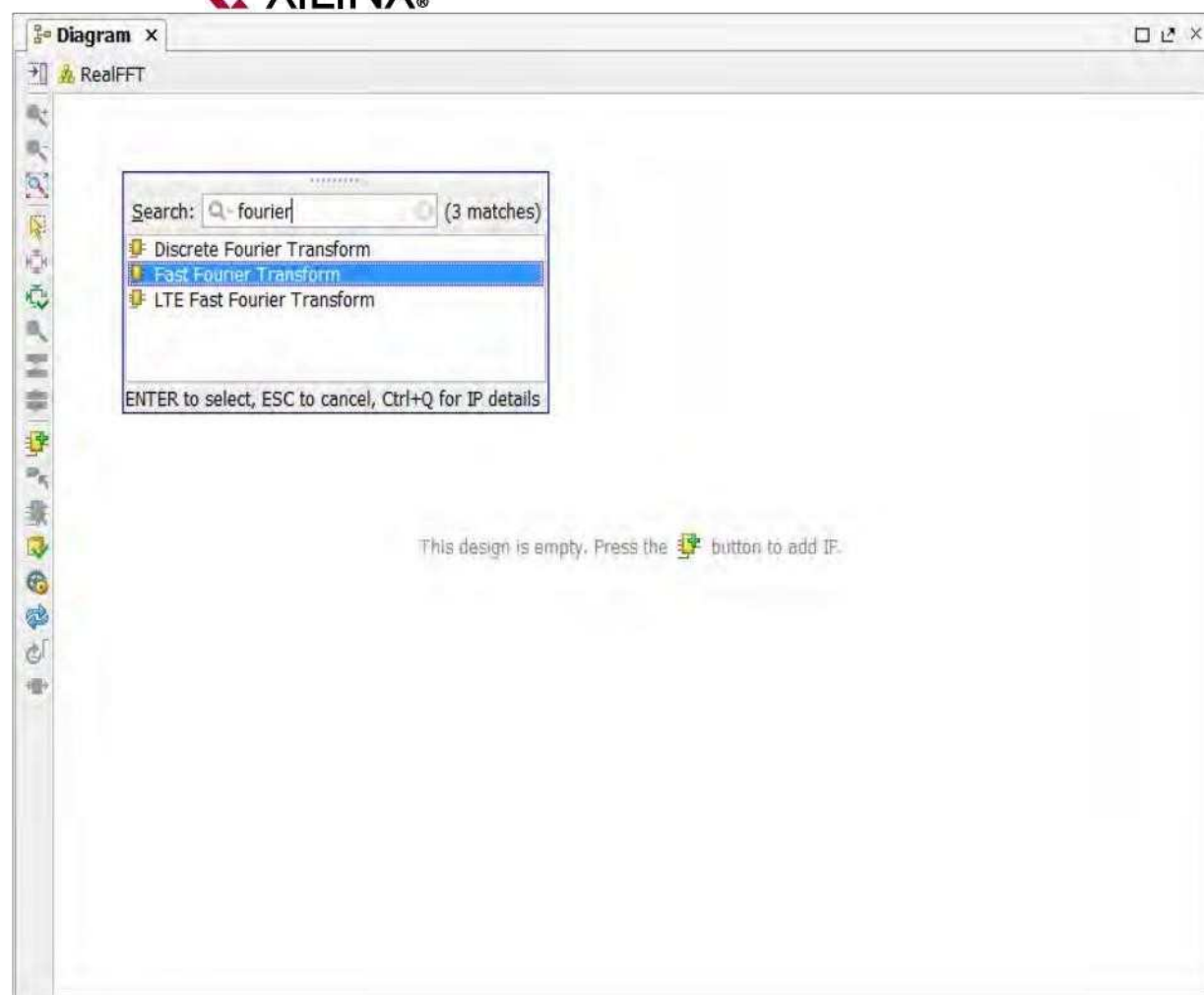


图9-11: 添加 Xilinx FFT IP

Xilinx IP 块 FFT 现在在设计中被实例化，如图 9-12 所示。

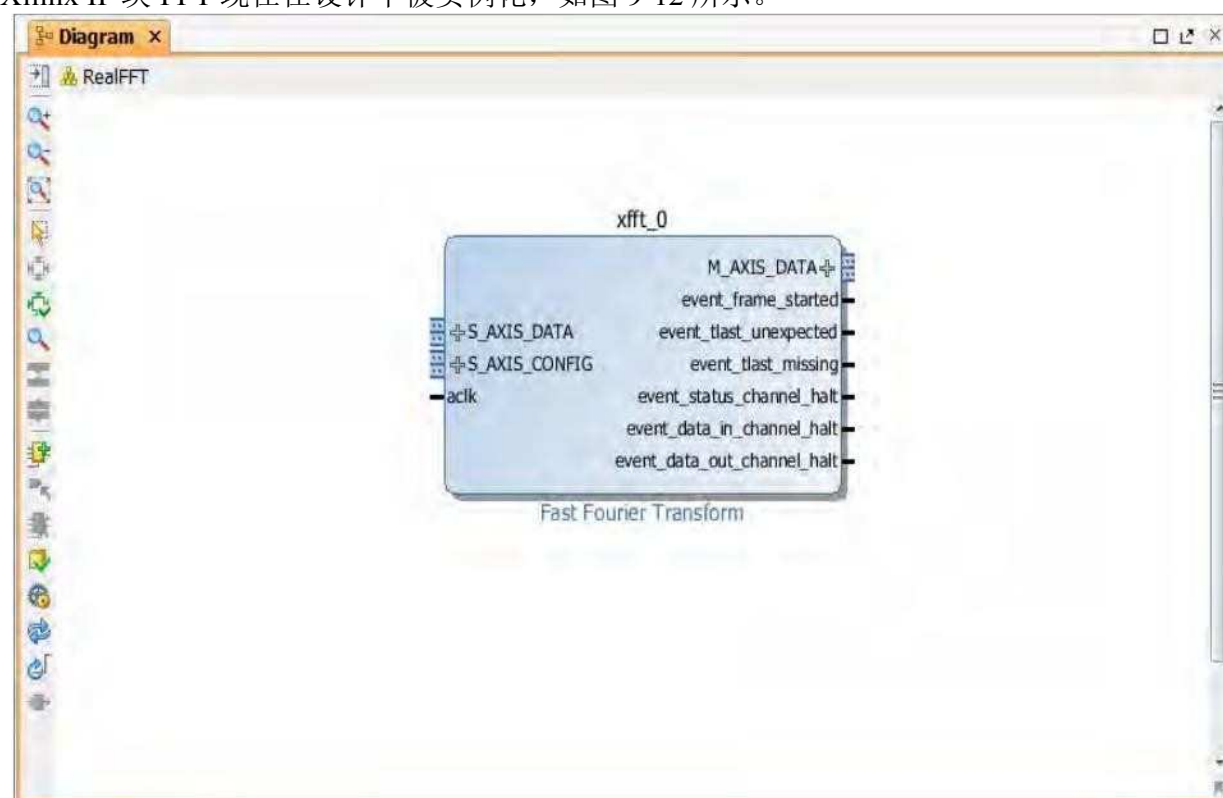


图9-12: Xilinx FFT IP

3. 双击新的快速傅里叶变换 IP 符号，打开重新定制 IP 对话框。
4. 在 Configuration 选项卡上(图 9-13):
 - a. 将变换长度 (*Transform Length*) 更改为 **512**。
 - b. 在架构选择一节中选择**流水线、流式**(*Pipelined, Streaming*) I/O。

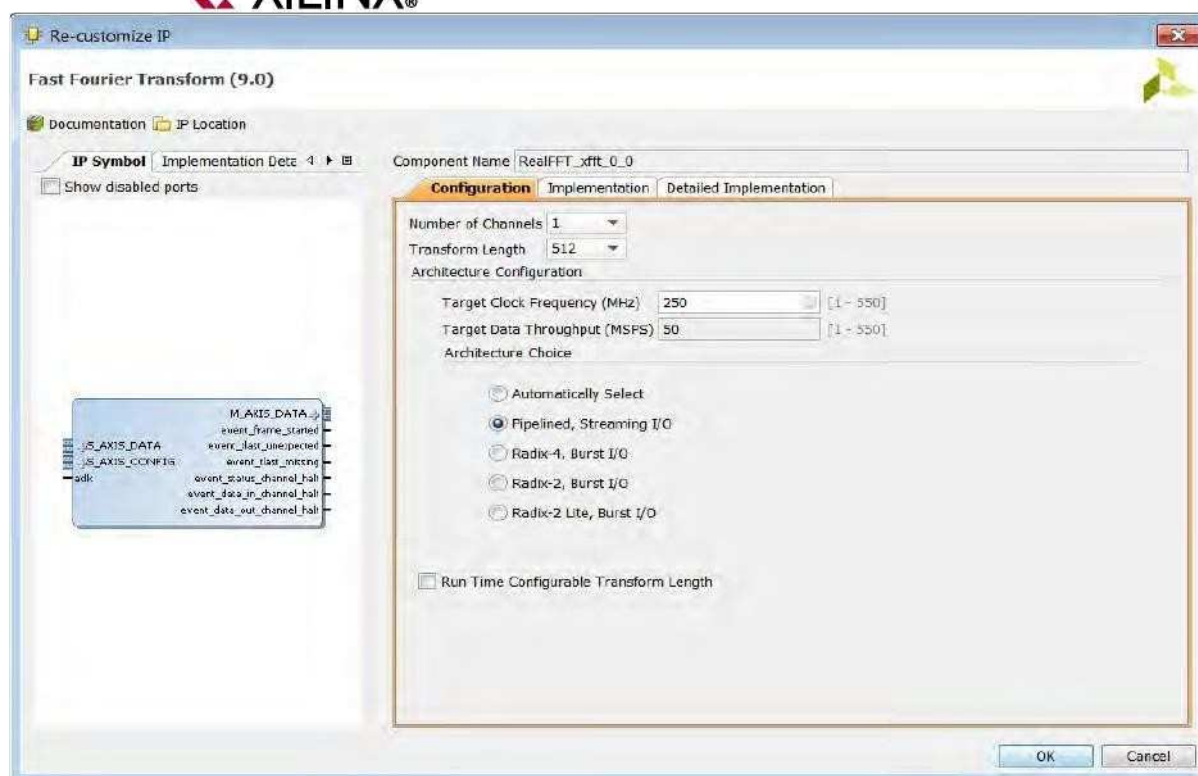


图 9-13:Xilinx FFT 配置

5.a. 在控制信号组中选择

- a. **ARESETN** (active low)。
- b. 确认选择非实时 (**Non Real Time**) 作为 **Throttle Scheme** 方案。
- c. 单击“确定”退出“重新自定义 IP(Re-customize IP)”对话框。

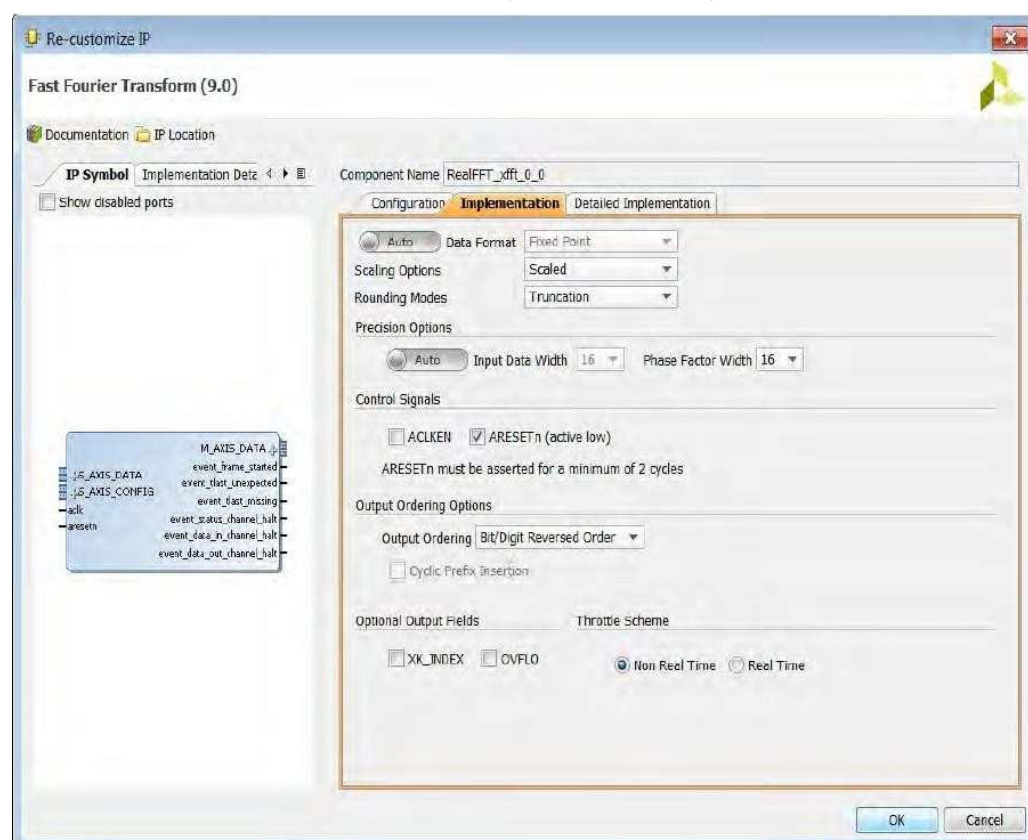


图 9-14:Xilinx FFT 实现

将生成的每个 HLS 块的一个实例添加到设计中。

6. 右键单击画布中的任何空间并选择 **Add IP**(图 9-15)。

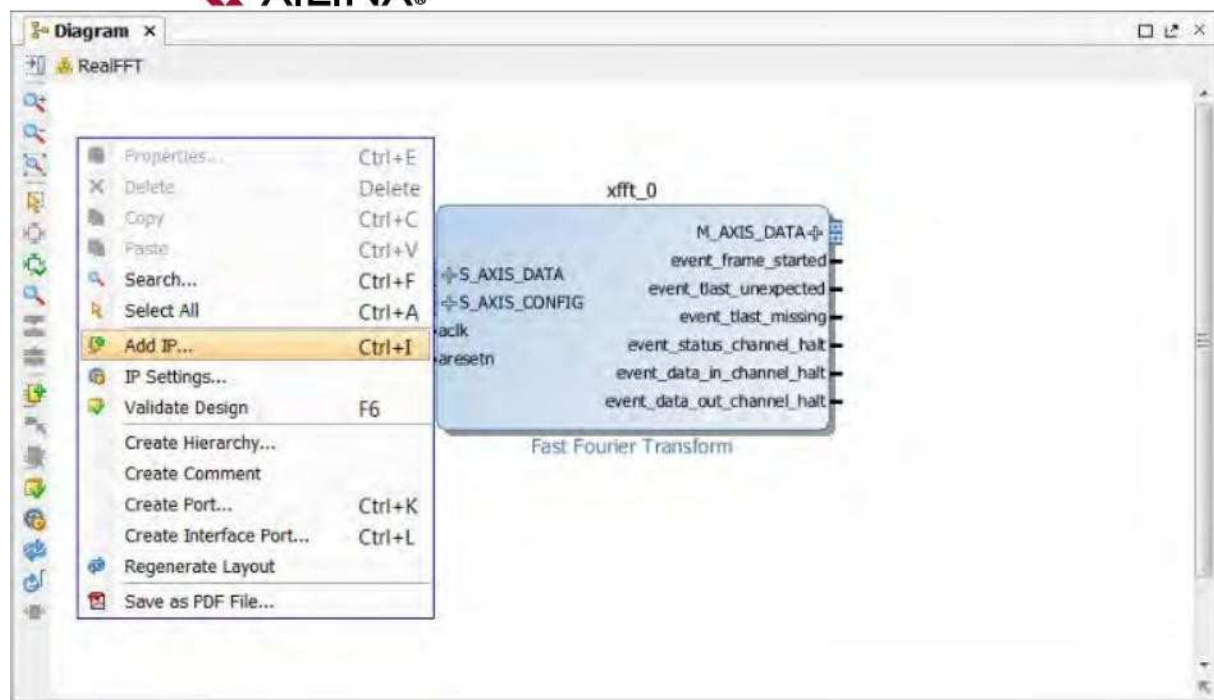


图 9-15: 添加 IP 块

7. 在“搜索文本输入”框中键入 **hls**.

- a. 突出两个 IPs。(点击控制键并同时选择)
- b. 回车。

设计块现在有三个 IP 块，如图 9-16 所示。

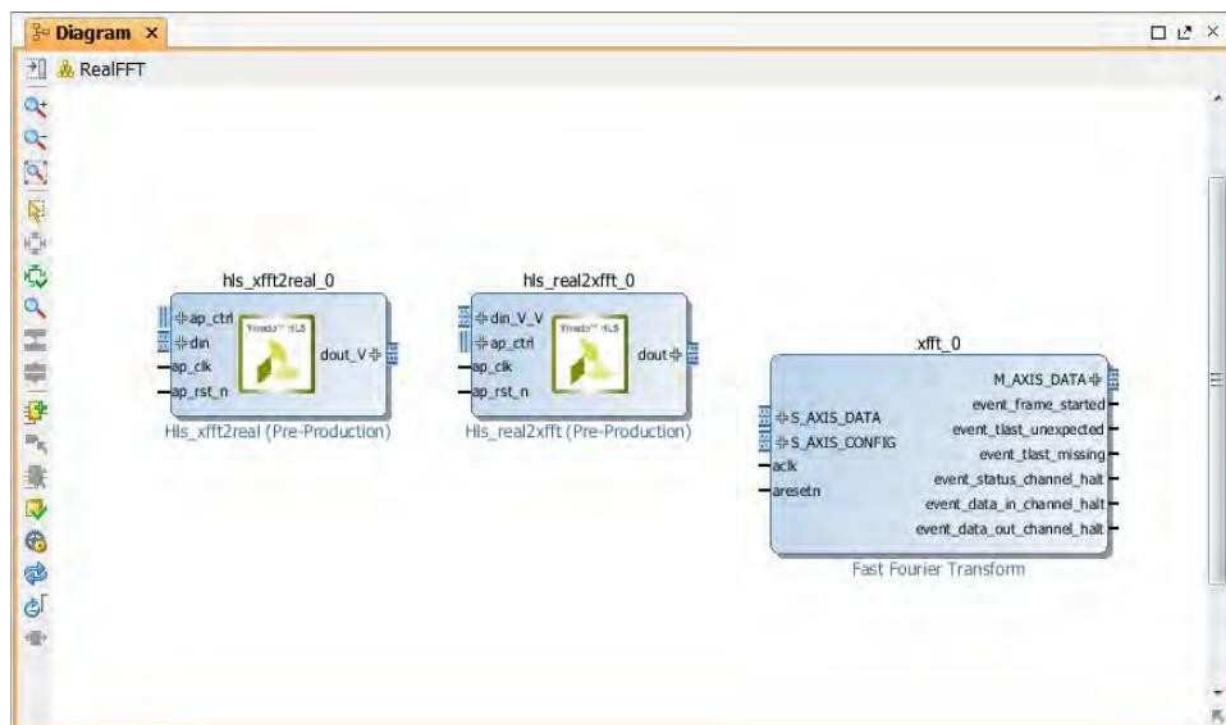


图 9-16: RealFFT IP 块

下一步是将 **HLS** 块连接到 **FFT** 块和端口。

8. 光标悬停在 **Hls_real2xfft** 块的 **dout** 接口连接器上，直到出现铅笔光标
 - a. 左键单击并按住鼠标按钮启动连接。
 - b. 将连接线拖到 **FFT** 块的 **S_AXIS_DATA** 端口连接器并释放（当绿色复选标记出现在其旁边时）。
 9. 以类似的方式，将 FFT 的 **M_AXIS_DATA** 接口连接到 **Hls_xfft2real** 模块的 **din** 接口。
- 这两个连接如图 9-17 所示。

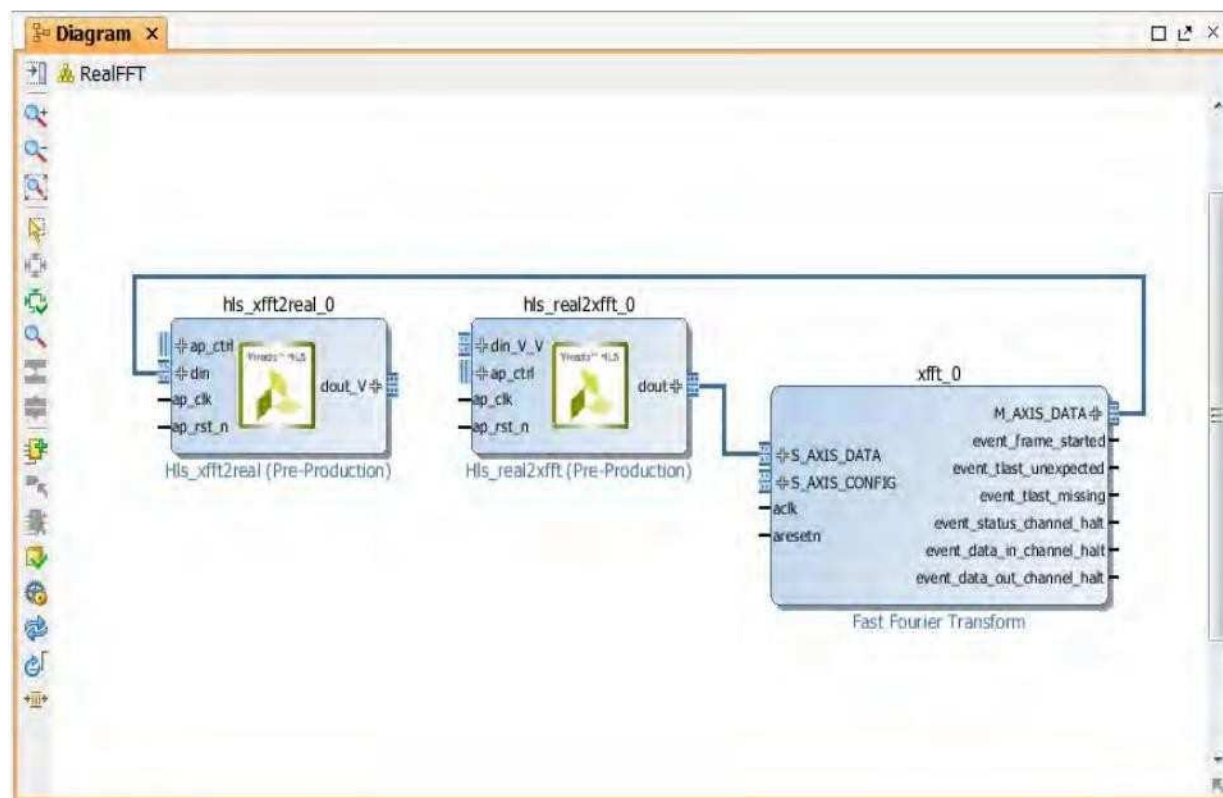


图 9-17: 连接 IP 块上的端口

要为设计创建 I/O 端口，需要进行一些外部连接。

10. 右键单击 hls_real2xfft 块上的 din_V_V 接口连接器，然后选择“设为外部”（图 9-18）。

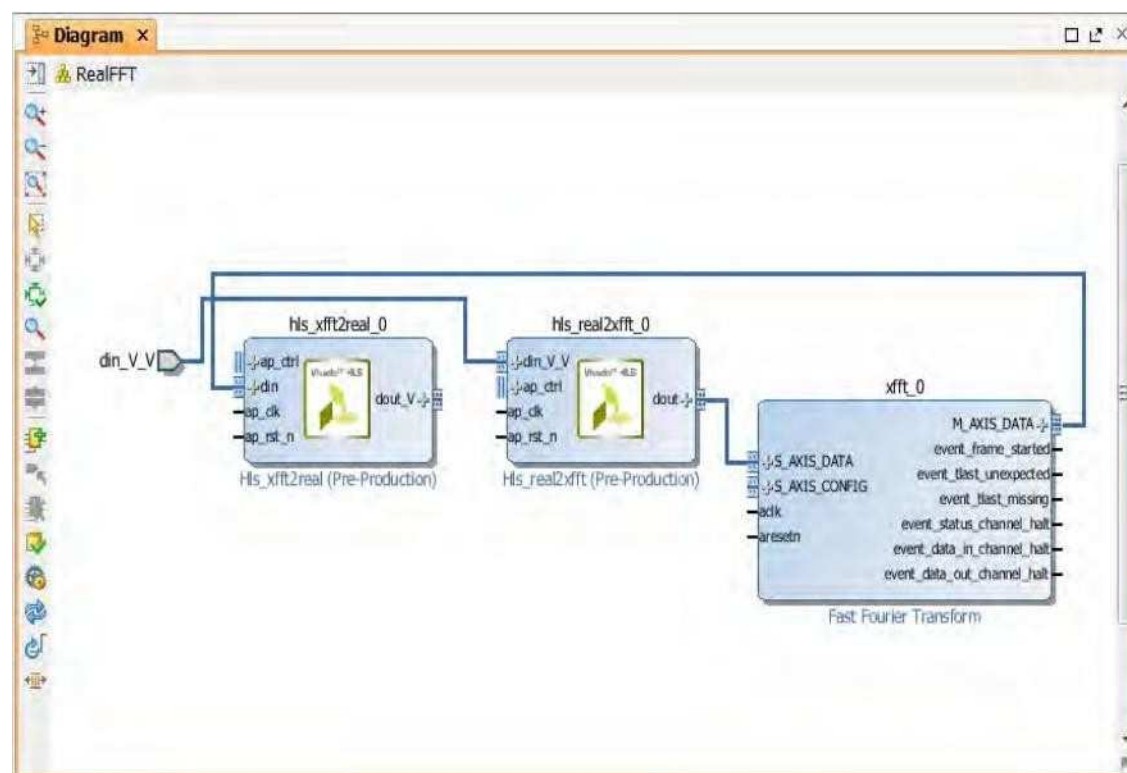


图 9-18: 建立外部连接

11. 给新接口端口一个唯一的名称。

a. 单击端口符号以突出显示它。

b. 在“外部接口属性”窗格(图 9-19)中，单击“名称”文本输入框以突出显示 **din_V_V**。

c. 键入 **real2xfft_din** 并回车。

重要提示: 如果不执行此重命名步骤，属性更改可能不会生效。



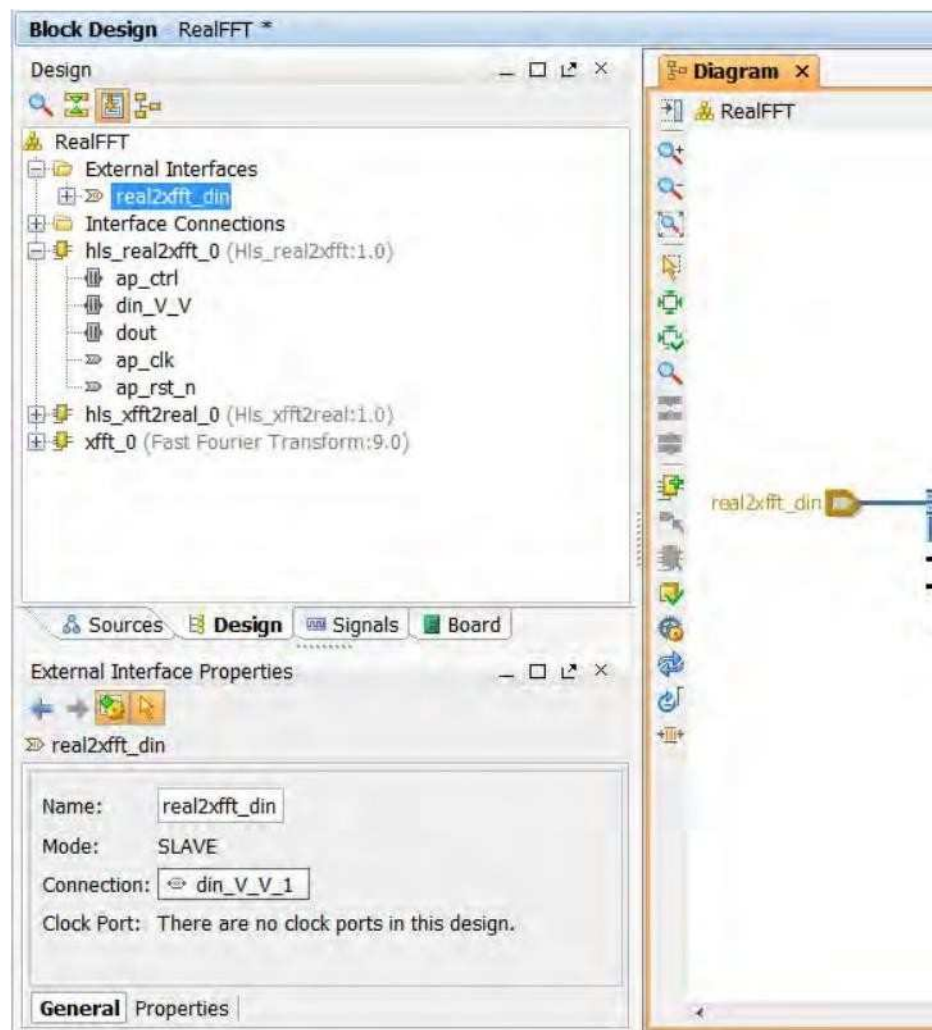


图 9-19: 港口命名

12. 以与前一步相似的方式:
 - a. 将 *Hls_xfft2real* 块的 *dout_V* 接口设置为外部, 并将其重命名为 *xfft2real_dout*。
13. 右键单击 FFT 模块的 *acclk* 连接器, 然后选择 “*Make External*”。
14. 右键单击 FFT 块的 *aresetn* 连接器, 然后选择 “*Make External*”。
15. 将两个 HLS 块的 *ap_start* 端口绑高。
 - a. 右键单击画布 (*canvas*) 并选择 *Add IP*。
 - b. 在搜索文本输入框 (*Search text entry box*) 中输入 *const*。
 - c. 选择 *Constant IP*。
 - d. 双击 “*Constant IP*” 符号 (图 9-21), 并确认 “*Const Width*” 和 “*Const Val*” 设置为 *1*。
 - e. 点击确定关闭重定义 IP 对话框。

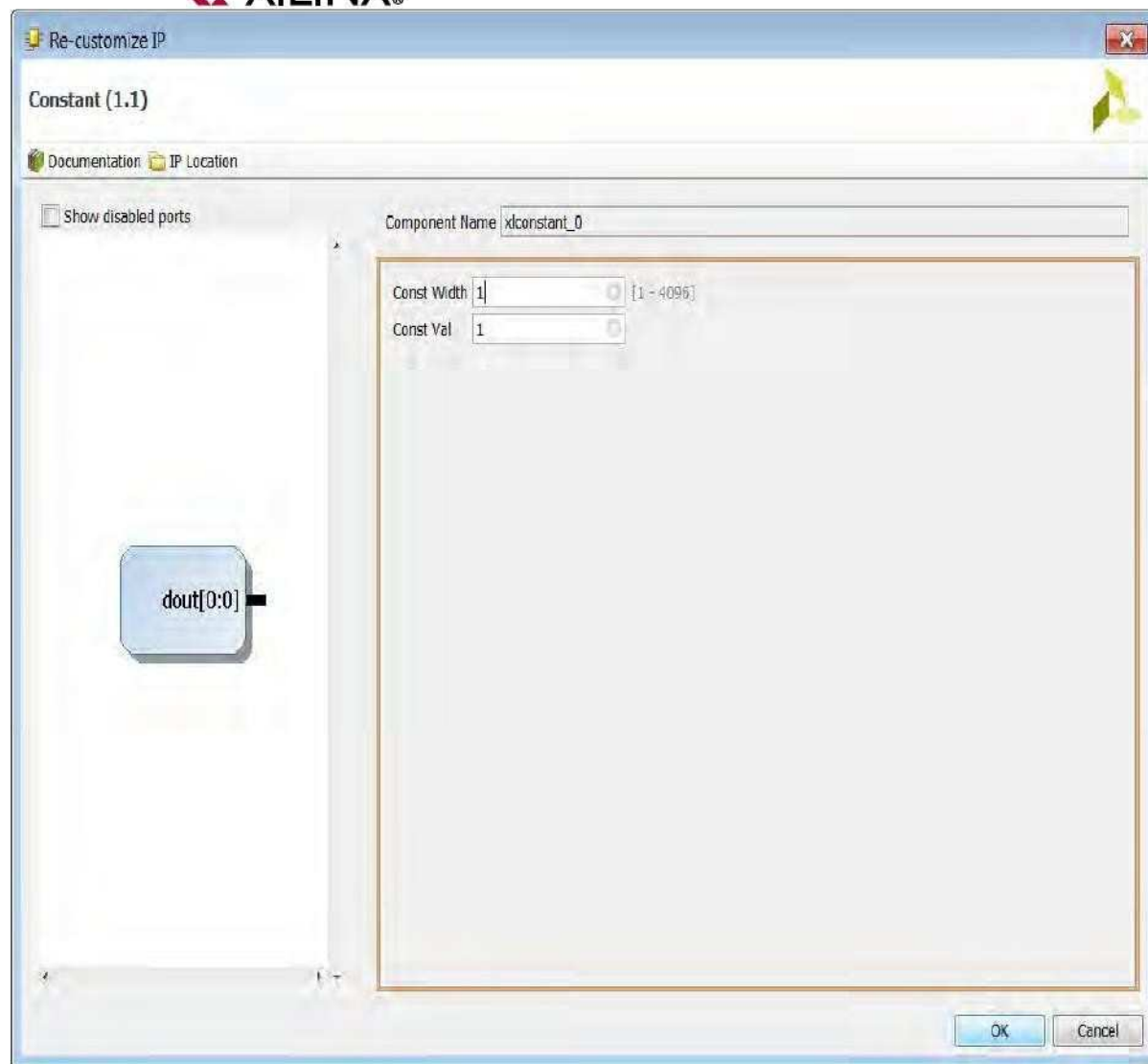


图 9-20: 不变的 IP 属性

- f. 在 *hls_xfft2real* 和 *hls_real2xfft* 上展开 *ap_ctrl* 总线端口(单击与每个端口相关联的加号)。
- g. 将两个 HLS 块中的 *ap_start* 连接到 *Constant* 块

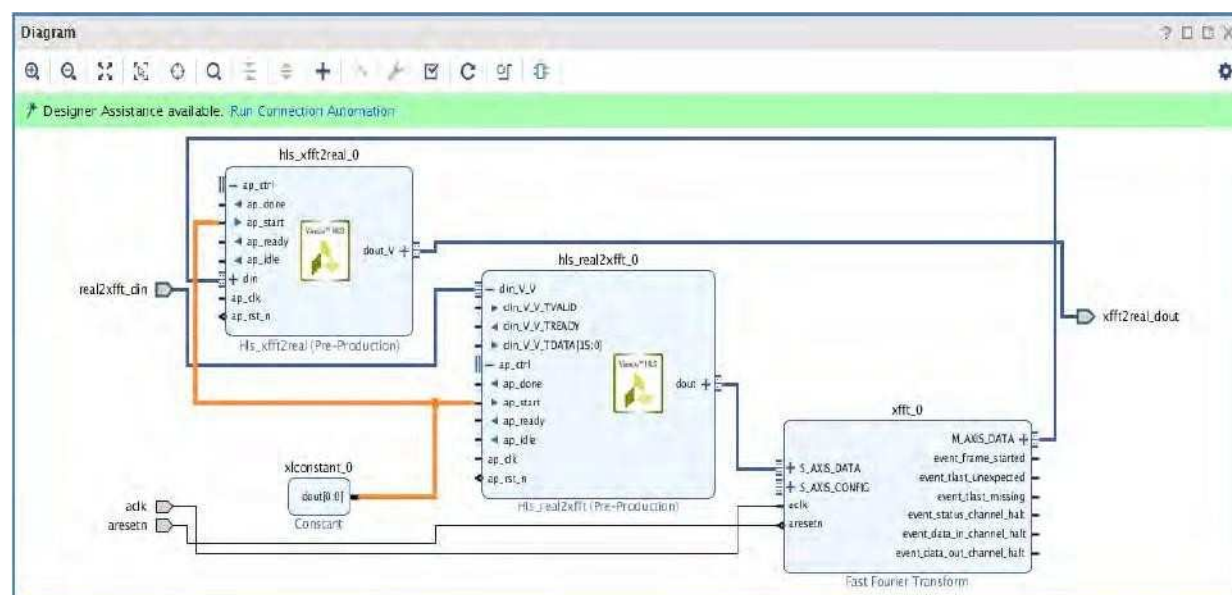


图 9-21: 将 AP_START 连接到常量

16. 建立其余的连接。

- a. 单击并从 *hls_real2xfft* 和 *hls_xfft2real* 块的 *aclk* 连接器拖动到 *aclk* 外部端口 (或 *FFT* 块上的 *aclk* 连接器或 “wire” 连接它们的任何位置)。
- b. 将 *hls_real2xfft* 和 *hls_xfft2real* 块的 *ap_rst_n* 连接到 *aresetn* 网络

17. 单击重新生成的布局图标 (*Regenerate Layout icon*) 来清理和重新组织块设计。

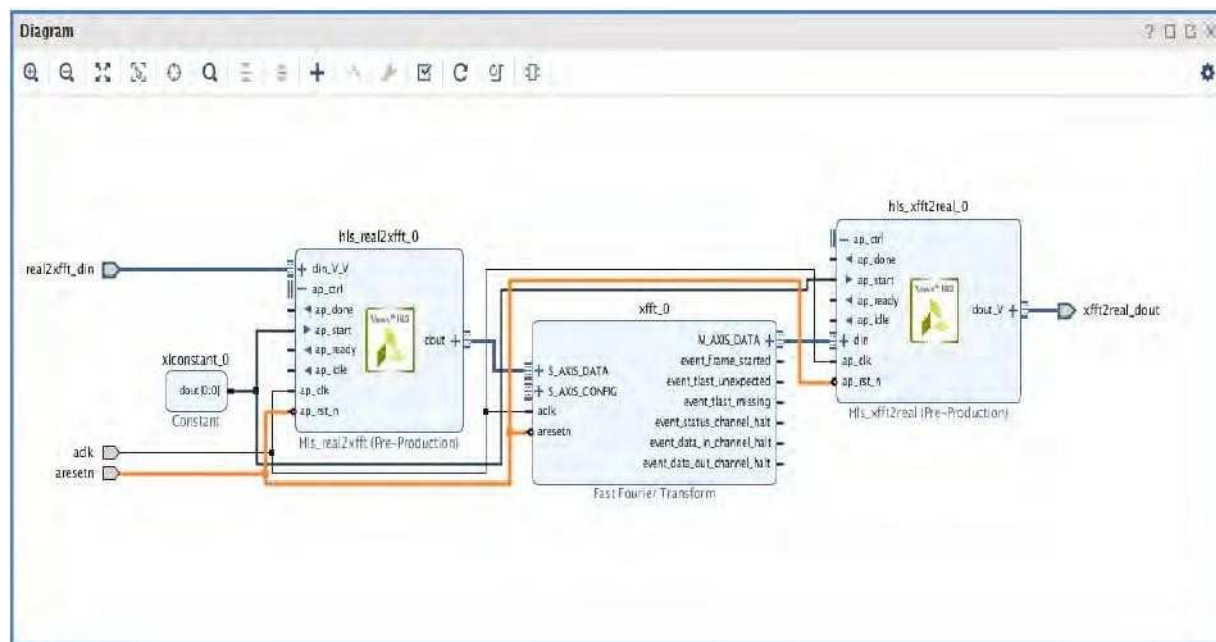


图 9-22: 重新生成的设计图

18. 单击 *Validate Design* 按钮以验证设计是否正确。

验证设计将显示一些警告。它们与 FFT 的 *s_axis_config* 引脚相关。

- a. XFFT 配置接口未连接，因为本设计始终在核心的默认模式下运行。
- b. 单击 OK 关闭消息。

19. 单击 **文件>保存块设计 (File > Save Block Design)**。

20. 关闭模块设计。

21. 下一步是生成输出产品。

- a. 在“项目管理器”窗格的“源”选项卡中（图 9-24），右键单击 *RealFFT.bd*，然后选择“生成输出产品”。
- b. 在结果对话框中单击“*Generate*”以启动所有输出产品的生成。
- c. 选择 *ok* 以忽略上面讨论的警告。

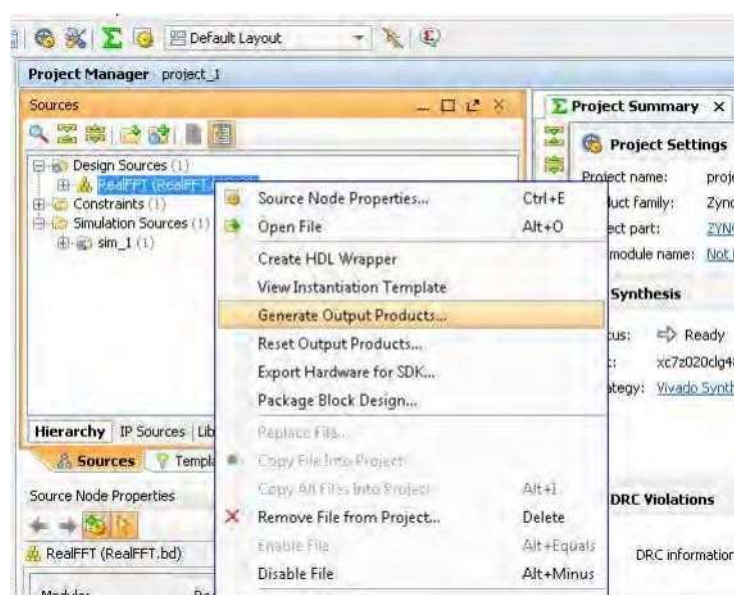


图 9-23: 生成输出产品

22. 创建一个 *HDL Wrapper* (包装器)。

- a. 在“项目管理器”窗格的“源”选项卡中，右键单击 *RealFFT.bd*，然后选择“*Create HDL Wrapper*”。这与上一步中描述的过程和菜单相同。

b. 点击 OK，让 Vivado 管理包装器。

第五步: 验证设计

创建最终设计的下一步是使用按实验练习中提供的HDL测试平台来验证设计: *realfft_rtl_tb.v*。

1. 在 Project Manager 窗格的 Sources 选项卡中右键单击 *Simulation Sources* (图 9-24)。
2. 选择 *Add* 来源。

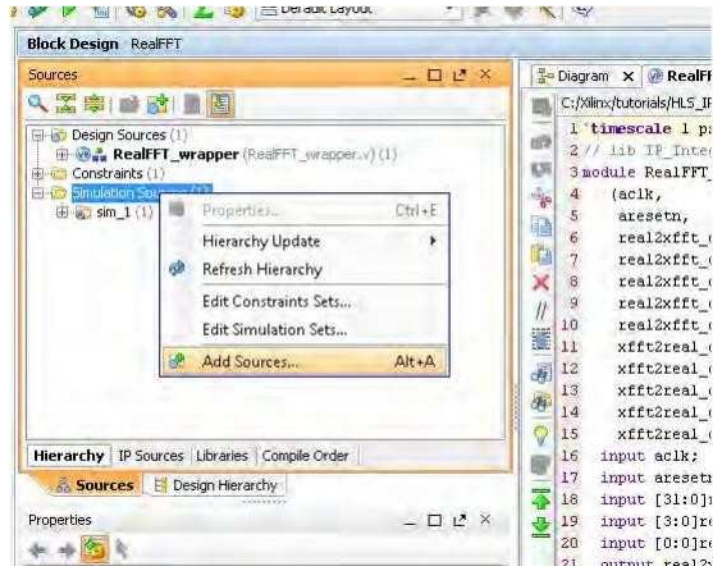


图 9-24: 添加模拟源

3. 在“添加源”对话框中选择“*Add or Create Simulation Sources*”。
4. 单击 Next。
5. 在“添加源”对话框中，单击“+”符号图 9-25 并选择“*Add Files*”。

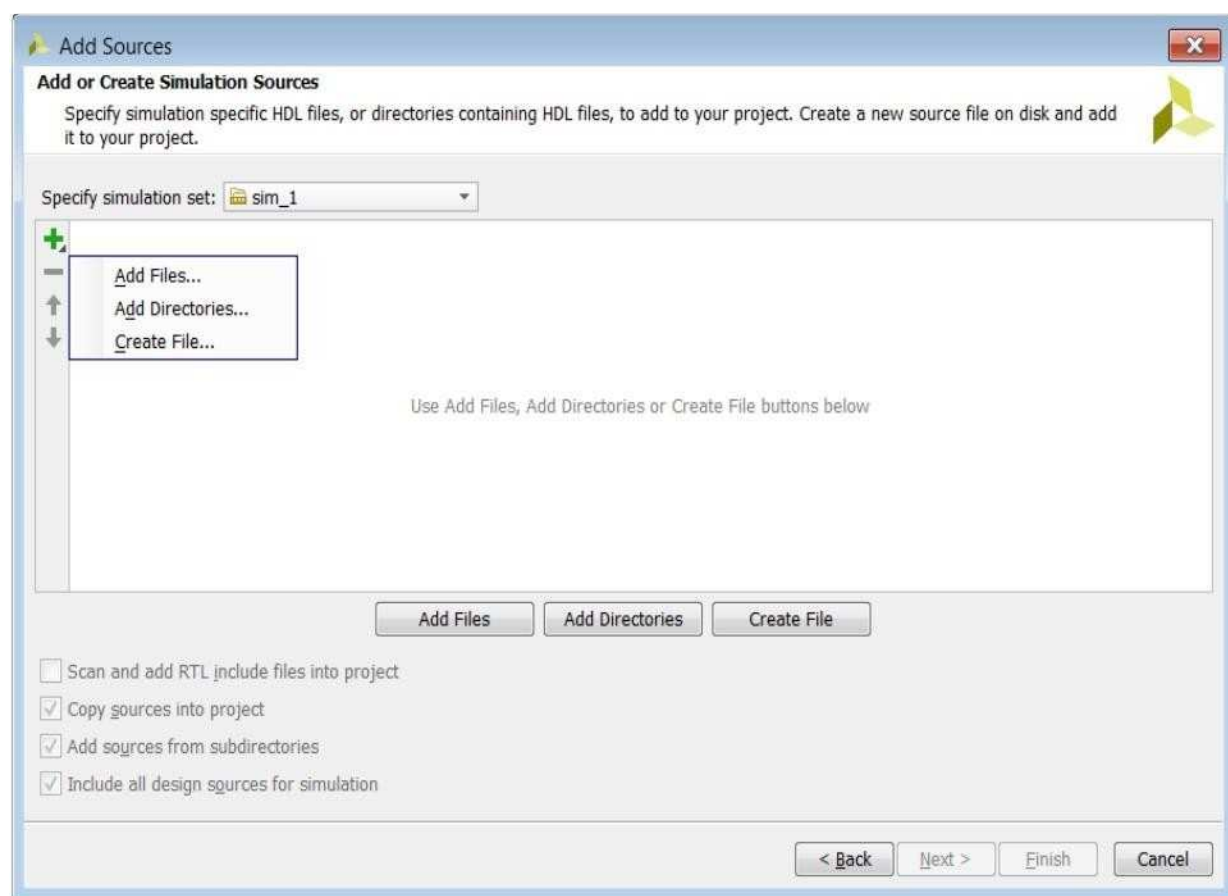


图 9-25: 添加源对话框窗口 6。

6. 在教程目录 *Using_IP_with_IPI\lab1\verilog_tb* 中浏览到文件 *realfft_rtl_tb.v*。

7.选择它并单击 **OK**。

8.选择复选框将 *Copy sources into the project* (图 9-26)。

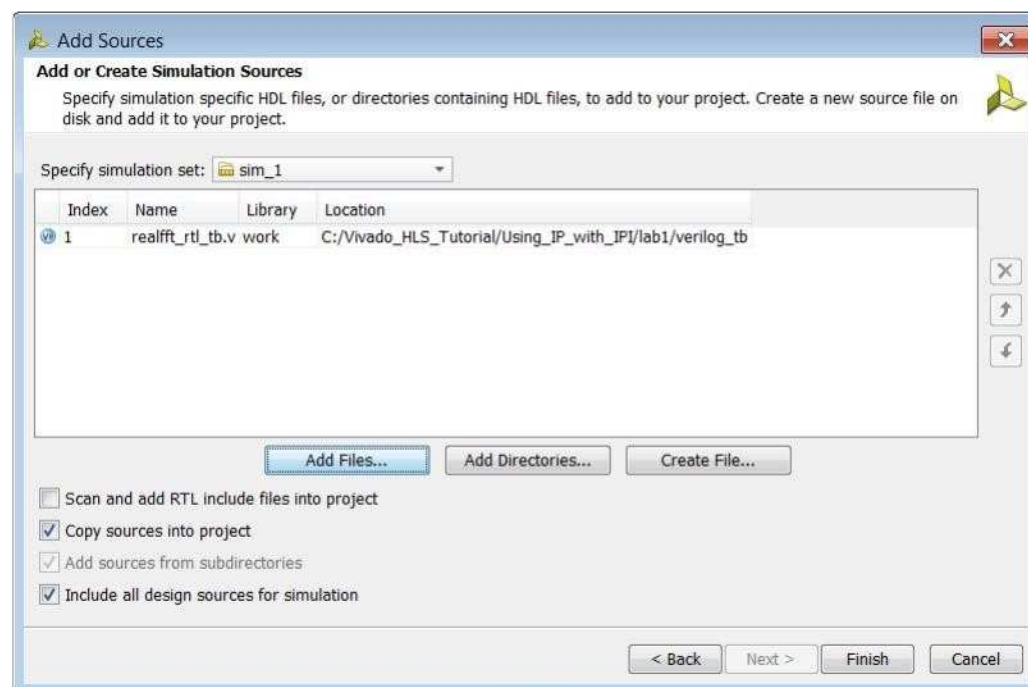


图 9-26: 复制设计源文件

注意: 当您将设计源文件复制到项目中时，对文件的编辑不会影响到原始源文件。

9.单击 **Finish**。

10.在流导航器中单击 **Run Simulation**(图 9-27)并选择 *Run Behavioral Simulation*。



图 9-27: 执行模拟

11.模拟开始后，单击 **Run All** 图标完成模拟。



图 9-28: 运行仿真得出结果

结论

在本教程中，您学习了：

如何使用 Tcl 脚本创建 Vivado HLS IP。

如何导入一个创建的设计使用 IP intergrator(集成器)，包括 Xilinx IP 和 Vivado IP 块。

在 Zynq SoC 设计中使用 HLS IP

概述

高级综合设计的一个**常见用途**是为 CPU 创建一个加速器——将 CPU 上执行的代码移动到 FPGA 可编程逻辑中以提高性能。本教程展示了如何将使用高级综合创建的设计合并到 Zynq 设备中。

本教程包括两个实验练习：

实验室 1 描述

您可以创建并配置一个简单的 HLS 设计，使其与 Zynq 设备上的 CPU 一起工作。本实验使用的 HLS 设计很简单，因此**本教程的重点是解释与 CPU 的连接，以及如何配置由高级综合创建的软件驱动程序来控制设备和管理中断。**

实验室 2 描述

该实验演示了一种通用的高性能连接方案，用于连接使用 CPU 内存中的数据和/或以流方式为其生成数据的硬件加速器块。实验强调软件需求以避免缓存一致性问题。

教程设计描述

您可以从 Xilinx 网站下载教程设计文件。请参阅定位教程设计文件中的信息。本教程使用教程目录中的设计文件

Vivado_HLS_Tutorial \ Using_IP_with_Zynq。

样本设计是一个简单的**多重累积块**。**本教程的重点是软件驱动程序的方法、连接和集成。**(本教程并不关注设计本身的逻辑。)

实验 1:在 Zynq 设备上实现 Vivado HLS IP

这个实验练习在 Zynq 设备上实现的设计中集成了高级综合 IP 和由 HLS 创建的软件驱动程序来控制 IP。



重点:本教程中的图形和命令使用教程数据目录

将 Vivado HLS Tutorial 解压缩并放置在位置 C:\Vivado HLS Tutorial 中。如果将教程数据目录解压缩到不同的位置，或者在 Linux 系统上，请将引用的少数路径名调整到您选择的位置，以放置 Vivado_HLS_Tutorial 目录。

步骤 1:创建一个 Vivado HLS IP 块

1.打开 Vivado HLS 命令提示符。

在 Windows 上使用 **Start > All Programs > Xilinx Design Tools > Vivado 2018.3 > Vivado HLS > Vivado HLS 2018.3 Command Prompt**

- 在 Linux 上,打开一个新的 shell。
- 2.使用命令提示符窗口,将目录更改为
- Vivado_HLS_Tutorial \ Using_IP_with_Zynq \ lab1 \ hls_macc**(图 1)。
- 3.输入 **vivado_hls -f run_hls.tcl** 来创建 HLS IP(图 10-1)。

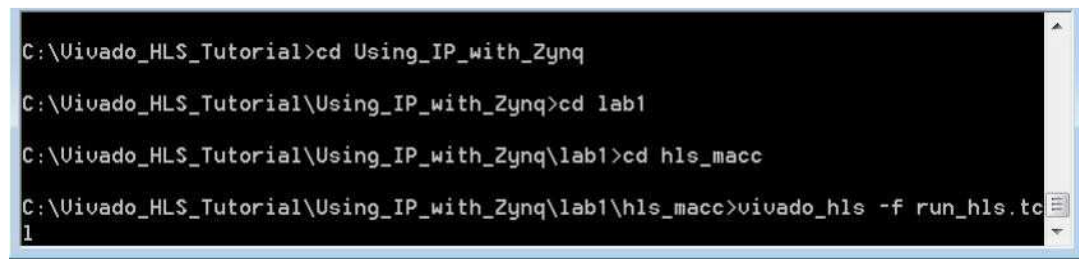


图 10-1:创建 HLS 设计

脚本完成后,将出现一个 Vivado HLS 项目目录 **vhls_prj**, 其中包含 **HLS IP**, 其中包括供 Vivado 设计使用的 **Vivado IP Catalog** 存档。

本教程练习的其余部分显示了如何使用 **IP Integrator** 将 Vivado HLS IP 模块集成到 Zynq 设计中。

步骤 2:创建一个 Vivado Zynq 项目

- 1.启动 Vivado 设计套件(不是 Vivado HLS):
- 在 Windows 上使用启动>所有程序> Xilinx 设计工具> Vivado 2019.1 > Vivado 2019.1。
- 在 Linux 上,在 shell 中键入 vivado。
- 2.在欢迎屏幕中,单击 **Create New Project**(图 10-2)。



- 在新建项目向导: a.单击 **Next**。
- b. 在“项目位置”文本输入框中,浏览到教程文件目录 **Using_IP_with_Zynq\lab1** 的

- 位置，然后单击 “*Next*”（图 10-3）。
- 在 “项目类型” 页上，选择 “RTL 项目”，此时 *Do not specify sources at this time* (如果它不是默认的)。
 - 单击 *Next*。

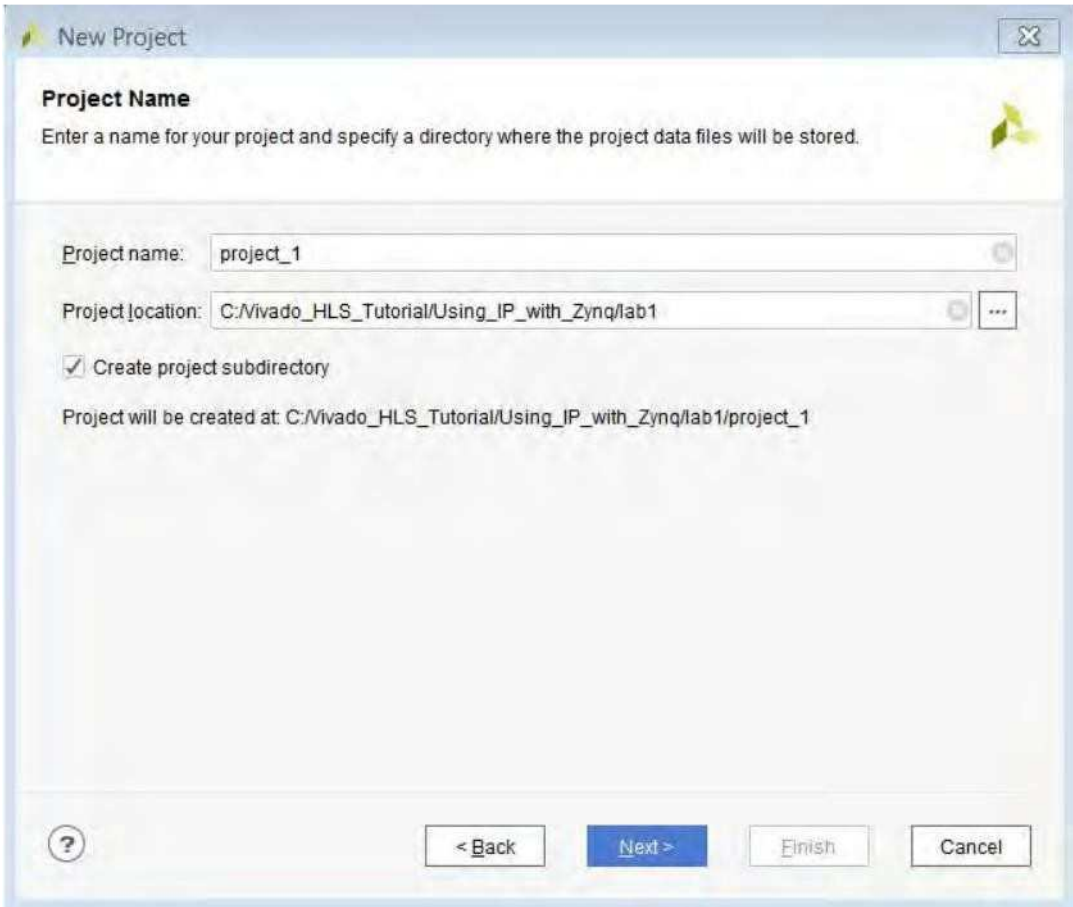


图 10-3: 指定 Vivado 项目目录

- 默认部分页面:
 - 单击 *Boards*。
 - 选择 ZYNQ-7 ZC702 评估板(图 10-4)。

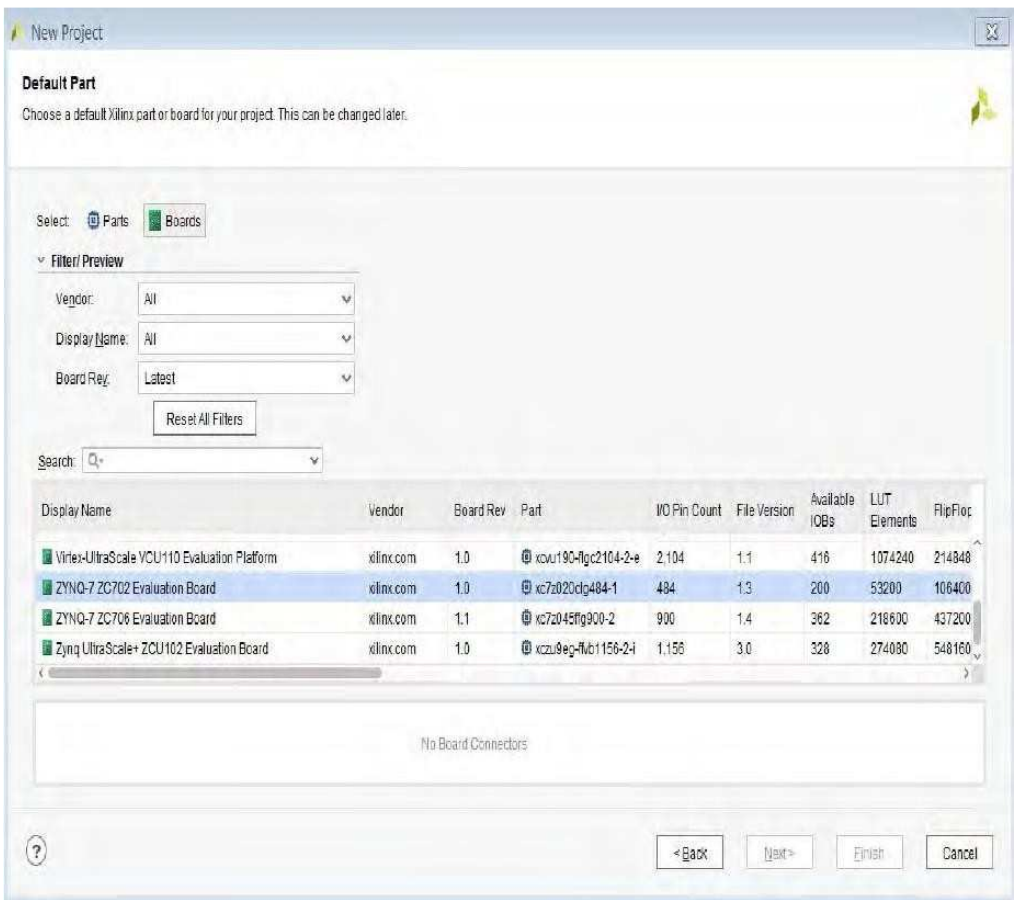


图 10-4: 指定 Vivado 项目细节

- c. 单击 Next。
 - d. 在新建项目摘要页面上单击 Finish。
- 项目工作区打开，如图 10-5 所示。

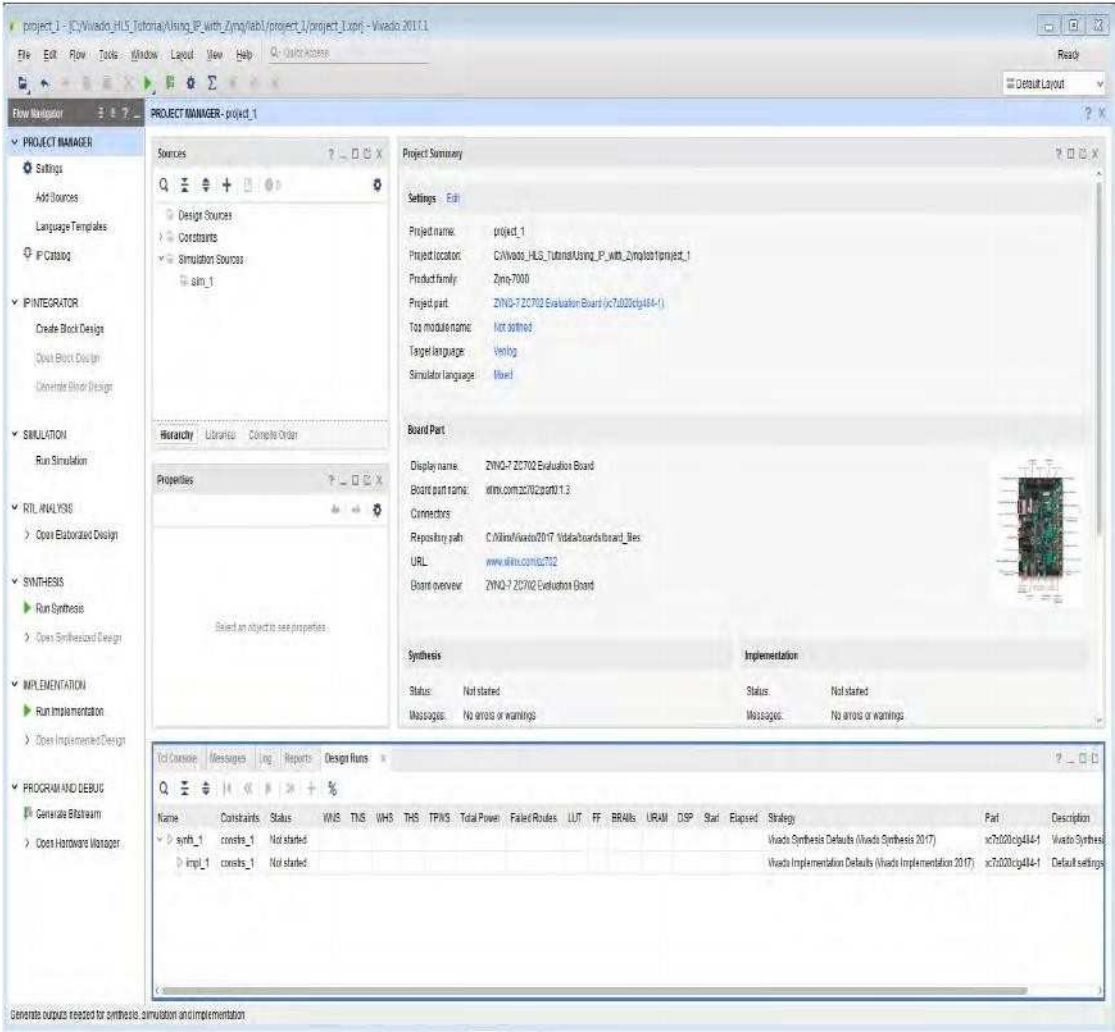


图 10-5: 最初的 Vivado Zynq 项目

步骤 3: 将 HLS IP 添加到 IP 目录

1. 在 **Flow Navigator** 窗格的 **Project Manager** 区域，单击 **IP Catalog**。

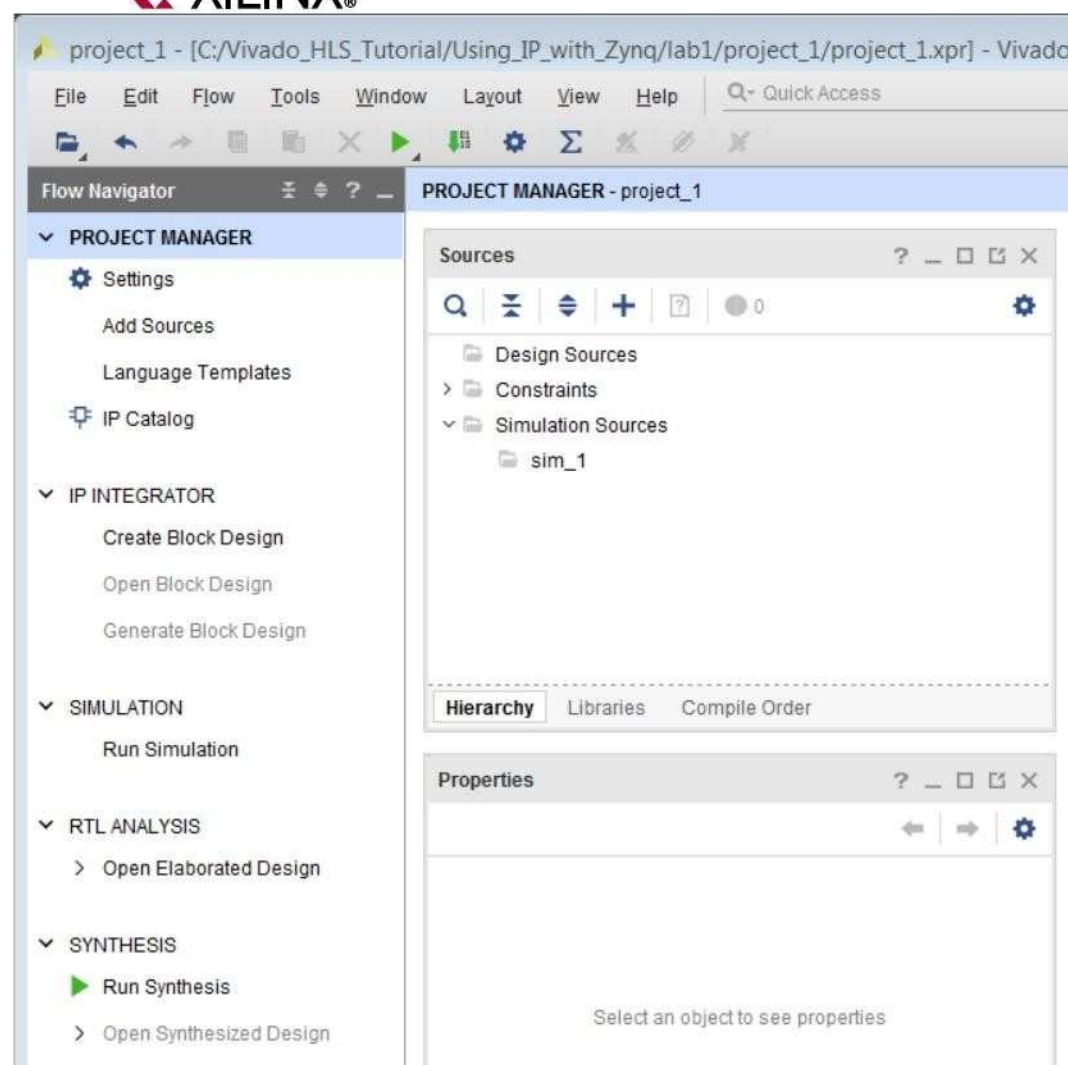


图 10-6: 打开工作区的主窗格中

出现的 IP 目录。

2. 在空白处右键单击，并选择 **Add Repository**。



图 10 - 7: 结 打开 IP 目录设置

3. 右键单击 **IP Catalog** 画布并选择 **Add Repository**。
4. 在 “**IP Repositories**” 对话框中:
 - a. 浏览到 Vivado HLS 创建的 IP 位置，使用 **Using_ip_with_zynq\lab1\hls_macc\vhls_prj\solution1\impl\ip** 并单击 **Select**。

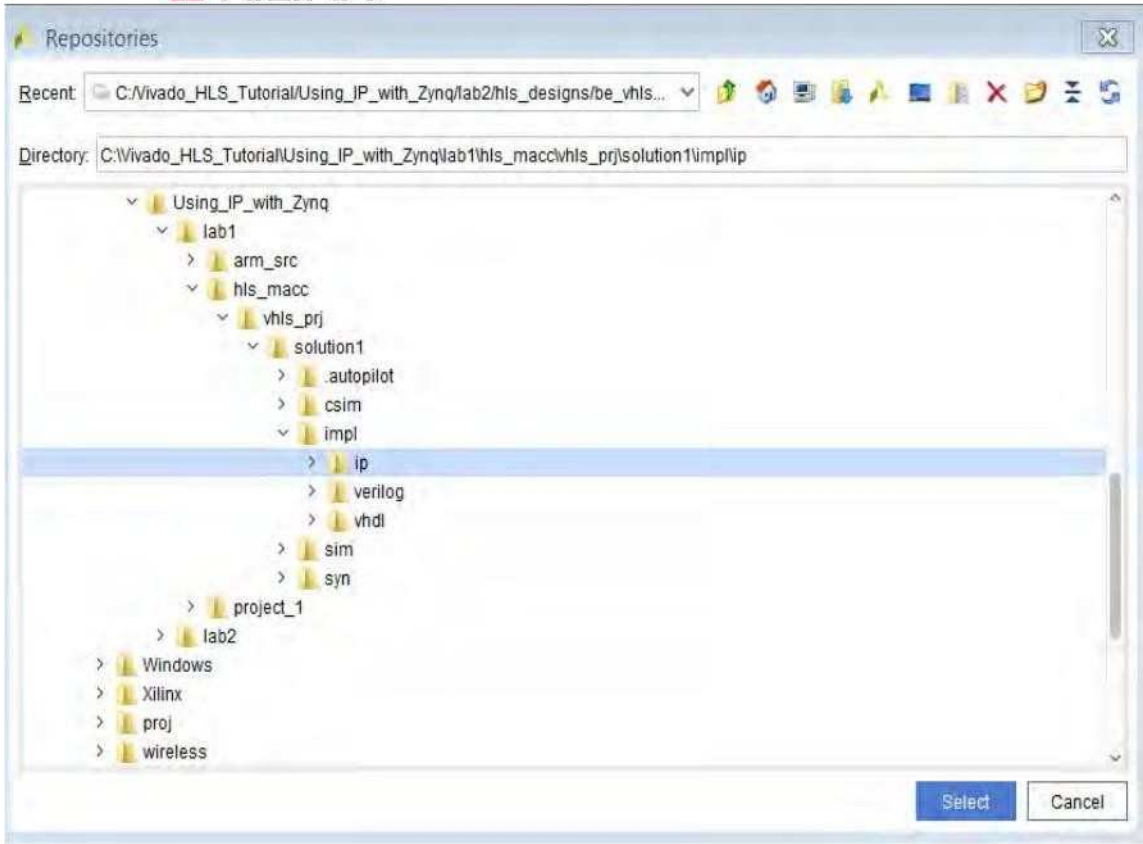


图 10-8: IP 存储库

5.单击 **OK** 关闭 IP 存储库管理器。

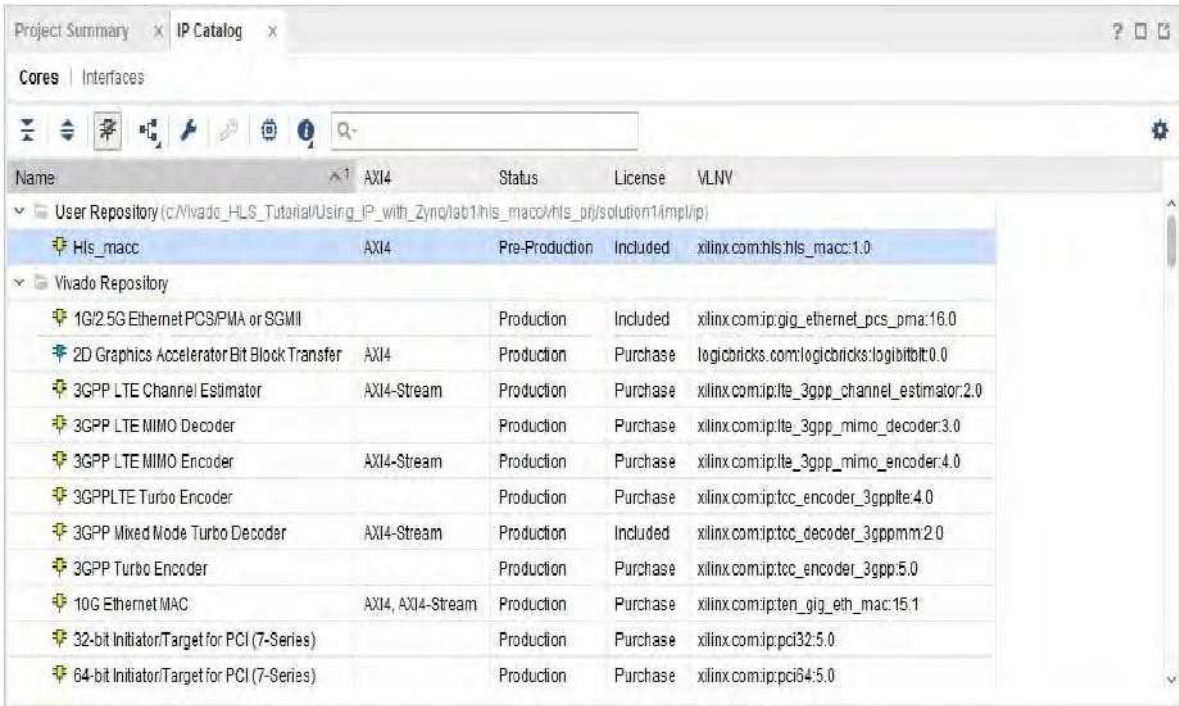


图 10-9: 储存库中的 HLS IP

6.现在在 IP 目录中有一个 HLS IP, **Hls_macc**.

步骤 4: 创建一个 IP 集成器块设计的系统

1.在流导航器的 IP 集成器区域, 单击 “**创建块设计**” 并在对话框中键入 **Zynq_Design**。

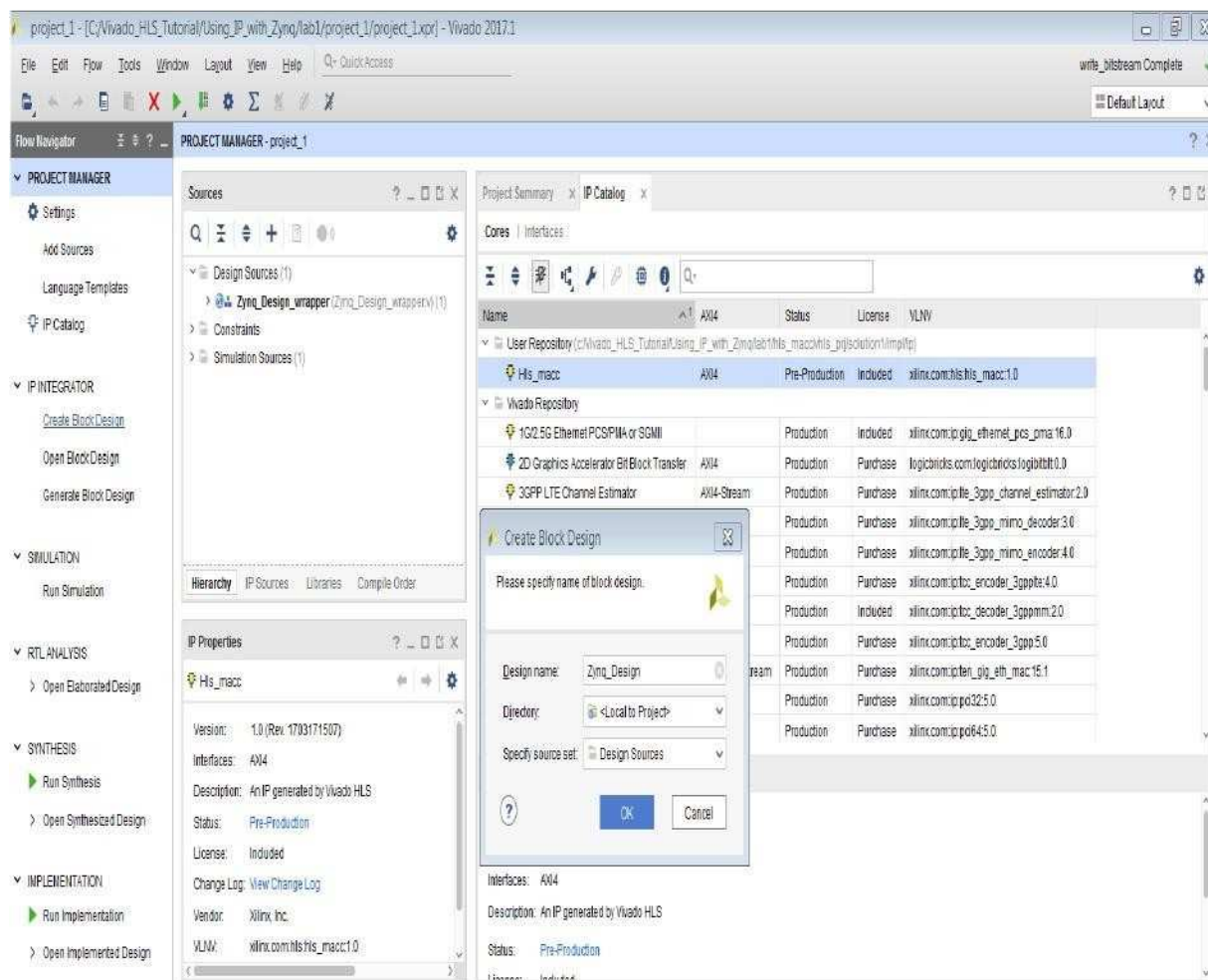


图 10-10: 创建 Zynq 设计

块设计视图在主窗格中打开，其中有一个新的 **Diagram** 选项卡，其中包含一个空白块设计画布。

2. 按下主屏幕上的添加 IP 按钮，打开 IP 搜索对话框。

- a. 在搜索文本输入框中键入 **zynq**。
- b. 选择 **ZYNQ7** 处理系统，回车。

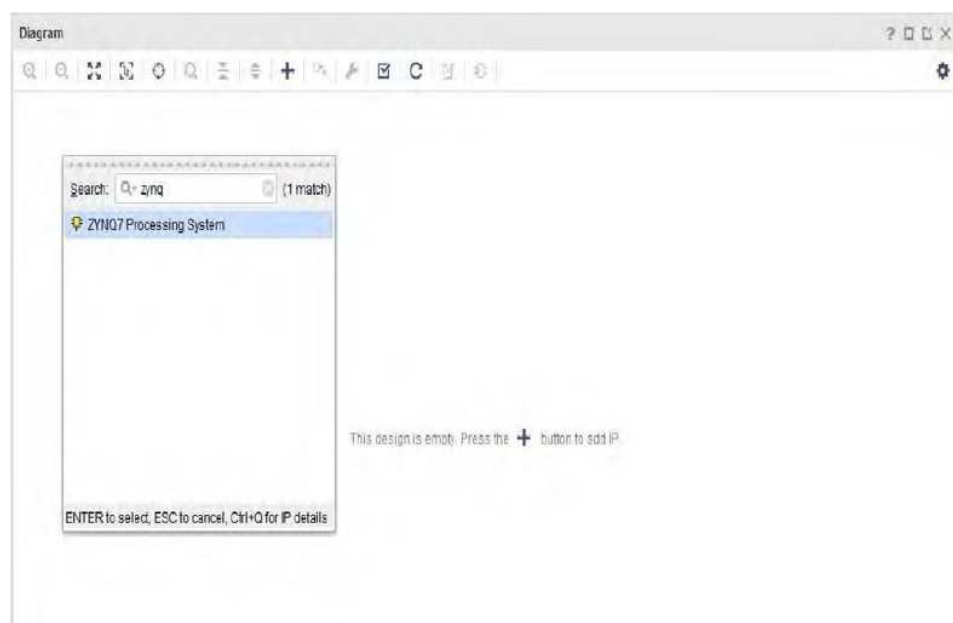


图 10-11: 在设计中添加一个 CPU 处理器

画布上出现了 **ZYNQ7** 处理系统的 IP 符号。

3. 双击 **ZYNQ IP** 符号打开相关的重新自定义 IP 对话框。

- a. 点击 **Presets** 图标，选择 **ZC702** (图 10-12)。

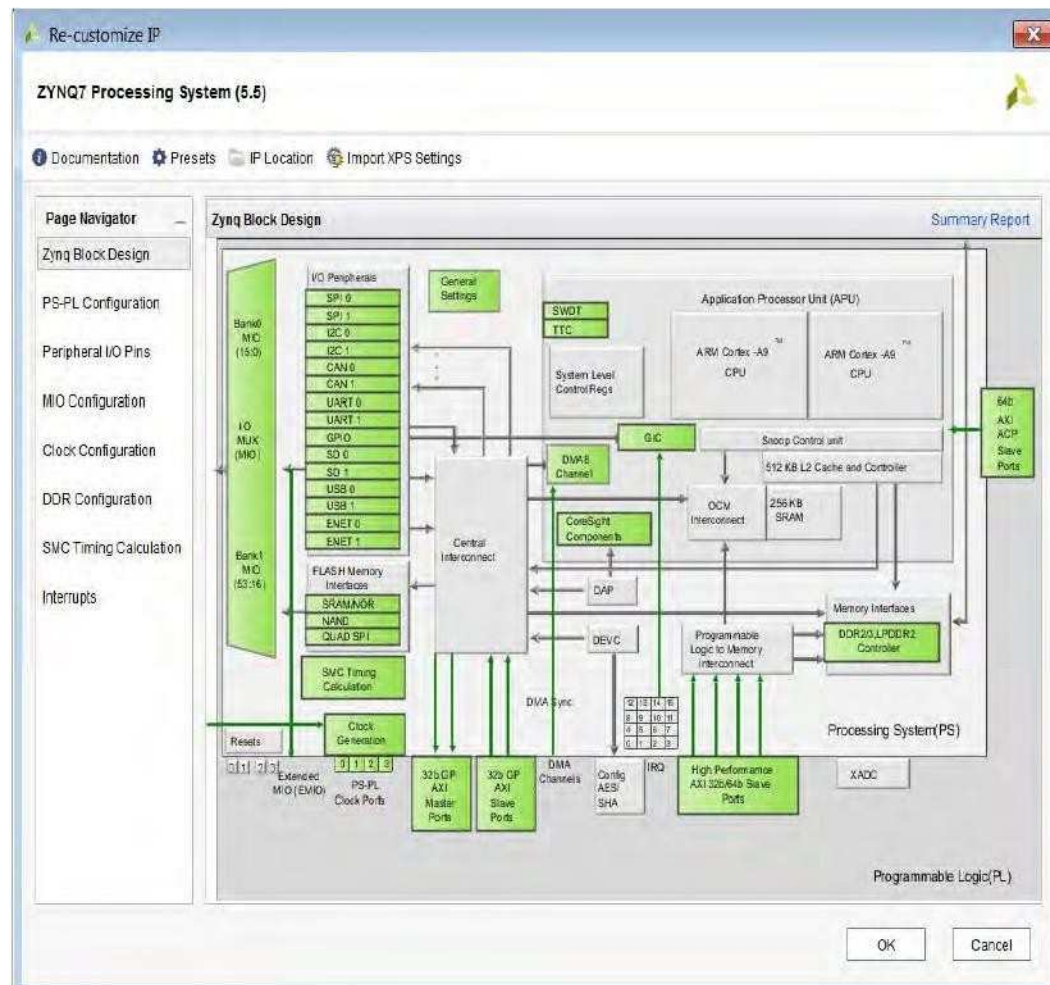


图 10-12:配置 Zynq SoC 。

在页面导航窗格中单击 **MIO** 配置。展开应用处理器单元树视图。取消对 **Timer 0** 的选择(如果选择了其他定时器)。

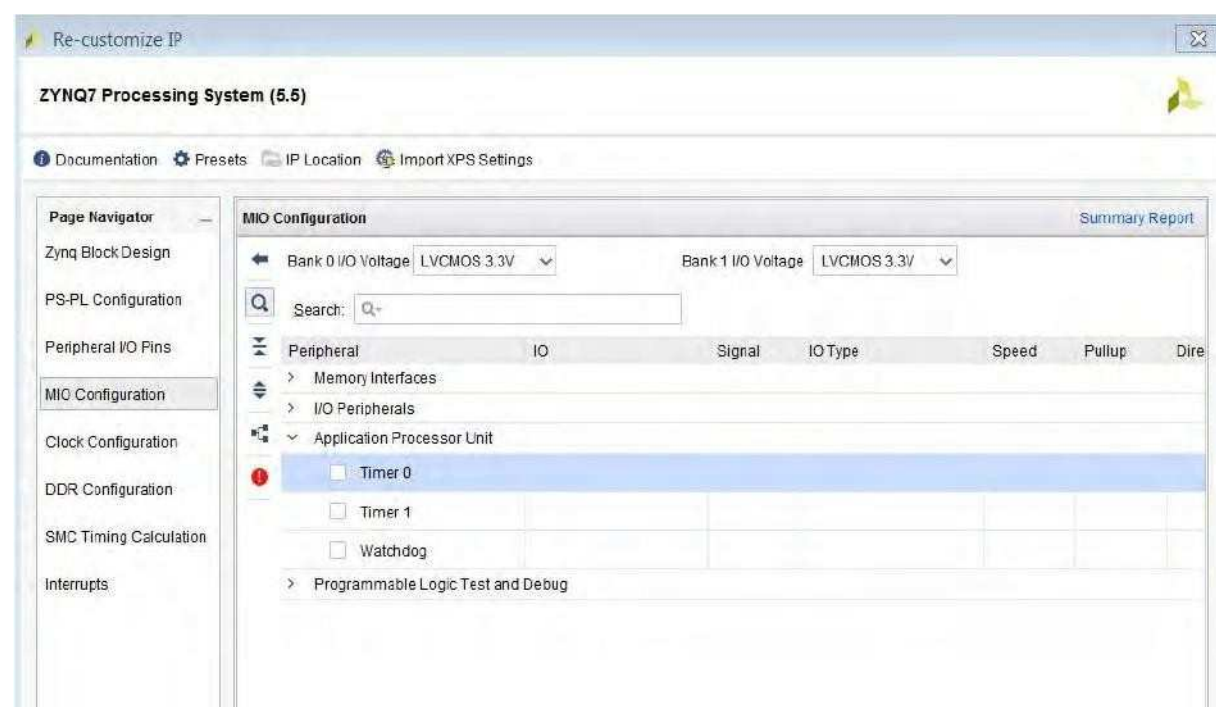


图 10-13:Zynq SoC MIO 配置

- 5.在页面导航窗格中单击 **Interrupts**。
 - a.选择 **Fabric Interrupts** 并展开它的树视图并展开 **PL-PS 中断端口**。
 - b.选择 **IRQ_F2P[15:0]**，点击 **OK** 关闭重定义 IP 对话框。

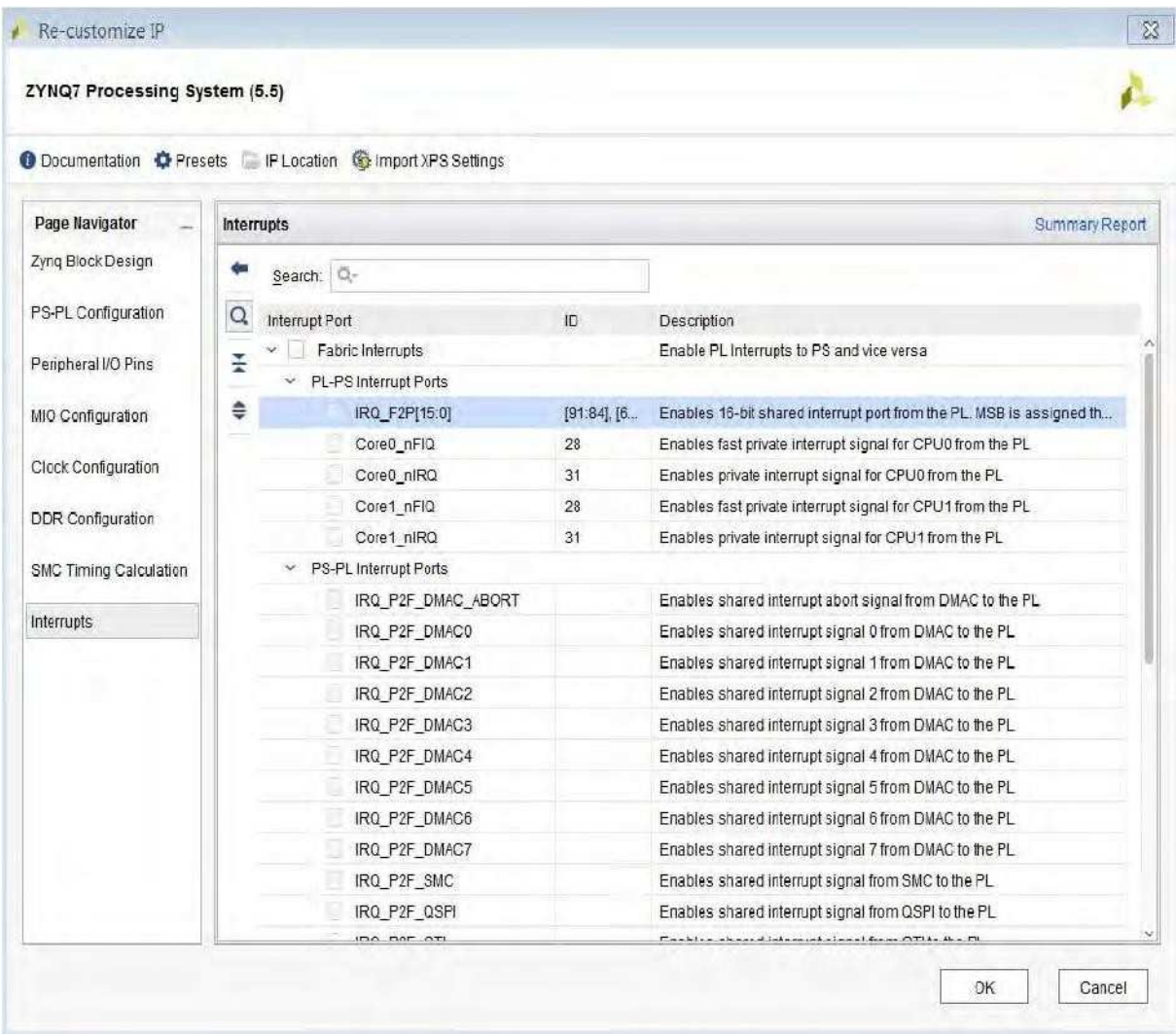


图 10-14: Zynq SoC 中断配置

IPI 为某些任务提供了自动化的设计帮助，例如为 ZYNQ PS7 建立正确的 **DDR** 内存外部连接和固定 I/O。

- 6.单击标题栏下的 **Run Block Automation 链接**(图 10-15)。
 - a.确保选择 **processing_system7_0**。
 - b. 确保取消选择 “**Apply Board Presets**”。如果仍然选择此选项，它将重新应用在步骤 4 中禁用的计时器，并在图 10-15 中的 Zynq 块上导致其他端口在结果对话框中单击 OK 完成。

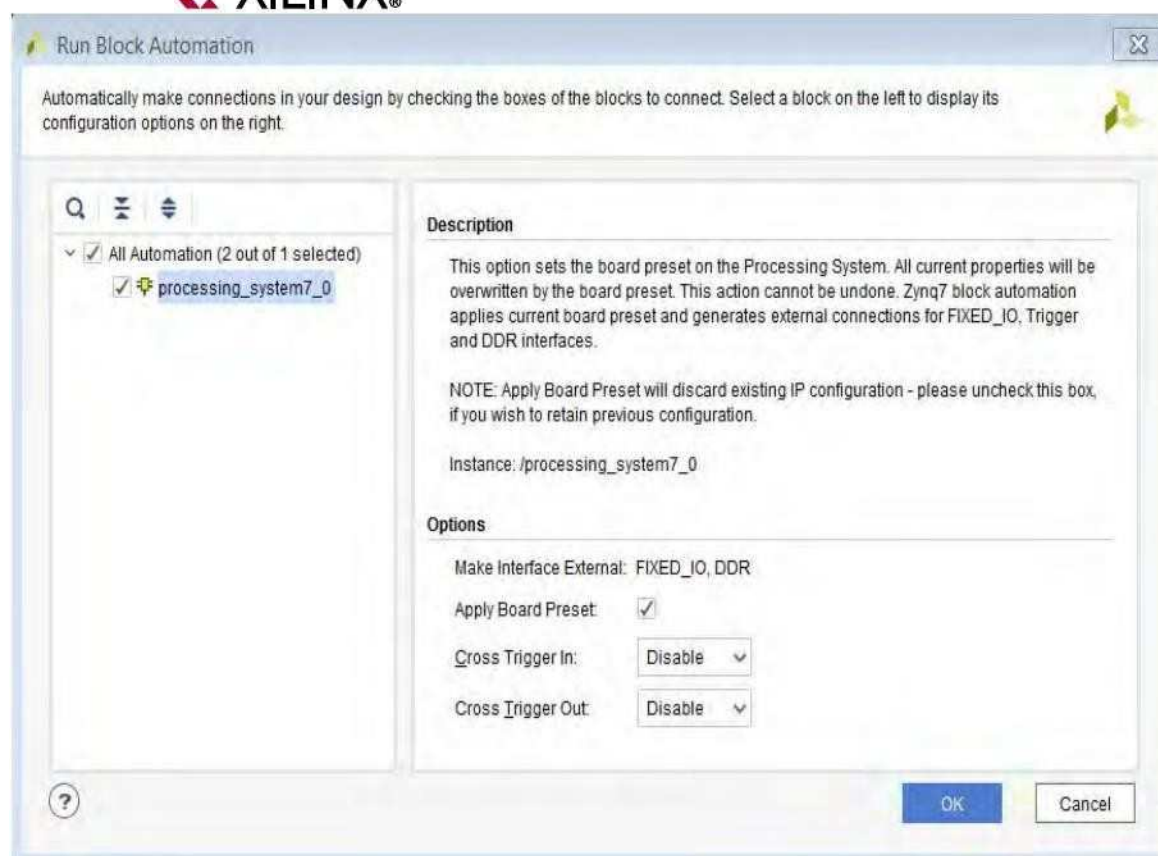


图 10-15: 运行自动化

7. 在设计中加入 HLS IP:

- 在画布的空白处右键单击，从上下文菜单中选择 **Add IP**。
- 在搜索文本输入框中输入 **hls** 并按回车键将其添加到设计中 (图 10 到 16)。

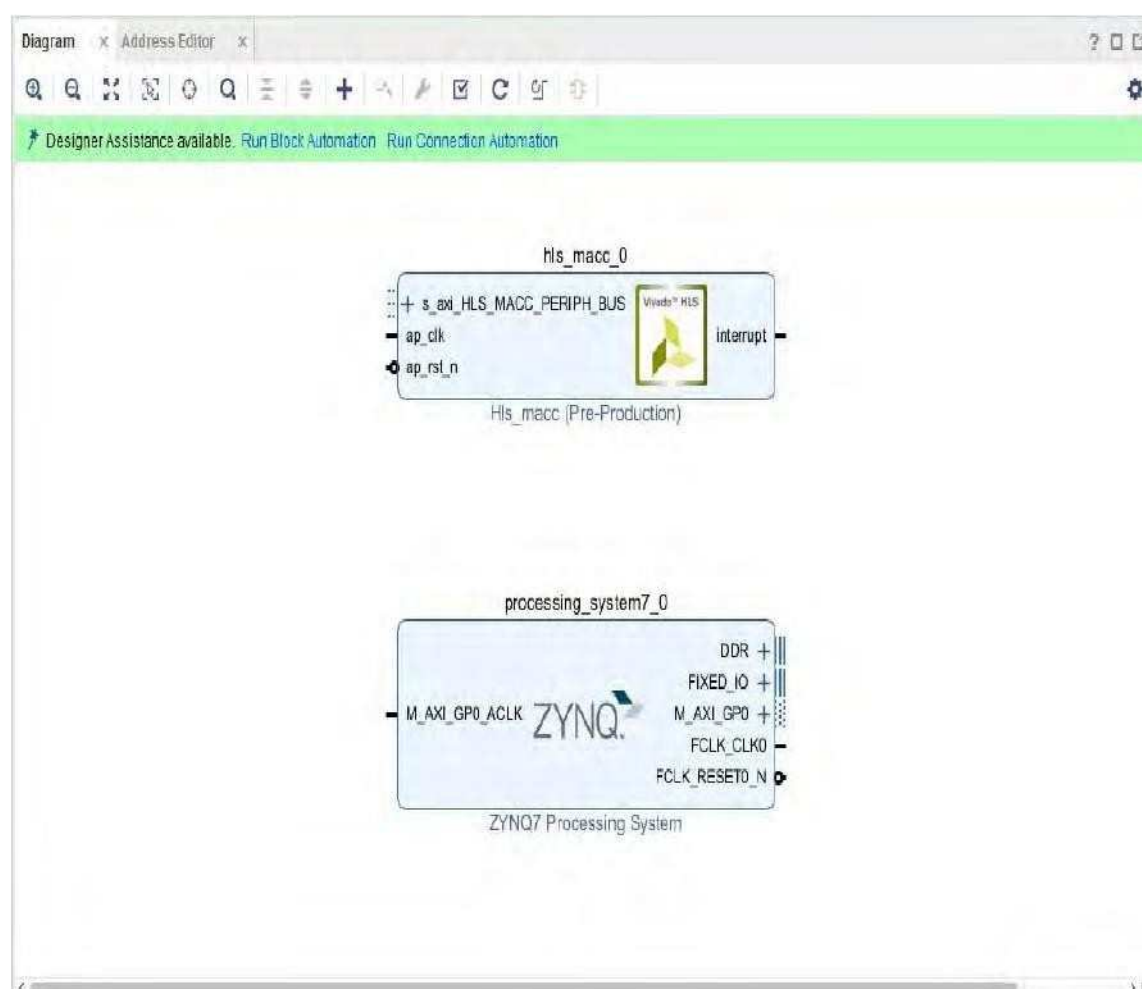


图 10-16: 处理器和 HLS IP

设计人员也可以协助实现 IP 块的自动连接。

- 单击画布顶部的 **Run ConnectionAutomation** 链接。
- 选择 **/hls_macc_0/S_AXI_HLS_MACC_PERIPH_BUS**，然后在出现的对话框中单击“确定”，将 HLS IP 自动连接到 Zynq 处理器的 **M_AXI_GPO** 接口。

这将添加一个 **AXI 互连**（块实例：*processing_system7_0*），一个 *Proc Sys Reset* 块，并进行所有必需的与 AXI 相关的连接，以创建如图 10-18 所示的设计。

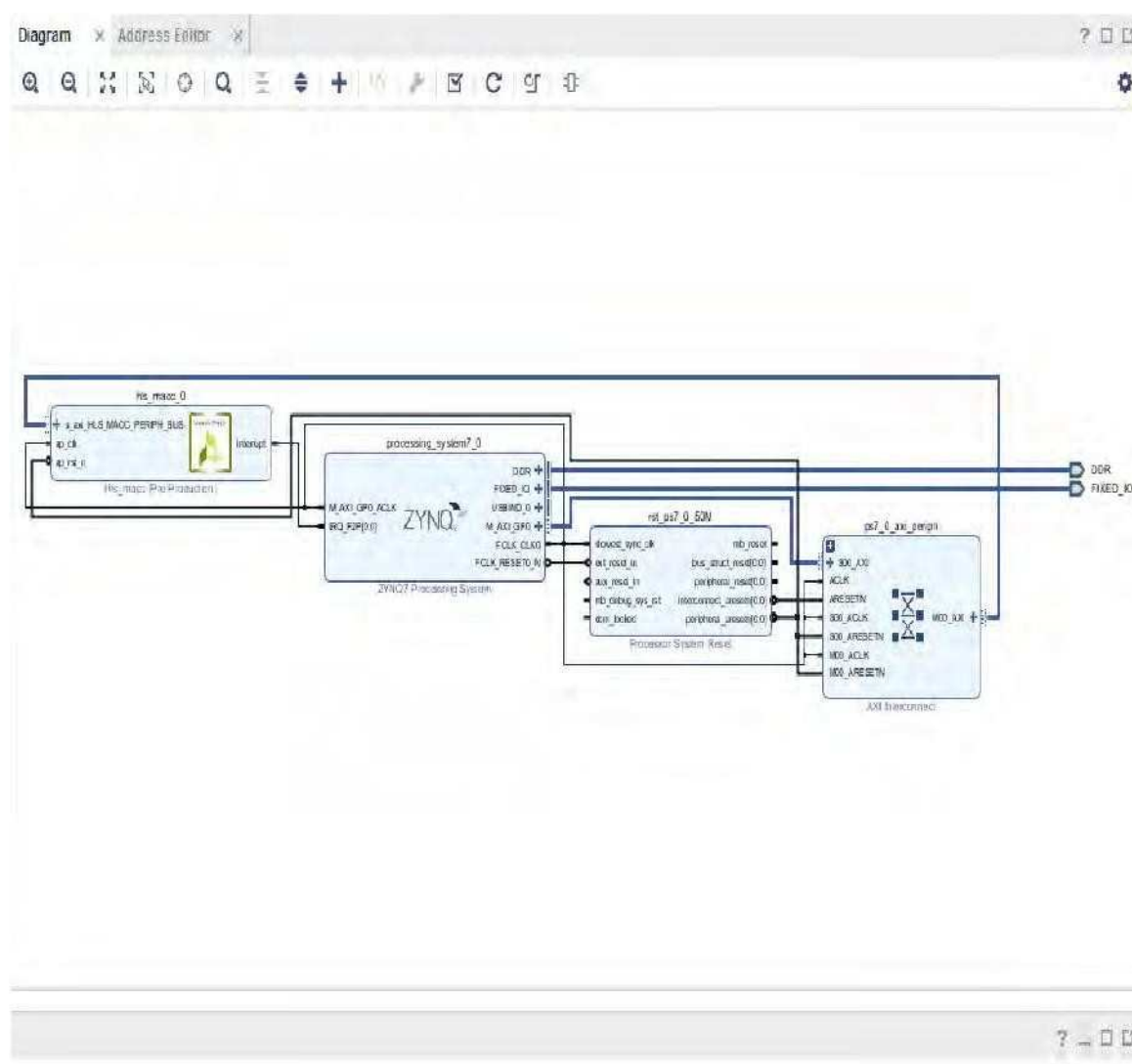


图 10-17: 使用 AXI4 互连进行设计

剩下的**唯一必要连接**是从 *HLS* 中断端口到 *PS7_IRQ_F2P* 端口

10. 将鼠标悬停在 *hls_macc_0* IP 符号上的**中断引脚**上。当光标变为铅笔形状时，单击并拖动到 *PS7* 的 *IRQ_F2P [0: 0]* 端口并释放，完成连接。
11. 选择 “*Address Editor*” 选项卡，并确认已为 *hls_macc_0* 外设分配了主地址范围。如果没有，请单击 “*Auto Assign Address*” 图标。

Diagram x Address Editor x

Q [Icons]

Cell	Slave Interface	Base Name	Offset Address	Range	High Address
processing_system7_0					
Data (32 address bits : 0x40000000 [1G])					
his_macc_0	s_axi_HLS_MACC_PERIPH_BUS	Reg	0x43C0_0000	64K	0x43C0_FFFF

图10-18:地址编辑器

框图设计输入流程的最后一步是验证设计。

- 12.单击工具栏中的 **Validate Design** 图标。
- 13.验证成功后，保存块设计。

第五步:实施系统

在继续进行系统设计之前，您必须生成实现源，并创建一个 **HDL Wrapper** 作为综合和实现的顶层模块。

- 1.在流导航器中单击 **Project Manager** 返回到 Project Manager 视图。
- 2.在主工作区窗格中的 **Sources** 浏览器中，一个名为的框图对象位于 Design Sources 树视图的顶部(图 10-19)。 **Zynq_Design** 右键单击此对象并选择 **Generate Output Products**。
- 3.在 “**Generate**” 对话框中，单击 “**Generate**” 以启动生成所需源文件的过程。

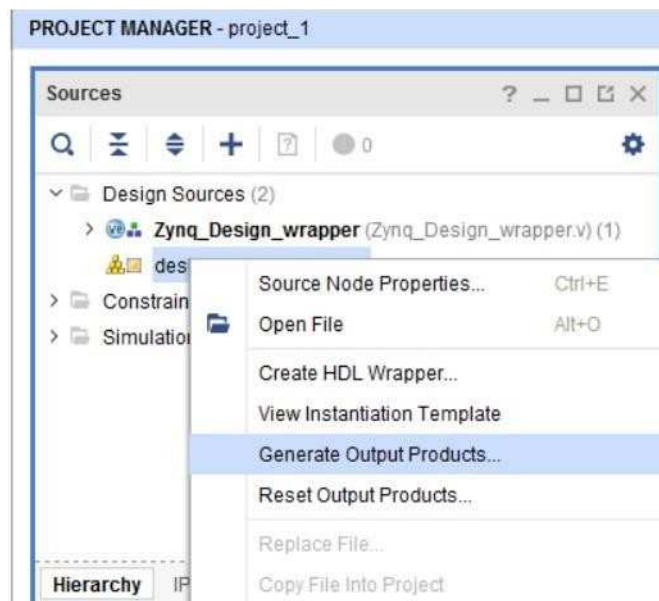


图10-19:生成输出产品

- 4.再次右键单击该对象 **Zynq_Design**，选择 **Create HDL Wrapper**，然后单击OK 以退出结果对话框。

设计源树的顶层成为文件 **Zynq_Design_wrapper.v**。设计现在已经准备好进行综合、实现，并生成一个 FPGA 编程 **bit** 流。

- 5.单击 **Generate Bitstream** 启动流的其余部分。
 - a.单击 **Yes** 实现设计。
- 6.在位流生成完成后出现的对话框中，选择 **Open implementation Design** 并单击 **OK**。

步骤 6:开发软件并在 ZYNQ 系统上运行

现在可以将设计导出到 Xilinx SDK 了。在 SDK 中，您可以创建在 ZC702 板上运行的软件(如果可用)。HLS 块的驱动程序是在 Vivado IP Catalog 包的 HLS 导出期间生成的。该驱动程序必须在 SDK 中可用，以便 PS7 软件可以与块通信。

- 1.从 Vivado 文件菜单中选择 **Export > Export Hardware**。
- 注意:**为了成功完成这一步，**IPI 块设计和实现的设计都必须在 Vivado 工作区中打开。**
- 2.在 **Export Hardware** 对话框(图 10-20)中，确保包含的 bit 流是启用并单击 **OK**。

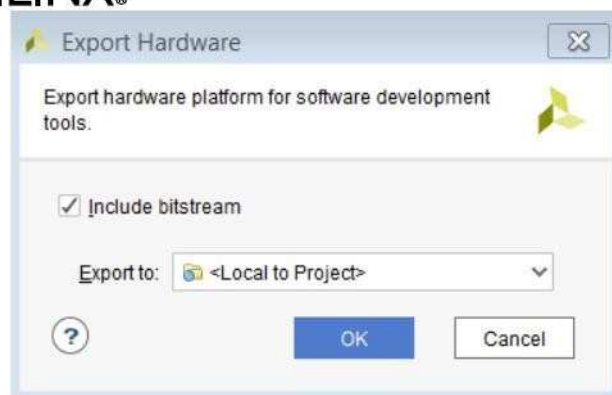


图 10-20: 导出硬件对话框

4. 单击 OK 打开 SDK。
5. 从 SDK 文件菜单中，选择 **New > Application Project**。
 - a. 在新建项目对话框中输入项目名称 **Zynq_Design_Test**
 - b. 单击 **Next**。
 - c. 选择 **Hello World** 模板。
 - d. 单击 **Finish**。

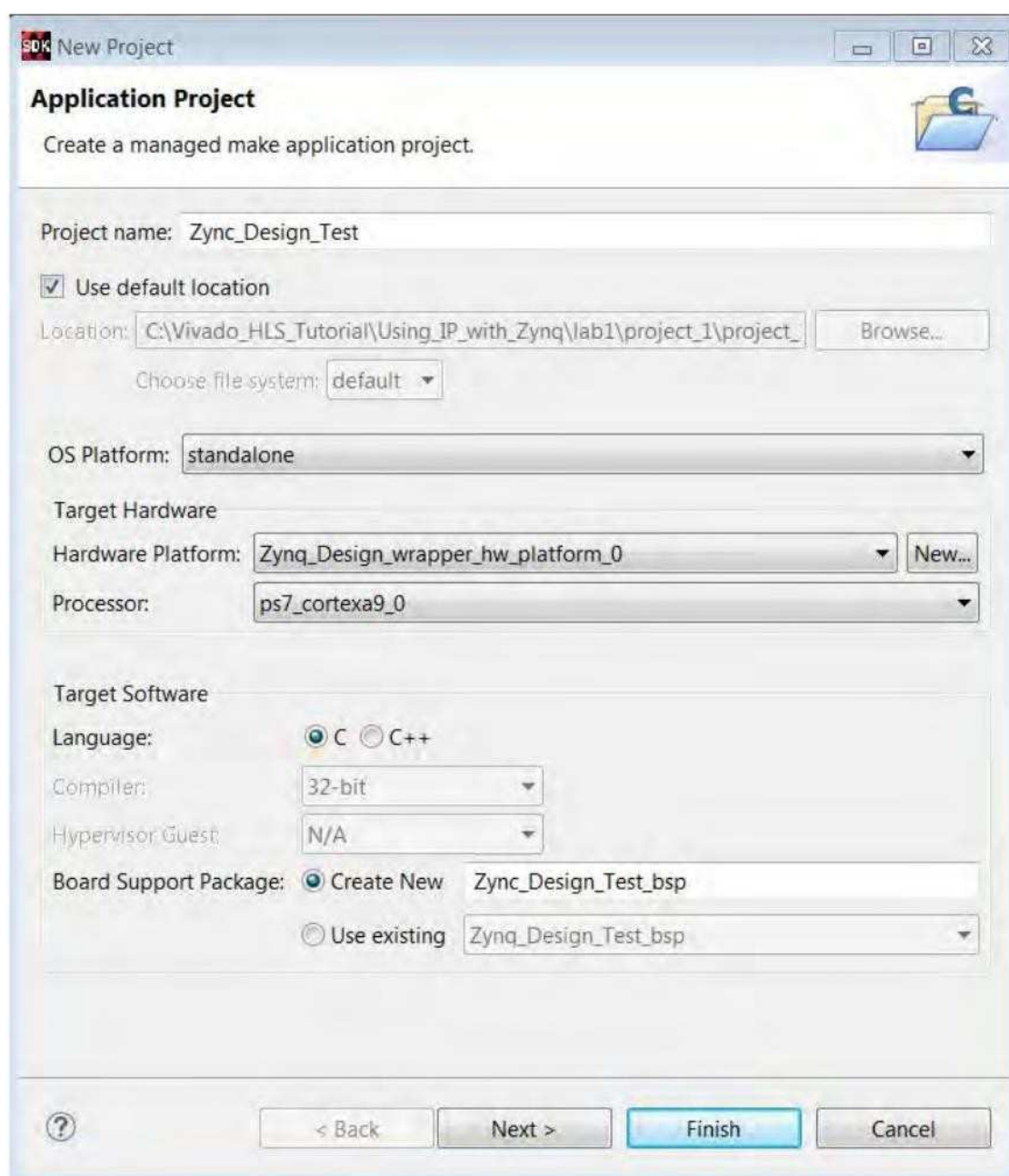


图 10-21: 应用程序项目

- 6.启动 ZC702板并测试 Hello World应用程序。确保板有所有的连接,让您下载的位流在 FPGA 设备上。参见 ZC702 开发板附带的文档。
- 7.点击 **Xilinx Tools > Program FPGA**(或工具栏图标)。
请注意, **Done LED**(DS3)现在已经打开。
- 8.点击 **SDK**终端, 点击 **add**按钮向终端添加一个端口。
 - a.点击 Connect 图标(图 10-22)。

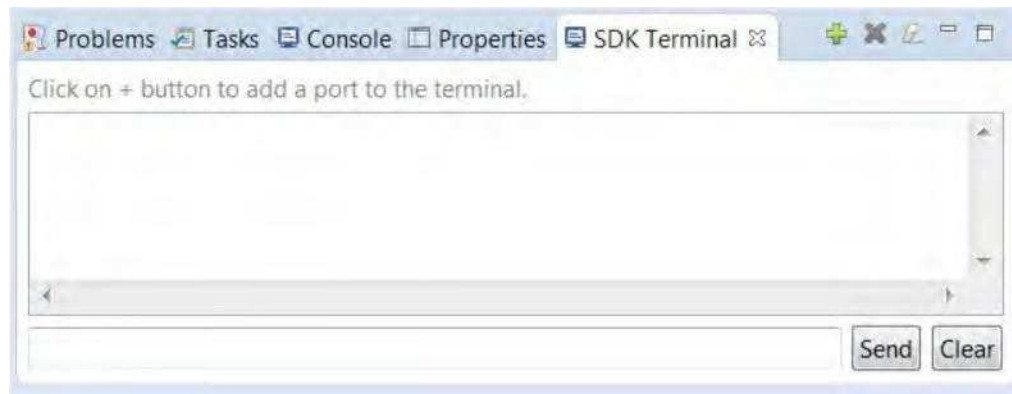


图 10-22:Connect 图标

- b.选择 **Connection Type > Serial**。
- c.选择 **USB UART** 电缆连接到的 **COM** 端口(一般不是 COM1 或 COM3)。在 Windows 上,如果你不确定,打开设备管理器,在端口(COM & LPT)下用 Silicon Labs 驱动程序识别端口。
- d.将波特率更改为 **115200**(图 10-23)。
- e.单击“确定”退出“终端设置”对话框。

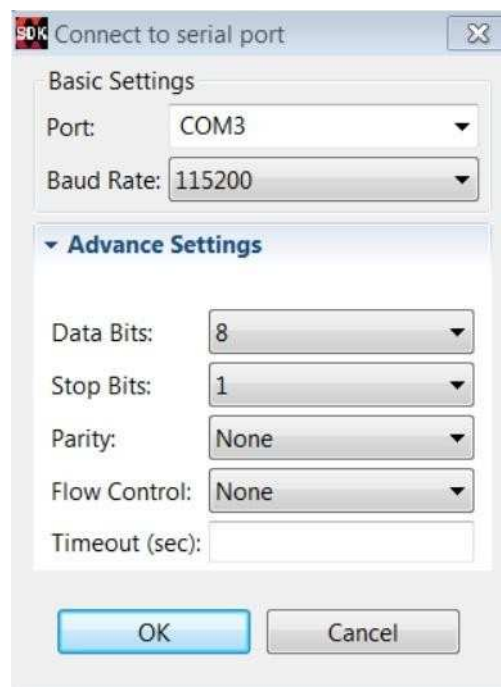


图 10-23:终端设置

- 9.在资源管理器窗格中右键单击应用程序项目 **Zynq_Design_Test**(图 10-24)。
 - a.在硬件上单击 **Run** 作为>启动。

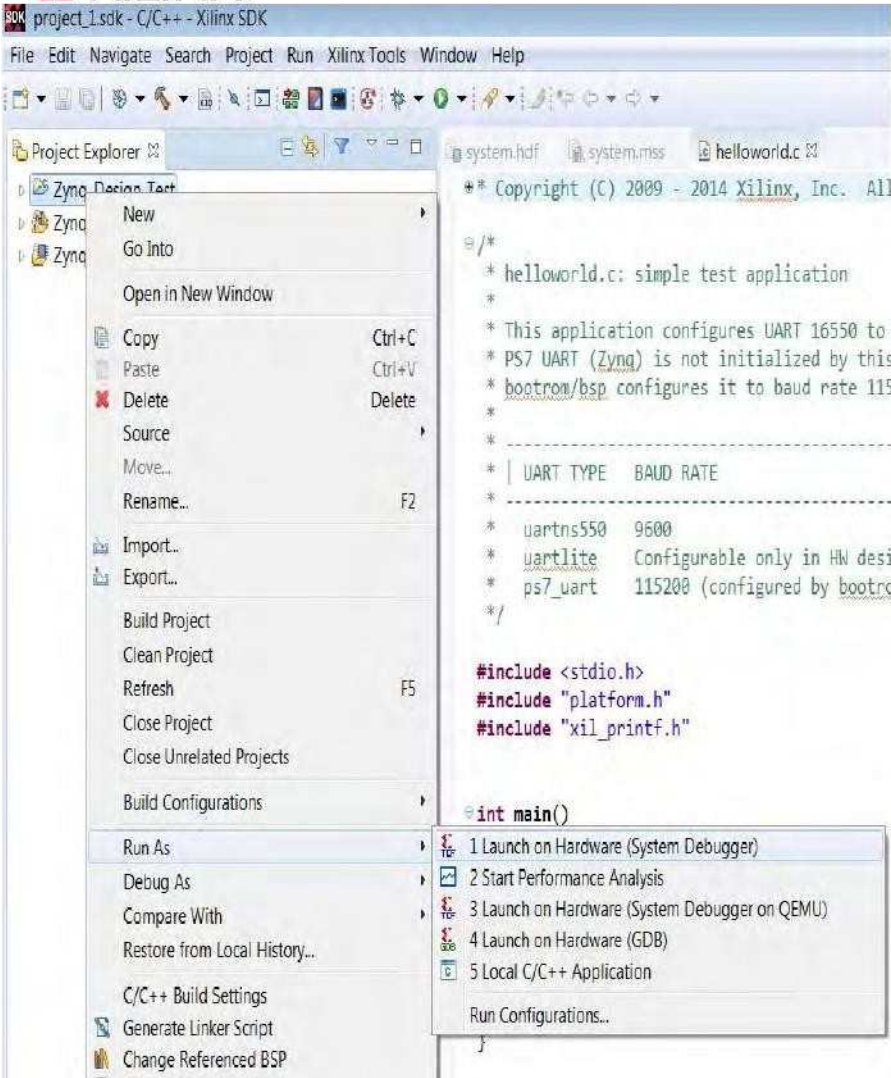


图 10-24: 运行应用程序项目

10. 切换到 **Terminal** 选项卡，确认已经收到 **Hello World**。



图 10-25: 控制台输出

第七步: 修改软件与 HLS 块通信

完全修改过的源文件可以在教程文件集的目录 **arm_code** 中找到。修改将在下面详细讨论。

1. 打开源文件 **helloworld.c**。
2. 几个 **BSP(和标准 C)头文件** 需要包括:

```
#include <stdlib.h> // Standard C functions, e.g. exit()
#include <stdbool.h> // Provides a Boolean data type for ANSI/ISO-C
#include "xparameters.h" // Parameter definitions for processor peripherals
#include "xscugic.h" // Processor interrupt controller device driver
#include "xHls_macc.h" // Device driver for HLS HW block
```

3. 为 **HLS 块**和**中断控制器实例数据**定义变量。这些变量将作为相应硬件中的句柄传递给 驱动程序 API 调用。

```
// HLS macc HW instance
XHls_macc HlsMacc;
//Interrupt Controller Instance
XScuGic ScuGic;
```

4. 定义与中断服务例程(ISR)交互的全局变量。

```
volatile static int RunHlsMacc = 0;
volatile static int ResultAvailHlsMacc = 0;
```

5. 定义一个函数来封装所有对 HLS 块的一次运行 API 初始化函数调用。

```
int hls_macc_init(XHls_macc *hls_maccPtr)
{
    XHls_macc_Config *cfgPtr;
    int status;

    cfgPtr = XHls_macc_LookupConfig(XPAR_XHLS_MACC_0_DEVICE_ID);
    if (!cfgPtr) {
        print("ERROR: Lookup of accelerator configuration failed.\n\r");
        return XST_FAILURE;
    }
    status = XHls_macc_CfgInitialize(hls_maccPtr, cfgPtr);
    if (status != XST_SUCCESS) {
        print("ERROR: Could not initialize accelerator.\n\r");
        return XST_FAILURE;
    }
    return status;
}
```

6. 定义一个帮助函数来包装启用它的中断和启动块所需的 HLS 块 API 调用。

```
void hls_macc_start(void *InstancePtr) {
    XHls_macc *pAccelerator = (XHls_macc *)InstancePtr;
    XHls_macc_InterruptEnable(pAccelerator, 1);
    XHls_macc_InterruptGlobalEnable(pAccelerator);
    XHls_macc_Start(pAccelerator);
}
```

为了使处理器响应由外设产生的中断，需要一个中断服务例程。每个附加中断到 PS 的外设必须定义一个 ISR 并向 PS 的中断处理程序注册。

ISR 负责清除外围设备的中断，在本例中，设置一个标志，指示结果可以从外围设备检索。一般来说，ISRs 应该被设计成轻量级的，并且尽可能的快，本质上做最少的必要工作来服务中断。诸如检索数据之类的任务应该留给主应用程序代码。

```
void hls_macc_isr(void *InstancePtr) {
    XHls_macc *pAccelerator = (XHls_macc *)InstancePtr;

    //Disable the global interrupt
    XHls_macc_InterruptGlobalDisable(pAccelerator);
    //Disable the local interrupt
    XHls_macc_InterruptDisable(pAccelerator, 0xffffffff);

    // clear the local interrupt
    XHls_macc_InterruptClear(pAccelerator, 1);

    ResultAvailHlsMacc = 1;
    // restart the core if it should run again
    if(RunHlsMacc){
        hls_macc_start(pAccelerator);
    }
}
```

7. 定义一个例程来设置 PS 中断处理程序并注册 HLS 外设的 ISR。

```
int setup_interrupt()
{
    //This functions sets up the interrupt on the Arm
    int result;
    XScuGic_Config *pCfg = XScuGic_LookupConfig(XPAR_SCUGIC_SINGLE_DEVICE_ID);
    if (pCfg == NULL) {
        print("Interrupt Configuration Lookup Failed\n\r");
        return XST_FAILURE;
    }
}
```

```

result = XScuGic_CfgInitialize(&ScuGic,pCfg,pCfg->CpuBaseAddress);
if(result != XST_SUCCESS){
    return result;
}
// self-test
result = XScuGic_SelfTest(&ScuGic);
if(result != XST_SUCCESS){
    return result;
}
// Initialize the exception handler
Xil_ExceptionInit();
// Register the exception handler
//print("Register the exception handler\n\r");
Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT,
    (Xil_ExceptionHandler)XScuGic_InterruptHandler,&ScuGic);
//Enable the exception handler
Xil_ExceptionEnable();
// Connect the Adder ISR to the exception table
//print("Connect the Adder ISR to the Exception handler table\n\r");
result = XScuGic_Connect(&ScuGic, XPAR_FABRIC_HLS_MACC_0_INTERRUPT_INTR,
    (Xil_InterruptHandler)hls_macc_isr,&HlsMacc);
if(result != XST_SUCCESS){
    return result;
}
//print("Enable the Adder ISR\n\r");
XScuGic_Enable(&ScuGic,XPAR_FABRIC_HLS_MACC_0_INTERRUPT_INTR);
return XST_SUCCESS;
}

```

8.定义 HLS 硬件功能的软件模型，您可以与之比较参考结果。

```

void sw_macc(int a, int b, int *accum, bool accum_clr)
{
    static int accum_reg = 0;
    if (accum_clr)
        accum_reg = 0;
    accum_reg += a * b;
    *accum = accum_reg;
}

```

9.修改 main()以使用 HLS 设备驱动程序 API 和上面定义的函数来测试 HLS 外围硬件。

```

int main()
{
    print("Program to test communication with HLS MACC peripheral in PL\n\r");
    int a = 2, b = 21;
    int res_hw;
    int res_sw;
    int i;
    int status;

    //Setup the matrix mult
    status = hls_macc_init(&HlsMacc);
    if(status != XST_SUCCESS){
        print("HLS peripheral setup failed\n\r");
        exit(-1);
    }
}

```

```

}
//Setup the interrupt
status = setup_interrupt();
if(status != XST_SUCCESS){
    print("Interrupt setup failed\n\r");
    exit(-1);
}

//set the input parameters of the HLS block
XHls_macc_SetA(&HlsMacc, a);
XHls_macc_SetB(&HlsMacc, b);
XHls_macc_SetAccum_clr(&HlsMacc, 1);

if (XHls_macc_IsReady(&HlsMacc))
    print("HLS peripheral is ready. Starting... ");
else {
    print("!!! HLS peripheral is not ready! Exiting...\n\r");
    exit(-1);
}

if (0) { // use interrupt
    hls_macc_start(&HlsMacc);
    while(!ResultAvailHlsMacc)
        ; // spin
    res_hw = XHls_macc_GetAccum(&HlsMacc);
    print("Interrupt received from HLS HW.\n\r");
} else { // Simple non-interrupt driven test
XHls_macc_Start(&HlsMacc);
do {
    res_hw = XHls_macc_GetAccum(&HlsMacc);
} while (!XHls_macc_IsReady(&HlsMacc));
print("Detected HLS peripheral complete. Result received.\n\r");
}

//call the software version of the function
sw_macc(a, b, &res_sw, false);

printf("Result from HW: %d; Result from SW: %d\n\r", res_hw, res_sw);
if (res_hw == res_sw) {
    print("*** Results match ***\n\r");
    status = 0;
}
else {
    print("!!! MISMATCH !!!\n\r");
    status = -1;
}

cleanup_platform();
return status;
}

```

10. 保存修改后的源文件。当您保存文件时，SDK 会自动尝试重新构建应用程序可执行文件。如果构建失败，修复任何未解决的问题。

在硬件上运行新应用程序，并验证它是否按预期工作。确保 TCF 硬件服务器正在运行，FPGA 已编程，终端会话正在运行。

连接到 UART。然后在硬件上启动，就像前面的 Hello World 应用程序代码一样。

成功后，终端会话类似于图 10-26。

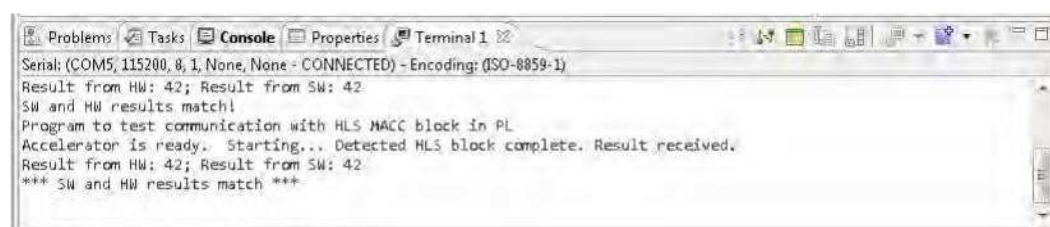


图 10-26: 带有更新的 C 程序的控制台输出

实验 2:在 Zynq CPU 和 HLS 加速器块之间的数据流

本实验说明了一种通用的高性能连接方案，用于以流方式连接硬件加速器块，这些硬件加速器块消耗源自 CPU 内存的数据和/或生成发往该内存的数据。

- 本教程使用与教程“在 IP Integrator 中使用 HLS”的 **lab1** 中创建的 Vivado HLS 和 XFFT IP 块相同的知识。在这个实验练习中，这些块通过一个 **AXI DMA IP 核** 连接到 Zynq7 处理系统上的 **HP0** 从 **AXI4 端口**。
- 硬件加速器块是自由运行的，不需要驱动程序;只要数据是由 CPU(通常简称为处理系统或 PS)推入和拉出的。
- 实验强调软件需求以避免缓存一致性问题。

步骤 1:生成 HLS IP

1. 从 **lab1** 中使用的 Vivado HLS 命令提示符，切换到如图 10-26 所示的目录 **lab2**。
2. 运行 Vivado HLS，通过键入 **vivado_hls -f run_hls.tcl** 创建两个 HLS IP 块。

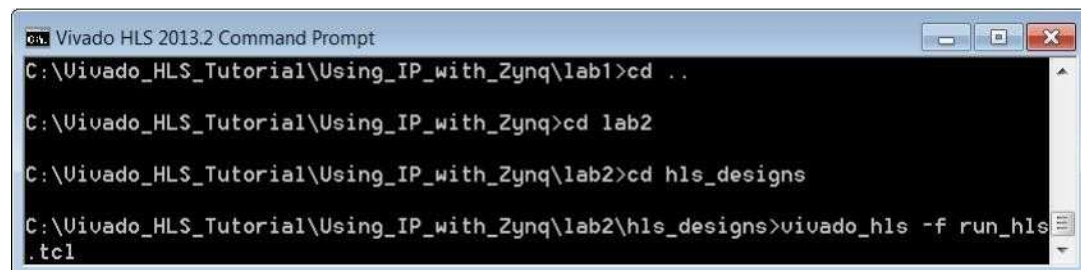


图 10-27:Zynq 实验室 2 的设置

脚本完成后，将有两个 Vivado HLS 项目目录 **fe_vhls_prj** 和 **be_vhls_prj**，其中包含 HLS IP，包括用于 Vivado 设计的 Vivado IP Catalog 存档。

- “**前端**” IP 存档位于 **fe_vhls_prj/IPXACTExport/impl/ip/**
- “**后端**” IP 存档位于 **fe_vhls_prj/IPXACTExport/impl/ip/**

步骤 2:创建一个 Vivado 设计套件项目

1. 启动 Vivado 设计套件(不是 Vivado HLS):

在 Windows 上使用 **Start > All Programs > Xilinx Design Tools > Vivado 2018.3 > Vivado HLS > Vivado HLS 2018.3 Command Prompt**

2. 在欢迎屏幕中，选择 **Create New Project**。
3. 单击“新建 Vivado 项目向导”的第一页上的“**Next**”。
4. 单击项目位置文本输入框右侧的省略号按钮，并浏览到教程目录 **lab2**。
5. 如果尚未指定项目名称，则将其设置为 **project_1**。
6. 单击 **Next** 以移动到向导的项目类型页面。
 - a. 选择 RTL 项目。
 - b. 选择此时不要指定来源(如果不是默认);只需单击 Next。
7. 在默认的部件页面，在“指定”下，单击“board”并选择 ZYNQ-7 ZC702 评估板。单击 Next。
8. 在新项目摘要页面上，单击 Finish 完成新项目设置。

步骤 3:将 HLS IP 添加到 IP 存储库

1. 在 Flow Navigator 窗格的 Project Manager 区域，单击 *IP Catalog*。
2. 单击 IP Catalog 窗格，右键单击并选择 *Add Repository*。
3. 在“IP 知识库”对话框中：
 - a. 浏览到教程目录 *lab2*。
 - b. 单击 *Create New Folder* 图标。
 - c. 在结果对话框中输入 *vivado_ip_repo*
 - d. 单击 OK。
 - e. 单击 Select 关闭 IP Repository 窗口。
4. 返回 IP 设置对话框：
 - a. 单击“+”符号添加 IP
 - b. 在 IP 知识库对话框中，浏览到 HLS IP *lab2/hls_designs/fe_vhls_prj/IPXACTExport/impl/IP/*的位置，或者，如果使用前面教程中创建的 IP，浏览到相应的路径。
 - c. 选择 *xilinx_com_hls_hls_real2xfft_1_00_a.zip* 文件。
 - d. 单击 OK。
5. 按照相同的过程将目录 *lab2/hls_designs/be_vhls_prj/IPXACTExport/impl/IP/*中的第二个 HLS IP 包添加到存储库：*xilinx_com_hls_hls_xfft2real_1_00_a.zip*。
6. 新的 HLS IP 现在出现在 IP 设置对话框中。
7. 单击 OK 退出对话框。
8. 现在在 IP 目录中有 HLS IP (*Hls_real2xfft* 和 *Hls_xfft2real*)。


 发送反馈

步骤4:创建顶级块设计

1. 在流导航器中，单击 IP Integrator 下的 Create 框图。
 - a. 在结果对话框中，命名设计 *Zynq_RealFFT*。
 - b. 单击 *OK*。
2. 在 Diagram 选项卡中，单击 *Add IP* 按钮来添加 IP
 - a. 在搜索框中键入 *fourier*。
 - b. 选择快速傅里叶变换并双击鼠标。
3. 双击新的快速傅里叶变换 IP 符号，打开重新定制 IP 对话框。在配置选项卡：
 - a. 将变换长度更改为 512。
 - b. 将目标时钟频率更改为 100 MHz。
 - c. 在体系结构选择部分，选择流水线，流式 I/O。
4. 选择 Implementation 选项卡：
 - a. 在控制信号组中选择 ARESETN (active-Low)。
 - b. 验证在输出排序选项中选择了位/位颠倒的顺序。
 - c. 确认选择非实时作为节气门方案。
 - d. 单击 OK 退出重定义 IP 对话框
5. 将生成的每个 HLS 块的一个实例添加到设计中。
 - a. 右键单击画布上的任意空间，选择 Add IP。
 - b. 在搜索文本输入框中键入 *hls*。
 - c. 突出两个 ip。(点击控制键并选择两者)
 - d. 回车。

6. 将 HLS 块连接到 FFT 块。
 - a. 将鼠标移到块的接口 *dout hls_real2xfft* 连接器上，直到出现铅笔光标。
 - b. 左键单击并按住鼠标按钮启动连接。
 - c. 将连接线拖放到 FFT 块的输入端口 *S_AXIS_DATA* 连接器上，当旁边出现绿色的复选标记时松开。
7. 以类似的方式：
 - a. 将 FFT 的 *M_AXIS_DATA* 接口连接到 *hls_xfft2real* 模块的 *din* 输入接口。
8. 将数据处理块放到它们自己的层次结构中。
 - a. 按 **Ctrl+ a** 键选择当前字母组合中的所有字母。
 - b. 右键单击画布，从上下文菜单中选择 **Create Hierarchy**。

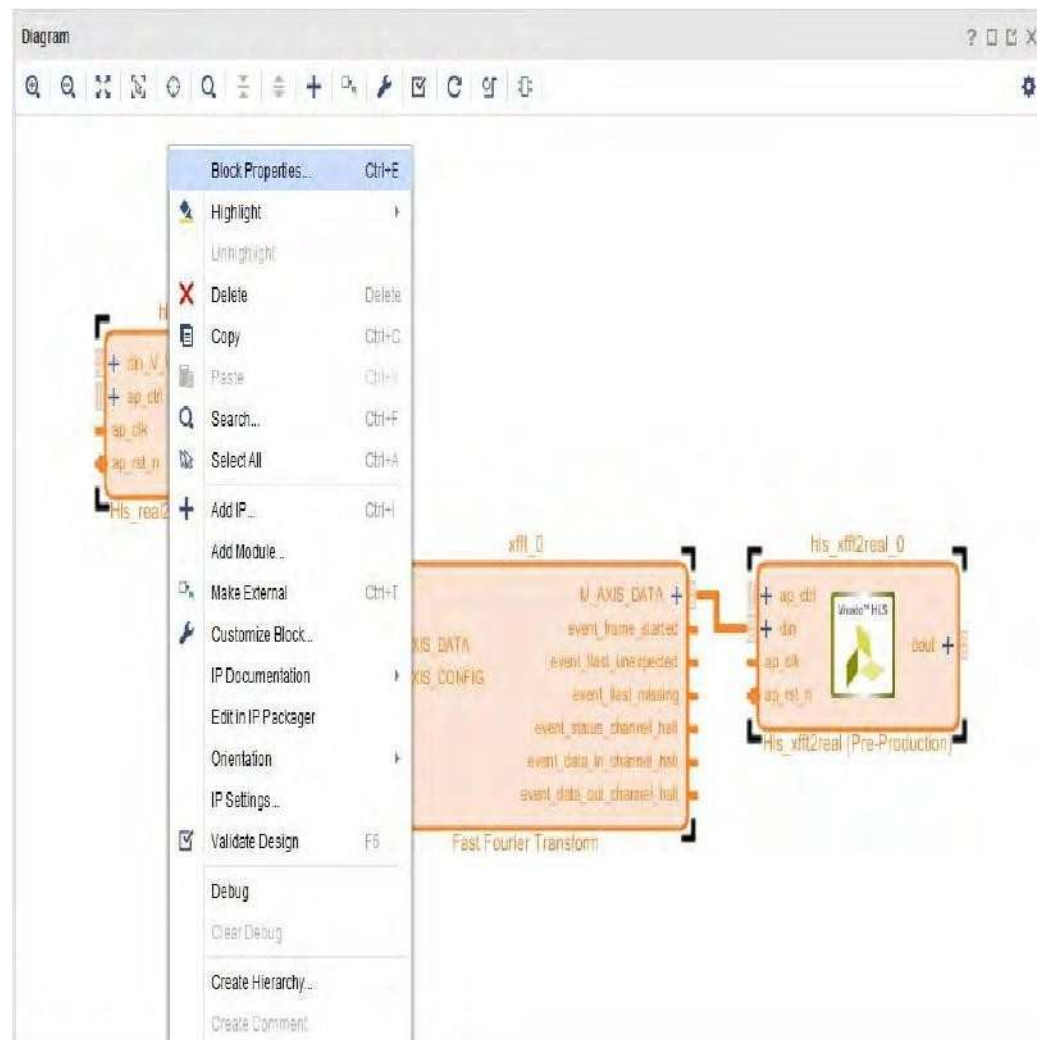


图 10-28: 创建一个层次结构块

- c. 在“创建层次结构”对话框中，输入单元格 **RealIFFT** 名称。
- d. 确保选中 **Move '3' selected blocks to new hierarchy** 选项中，如图 10-29 所示

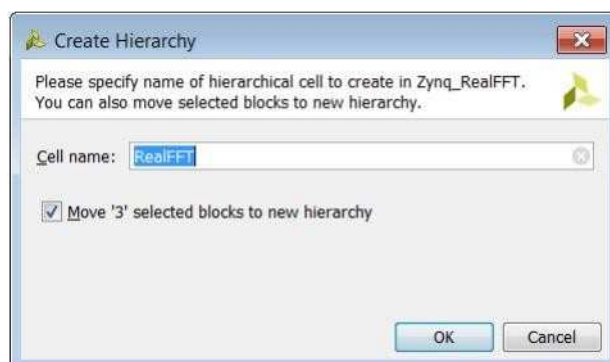


图10-29: 名称层次结构块

e. 单击 OK。图如图 10-30 所示。

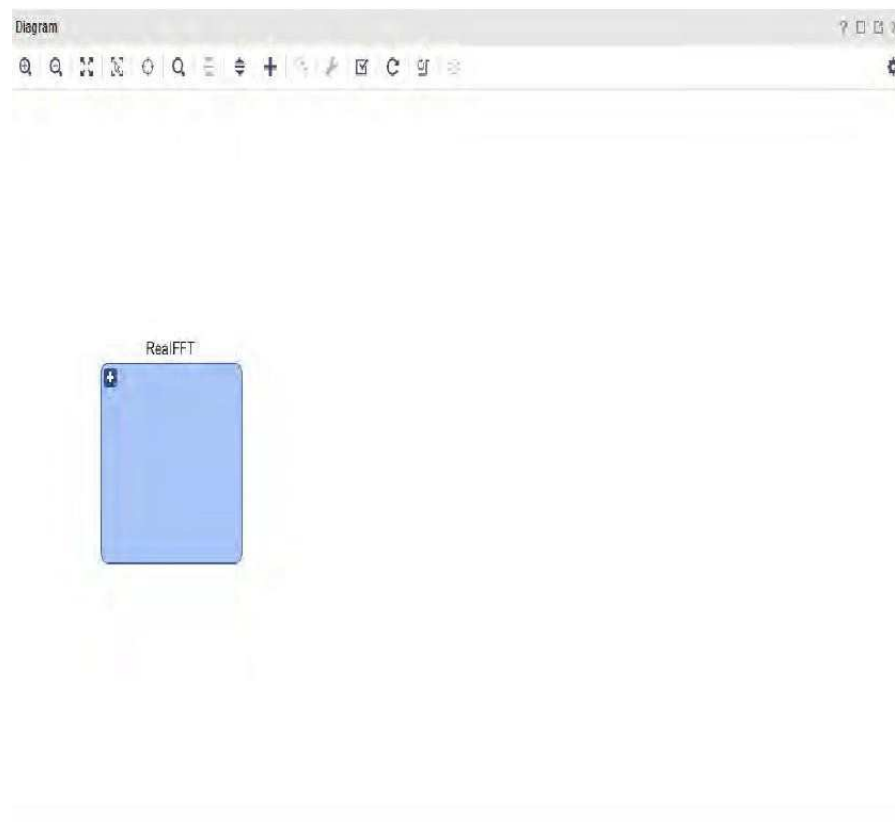


图10-30: 新的层次结构块

将大头针添加到层次结构 **RealFFT** 块中，以便您可以在顶层连接它。

9. 双击该 **RealFFT** 块以打开其关系图。

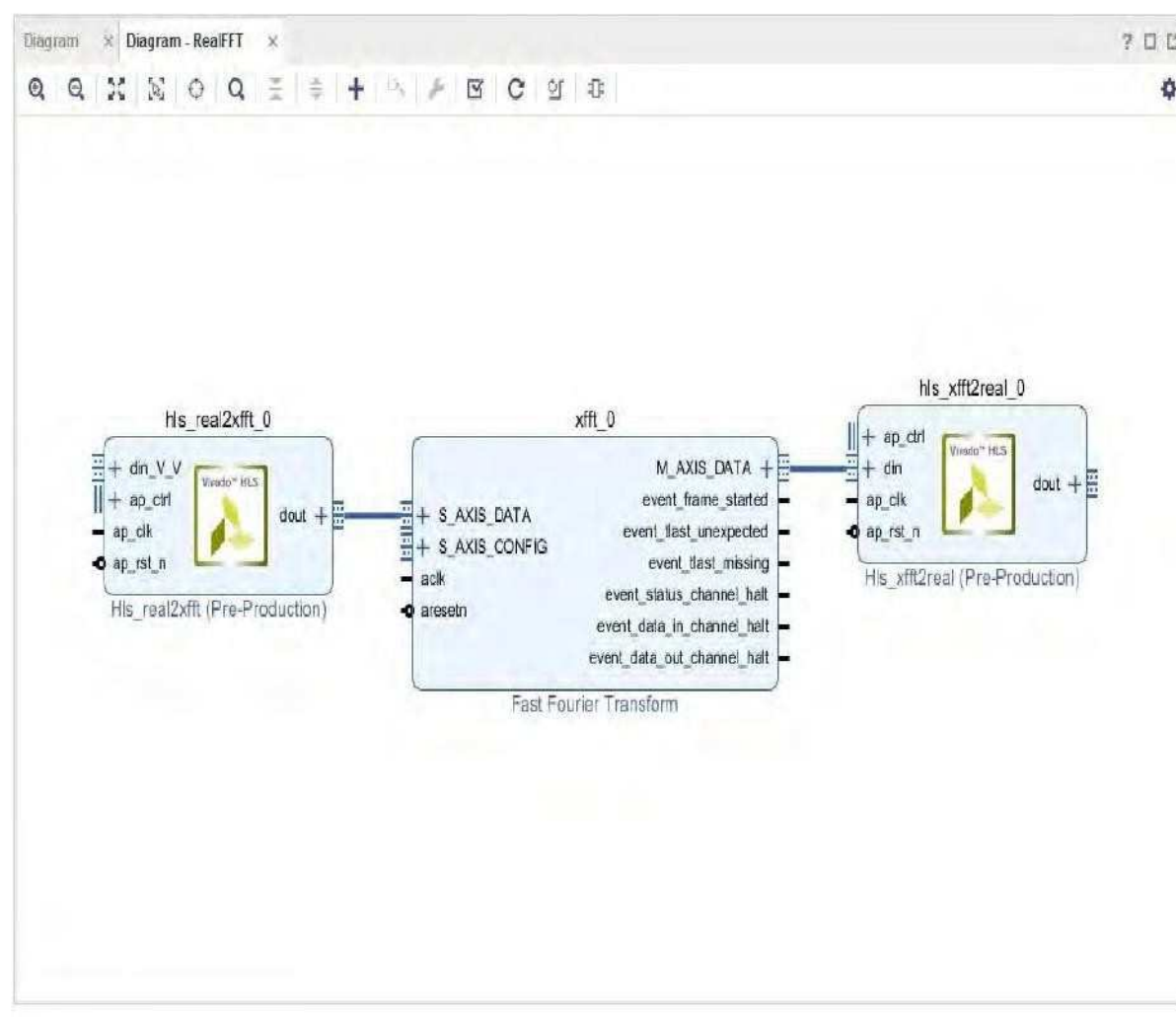


图 10-31:RealFFT 图

10. 右键单击 **hls_real2xfft_0** 块的 **din_V_V** 引脚，然后从上下文菜单中选择“创建接口引脚”。

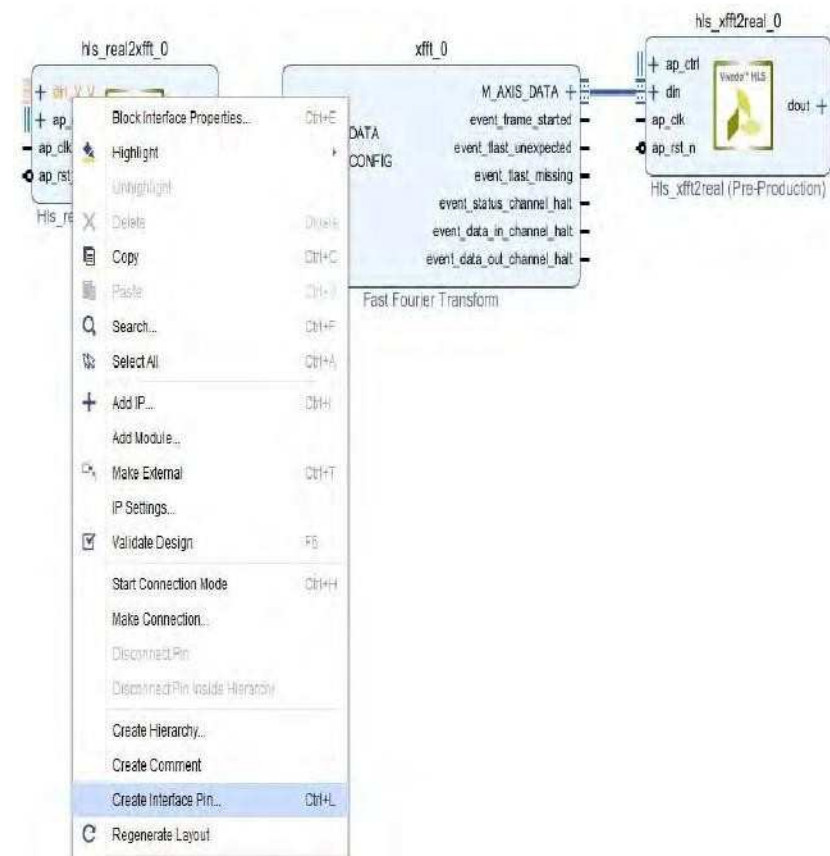


图10-32: 创建接口大头针 11。

11. 在“创建接口Pin”对话框中，将接口名称更改为 **realfft_s_axis_din**。
- 接受所有其他默认设置并单击 OK



图 10-33: 命名接口大头针

12. 右键单击 **hls_real2xfft_1** 块的 **ap_clk** 引脚，然后从上下文菜单中选择“创建引脚”。

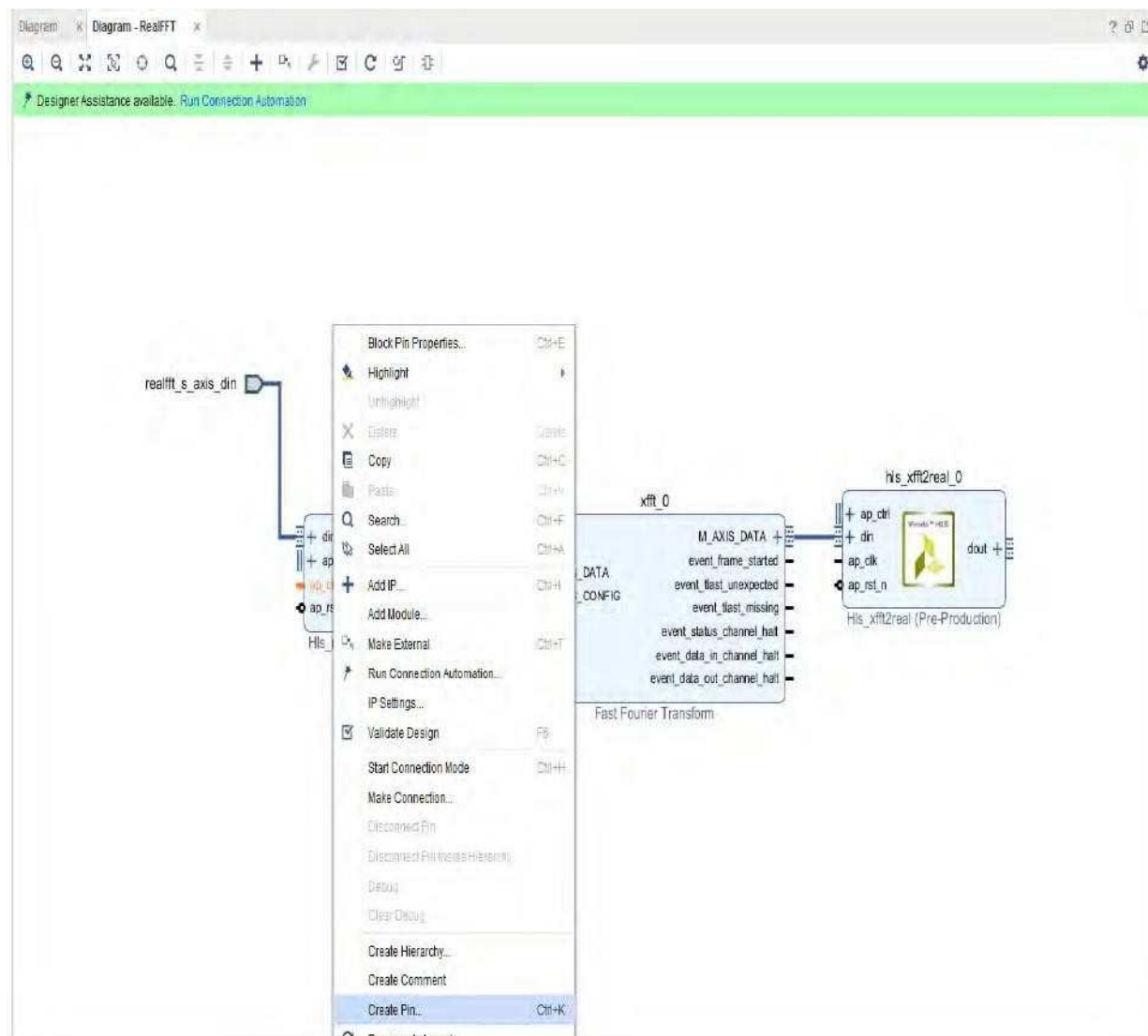


图 10-34: 创建一个时钟针

创建此时钟针后，将显示如图 10-35 所示 **RealFFT** 的关系图。

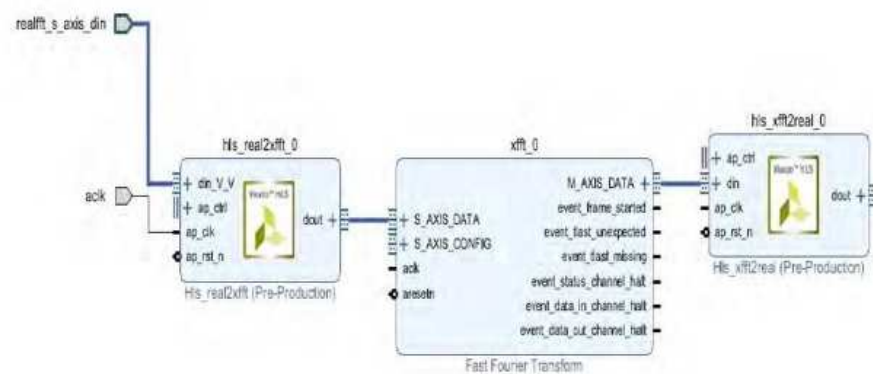


图10-35: 带有接口插脚和时钟插脚

13 的 RealFFT 图。步骤 10 至 12 的程序如下：

- 创建一个称为 `realfft_m_axis_dout` 的接口引脚，该接口引脚连接到 `hls_xfft2real` 组件的 `dout_V` 引脚。
- 为 `aresetn` 创建一个引脚（来自任何一个块）。

在这一步之后，**RealFFT** 如图 10-36 所示。

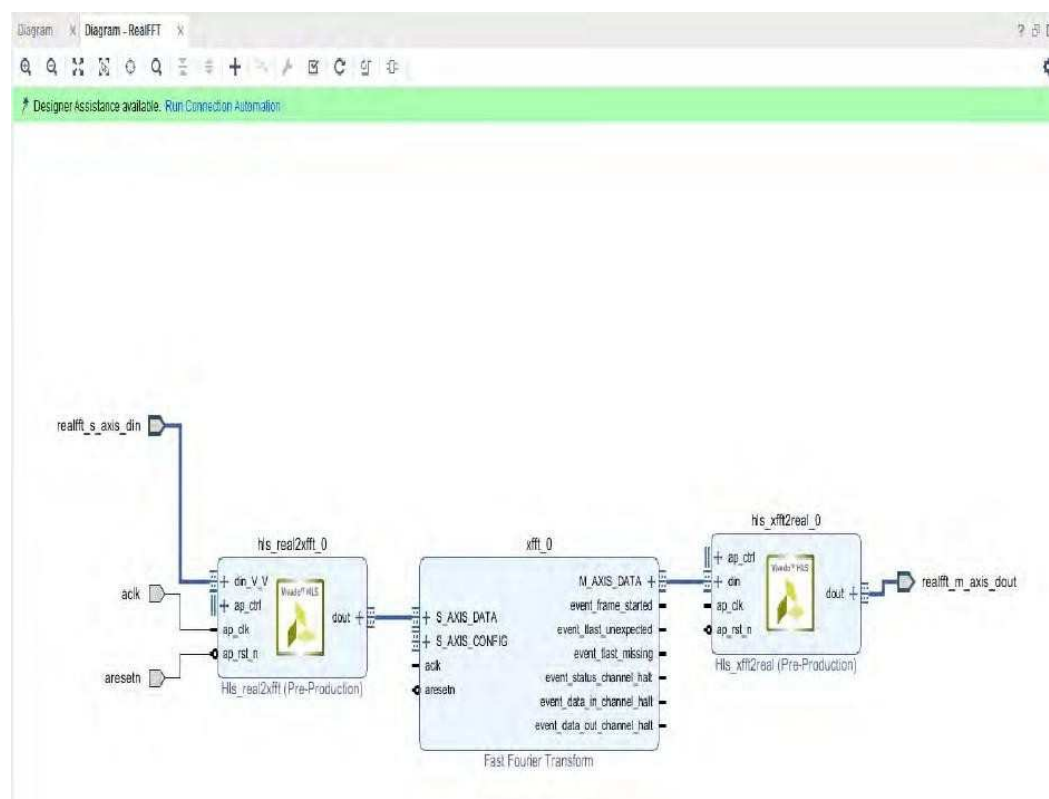


图 10-36: 带有所有引脚的 RealFFT 图

完成 RealFFT 模块的内部连接。HLS 模块的 **ap_start** 引脚连接到高电平，所有模块上的 **aclk** 和 **aresetn** 引脚连接在一起。

14. 右键单击画布并从上下文菜单中选择 Add IP。

a. 在搜索框中键入 **const** 并回车。

b. 双击组件 **xlconstant_0**，确认 Customize IP 对话框中的 Const Val 字段设置为 1。

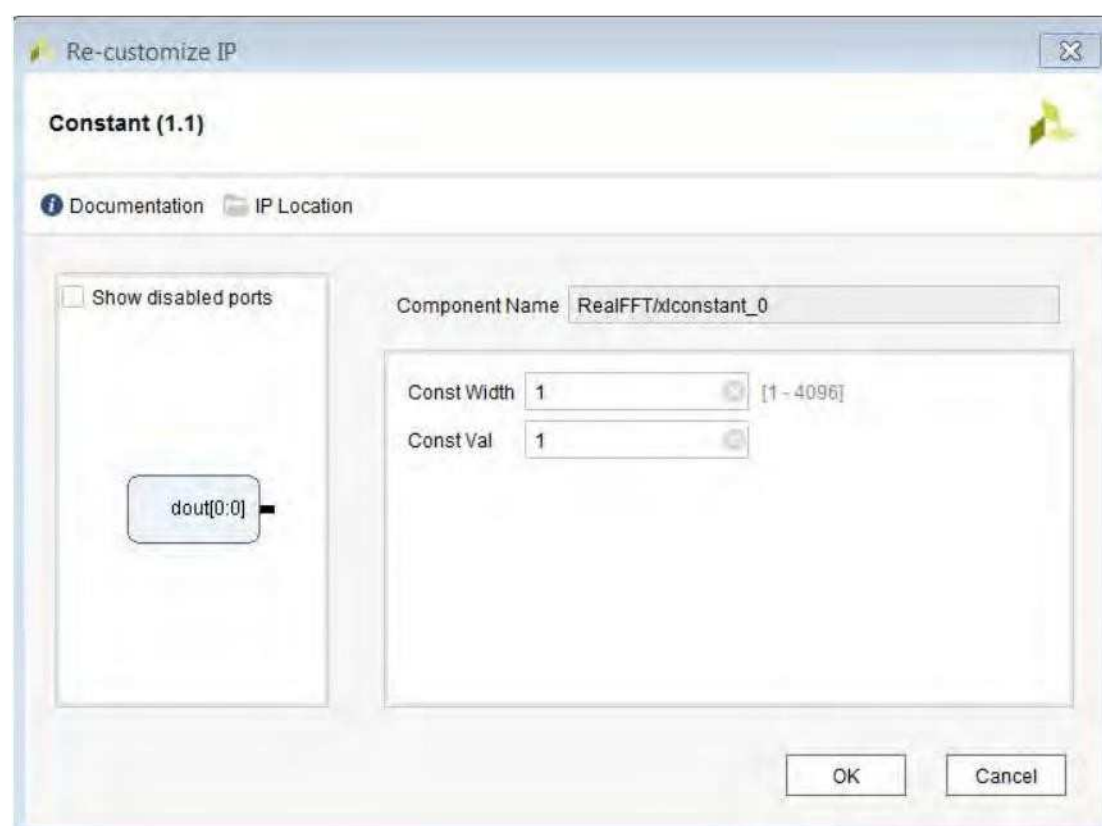


图 10-37: 创建一个固定的连接

15. 通过点击和块符号旁边的+号来扩展界面:
 - a. 连接的输出引脚 **xlconstant_0** 到的引脚 **ap_start**
hls_real2xfft_0。
 - b. 连接的输出引脚 **xlconstant_0** 到引脚 **x ap_start**
hls_xfft2real_0。
 16. 同样, 将所有其余组件 **dout_V** 和 **reset** 引脚分别连接到 **RealFFT** 框图的 **ac1k** 和 **aresetn** 引脚。
 17. 添加另一个块 **xlconstant**, 并将其配置为 Const 宽度为 16, Const Val 为 0。
 18. 扩展 FFT 块的 **S_AXIS_CONFIG** 接口, 并将 **s_axis_config_tdata** 和 **s_axis_config_tvalid** 连接到新的常量块
- 将组件的所有其他输出引脚断开。最终的关系 **RealFFT** 图显示在图 10-38 中。

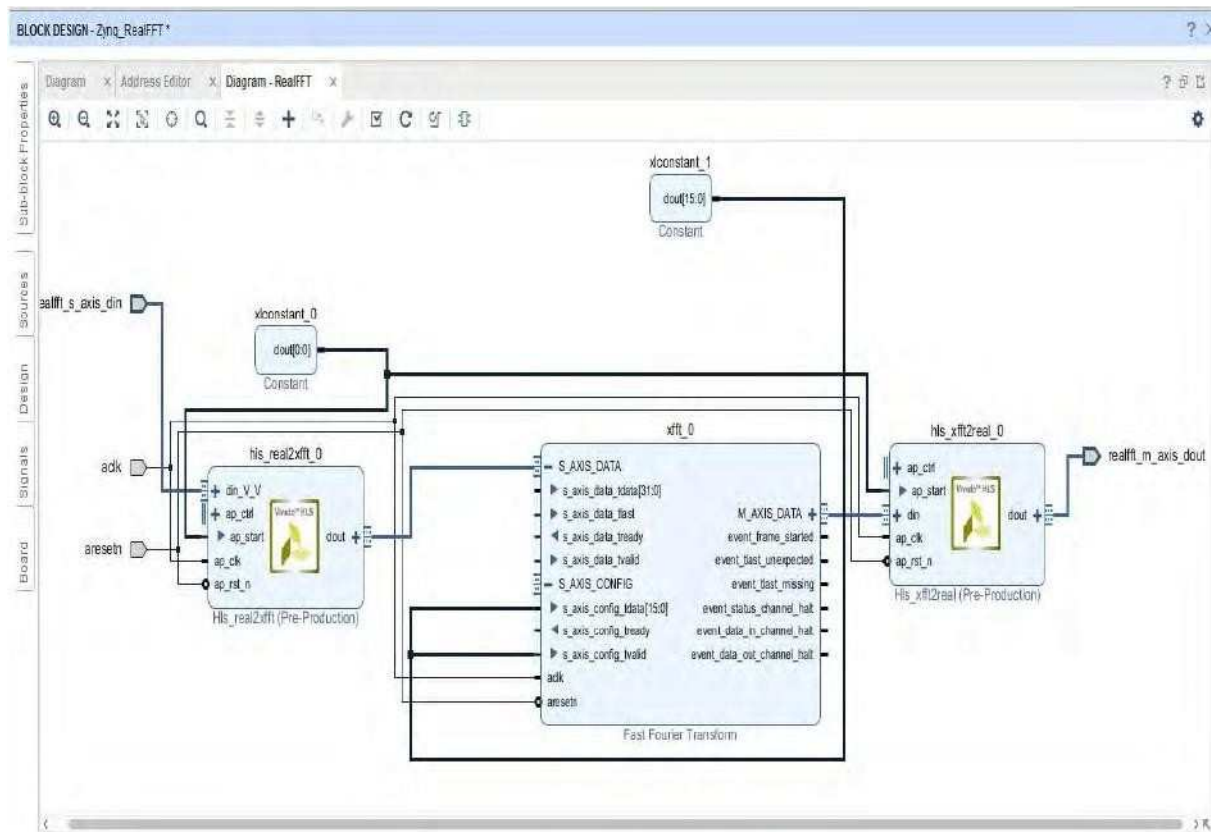


图 10-38: 最终的 RealFFT 图

19. 关闭“RealFFT 图”选项卡, 并返回到顶级 Zynq_RealFFT 图。
20. 创建 Zynq 系统。
 - a. 右键单击顶层图的画布, 从上下文菜单中选择 Add IP。
 - b. 在搜索框中输入 zynq, 选择 ZYNQ7 处理系统, 回车。
 - c. 注意, 设计器辅助是可用的, 然后单击运行块自动化链接。通过单击 OK 接受对话框中的默认设置。
 - d. 双击 **processing_system7_0** 组件, 进入 ZYNQ7 的重新定制 IP 向导。
 - e. 点击向导界面顶部附近的预设按钮, 选择 ZC702 开发板模板, 点击确定。
 - f. 在向导左侧的 PageNavigator 窗格中单击 PS-PL Configuration。
 - g. 扩展 HPSlaveAXI 接口类别, 勾选 S AXI HP0 接口, S AXI HP0 数据宽度为 64。

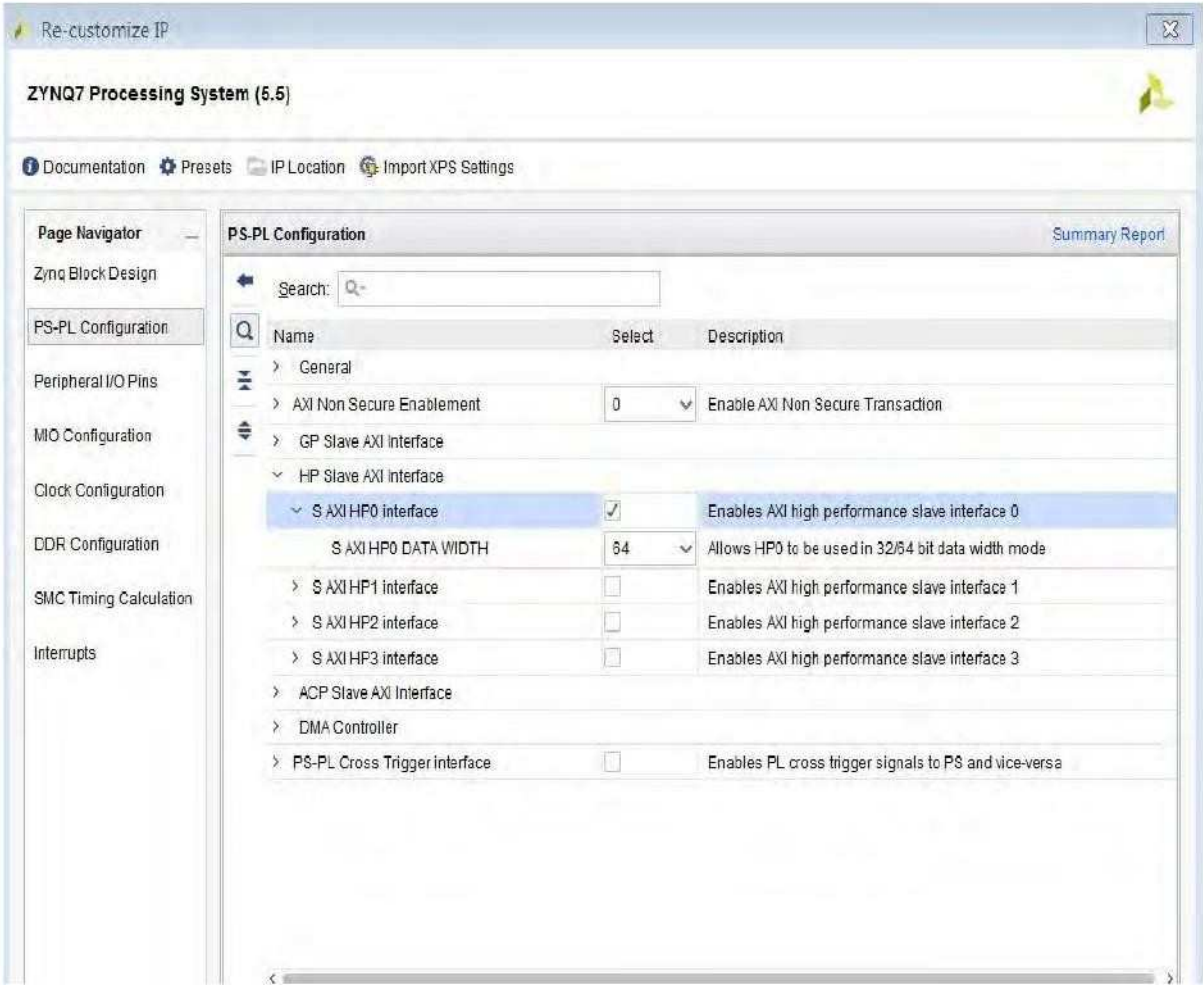


图 10-39: 配置端口 HP0

h. 在页面导航器中选择时钟配置，展开 PL Fabric 时钟，并将请求的频率更改为 100 (MHz)。

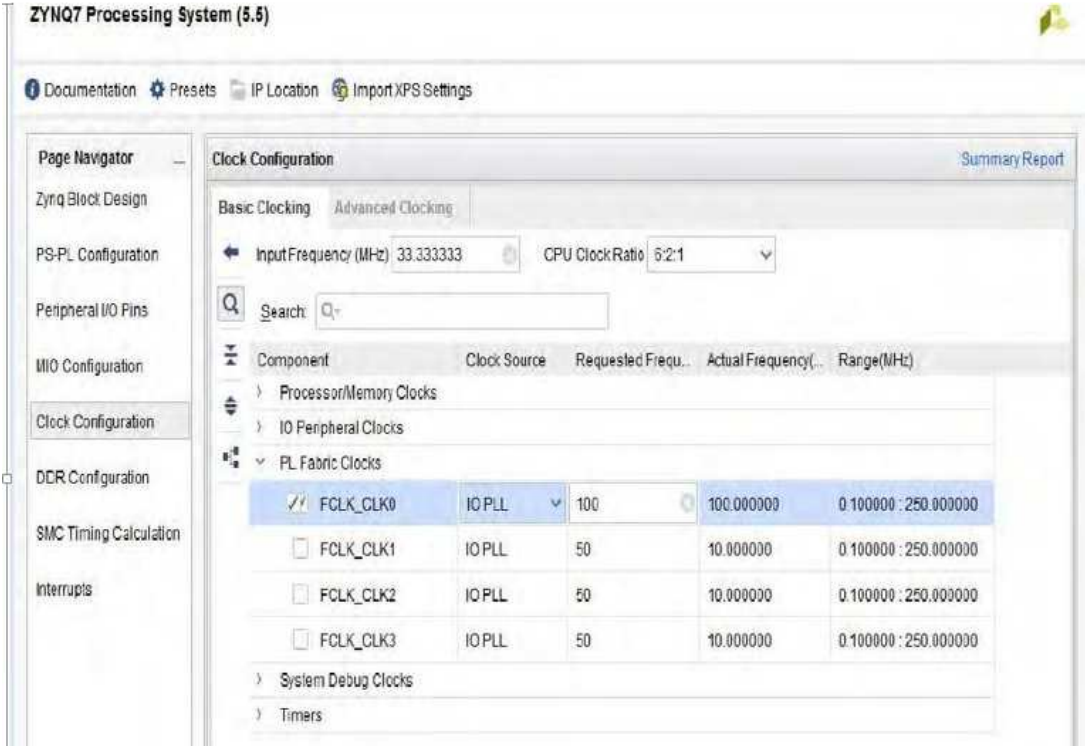


图 10-40: 配置时钟

i. 所有其他设置保持默认;单击 OK 应用定制。

21. 建立从 RealFFT 块的 `realfft_s_axis_din` 到 Zynq SoC 的 `S_AXI_HP0` 的连接，在“建立连接”对话框中接受默认设置，然后单击“确定”。
- IPI 将放置几个需要自动完成连接的新块，包括一个 AXI DMA 核心、一个 AXI 互连和一个处理器系统复位块。

22. 从 RealFFT 块的 `realfft_m_axis_dout` 连接到 Zynq 的 S_AXI_HP0 接口。在“建立连接”对话框中接受默认设置将使 IPI 使用现有的 AXI DMA（具有未使用的写通道）和 AXI 互连来建立“S2MM”连接。
23. 请注意，设计器辅助也是可用的。在 `/axi_dma/S_AXI_LITE` 上运行连接自动化，然后在结果对话框中单击 OK。
24. 将 RealFFT 分层模块的 `aclk` 和 `aresetn` 端口分别连接到 `nets_processing_system7_0` 引脚 `FCLK_CLK0` 和 `rst_processing_system7_0_100M` 引脚 `peripheral_aresetn`。
25. 要完成设计，请运行 **Validate design**。当验证成功完成时，框图应该如图 10-41 所示。

第五步:实施系统

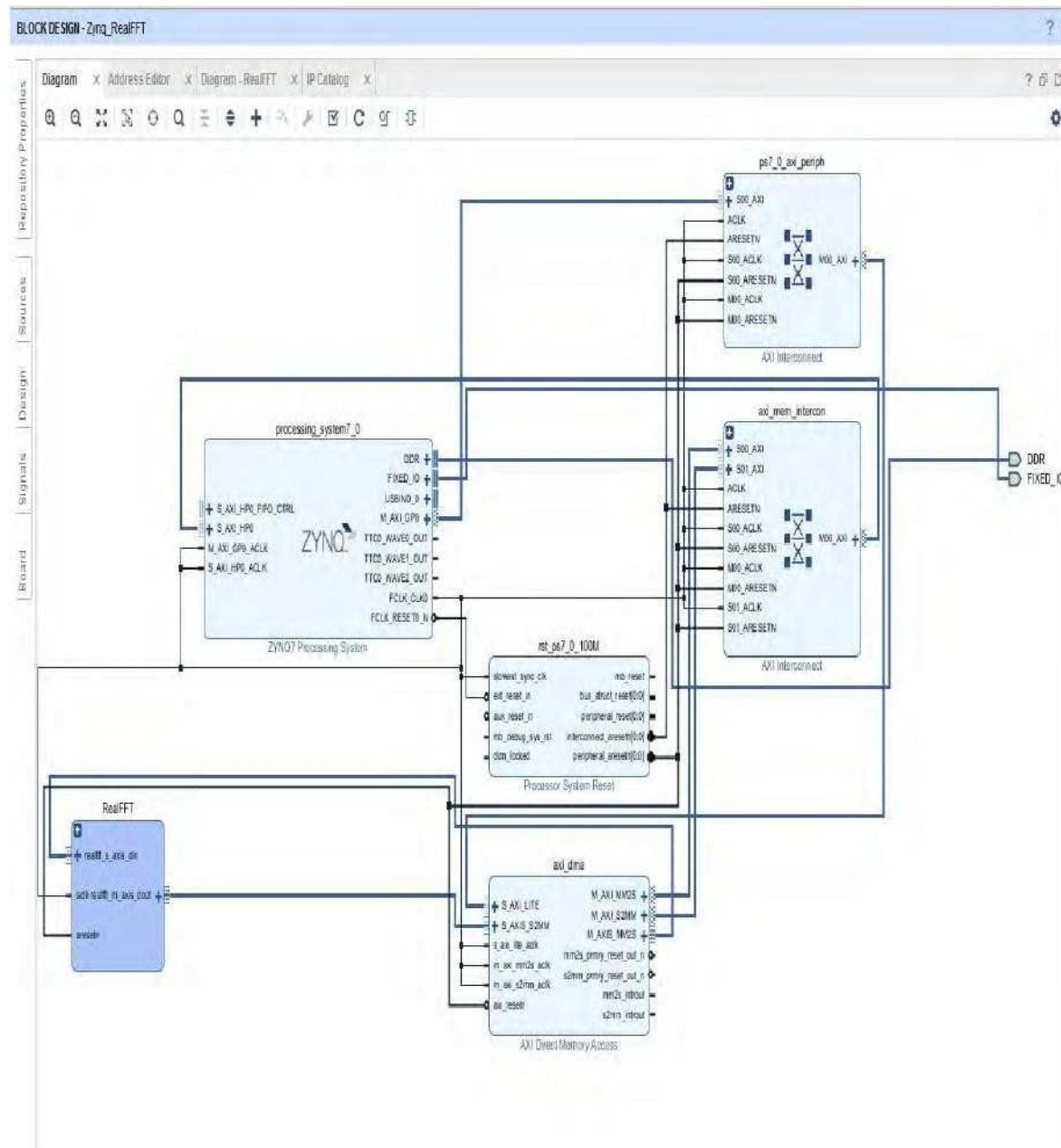


图 10-41: 内部连接的 Zynq 图

1. 在继续进行系统设计之前，您必须生成实现源，并创建一个 HDL 包装程序作为合成和实现的顶级模块。
- 2 在 Flow Navigator 中单击 Project Manager 返回到 Project Manager 视图。
在主工作区窗格中的 Sources 浏览器中，一个名为的框图对象出现在 Design Sources 树视图的顶部。Zynq_RealFFT 右键单击此对象并选择 Generate Output Products。
3. 在生成的对话框中，单击 OK 开始生成所需的源文件。
4. 再次右键单击该对象 Zynq_RealFFT，选择 Create HDL Wrapper，然后单击 OK 以退出

结果对话框。

设计源树的顶层成为文件。Zynq_RealFFT_wrapper.v 现在可以为设计合成、实现和生成 FPGA 编程位流了。

5.单击 **Generate Bitstream** 启动流的其余部分。

6.在位流生成完成后出现的对话框中，选择 **Open implementation Design** 并单击 **OK**。

步骤6:设置 SDK 并测试 ZYNQ 系统

现在可以将设计导出到 Xilinx SDK 了。在 SDK 中，您可以创建在 ZC702 板上运行的软件(如果可用)。HLS 块的驱动程序是在 Vivado IP Catalog 包的 HLS 导出过程中生成的，必须在 SDK 中为 PS7 软件提供与该块通信的驱动程序。

1.从 **Vivado File** 菜单中选择 **Export > Export Hardware for SDK**。 **注意:为了成功完成这一步，IPI 块设计和实现的设计都必须在 Vivado 工作区中打开。**

2.在 **SDK 导出硬件**对话框中，确保选中了 **Include Bitstream** 选项，然后单击 **OK**。

3.从 **Vivado** 文件菜单中选择 **Launch SDK**。

4.单击 **OK** 启动 SDK。

5.创建一个 **Hello World** 应用程序(也创建 **BSP**)。

a.选择 **文件>新建>应用项目**。

b.输入项目名称 **Zynq_RealFFT_Test**。

c.单击 **Next**。

d.选择 **Hello World**(如果它不是默认的)。

e.单击 **Finish**。

6.启动 ZC702 板并对 FPGA 进行编程。

确保板有所有的连接，让您下载的位流在 FPGA 设备上。请参阅 ZC702 开发板附带的文档。

7.点击 **XilinxTools > 程序 FPGA**。完成的 **LED (DS3)**继续工作。

8.在工作区底部的选项卡中设置一个终端:

a.单击 **Connect** 图标。

b.选择 **连接类型>串行**。

c.选择 **USB UART** 电缆连接到的 **COM** 端口(一般不是 **COM1** 或 **COM3**)。在 Windows 上，如果你不确定，打开设备管理器，在端口(**COM & LPT**)下用 **Silicon Labs** 驱动程序识别端口。

d.将波特率更改为 **115200**。

e.点击 **确定**退出终端设置对话框。

f.在标签标题栏中检查终端是否通过消息连接。

9.在资源管理器窗格中右键单击应用程序项目。Zynq_Design_Test a.在硬件上选择 **Run** 作为>启动。

10.切换到 **Terminal** 选项卡，确认 **Hello World** 已经收到。

11.该项目使用 **C math library(libm)**，因此必须调整构建设置以链接到它。

a. 在 **project Explorer** 窗格中右键单击项目 **Hello World**，并选择 **C/C+ Build Settings**(图 10-42)。

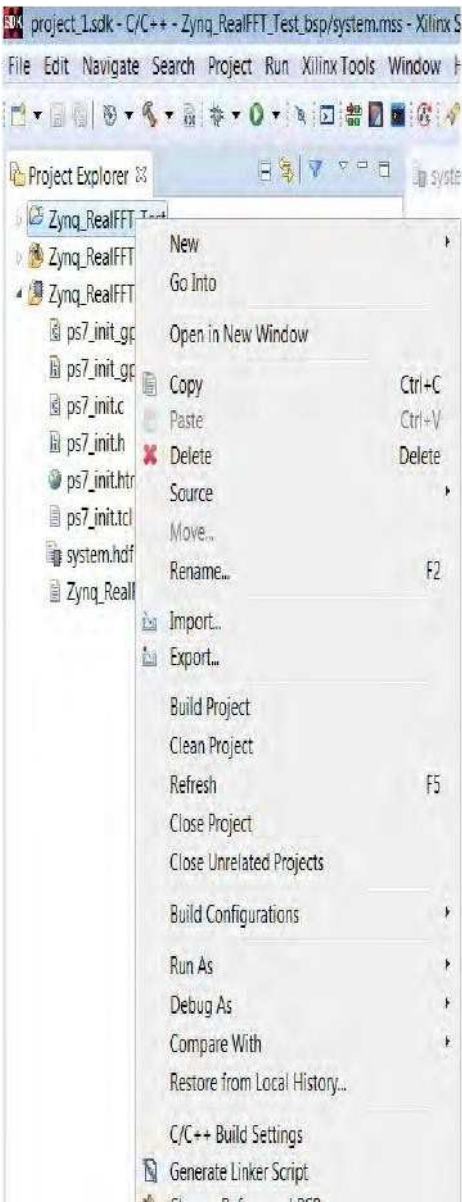


图 10-42: 指定 C/ c++ 构建设置

- b. 添加 Arm gcc 链接器库。
- i. In the Tool Settings tab, select **'Arm gcc linker' > Libraries.**
 - ii. Click the **Add** icon.

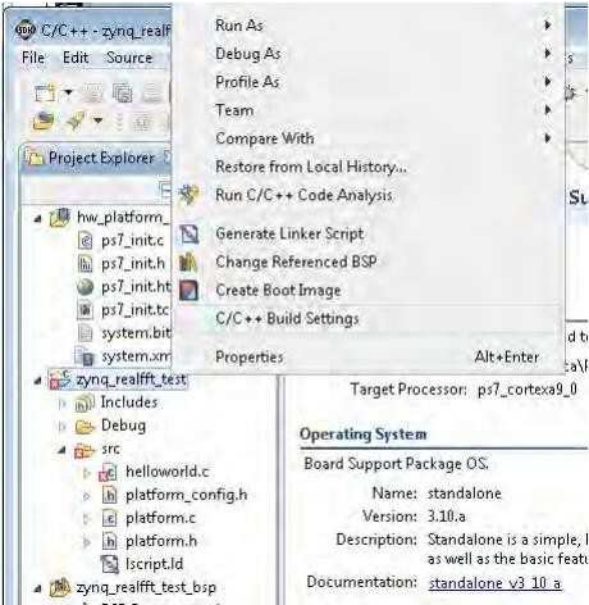


图 10-43: C/ c++ 构建设置

- c. 在“输入值”对话框的文本字段中输入并单击“确定”。

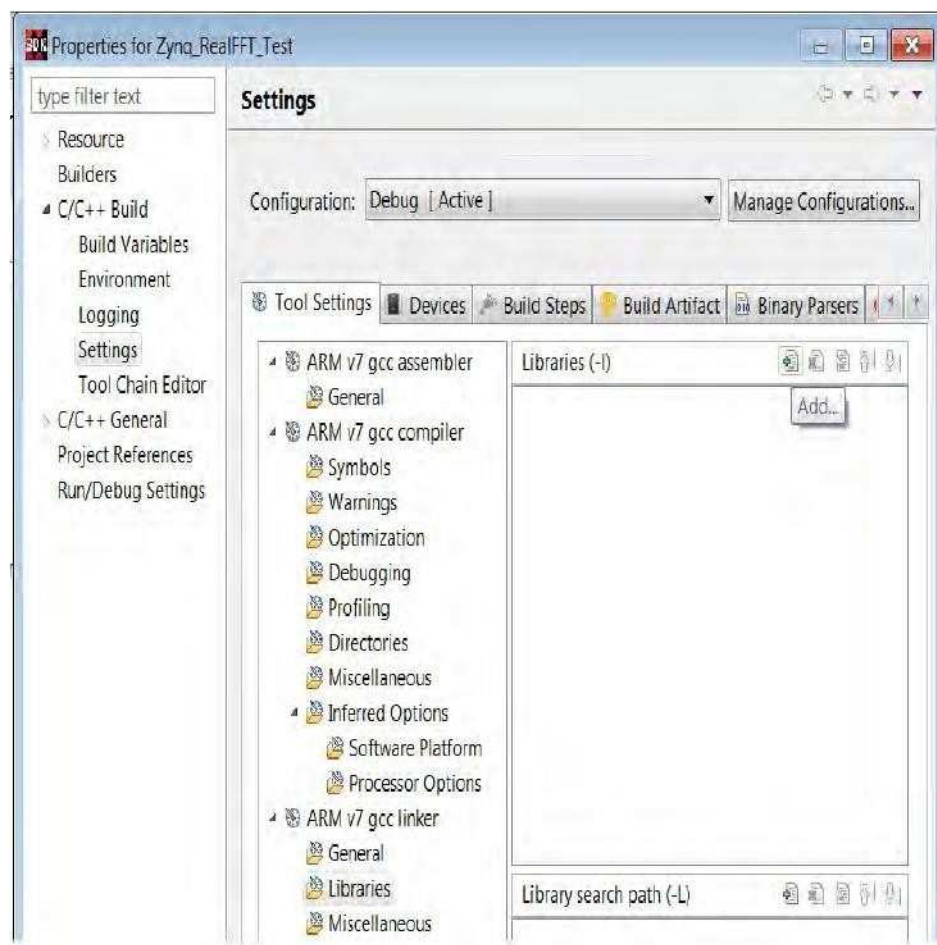


图 10-44: 库设置

d. 单击 OK 退出对话框的属性。 **zynq_realfft_test**

第七步: 修改软件与 HLS 块通信

完全修改过的源文件可以在教程文件集的目录 **arm_code** 中找到。修改将在下面详细讨论。

1. 打开源文件 **helloworld.c**。

2. 必须包含几个 BSP(和标准 C)头文件:

```
#include <stdlib.h> // Std C functions, e.g. exit()
#include <math.h>   // libm header: sqrt(), cos(), etc
#include "xparameters.h" // System parameter definitions
#include "xaxidma.h" // Device driver API for AXI DMA
```

3. 定义 FFT 的(真实数据)变换长度:

```
#define REAL_FFT_LEN 1024
```

4. 定义一个自定义的复杂数据类型, 包含 16 位实数和虚数成员:

```
typedef struct {
    short re;
    short im;
} complex16;
```

5. 在定义 main() 之前声明辅助函数; 他们将在以后定义。

init_dma() 函数包装了所有一次运行的初始化 AXI DMA 驱动程序 API 调用, 并在错误条件返回或退出之前检查硬件初始化是否成功。generate_waveform() 函数使用简单的周期性波形填充数组, 以用作 RealFFT 加速器的输入激励。

```
int init_dma(XAxiDma *axiDma);
void generate_waveform(short *signal_buf, int num_samples);
```

6. 修改 main() 以生成输入数据并将其发送到 RealFFT 加速器, 并通过 AXI DMA 引擎从中接收光谱数据。特别重要的部分将详细讨论。

```
// Program entry point
int main()
{
```

- a. Declare an XAxiDma instance to use as a handle to the AXI DMA hardware:

```
// Declare a XAxiDma object instance
XAxiDma axiDma;
```

- b. Declare variable for local data storage:

```
// Local variables
int i, j;
int status;
static short realdata[4*REAL_FFT_LEN];
volatile static complex16 realspectrum[REAL_FFT_LEN/2];
```

- c. Run platform and DMA initialization functions:

```
// Initialize the platform
init_platform();
print("-----\n\r");
print("- RealFFT PL accelerator test program -\n\r");
print("-----\n\r");

// Initialize the (simple) DMA engine
status = init_dma(&axiDma);
if (status != XST_SUCCESS) {
    exit(-1);
}
```

- d. Generate a stimulus waveform:

```
// Generate a waveform to be input to FFT
for (i = 0; i < 4; i++)
    generate_waveform(realdata + i * REAL_FFT_LEN, REAL_FFT_LEN);
```

- e. Before making the DMA transfer request, the buffer containing the data must be flushed from the processor's data cache. Without this step, the DMA might pull stale data from the DRAM.

```
// *IMPORTANT* - flush contents of 'realdata' from data cache to memory
// before DMA. Otherwise DMA is likely to get stale or uninitialized data
Xil_DCacheFlushRange((unsigned)realdata, 4 * REAL_FFT_LEN * sizeof(short));
```

- f. Request DMA transfer from PS to PL. Enough data to fill the front-end block and the FFT processing pipelines must be sent in order for spectral data to be ready when the PL to PS transfer is requested. Therefore, four data sets are sent before the first output set is requested:

```
// DMA enough data to push out first result data set completely
status = XAxiDma_SimpleTransfer(&axiDma, (u32)realdata,
    4 * REAL_FFT_LEN * sizeof(short), XAXIDMA_DMA_TO_DEVICE);

// Do multiple DMA xfers from the RealFFT core's output stream and
// display data for bins with significant energy. After the first frame,
// there should only be energy in bins around the frequencies specified
// in the generate_waveform() function - currently bins 191-193 only
for (i = 0; i < 8; i++) {
```

- g. Request DMA transfer of a frame of FFT spectral data from PL to PS then poll for completion of the transfer before proceeding.

```
// Setup DMA from PL to PS memory using
// AXI DMA's 'simple' transfer mode
status = XAxiDma_SimpleTransfer(&axiDma, (u32)realspectrum,
REAL_FFT_LEN / 2 * sizeof(complex16), XAXIDMA_DEVICE_TO_DMA);
// Poll the AXI DMA core
do {
    status = XAxiDma_Busy(&axiDma, XAXIDMA_DEVICE_TO_DMA);
} while(status);
```

- h. Before attempting to use the spectral data, the processor's data cache copy of the buffer must be invalidated to avoid use of stale data.

```
// Data cache must be invalidated for 'realspectrum' buffer after DMA
Xil_DCacheInvalidateRange((unsigned)realspectrum,
REAL_FFT_LEN / 2 * sizeof(complex16));
```

- i. Push another set of stimulus data to the PL in order to start the accelerator processing the next frame:

```
// DMA another frame of data to PL
if (!XAxiDma_Busy(&axiDma, XAXIDMA_DMA_TO_DEVICE))
    status = XAxiDma_SimpleTransfer(&axiDma, (u32)realdata,
REAL_FFT_LEN * sizeof(short), XAXIDMA_DMA_TO_DEVICE);
printf("\n\rFrame #d received:\n\r");
```

- j. Do something to verify that the accelerator is functioning. In this case, the spectral data is scanned for bins that contain significant energy. The expectation is to detect only energy in bins around the single tone (192) generated by the generate_waveform() function.

7. 定义生成波形数据集的辅助函数。此版本仅用一个单音填充缓冲区，每个 num_samples 数据窗口具有 192 个周期的单音，其值采用 S.15 定点格式。

```
void generate_waveform(short *signal_buf, int num_samples)
{
    const float cycles_per_win = 192.0f;
    const float phase = 0.0f;
    const float ampl = 0.9f;
    int i;
    for (i = 0; i < num_samples; i++) {
        float sample = ampl *
            cosf((i * 2 * M_PI * cycles_per_win / (float)num_samples) + phase);
        signal_buf[i] = (short)(32767.0f * sample);
    }
}
```

8. 定义一个例程来设置和初始化 AXI DMA 引擎，包装所有只需要在启动时运行一次的驱动程序 API 调用。

```
int init_dma(XAxiDma *axiDmaPtr){
    XAxiDma_Config *CfgPtr;
    int status;
    // Get pointer to DMA configuration
    CfgPtr = XAxiDma_LookupConfig(XPAR_AXIDMA_0_DEVICE_ID);
    if(!CfgPtr){
        print("Error looking for AXI DMA config\n\r");
        return XST_FAILURE;
    }
    // Initialize the DMA handle
    status = XAxiDma_CfgInitialize(axiDmaPtr,CfgPtr);
    if(status != XST_SUCCESS){
        print("Error initializing DMA\n\r");
        return XST_FAILURE;
    }
    //check for scatter gather mode - this example must have simple mode only
    if(XAxiDma_HasSg(axiDmaPtr)){
        print("Error DMA configured in SG mode\n\r");
        return XST_FAILURE;
    }
    //disable the interrupts

    XAxiDma_IntrDisable(axiDmaPtr, XAXIDMA_IRQ_ALL_MASK,XAXIDMA_DEVICE_TO_DMA);
    XAxiDma_IntrDisable(axiDmaPtr, XAXIDMA_IRQ_ALL_MASK,XAXIDMA_DMA_TO_DEVICE);

    return XST_SUCCESS;
}
```

9.保存修改后的源文件。保存文件后，SDK 会自动尝试重新构建应用程序可执行文件。

10.在硬件上运行新应用程序，并验证它是否按预期工作。确保 FPGA 已编程并将终端会话连接到 UART。然后在硬件上启动，就像前面的 Hello World 应用程序代码一样。

结论

在本教程中，您学习了：

如何使用 Tcl 脚本创建 Vivado HLS IP。

- 如何将一个作为 IP 的 HLS 设计导入到 IP 集成器中。
- 如何使用 AXI4-Lite 接口和 axi4 流接口将 HLS IP 连接到 Zynq SoC。
- 如何在软件中配置带有 AXI4-Lite 的 HLS IP。
- 如何在软件中使用 AXI4-Stream 控制 DMAs。

在 DSP 系统生成器中使用 HLS IP

概述

高阶综合产生的 RTL 可以封装成 IP，并在 DSP (Vivado)的系统生成器中使用。本教程展示了这个过程是如何执行的，并演示了如何设计可以用于 DSP 的系统生成器。

本教程由一个实验练习组成。

实验室 1 描述

使用 Vivado HLS 生成设计，并将设计与 DSP 系统生成器一起使用。然后将 HLS IP 加入到系统生成器中进行 DSP 设计和 RTL 仿真。

教程设计描述

您可以从 Xilinx 网站下载教程设计文件。请参阅定位教程设计文件中的信息。本教程使用教程目录

Vivado_HLS_Tutorial\Using_IP_with_SysGen 中的设计文件。

该示例设计是一个 FIR 过滤器，它使用高层综合 `hls::stream` 类建模的流接口。该设计在功能层面实现了完全的流水线化。优化指令作为实用程序嵌入到 C 代码中。

实验一:为系统产生器封装 HLS IP

本实验将高层综合 IP 集成到 DSP 的系统生成器中。重点:本教程中的图形和命令使用教程数据目录

录



Vivado_HLS_Tutorial 是否拉开并放置在位置上 C:\Vivado_HLS_Tutorial.

如果将教程数据目录解压缩到不同的位置，或者在 Linux 系统上，请将引用的少数路径名调整到您选择的目录位置。Vivado_HLS_Tutorial

步骤 1:创建一个 Vivado HLS IP 块

使用提供的 Tcl 脚本为 Vivado IP 目录创建两个 HLS 块。脚本运行 HLS C-synthesis，运行 RTL co-simulation，并打包 IP。

1.打开 Vivado HLS 命令提示符。

在 Windows 上，打开>所有> Xilinx 设计工具> Vivado 2019.1 > Vivado HLS > Vivado HLS 2019.1 命令提示符。

在 Linux 上，打开一个新的 shell。

2.使用命令提示符窗口，将目录更改为

Vivado_HLS_Tutorial \ Using_IP_with_SysGen \ lab1 .

3.键入 **vivado_hls -f run_hls.tcl** 以创建 HLS IP。

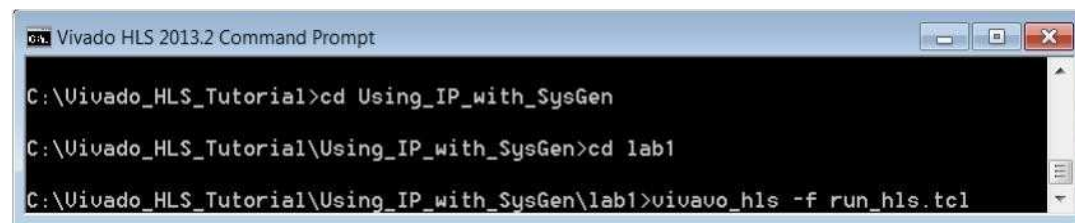


图 11-1: 创建 HLS 设计

用于创建此 IP 的 Tcl 脚本的一个关键方面是命令 **export_design-format sysgen**. 此命令为系统生成器创建一个 IP 包。当脚本完成时，有一个 Vivado HLS 项目目录，其中包含 HLS IP，包括用于 DSP 设计的系统生成器的 IP 包。

本教程的其余部分将展示如何将 Vivado HLS IP 块集成到系统生成器设计中。

步骤 2: 打开系统生成器项目

1. 打开系统发生器的 DSP。

在 Windows 上使用桌面图标。在 Linux 上，打开一个新的 **shell** 并输入 **sysgen**。



图 11-2: 系统生成器图标

2. 当 **Matlab** 调用时，单击 **Open toolbar** 按钮，如图 11-3 所示。



图 11-3: 打开系统生成器设计

3. 导航到教程目录 **Vivado_HLS_Tutorial\Using_IP_with_SysGen\lab1** 并选择如图 11-4 所示的文件 **fir_sysgen.slx**

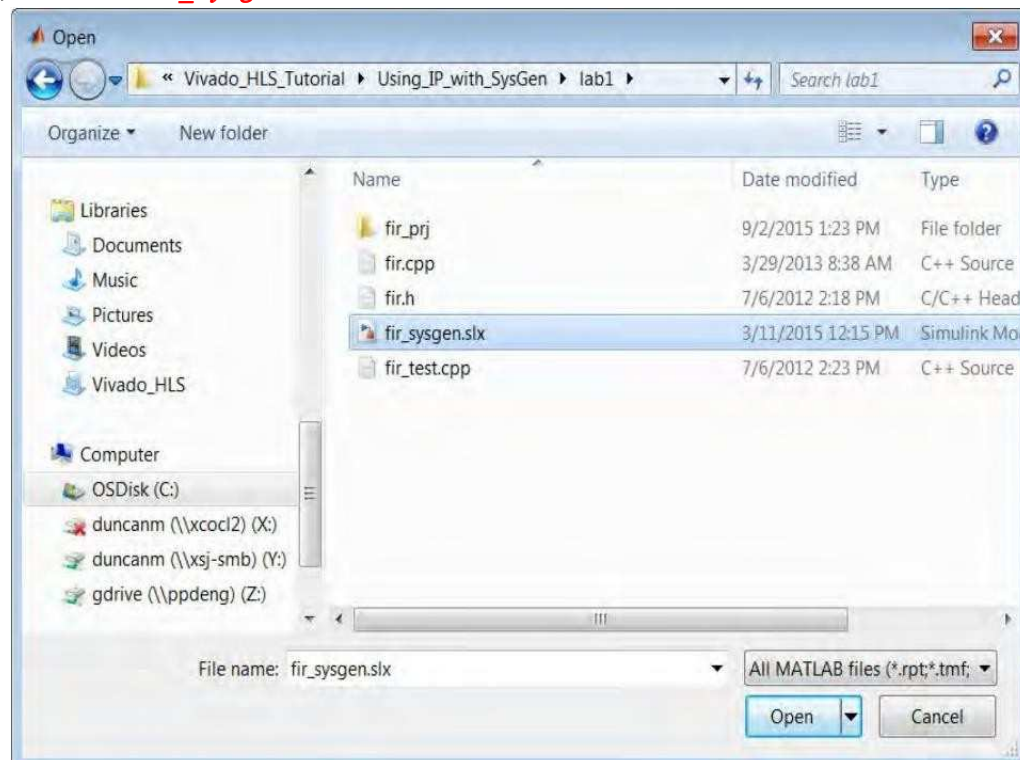


图 11-4: 选择文件 fir_sysgen.slx

当系统生成器调用时，除了 HLS IP 之外的所有块和端口都已在设计中实例化。

4.右键单击画布并选择 **Xilinx BlockAdd**，如图 11-5 所示。

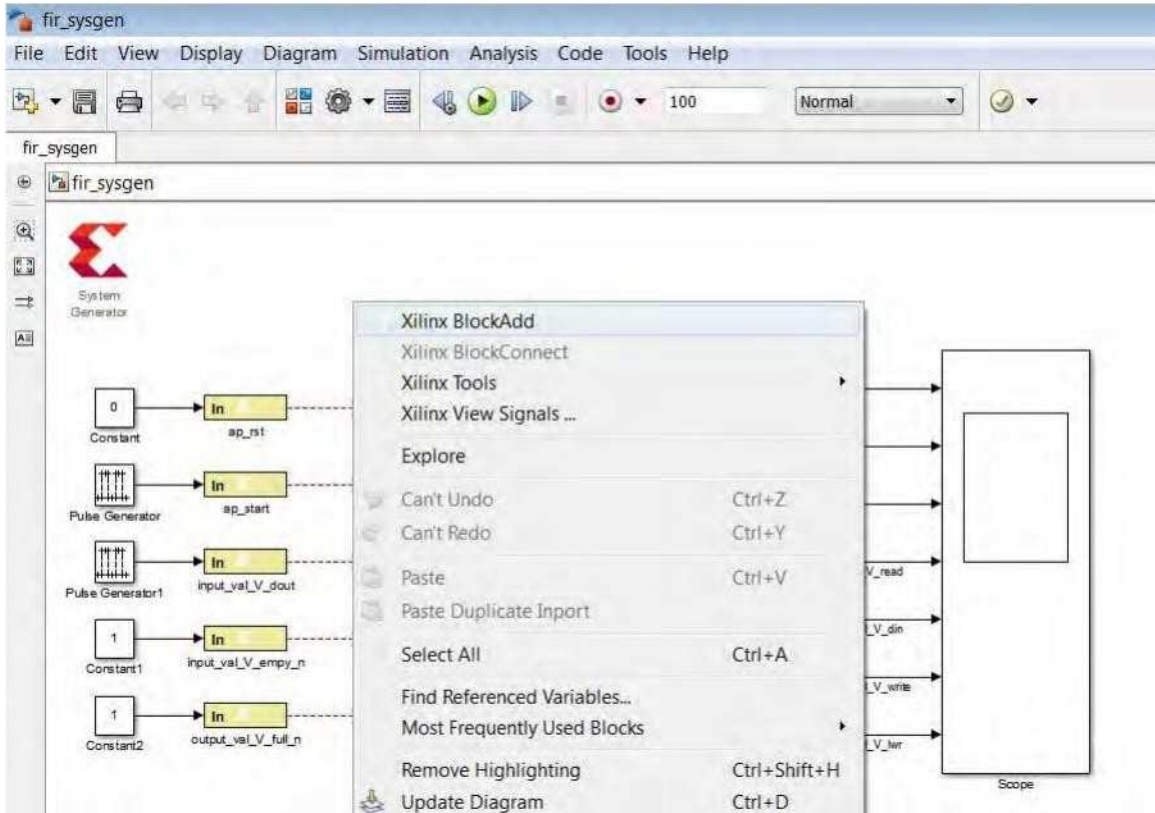


图 11-5: 添加新块

5.在 **Add** 块字段中键入 **hls**

6.选择 **Vivado HLS**。



图 11-6: 选择一个 Vivado HLS IP 块

7.双击 **Vivado HLS** 模块，打开 Vivado HLS 对话框。

8.导航到项目 **fir_prj** 并单击 **Choose** 以选择文件夹 **solution1**。

重要提示:系统生成器为 DSP 使用解决方案文件夹的位置来识别 IP。



9.单击OK 以加载IP 块，如图11-7 所示。

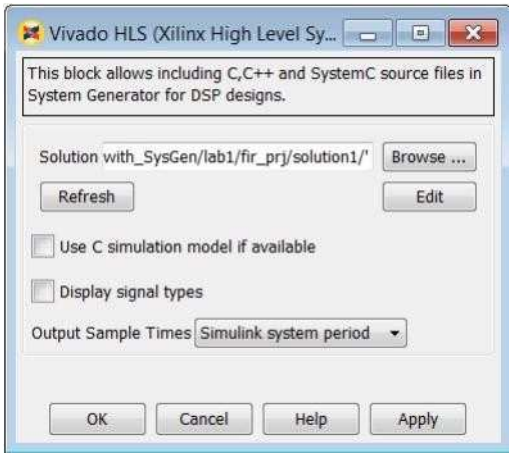


图 11-7:选择 FIR IP 块

FIR IP 块被实例化到设计中。

10.将设计 I/O 端口连接到 FIR IP 块上的端口，如图 11-8 所示。

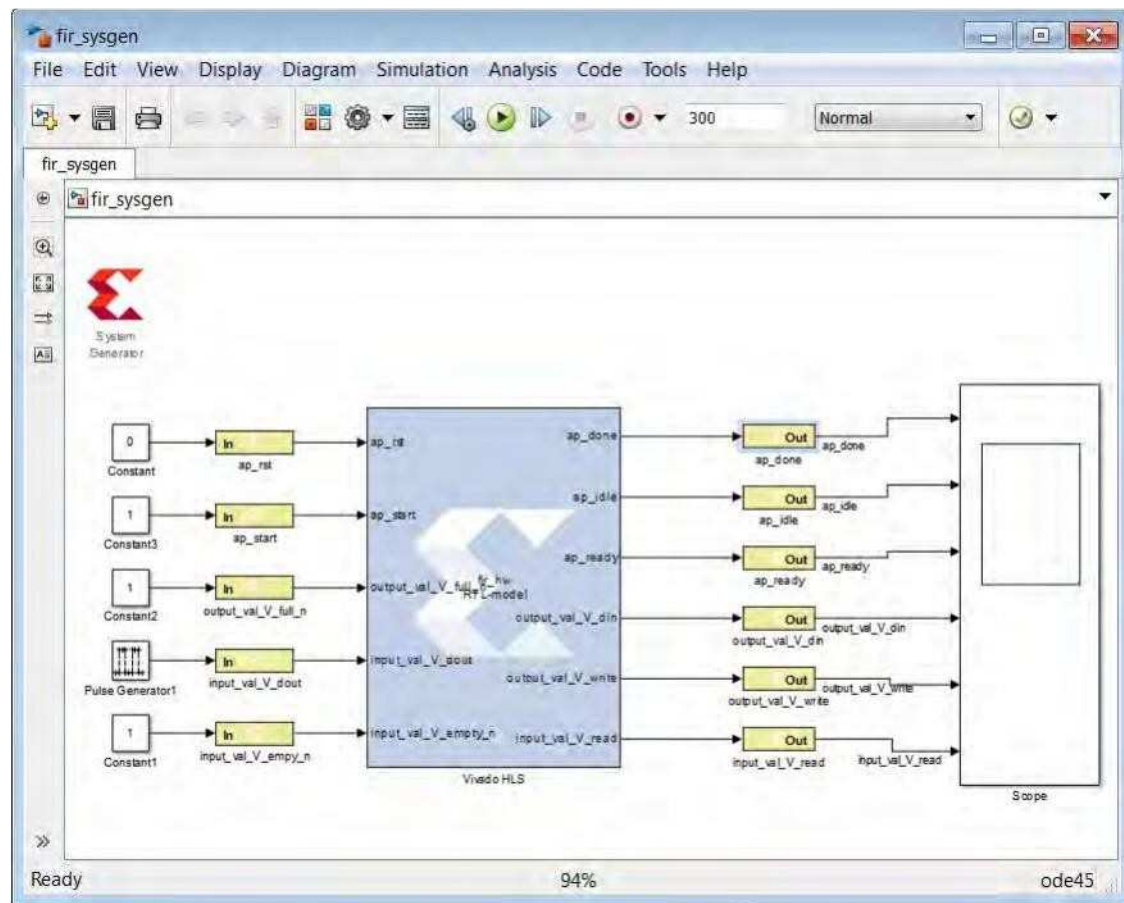


图 11-8:所有连接的设计

确保模拟停止时间为 **300**。

12.单击工具栏上的 Run 按钮执行模拟。

13.双击 Scope 模块查看仿真波形。

结论

在本教程中，您学习了：

- 如何使用 Tcl 脚本创建 Vivado HLS IP。
- 如何导入一个作为 IP 的 HLS 设计到 DSP 的系统生成器

附录 A

附加资源和法律通知

Xilinx 资源

有关答案、文档、下载和论坛等支持资源，请参阅 Xilinx 支持。

解决方案中心

请参阅 Xilinx 解决方案中心，以获得设计周期所有阶段的设备、软件工具和知识产权方面的支持。主题包括设计帮助、建议和故障排除技巧。

文档导航器和设计中心

Xilinx 文档导航器提供对 Xilinx 文档、视频和支持资源的访问，您可以过滤和搜索这些资源来查找信息。打开 Xilinx 文档导航器(DocNav):

- 从 Vivado IDE 中，选择 Help > 文档和教程。
- 在 Windows 上，选择启动>所有程序> Xilinx 设计工具> DocNav。
- 在 Linux 命令提示符处，输入 docnav。

Xilinx 设计中心提供了到按设计任务和其他主题组织的文档的链接，您可以使用这些链接来学习关键概念和解决常见问题。进入设计中心:

- 在 Xilinx 文档导航器中，单击 Design hub 视图选项卡。
- 在 Xilinx 网站上，查看设计中心页面。

注意: 有关文档导航器的更多信息，请参阅 Xilinx 网站上的文档导航器页面。

参考文献

- 1.介绍 FPGA 设计与 Vivado 高级合成(UG998)
- 2. Vivado®设计套件用户指南:高级合成(UG902)
- 3.Vivado 设计套件用户指南:发布说明、安装和许可(UG973) 4。 [Vivado设计套件文档](#)

培训资源

Xilinx 提供了各种培训课程和 QuickTake 视频，以帮助您更多地了解本文中介绍的概念。利用这些连结浏览相关的培训资源:

- 1.基于 c 的设计:高级综合与 Vivado HLS 工具培训课程 2。基于 c 语言的硬件设计人员 HLS 编码培训课程
- 3.基于 c 语言的软件设计人员 HLS 编码培训课程
- 4.Vivado 设计套件 QuickTake 视频教程
- 5。 [Vivado 设计套件 QuickTake 视频教程:Vivado 高级合成](#) 6。 [Vivado 设计套件 QuickTake 视频:开始与高级合成](#) 7。 [Vivado 设计套件 QuickTake 视频:验证您的 Vivado HLS 设计](#) 8。 [Vivado 设计套件 QuickTake 视频:创建不同类型的项目](#)

请阅读:重要的法律通知

本协议项下披露给您的信息(“资料”)仅供您选择和使用 Xilinx 产品之用。在适用法律允许的最大范围内:(1)材料是“按原样”提供的，且存在所有缺陷，Xilinx 特此放弃所有明示、暗示或法定的保证和条件，包括但不限于适销性、不侵权或适合任何特定目的的保证;和(2)Xilinx 概不负责(无论是在合同或侵权行为,包括疏忽,或在任何其他责任理论)的任何损失或损害的任何种类或性质有关,引起,或与材料(包括使用的材料),包括任何直接、间接、特殊、偶然或间接损失或损害(包括数据丢失、利润、商誉、或因第三方提起的任何诉讼而遭受的任何类型的损失或损害)，即使该等损失或损失是合理预见的，或 Xilinx 已被告知可能发生相同的情况。Xilinx 没有义务纠正材料中包含的任何错误，或通知您材料或产品规格的更新。未经事先书面同意，您不得复制、修改、分发或公开展示资料。某些产品受 Xilinx 有限保修条款的约束，请参阅 Xilinx 的销售条款，该条款可在 <https://www.xilinx.com/legal.htm#tos> 查看;IP 核可能受到 Xilinx 颁发给您的许可证中包含的保证和支持条款的约束。Xilinx 产品的设计或意图不是故障安全的，或用于任何需要故障安全性能的应用程序;您承担在此类关键应用程序中使用 Xilinx 产品的全部风险和责任，请参阅 Xilinx 的销售条款，该条款可通过 <https://www.xilinx.com/legal.htm#tos> 查看。

汽车应用程序声明

汽车产品(标识为“XA”零件号)不保证使用安全气囊或用于应用程序部署的影响控制的车辆(“安全应用程序”),除非是一个安全的概念或冗余特性符合 ISO 26262 汽车安全标准(“安全设计”)。客户应在使用或分发任何包含产品的系统之前，对该系统进行彻底的安全测试。在没有安全设计的情况下在安全应用程序中使用产品完全由客户承担风险，仅受有关产品责任限制的适用法律和法规的约束。

Xilinx, Inc.版权所有 Xilinx、Xilinx 标识、Artix、ISE、Kintex、Spartan、Virtex、Vivado、Zynq 等指定品牌均为 Xilinx 在美国及其他国家的商标